



Language specification, module plan, roadmap, verification and tooling contracts

11 documents, 12 source files

source revision: commit af5f88f + uncommitted changes

generated: 2026-09-06 15:34 Romance Daylight Time

generator: scripts/build-docs-pdf.py

Contents

Language specification	11
1. What neper is	11
Who writes neper	11
Goals	11
Non-goals	11
The name	11
2. Files and modules	12
3. Lexical structure	13
Literals, indices and ranges	14
Keywords	14
Naming	15
4. Types	15
Primitives	15
Aliases	15
Narrow floats	15
No implicit conversions	16
Casts	16
Composites	16
Places, & and slicing	17
Tagged unions	18
Vectors (SIMD)	19
Strings	21
Formatting	23
Pointers and nil	24
5. Declarations	24
Attributes	24
Local initialisers	25
Functions	25
Multiple return values	25
External functions	26
e.os	28
6. Statements and expressions	31
Operators	33
Implicit operations	34
Conditional compilation	34
7. Errors	35
Error names and values	35
Zero values	35

Where try may appear	36
8. Memory	37
9. Compile-time parameters	39
Compile-time evaluation	41
Protocols	41
Compile-time introspection	43
Argument packs	45
10. The GPU profile	45
Platforms	45
What runs where	46
Restrictions inside @gpu	46
Kernels and workgroups	47
Device types and slices	47
Shared memory	48
Barriers, memory ordering and atomics	49
Subgroups	50
Capabilities	50
Host side	51
Device discovery and selection	53
Queues and synchronisation	55
CPU execution model	56
11. Build modes and safety	57
Trap protocol	58
Debug fills	58
Floating point	58
12. Compiled modules (.em)	60
Contents	60
Incremental rebuilds	60
13. Toolchain	62
Diagnostics	63
Program entry	63
Testing	64
Linking	66
Target CPU levels	67
Debug information	68
Editor integration	69
Standard library	69
14. Invariants for tooling and generated code	70
15. Compile speed and parallelism	70
Single-thread budget	71
Parallelism	71

16. Standard library: time	72
17. Open questions	73

Module architecture 75

1. Namespace policy	75
2. Delivery tiers	75
3. Dependency layers	76
4. Canonical grouping	77
Layer 0 — foundations	77
Layer 1 — pure primitives	77
Layer 2 — containers and pure domains	77
Layer 3 — platform boundary	77
Layer 4 — host services	77
Layer 5 — language-owned runtimes	78
Layer 6 — application and protocols	78
5. Optional package reservations	78
6. Protocol dependencies	79
7. API ownership	79

Module API catalogue 80

1. Language foundations	80
e.mem	80
e.meta	80
e.math	81
e.simd	81
e.atomic	82
2. Pure foundations	82
e.bytes	82
e.str	82
e.path	84
3. Containers and traversal	84
e.data.list	84
e.data.deque	84
e.data.stack	85
e.data.queue	85
e.data.linked	85
e.data.ring	85
e.data.map	86
e.data.sort	86
e.data.heap	86
e.data.tree	87

e.data.disjoint_set	87
e.data.graph	87
e.data.slot_map	88
e.data.iter	88
4. Pure algorithms, text and cryptography	89
algo.rand	89
algo.uuid	89
algo.hash	89
algo.deflate	90
algo.graph	90
algo.stat	90
algo.bitset	91
algo.complex	91
algo.decimal	91
algo.bignum	92
algo.linalg.matrix	92
algo.linalg.tensor	92
text.encoding	92
text.utf8	93
text.unicode	93
text.normalize	93
text.collate	93
text.locale	94
text.template	94
text.regex	94
gfx.geometry	94
gfx.paint	95
gfx.image	95
text.shape	96
text.layout	96
ui.style	96
ui.layout	96
crypto.hash	97
crypto.mac	97
crypto.kdf	97
crypto.aead	98
crypto.sign	98
crypto.kx	98
crypto.random	98
crypto.x509	98
5. Platform and host services	99

e.os	99
e.cancel	101
e.io	101
text.io	102
e.fs	102
e.fs.mmap	103
e.fs.watch	104
e.proc	104
e.thread	104
e.sync	104
e.channel	105
e.concurrent.queue	105
e.concurrent.map	106
e.task	106
e.time	106
e.time.calendar	107
e.tz	108
6. Language-owned runtimes	108
e.asset	108
e.gpu	108
e.gpu.tensor	109
e.test	109
e.test.support	109
e.test.coverage	110
e.test.fuzz	110
e.debug	110
7. Application facilities and networking	110
e.metrics	110
e.log	111
e.cli	111
e.async	111
e.async.io	112
e.net	112
e.net.tls	113
e.net.http	113
e.net.ws	114
e.db	115
8. Declarative GPU UI	115
gfx.scene	115
ui.asset	116
ui.window	116

ui.input	117
ui.widget	117
ui.animation	118
ui.accessibility	118
ui.testing	118
ui.app	119
9. Interchange formats	119
fmt.json	119
fmt.csv	120
fmt.ini	120
fmt.uri	120
fmt.mime	121
fmt.asn1	121
fmt.pem	121
fmt.multipart	121
fmt.quoted_printable	122
fmt.mail	122
fmt.gzip	122
fmt.zstd	122
fmt.bzip2	123
fmt.lzw	123
fmt.zlib	123
fmt.zip	123
fmt.tar	123
fmt.yaml	124
fmt.xml	124
fmt.html	124
fmt.html.template	125
fmt.png	125
fmt.jpeg	125
fmt.webp	126
fmt.bson	126
fmt.msgpack	126
fmt.protobuf	126
10. External packages	127

Roadmap 128

S0 — Specification, API and harness freeze	128
M0 — Bootstrap compiler, walking skeleton	129
neper-0 — the bootstrap subset	129
M1 — The full CPU language	134

M2 — Self-hosting and .em modules	134
M2.5 — LLM language and compiler hardening	135
M3 — GPU	136
M4 — Breadth and depth	137
Later toolchain-library waves — unnumbered	137
M5 — Native Metal	138
M6 — pacman package manager	138
Cross-milestone verification and release claims	139
Deliberately not scheduled	140
Decisions	141
D84 — standard-library composition and stable dependency closure	169
Consequences accepted	169
Diagnostic fixtures	170
tests/selfhost/fixtures/diagnostics/src/broken.e	170
tests/selfhost/fixtures/diagnostics/src/main.e	170
General-purpose verification	171
1. Claim under test	171
2. Questions that must be answered with programs	171
Memory and lifetime	171
Resource cleanup	172
Abstraction	172
Errors and diagnostics	172
Concurrency and services	172
Interoperability and platforms	172
Project scale	172
3. Reference workload corpus	173
4. LLM-generation evaluation	173
Task set	174
Metrics	174
Initial acceptance thresholds	174
Post-M2 evaluation before M3	174
5. Semantic and implementation verification	175
Conformance	175
Determinism	175
Adversarial safety	175
Portability	175
Operational behavior	175

6. Replacement matrix	176
7. Release gates	176
Language-complete gate	176
Tool-complete gate	176
General-purpose gate	176
Performance-language gate	176
8. Decision rule when verification fails	176
Grammar	178
pacman package manager	183
1. Boundary and principles	183
2. Tier and namespace policy	183
3. Project manifest: project.yaml	183
4. Lockfile: project.lock	185
5. Resolution	185
6. Sources: registry, Git, and path	186
Registries	186
Git	186
Paths	186
7. Global immutable cache	186
8. CLI	187
9. Tasks, not lifecycle hooks	187
10. Publishing and trust	188
11. Failure, offline, and reproducibility rules	188
12. Implementation sequence and acceptance	188
13. Explicit deferrals	189
Tooling protocol	190
1. Version and stream envelope	190
2. Sources and spans	191
3. Diagnostics and fixes	191
4. Token and syntax commands	192
5. Symbol and reference index	192
6. Formatting contract	193
7. Test, build and command results	193
8. Generated source maps	193
9. Conformance and compatibility	194
UI framework	195

1. Goal and boundary	195
2. Architecture	195
3. Declarative code	196
4. Reconciliation and state	197
5. Layout, paint and text	197
6. Input, frames and accessibility	197
7. Platform and GPU prerequisites	198
8. Resource limits and failure	198
9. Testing and delivery	198
Source manifest	199

Language specification

docs/spec.md | 284280 bytes | sha256 ceb18f69bd845468

The neper language — specification (draft 0.1)

Status: design draft. Implementation coverage is tracked milestone by milestone in `roadmap.md`; implemented subsets do not imply full conformance.

The post-M2 design revision is scheduled in `post-m2-11m-hardening.md` as the mandatory M2.5 gate before M3. Its ownership, lifetime, check-policy and tooling proposals do not change the current rules below until versioned normative amendments land during M2.5.

Neper is a compact, ahead-of-time general-purpose language for model-generated software, with explicit memory, deterministic semantics, inexpensive abstraction, native interoperability, and first-class CPU/GPU execution.

1. What neper is

A small procedural language, designed from the start to be **written by language models and read by people**. Records, structs and unions live on the stack. Memory comes from arenas you create explicitly. Code compiles straight to machine code for CPUs and to SPIR-V/PTX for GPUs, with no LLVM in the path.

Who writes neper

The expected author is a model; the expected reader is a human reviewing what that model produced, or a tool searching and amending it. neper is meant to be read fluently and audited quickly. It is **not** optimised for being typed by hand all day, and that trade is deliberate rather than incidental.

The consequence runs through every decision in this document: **where verbosity buys unambiguity, verbosity wins**. Explicit casts, qualified call sites, no overloading, no macros, one definition per name — each costs keystrokes and each removes a question the reviewer would otherwise have to answer by reading more code. Keystrokes are spent by a generator; ambiguity is paid for by the reviewer and by the next model to touch the file. They are not the same currency.

Goals

1. **Built for generated code** — every construct is unambiguous to produce, to search, to amend in place, to debug and to test (§14). A model should not be able to write a line whose meaning depends on context it cannot see.
2. **Readable by a human reviewer** — no hidden control flow, no implicit allocation, no invisible conversions. What is on the page is what runs.
3. **Outstanding execution speed** — direct control over layout and SIMD, and a direct emitter.
4. **Outstanding compile speed** — target: 1M+ lines/sec/core, whole-project rebuild in well under a second. Generated code is regenerated often; a slow compiler taxes every iteration.
5. **One language, CPU and GPU** — the same syntax, with a restricted GPU profile.
6. **Easy to debug and test** — checks on in debug builds, precise machine-readable diagnostics, built-in testing (§13), GPU kernels runnable on the CPU.

Non-goals

No classes, inheritance, interfaces, closures, exceptions, garbage collection, operator overloading, function overloading, textual macros, RTTI, or implicit numeric conversions. Each of these either costs performance, costs compile time, or makes a name ambiguous to a grep.

Also a non-goal: **terseness for the typist**. Shorthand that saves a human keystrokes at the cost of a reader's certainty is a bad trade here, because the typist is not the constraint.

The name

A neper (Np) is the natural-log unit of ratio — the quiet sibling of the bel. Fitting for a language about amplification with nothing wasted.

2. Files and modules

- Source files use the extension `.e`.
- **One file is one module.** The path below its **source root** is the module name, `/` becoming `.`: `lib/e/mem.e` is module `e.mem`, `src/main.e` is module `main`, `src/parse/expr.e` is `parse.expr`. The source roots are exactly `lib/` and `src/` under the project root; a file elsewhere is not a module unless it is the one named to `neper run` or `neper build` (below). A per-target suffix (§6) is not part of the name.
- A file that declares `fn main` is a **program root** when it is the file named to `neper run` or `neper build`. `neper run <file.e>` (§13) takes one; `neper build <file.e>` takes one or any other `.e` file, and compiles a file without `main` — `examples/vec3.e`, `examples/list.e` — together with the modules it reaches to `.em` files and no executable. The named file is a module whether or not it lies under a source root: under one it is named from that root as any module is; elsewhere its bare filename is its module name, so `examples/hello.e` is module `hello`, `examples/` need not be a package, and a program root may be run from any directory. A program root is the **root module** of the executable it produces and holds `main`. `main` is special only there: in any other module a `fn main` is an ordinary function — any signature, called and qualified like any other, `lex.main` — and nothing about it is checked or run unless that module is the one named on the command line. The root that `neper test` synthesises (§13) reaches the module under test through a compiler-internal name that appears in no source file and is exempt from the import-name rule below — generated code is not subject to source-level invariants — which is also how a module named `main` is tested.
- The **project root** is the nearest ancestor directory of the named file — or, for a command that names no file (`neper fmt`, `neper test`, `neper index`, §13), of the working directory — that contains a `lib/` or a `src/`; when none does, it is the file's own directory or the working directory, and the project has no modules of its own. The toolchain's own `lib/`, beside the `neper` binary, is a third source root on every command: it is where `e.*` lives, which is how `use e.mem` resolves from any directory. A module name is looked up under the project's `lib/` and `src/` first and under the toolchain's `lib/` only when neither defines it, so a project's `lib/e/mem.e` replaces the toolchain's `e.mem` — which is how `lib/e` is built and tested in place: from the repository, `neper test` runs the tests of every module under the project's two roots, `e.*` among them. A module defined under both project roots — `lib/x.e` and `src/x.e` — is a compile error naming both files.
- There is no `module` declaration. The filename is the single source of truth, so a glob locates any module without reading a byte of it.
- There are no headers, no forward declarations, and no separate interface files. Module scope is order-independent: a declaration may be inserted anywhere.

Imports

```
use e.mem
use e.io
use vendor.collections.map as vendor_map
```

`use` binds the **last segment** as the qualifier unless an explicit alias is written. Every cross-module reference is qualified at the point of use:

```
var a = mem.arena_from(buf)
let item = vendor_map.get(...)
```

There are no wildcard imports. An alias is written `use <module> as <qualifier>`, is local to that importing module, must be `snake_case`, and may not equal another qualifier or any value/function/error name there. A qualifier may equal a type name, because those are separate position-selected namespaces (below). If two imported modules would bind the same last segment, at least one must be aliased. Thus every written qualifier still means exactly one module while independently generated dependencies do not have to coordinate their final path segments across the ecosystem. `neper fmt` preserves an explicit alias even when it equals the default qualifier: source generators may use that form to make their naming policy explicit.

The `use` graph is a DAG. A cycle — `a` uses `b` and `b` uses `a`, directly or through any number of modules — is a compile error naming every module in the cycle. Nothing is gained by allowing one: module scope is order-independent within a file, but a module's interface (§12, §15) is built from the interfaces of the modules it uses, and a cycle has no first module to build.

A qualifier is a module-scope value-namespace name and falls under §14 invariant 1: a local, parameter, function, value or error declaration that reuses it is a compile error, so `mem.alloc[u8](a, n)` can never be a call through a field of some local named `mem`. Module qualifiers and type names are separate namespaces resolved by position — `str` the qualifier in `str.eq(x, y)` and `str` the type in `name: str coexist (§4)` — and the builtin target namespace (§6) is reserved the same way as a qualifier.

A name before a `.` is resolved in a fixed order: as a *use* qualifier first, then as a type — `u8.trunc(x)` (§4), `Node.Tag` (§4), `Kind.Int` (§4) — then as a comptime type parameter or a comptime type value, whose protocol function is looked up at the instantiation (`T.hash(v)`, §9), then as a value whose field is named. A protocol receiver need not be a name at all: any expression whose type is `type` and whose value is comptime-known stands in that position, `f.ty.format(x, b)` (§9). The order can never be ambiguous: the first three sets are disjoint, and a name in the fourth is a type value only when its type is `type`, which has no fields. A qualifier cannot be redeclared (above); a comptime type parameter is a parameter, so §5 forbids it from reusing a module-scope type name, a *use* qualifier or a builtin type name; and the **builtin type names** — the primitive type names of §4, `Vec`, `Mask` and `Atomic` — are reserved: a *type*, *fn*, *const*, *var*, parameter or local reusing one is a compile error, and no module may be named after one. An alias is not reserved: `str` is an alias (§4), which is why `e.str` is a legal module.

3. Lexical structure

The normative token and concrete-syntax grammar is `grammar.ebnf`. This section explains its consequences; where an example and the grammar disagree, the grammar controls concrete syntax and this document controls typing and semantics. A conforming implementation publishes the grammar revision it accepts through `neper info --json` (§13).

- A source file is valid UTF-8. One leading UTF-8 BOM is accepted and ignored; `neper fmt` removes it. Before tokenization, CRLF and bare CR are normalized to LF. NUL is always a source error. Every other ASCII control character except LF and TAB is forbidden everywhere; normal strings and characters represent controls only with the escapes below. TAB is permitted as data inside a raw string or comment, but outside a literal or comment it is an error rather than indentation whose width differs by tool. Horizontal source whitespace is otherwise ASCII space only. Line numbers count normalized LF-separated lines from one.
- Tokens use **longest match**. In particular `...` precedes `..` and `.`, `<<` and `>>` precede `<` and `>`, and the wrapping operators `+`, `-`, `*` are single tokens. A numeric suffix is part of its literal token and may have no intervening whitespace. `1..2` is the three tokens `1`, `..`, `2`; a decimal float therefore requires digits on both sides of its dot. A leading sign is always a separate prefix token.
- Comments: `//` to end of line. No block comments (they break line-oriented edits). A consecutive run of `///` lines immediately before a declaration, field or member at the same indentation is its documentation; one optional space after `///` is removed and the lines join with LF. A blank line or ordinary `//` breaks attachment. Documentation is returned by `neper index`, so API-reading tools do not infer it from nearby trivia.
- Identifiers: `[A-Za-z_][A-Za-z0-9_]*`. Case-sensitive. `_` alone is reserved and is not an identifier: it is the discard in a `let` (§5) and the inferred length in an array literal (§4), and it can be neither declared nor referenced.
- Integer literals: `123`, `0xFF`, `0b1010`, `0o777`, `1_000_000`, with an optional type suffix: `42u64`, `3i32`, `255u8`, `1usize`.
- Float literals: `1.5`, `1e-9`, with an optional type suffix: `1.5f32`, `2.0f64`, `1.0f16`, `1.0bf16`.
- Source is UTF-8. String literals: `"..."` with the escapes `\n` `\t` `\r` `\\` `\"` `\'` `\0` `\xNN`; the bytes between the quotes are stored as written, so a non-ASCII character is its UTF-8 sequence. A string literal is a `[]const u8` pointing at read-only data. No null terminator unless written.
- A raw string is `r"..."` or `r#"..."#` through eight `#` characters. It has no escapes, may contain normalized newlines, and ends only at `"` followed by the same number of `#` characters. Its value is the UTF-8 bytes between the delimiters and its type is `[]const u8`. `neper fmt` chooses the smallest delimiter count that leaves the bytes unchanged, so generated JSON, regular expressions and source text need no escape-heavy construction. A raw string cannot serve where a rule specifically requires a one-line string literal, such as `@import` or `unreachable`'s optional message.
- Character literals: `'a'` is a `u8`. It takes the string escapes — `'\'`, `'\n'`, `'\xNN'` — and must denote exactly one byte, so a non-ASCII character between the quotes is a compile error; write its bytes with `\xNN`.

- The tokenizer emits every normalized newline as a `NEWLINE` token. The parser treats a `NEWLINE` inside an unclosed `(` or `[`, inside the `{ }` of a **literal** — a struct, union, array, union enum or Vec literal, whose `{` follows a type name — or inside the `{ }` of a **type body** — the `{` after struct, union, enum T or union enum T in a type declaration — is whitespace: that is how a call, a literal, a signature or a type declaration continues across lines, and it is the only place neper fmt breaks a line — at 100 columns, one element per line (§14). The `{` that opens a **block** — after a signature, if, else, while, for, switch, when, defer (its block form, §6) or @nocheck — does not suspend the rule: every newline inside a block ends a statement. A newline anywhere else ends the statement; there is no backslash continuation.
- **No continuation crosses a declaration.** A `(`, `[`, or a literal's or type body's `{` that is still open when a column-0 declaration keyword is reached — use, type, const, var, fn, error, extern, or an attribute line (§5) — is a compile error, reported at the **opening** bracket and naming the keyword that ended it:

```
src/parse.e:312:24: error[E-SYNTAX-0012]: `` opened here is still unclosed at `fn` on line 318
```

§14 invariant 2 guarantees nothing else begins a line at column 0, so the parser has a barrier it can trust: one dropped bracket costs one precise diagnostic instead of a cascade that reinterprets the rest of the file. It is the failure every implicit-continuation language has, and the invariant that makes `^fn` greps exact is what removes it here.

- **No continuation crosses its containing block.** A soft delimiter opened inside a block that remains open when the closing that block is reached produces one error at the opening delimiter; the parser then resumes after that `}`. In a switch, case and default at the switch body's brace depth are additional barriers. Recovery affects only which later diagnostics are suppressed, never whether an otherwise valid program is accepted. The conformance corpus (§13) fixes the required primary diagnostic and recovery point for every delimiter.
- Blocks use `{ }`. Braces are required — there is no single-statement if.

Literals, indices and ranges

An unsuffixed integer or float literal is an **untyped compile-time value**. It takes the type its immediate context demands — the other operand of a binary operator (except a shift count, §6), the parameter it is passed to, the field or element it initialises, the annotation on its binding, the declared return type where it stands after `ret`, the subject's type where it stands as a switch case label, the left side's type where it stands on the right of a compound assignment (`x += 1`), the comptime parameter it is bound to in a `[...]` argument, or the parameter type a checked comptime signature (§9) supplies — and the value must be representable in that type, or it is a compile error. With no context there is no default: `let x = 3` and `const N = 4096` are compile errors, not `i32`. Write the annotation or the suffix: `let x: i32 = 3`, `let x = 3i32`. A suffixed literal has its suffix's type and supplies context to whatever it meets. An expression built only from untyped literals and operators — `64*1024` — is itself an untyped compile-time value: it is folded exactly, at unbounded precision, and takes its type from context exactly as a single literal does, so `[64*1024]u8` is well-typed because an array length is `usize` context. An array length is an expression that must have type `usize` once its literals are typed: `[N*2]u8` is well-typed with `const N: usize` and a compile error with `const N: u32`, since nothing widens it (§4).

Expression	Type
<code>s[i]</code> on a slice or array	the index <code>i</code> is <code>usize</code>
<code>s.len</code>	<code>usize</code>
<code>a..b</code>	the type of its bounds, which must agree after literal typing; two untyped bounds are a compile error
<code>for i in a..b</code>	<code>i</code> has the range's type
<code>for i, v in s</code>	<code>i</code> is <code>usize</code>
<code>gpu.gid, gpu.lid, gpu.wgid</code> fields <code>(.x .y .z)</code>	<code>u32</code> (§10)

So for `i in 0..n` with `n: usize` is well-typed, for `i in 0..1000` is not (write `0usize..1000` or `0i32..1000`), and indexing a slice with a `u32` invocation id needs `usize(i)` written down. Under §11 the width of an expression decides where it wraps, so the type of every literal has to be legible from the line it appears on and the one signature that line calls — never from anything further away (D27).

Keywords

```
use type const var let fn ret if else while for in switch case default
break continue defer try struct union enum error when true false nil ok
```

as zero undef extern unreachable shared

`unreachable` is a keyword because `unreachable()` is a builtin that needs no use and cannot be redeclared (§11). `shared` is a keyword because `shared var` is a statement and `[]shared T` a type, both legal only in device code (§10).

Naming

Naming is part of the language, not a style guide. `neper fmt --check` rejects a violation the same way it rejects bad indentation. The point is predictability: if you know something is a function, you know how it is spelled, so `grep "^fn parse_"` finds every parser entry point without guessing at casing.

Kind	Convention	Example
Functions	snake_case	<code>parse_expr</code> , <code>is_power_of_two</code>
Variables, parameters, fields	snake_case	<code>node_count</code> , <code>arena</code>
Modules and files	snake_case	<code>e/map.e</code>
Types	PascalCase	<code>Vec3</code> , <code>Arena</code> , <code>HashMap</code>
Enum members	PascalCase	<code>.Float</code> , <code>.NotStarted</code>
Errors	PascalCase	<code>error NotFound</code>
Constants (const)	SCREAMING_SNAKE	<code>MAX_NODES</code>
Comptime parameters of kind <code>type</code> or <code>fn</code> (§9)	PascalCase	<code>T</code> , <code>Ctx</code> , <code>K</code>
Comptime parameters of a value kind — an integer, <code>str</code> , <code>bool</code> , an array, a comptime-only struct value (§9)	SCREAMING_SNAKE	<code>N</code> , <code>FMT</code> , <code>IDX</code> , <code>FIELD</code>
Comptime binding from an unrolled <code>for</code> (§9)	snake_case	<code>f in for f in meta.fields[T]()</code>

Casing therefore tells you the kind of a name on sight, with no lookup: `Foo` is a type, an error or a type-kind comptime parameter, `F00` is a constant or a value-kind one, `foo` is a function or a value. The `SCREAMING_SNAKE` rule covers comptime **parameters** only: the binding an unrolled comptime `for` introduces (§9) is a local binding like any other and is spelled like one, even though its value is known at compile time.

4. Types

Primitives

Category	Types
Signed	<code>i8 i16 i32 i64 isize</code>
Unsigned	<code>u8 u16 u32 u64 usize</code>
Float	<code>f16 bf16 f32 f64</code>
Other	<code>bool void err type</code>

Aliases

`str` is an alias for `[]const u8` — not a primitive, and not a distinct type: there is no string type, and `str` is not a reserved name (§2). It is the alias form of `type` (Composites, below) applied by the language itself.

Narrow floats

`f16` is IEEE 754 binary16; `bf16` is bfloat16 — same exponent range as `f32`, eight bits of significand precision. Both are **storage types first**: they halve memory traffic, which is usually the point. Arithmetic on them is defined as *convert the operands to f32, perform the named f32 operation, then round once to the narrow type after every operation*. This widen-operate-narrow sequence is the language result, including for division and square root; it is deterministic across targets but is not claimed to equal a hypothetical directly-rounded native narrow operation in the double-rounding edge cases.

Where a target has native narrow arithmetic, the emitter may use it only when that instruction is proven to implement the same widen-operate-narrow result; otherwise it widens, operates and narrows explicitly. `bf16` therefore computes via `f32` on SPIR-V and, despite native instructions from `sm_80`, uses only instructions or a sequence that preserves these semantics on PTX. §13 lists available instructions; availability alone never changes the language result.

There is no implicit conversion between any two float widths (No implicit conversions, below). `f32(h)` and `f16(x)` are written down like every other cast.

No implicit conversions

No implicit conversions, not even widening. `u32(x)` and `f32(y)` are explicit casts. This is verbose by design: it makes every conversion greppable and removes a whole class of silent bugs from generated code.

The sole exception is **losing mutability**: `[]T` converts to `[]const T` and `*T` to `*const T` implicitly. It is one-way, it can only remove a capability, and without it every read-only parameter would need a cast at every call site. These two, with pointer auto-dereference in `a.b`, the `Buf[T]`-to-slice mapping at a kernel launch, and the compiler-generated address of a mutable iterator, are the five implicit operations, kept in one place in §6.

Casts

`T(x)` is the checked conversion form. `T.trunc(x)`, `mem.cast` (Composites, below), `mem.bitcast` (§8 — the bytes of a value read as another type of the same size) and `simd.convert` (Vectors, below — lane-wise `T(x)`) are the only other conversion forms, each greppable by name, and there are no more. A cast is not a typing context for an untyped literal (§3): `u8(300)` is a compile error because `300` has no type to be cast from; write `300u16` or a value.

Between integer types `T(x)` is **checked**: the value must be representable in `T`. `u32(x)` with a negative `x`, or `u8(n)` with `n`: `u32 = 300`, is a narrowing cast that fails; in a debug build it traps (§11), in a release build it yields the C result — the source value sign- or zero-extended as its own type dictates, then cut to the width of `T`. A truncation that is *meant* is spelled `T.trunc(x)`: `u8.trunc(x)` keeps the low eight bits in every build mode and says so on the page, the counterpart of `+` for arithmetic. A widening cast between integer types can never fail and costs nothing beyond the extension.

Between and across the other scalar types the rules are these, and `simd.convert` applies them lane by lane:

Cast	Result
float → integer, <code>i32(f)</code>	rounds toward zero; a value outside T's range, or NaN, is a narrow check (§11): trap in debug, and in release the result saturates — T's minimum or maximum for an out-of-range value, 0 for NaN — on every target: aarch64 does so natively, x64 by a compare sequence around <code>cvttss2si</code> . There is no extra form
integer → float, <code>f32(i)</code>	rounds to nearest, ties to even; never fails
float → wider float	exact
float → narrower float	rounds to nearest, ties to even; overflow gives the infinity, never a check
<code>f16 ↔ bf16</code>	same width, different split: rounds to nearest, ties to even; overflow gives the infinity, never a check
bool → integer, <code>u8(b)</code>	0 or 1
integer → bool	not a cast: write <code>x != 0</code>

An integer-to-enum cast, `Kind(x)`, is checked against the declared members the same way: a value that names no member traps in every build mode. Unlike integer narrowing there is no useful C result—an invalid discriminant would break exhaustive switches—so this validity check is never removed. `u8(k)` on an enum `u8` is always representable. A cast between an enum and an integer of a **different width** goes through the backing type first, then an integer cast — `u32(u8(k))`, `Kind(u8(x))` — and a direct different-width cast is a compile error. Casting to or from `err` is not a cast at all; an `err` is only ever an error declaration or ok (§7).

Composites

<code>*T</code>	pointer
<code>*const T</code>	pointer to immutable data
<code>[]T</code>	slice — a { ptr, len } pair, 16 bytes on 64-bit
<code>[]const T</code>	slice of immutable data
<code>[N]T</code>	fixed array, passed by value
<code>*void</code>	opaque pointer — C's <code>void*</code> ; exists for extern signatures (§5)

`const` qualifies the *pointee*, not the binding — `let` already makes a binding immutable. There is no way to cast it away: if you need to mutate, copy. It qualifies the pointee of a pointer or slice type — `*const T`, `[]const T`, and §10's

address-space-qualified device forms `*const shared T` and `[]const shared T` — and those are the only positions it appears in.

`*void` cannot be dereferenced, indexed or sliced. A pointer converts to another pointer type — `*void` included — only through `mem.cast[*Foo](p)`, an intrinsic of `e.mem` that reinterprets the address and does nothing else. It **preserves** `const`: a `*const T` can become only a `*const U`, so the qualifier is never cast away by this route either. It is CPU-profile only: §10 bans it inside `@gpu`, where address spaces are distinct.

Places, & and slicing

Whether `&e`, `e[lo..hi]` or an assignment `e[i] = v / e.f = v` gets the mutable or the `const` form is decided by **the innermost pointer or slice step** on the path to the place — the last `*T`, `*const T`, `[]T` or `[]const T` that was walked through to reach it. Only when the path has no such step — the place is a value held directly in a binding: an array, a struct, a scalar — does the binding decide, and there `var` gives the mutable forms and `let`, a parameter and a `const` give the `const` forms. A parameter or `let` of pointer or slice type is an immutable binding whose pointee keeps the qualifier written in its type, so it contributes its own step.

Innermost step (or the binding, if there is none)	<code>&e</code>	<code>e[lo..hi]</code>	<code>e[i] = v, e.f = v</code>
<code>*T</code> , <code>[]T</code> , or a <code>var</code> binding with no step	<code>*T</code>	<code>[]T</code>	legal
<code>*const T</code> , <code>[]const T</code> , or a <code>let</code> / parameter / <code>const</code> binding with no step	<code>*const T</code>	<code>[]const T</code>	compile error

`*p = v`, assignment through a dereference, is under the same rule with `p`'s own type as the step: it requires `p: *T` and is a compile error on a `*const T`.

Worked through the shipped examples: in `fn slice[T](l: *List[T]) -> []T { ret l.items[0..l.len] }` the path is `l` (`*List[T]`) then `.items` (`[]T`); the innermost step is `[]T`, so the result is `[]T` and the declared return type is right. In `saxpy(..., y: []f32), y[k] = v, &y[k] (*f32)` and `y[0..n] ([]f32)` are all mutable, because `y`'s own type is the step. `let v = Vec3{...}; let p = &v; p.x = 1.0` has no step, so `p` is `*const Vec3` and the assignment is a compile error — the guarantee D18 was recorded for. `let targets = []str{...}` sliced as `targets[0..]` is a `[]const str`, and every read-only library parameter is declared `[]const T` (Strings, below) so that it fits without a cast.

`e[lo..hi]` is the slice from `lo` (inclusive) to `hi` (exclusive); `e[lo..]` runs to `e.len`. Bounds are `usize` (§3) and are bounds-checked in debug builds (§11). `.len` on a slice or an array is `usize`.

`[]const T` is **shallow**: with `T = []u8`, the elements of the outer slice cannot be reassigned, but the bytes they point at can be written through them. Deep immutability is spelled out, `[]const []const u8` — which is what `[]const str` already is.

Array literals name their type, like struct literals, and `_` infers the length:

```
let primes = [4]u8{ 2, 3, 5, 7 }
let names = []str{ "x64", "aarch64", "spv" }
```

An array literal supplies exactly `N` elements — a shorter or longer list is a compile error — and `_` takes the count from the list; a `Vec` literal (Vectors, below) supplies exactly its `N` lanes. A struct literal names every field exactly once, in any order, and `neper fmt` writes them in declaration order; an omitted field is a compile error, never zero, because there is no hidden `memset` (§5) — `= zero` is the spelling for all-zero. A union literal names exactly one member, `value{ i: 42 }`, and the bytes beyond that member are unspecified.

There is no tuple type. `(A, B)` appears only in a return signature, where it declares multiple return values rather than a value of some product type — see §5.

```
type Vec3 = struct {
    x: f32,
    y: f32,
    z: f32,
}

type Value = union {
    i: i64,
    f: f64,
    p: *u8,
}

type Kind = enum u8 {
```

```

    None,
    Int,
    Float,
}

```

- struct fields are laid out in declaration order with natural alignment. `@packed` removes padding. `@align(N)` sets alignment.
- union is untagged and unchecked, exactly like C, and it is the **escape hatch**: it exists for C ABI compatibility across extern (§5) — Win32's `INPUT`, Vulkan's `VkClearValue`, `sockaddr` — and for deliberate punning. A value that is one of several variants is a union `enum`, below, the default for a variant type; a bare union on the page signals FFI or a pun, and a pun alone never needs one, because `mem.bitcast[T](x)` (§8) reinterprets a value's bytes as `T` without a union. A bare union is banned in device code, and neither union form is a `gpu.Buf[T]` element type (§10).
- enum has an explicit backing integer type and is a distinct type from it. A member may carry an explicit value, `enum u8 { A = 3, B, C = 9 }`; a member without one takes the previous member's value plus one, the first defaulting to 0. Two members with one value, or a value outside the backing type, is a compile error. A member is written `.Int` where the context supplies the type; the qualified form `Kind.Int` is legal wherever `.Int` is and required where no context supplies the type — `let k = Kind.Int` (§2).
- type `X = T` with any other `T` — a primitive, a pointer, a slice, an array, a `Vec`, a named type — declares an **alias**: `X` is `T` itself, interchangeable with it everywhere with no conversion in either direction, not a distinct type. A distinct type over `u32` is a single-field struct — `struct { raw: u32 }` — which is what `e.time` (§16) and `e.os` (§5) do to keep a `Timestamp` or a `File` apart from its integer. `str` is exactly such an alias of `[]const u8` (Aliases, above).

Sized and storable types. A type is *sized* when `mem.size_of[T]()` and `mem.align_of[T]()` are defined. The integer and float primitives, `bool`, `err`, pointers, function pointers, enums, and recursively arrays, vectors, structs, bare unions, tagged unions and atomics of sized types are sized. `void`, `type`, multiple return lists, argument packs, `Mask[T, N]`, `Field` and `Member` are not. A type is *runtime-storable* when it is sized and is not a function type; only runtime-storable types may be a field, array element, slice element, `Atomic` element or `mem.alloc[T]` argument. Locals additionally admit the register-only `Mask[T, N]`. Functions and extern `fn` values may be held only through their function-pointer types.

An array length may be zero. `[0]T` has size zero and alignment `align_of[T]`; its stride is `size_of[T]` rounded up to `align_of[T]`. An empty struct has size one and alignment one, so distinct objects have distinct addresses. Empty bare unions, empty enums and empty tagged unions are compile errors. A struct, union, tagged union or enum may not declare two fields or members with the same name.

A type may refer to itself only through a pointer or function-pointer indirection. Any alias cycle or by-value size cycle through an array, struct, union or tagged-union payload is a compile error and the diagnostic prints the cycle.

Exact aggregate layout. For a non-packed struct, each field begins at the least multiple of its alignment at or after the previous field, the struct alignment is the maximum field alignment, and tail padding rounds its size to that alignment. An array uses the element stride above. A bare union has the maximum size and alignment of its members, rounded to that alignment. `@packed` gives every field alignment one and removes inter-field and tail padding; `@packed` with `@align(N)` keeps packed field offsets and raises only the aggregate alignment to `N`, adding tail padding as needed. Without `@packed`, `@align(N)` raises the aggregate alignment and tail rounding but does not change individual field alignment. Duplicate or non-power-of-two alignment attributes are errors. Target ABI chapters define pointer, `usize` and aggregate-call classification; these byte-layout rules do not depend on the host C compiler.

Primitive sizes are their names' bit widths and their natural alignment is their byte size, capped only where the target ABI chapter says so. `bool` is one byte with alignment one; `err` is four bytes with alignment four; `usize`, `isize`, data pointers and function pointers have the target pointer size and alignment. Floats use their IEEE storage widths. An enum has exactly the size and alignment of its backing integer. `Atomic[T]` has `T`'s size and at least `T`'s alignment, raised only when the target's atomic instruction requires it; that target-specific raising is part of the data-layout version and C ABI mapping.

Tagged unions

union `enum` is the tagged form. It is what a compiler is mostly made of — tokens, AST nodes, IR instructions — and it is the optional type: `union enum u8 { Some: T, None }`, with no keyword of its own.

```

type Node = union enum u8 {
    Lit: i64,
    BinOp: BinOp,
    Nil,
}

```

```

let n = Node{ Lit: 42 }
let z = Node{ Nil }

switch n {
case .Lit as v:
  ... // v: i64, a let binding of the payload
case .BinOp as op:
  ... // op: BinOp
case .Nil:
  ...
}

if n.tag == .Lit { ... } // the tag, an enum with the members' names
let v = n.Lit // payload access, tag-checked in debug (§11)

```

- The backing integer type is the tag's type. Tags are assigned in declaration order from 0. A member without a payload is a bare name.
- Layout on the CPU is the tag followed by one bare-union payload area whose size and alignment are the maxima of the payload members; the area begins at its natural alignment and the whole value receives normal tail padding — the same bytes as the hand-rolled `struct { kind: Kind, u: union { ... } }`. Placing the tag hides no cost; it only removes the chance of reading the wrong member. In device code the payloads do not overlap (§10); the layout is invisible to the program, so both are conforming.
- A literal names exactly one member: `Node{ Lit: 42 }`, `Node{ Nil }`. A member without a payload is written `.Nil` where the context supplies the type, as an `enum` member is, and `neper fmt` rewrites `Node{ Nil }` to `.Nil` wherever the context types it — so the canonical form (§14) has one spelling per position.
- `n.tag` reads the tag; its type is an `enum` of the backing type with the same member names, spelled `Node.Tag` where a type is needed. A `union enum` member named `Tag`, or a sibling type declaration whose name would collide with `<Union>.Tag`, is a compile error. `n.Lit` reads or writes the payload; in debug builds it is one compare against the tag, and a mismatch traps (§11). Assigning a whole value (`n = Node{ Nil }`) changes the variant.
- `case .Lit as v` binds the payload as a `let` for the case's block; `as` is omitted for a member without a payload.
- `switch` on a tagged union, like `switch` on an `enum`, must be exhaustive unless a `default` case is present (§6). Adding a variant therefore flags every `switch` that does not handle it.

Vectors (SIMD)

`Vec[T, N]` is the vector type: `N` lanes of `T`, a builtin generic in the form every other parameterised builtin takes (`Atomic[T]`, `gpu.Buf[T]`, `D5`). There are no `f32x8`-style names — one spelling, with both parameters visible to a `grep`. These are the primary route to vector performance; there is no auto-vectoriser in `v1`.

```

let a = simd.splat[Vec[f32, 8]](1.0)
let b = a * a + a
let c = Vec[i32, 4]{ 1, 2, 3, 4 }

```

The legal (T, N) pairs are a closed table. `T` is any integer or float primitive of §4 — not `usize`, `isize`, `bool` or a composite — and the vector's width `N * mem.size_of[T]()` (§8) is 16, 32 or 64 bytes. Nothing else is a vector type; a generator never has to guess whether `Vec[u64, 3]` exists (it does not).

T	Legal N	Width
i8 u8	16, 32, 64	16, 32, 64 bytes
i16 u16 f16 bf16	8, 16, 32	16, 32, 64 bytes
i32 u32 f32	4, 8, 16	16, 32, 64 bytes
i64 u64 f64	2, 4, 8	16, 32, 64 bytes

`mem.size_of[Vec[T, N]]()` is the width and its alignment equals its width, in a struct, an array and on the stack alike.

`Mask[T, N]` is the **mask type** of `Vec[T, N]`, legal for exactly the same pairs: one lane-wide `true/false` per lane, produced by comparisons and consumed by `select` and the masked loads and stores. Its representation is the target's — a same-width vector of all-ones/all-zeros lanes on SSE, AVX2 and NEON, a `k` register at `x64-v4` — and the program cannot see which. `& | ^ ~` apply to masks; `simd.any(m)`, `simd.all(m)` and `simd.bits(m) -> u64` (bit `i` for lane `i`) read one.

`Mask[T, N]` is **register-only**, on both profiles. It is legal as a `let` or `var` local, a parameter, a return value — passed and returned as the `Vec[T, N]` it masks, in the vector class (§5) — and an intrinsic operand, and nowhere else: not a struct field, an array element, a module-scope `var`, a `gpu.Buf[T]` element (§10) or the pointee of a pointer — `&m` is a compile error — and `mem.size_of[Mask[T, N]]()` is a compile error, because the type has no layout the program can name. A mask that must be stored is converted: `simd.bits(m)` gives the integer bitmask and `simd.mask[V](bits)` gives it back. `= zero` on a local `Mask` is every lane `false` (§7).

Operators apply lane by lane, both operands the same `Vec[T, N]`, and the result is that type. On float lanes `+` `-` `*` `/` are §11's IEEE operations, one rounding each, never contracted. On integer lanes only the **wrapping** forms `+%` `-%` `*%` exist: a vector unit has no overflow flag, so there is no debug check to elide, and the language does not let `+` silently mean `+%` — plain `+` `-` `*` on an integer vector is a compile error, and so is `/` or `%`, which no target implements as one instruction. `&` `|` `^` `~` apply to integer lanes; `<<` `>>` shift every lane by one scalar count, which must be `< width` (§11 shift). `==` `!=` `<` `<=` `>` `>=` are not defined on vectors; the `simd.cmp_*` intrinsics are, and they return a `Mask[T, N]`. `v[i]` reads or writes one lane, `i`: `usize`, bounds-checked in debug (§11).

The `simd` module is the operation surface, and every intrinsic that takes a vector is **comptime-generic on it** per D5: `simd.splat[Vec[f32, 8]](1.0)`, inferred from a vector argument where §9 allows. `V` is `Vec[T, N]`, `M` is `Mask[T, N]` for that `V`:

Intrinsic	Semantics
<code>splat[V](x: T) -> V</code>	every lane <code>x</code>
<code>load[V](s: []const T, off: usize) -> V</code>	lanes <code>s[off .. off+N]</code> , any alignment; <code>off + N <= s.len</code> is a bounds check (§11)
<code>store[V](s: []T, off: usize, v: V)</code>	the reverse, same check
<code>load_aligned[V](s: []const T, off: usize) -> V</code> , <code>store_aligned[V](...)</code>	as above, and <code>&s[off]</code> must be aligned to the vector's width — an align check in debug (§11); the emitter may select the aligned instruction
<code>load_masked[V](s: []const T, off: usize, m: M) -> V</code>	lanes whose mask lane is true are read, the rest are 0; only the enabled lanes must be within <code>s</code> , so a tail of a slice is loaded without a scalar loop
<code>store_masked[V](s: []T, off: usize, m: M, v: V)</code>	writes the enabled lanes only, same bounds rule
<code>shuffle[V, IDX: [N]u8](a: V, b: V) -> V</code>	lane <code>i</code> of the result is lane <code>IDX[i]</code> of the <code>2N</code> lanes of <code>a</code> then <code>b</code> ; <code>IDX</code> is a comptime array (§9), so the permutation is one instruction or a fixed sequence, never a runtime table
<code>cmp_eq cmp_ne cmp_lt cmp_le cmp_gt cmp_ge [V](a: V, b: V) -> M</code>	lane-wise comparison; on floats, IEEE ordered comparison — a NaN lane is false except under <code>cmp_ne</code>
<code>select[V](m: M, a: V, b: V) -> V</code>	lane <code>i</code> is <code>a[i]</code> where <code>m[i]</code> , else <code>b[i]</code>
<code>any[V](m: M) -> bool</code> , <code>all[V](m: M) -> bool</code> , <code>bits[V](m: M) -> u64</code>	true when any, or every, lane of <code>m</code> is; <code>bits</code> sets bit <code>i</code> for lane <code>i</code>
<code>mask[V](packed: u64) -> M</code>	the inverse of <code>bits</code> : lane <code>i</code> is true when bit <code>i</code> of <code>packed</code> is set, bits at or above <code>N</code> ignored — with <code>bits</code> , the one way to store a mask, which is register-only (above)
<code>reduce_add reduce_min reduce_max [V](v: V) -> T</code>	horizontal reduction, in a fixed pairwise tree in lane order — <code>(0+1)+(2+3)</code> and so on — so a float sum is bit-identical on every target under §11; min/max are IEEE minimum/maximum; integer <code>reduce_add</code> wraps
<code>convert[V, W](v: V) -> W</code>	lane-wise <code>T(x)</code> to <code>W</code> 's lane type, same <code>N</code> , with the Casts table above — an integer narrowing that does not fit is a narrow trap in debug and truncates in release; float-to-integer rounds toward zero and saturates in release
<code>gather[V](s: []const T, idx: Vec[u32, N]) -> V</code>	lane <code>i</code> is <code>s[idx[i]]</code> , each index bounds-checked in debug; legal when <code>(u32, N)</code> is in the table, so <code>N</code> in {4, 8, 16}
<code>fma[V](a: V, b: V, c: V) -> V</code>	float lanes only: the lane-wise form of <code>math.fma</code> , the only fused multiply-add (§11)
<code>pdep(x: u64, bit_mask: u64) -> u64</code> , <code>pext(x: u64, bit_mask: u64) -> u64</code>	the two scalar bit intrinsics §13 refuses to select on its own; a software bit-loop sequence on every target without the instruction — below x64-v3, and on aarch64 and x86-32

There is no other way to move a vector to or from memory: a `[]Vec[f32, 8]` slice and a `Vec` field are ordinary storage, and the intrinsics are how a slice of `T` is walked eight lanes at a time. In device code the slice parameter of `load`, `store`, `gather`

and the aligned and masked forms is generic over the address space, as §10's helpers are: it accepts `[]const T` and `[]const shared T` alike (`[]T` and `[]shared T` for a store), monomorphised per space.

Every vector width compiles on every target. A width the hardware supports natively lowers to one instruction; a wider one is split. `Vec[f32, 8]` becomes a single `vaddps ymm` under `--cpu=x64-v3` and a pair of SSE instructions under `x64-v1`; NEON is 16 bytes wide, so every 32- and 64-byte vector splits there; on SPIR-V a vector of more than four lanes splits into four-lane vectors (§10, Capabilities). Source stays portable, and the only thing that changes is how many instructions come out — see §13 for the CPU levels.

Vectors are a register class. A `Vec[T, N]` is passed and returned in vector registers — `xmm/ymm/zmm` at the level's width, NEON `v` registers — and is exempt from §5's hidden-reference rule whatever its size; a returned vector comes back in vector registers, as many as its width needs at the level, up to §5's per-target vector budget — two on x64 and aarch64, one on x86-32 — and beyond that budget the set goes to §5's caller slot like any aggregate. It never crosses an `extern` boundary by value (§5). In device code `Vec[T, N]` is a device storage and execution type when its lane type is (§10), and every `simd.*` intrinsic is legal except `pdep/pext`; the aligned forms are the unaligned ones there.

Because the width is written in the source rather than inferred by a vectoriser, the code you wrote is the code you get. That predictability is the trade for having no auto-vectorisation.

Strings

A string literal has type `[]const u8` and points at read-only data. It is not null-terminated, carries its own length, and costs no allocation.

```
let greeting = "hello" // []const u8, len 5
let name: str = "neper"
```

The `const` is what makes `s[0] = 'x'` a compile error rather than a segfault, which is why it exists at all.

Concatenation allocates, so it takes an arena — there is no `+` operator for strings, and never will be (§6). Four forms, no more:

```
// `a` is a *mem.Arena, as it is everywhere else (§8, §13)
let msg = try str.concat(a, greeting, name) // exactly two parts
let all = try str.join(a, parts, ", ") // a []const str, plus a separator
let s = try str.format("{}:{}".format(a, file, line)) // a format string (Formatting, below)

var b = try str.builder(a, 64) // incremental
try str.push(&b, greeting)
try str.push_u64(&b, 42)
let line = str.done(&b)
```

The push family is closed, one function per name (§14): `str.format` and `io.printf` expand every verb into `push` calls, or into one call to the argument type's `format` (§9, Protocols), and into nothing else. `io.printf`'s flushing belongs to its builder rather than to the expansion (below), so a user `format` body reached from a verb pushes exactly as any other code does:

Function	Pushes
<code>push(b: *Builder, s: []const u8)</code>	the bytes
<code>push_byte(b, c: u8)</code>	one byte
<code>push_i8(b, v: i8), push_i16(b, v: i16), push_i32(b, v: i32), push_i64(b, v: i64), push_isize(b, v: isize), push_u8(b, v: u8), push_u16(b, v: u16), push_u32(b, v: u32), push_u64(b, v: u64), push_usize(b, v: usize)</code>	decimal, one per integer type so that no cast is ever needed
<code>push_hex_u32(b, v: u32), push_hex_u64(b, v: u64), push_bin_u32, push_bin_u64</code>	<code>{x}</code> and <code>{b}</code> : the digits alone, no prefix and no leading zeros
<code>push_f32(b, v: f32), push_f64(b, v: f64)</code>	<code>{}</code> : the shortest digit string that round-trips to the value, in fixed notation when the decimal exponent is in <code>[-5, 15]</code> and scientific otherwise — <code>1.5e-7</code> — and <code>inf</code> , <code>-inf</code> , <code>nan</code> for the non-finite values
<code>push_f32_fixed(b, v: f32, prec: u8), push_f64_fixed(b, v: f64, prec: u8)</code>	<code>{.N}</code>
<code>push_bool(b, v: bool)</code>	<code>true</code> / <code>false</code>
<code>push_err(b, e: err)</code>	the qualified name (§7)

Every push returns `err` — `str.NotOnTop`, `below`; `mem.Exhausted` (§8) on a builder with no flush sink; the sink's own `err` on one with (below). Each push writes what the matching `printf` verb writes, so an expansion contains no source-level conversion. Three readings inside the formatter are not conversions: a narrow integer widened to the push width under `{x}/{b}`, a narrow float widened to `f32`, and a pointer's address read as a `u64` (Formatting, below). `push_byte` alone has no verb — it is called by name, and `{}` on a `u8` is decimal through `push_u8`. The rest of the surface:

```
type Sink    = struct { ctx: *void, write: fn(ctx: *void, bytes: []const u8) -> err }
type Builder = struct { arena: *mem.Arena, start: usize, len: usize, reserved: usize, sink: Sink, flushing: bool }
type Split   = struct { source: str, separator: str, off: usize, finished: bool }
error NotOnTop // something else has allocated from the arena since builder
error BadNumber // a parse function's input is malformed or out of range
error InvalidSeparator // split separator is empty
```

Sink is a context pointer and one ordinary neper function pointer (§5) — never an `extern fn`, so a sink crosses no C boundary and needs no `@cc`. A builder carries at most one, and a builder that carries one is a **flushing** builder. `Builder.start` is its byte offset in `arena`, `len` is written bytes and `reserved` is the current claim; the claim is on top exactly when `arena.off == start + reserved`. `flushing == false` means sink is ignored. These fields are readable for diagnostics but module functions require their invariants; callers do not replace them.

Function	Semantics
<code>fn builder(a: *mem.Arena, cap: usize) -> (Builder, err)</code>	claims the arena top with <code>cap</code> bytes reserved — an initial reservation, not a limit: the builder grows past it in place while it owns the top, and <code>mem.Exhausted</code> is the only bound
<code>fn builder_to(a: *mem.Arena, cap: usize, sink: Sink) -> (Builder, err)</code>	the same claim, carrying <code>sink</code> . Where builder's push returns <code>mem.Exhausted</code> , this one flushes : the bytes pushed so far go to <code>sink.write</code> , the claim is reset to its base, and the push proceeds — so <code>mem.Exhausted</code> never leaves a push on a flushing builder, a push larger than the whole buffer drains through the sink as it is written, and what a push can return is <code>str.NotOnTop</code> or whatever <code>sink.write</code> returned. <code>str.done</code> gives back the bytes still in the buffer, which the caller sends through the sink itself
<code>fn done(b: *Builder) -> str</code>	ends the claim; the bytes pushed so far
<code>fn concat(a: *mem.Arena, x: []const u8, y: []const u8) -> (str, err)</code>	exactly two parts
<code>fn join(a: *mem.Arena, parts: []const str, sep: []const u8) -> (str, err)</code>	many parts
<code>fn eq(x: []const u8, y: []const u8) -> bool</code>	byte equality
<code>fn parse_f32(s: []const u8) -> (f32, err), fn parse_f64(s: []const u8) -> (f64, err)</code>	the whole of <code>s</code> as a float, in the notation <code>push_f32</code> writes; <code>str.BadNumber</code> on anything malformed or out of range
<code>fn parse_i64(s: []const u8) -> (i64, err), fn parse_u64(s: []const u8) -> (u64, err)</code>	the whole of <code>s</code> as a decimal integer, an optional <code>-</code> on the signed form; <code>str.BadNumber</code> on anything malformed or out of range. Narrower widths go through a cast (§4), which is where a value that does not fit is reported
<code>fn parse_i64_radix(s: str, radix: u8) -> (i64, err), fn parse_u64_radix(s: str, radix: u8) -> (u64, err)</code>	whole-input parsing for radix <code>2..36</code> ; ASCII letters are case-insensitive and no prefix is recognized
<code>compare, compare_ascii_fold</code>	three-way byte-lexicographic comparison; the latter folds ASCII letters only
<code>starts_with, ends_with, contains, find, find_from, rfind, count</code>	byte-substring operations with <code>(index, found)</code> results; an empty needle matches every boundary
<code>trim, trim_start, trim_end, trim_bytes</code>	borrowed subslices; the first three remove ASCII whitespace
<code>split_once, split, split_next, lines</code>	borrowed, non-copyable traversal preserving empty fields; empty explicit separators return <code>InvalidSeparator</code> ; lines accept LF and CRLF and omit terminators
<code>replace, repeat</code>	checked arena allocation and non-overlapping replacement
<code>ascii_lower_in_place, ascii_upper_in_place, is_ascii_space, is_ascii_digit, is_ascii_alpha, is_ascii_alnum</code>	locale-free ASCII operations; Unicode equivalents remain in <code>text.*</code>

Fixed precision `N` is in `0..=99`; a larger runtime `prec` returns `BadNumber`, and a larger format-literal precision is a compile error. Fixed conversion rounds ties to even and always writes exactly `N` digits after the decimal point; non-finite values still

write `inf`, `-inf` or `nan` without a fractional suffix.

Parsing consumes every byte and accepts no surrounding whitespace or leading `+`. Integers require at least one ASCII digit and reject overflow. Floats accept the shortest/fixed decimal forms emitted above, an optional leading `-`, an optional lowercase `e` exponent with optional sign and at least one digit, and exactly the lowercase tokens `inf`, `-inf`, `nan`. They preserve negative zero, return the canonical NaN of §11, and reject decimal overflow or underflow that would produce an infinity or zero unless the input was the corresponding explicit token or exact zero.

And `e.io`, which is the one place text leaves the program: `fn print(s: []const u8) -> err` writes to `os.stdout()` (§5), looping over `os.write` until every byte is written or a call returns an `err`, which `print` returns; and `fn printf[FMT: str](args: ...) -> err` (Formatting, below) does the same through a **flushing builder over a 4 KiB stack arena** of its own — a `[4096]u8 = undef local under mem.arena_from` (§8), its top claimed by `str.builder_to` (above) with a sink whose write is `print`'s loop over `os.write` — so that every verb is a `str.push_*` and no caller's arena is involved. The flushing is the builder's, not `printf`'s: a push that fills the buffer drains it through the sink and carries on, a `str` argument longer than the whole buffer drains through it as it is pushed, and `printf` sends what `str.done` leaves through the sink once at the end. Output of any length therefore works; `mem.Exhausted` never reaches the expansion, so a user `format` (§9) called for a verb is an ordinary sequence of pushes; `str.NotOnTop` cannot arise because nothing else allocates from that arena; and only `os.write`'s errors propagate.

The builder is the fast path. While it owns the top of an arena, appending is a bump of the offset — the buffer grows in place with no reallocation and no copying, because nothing else can have allocated above it. That is a direct payoff from having no global allocator.

That precondition is **checked, in every build mode**: `str.push` (and every `str.push_*`) compares the arena's `off` with the builder's end and returns `str.NotOnTop` when anything else has allocated from the arena since the builder was created. It is one compare on the fast path, and it turns what would be a silent overwrite of the other allocation into a definite `err`. `str.done` ends the builder's claim on the arena top.

Slices are compared with `str.eq(x, y)` or `mem.eq[T](x, y)`, never `==` (§6).

Every library parameter that is only read is `[]const T`: `str.join`'s parts, `str.push`'s text, both operands of `str.eq` and `mem.eq[T]`, the source of `mem.copy[T]`, `io.print`. Because `[]T` converts to `[]const T` implicitly (above), a mutable slice, a `let` array and a literal all fit the same parameter; a parameter is `[]T` only where the function writes through it.

Formatting

The format string is a **comptime parameter**, so it is parsed at compile time and the call expands to straight-line pushes with no runtime parsing:

```
try io.printf("{} compiled {} modules in {}ms\n")(name, count, elapsed)
let s = try str.format["{}: {}: {}"](&a, file, line, col)
```

Arity and argument types are checked against the format string at compile time. A mismatched count or an unformattable type is a compile error, not a runtime one. **Formattable is every type §9 rule 4 supplies a format for, plus any type whose module declares a** `fn <t>_format(v: T, b: *str.Builder) -> err` — the format protocol of §9, spelled `fn vec3_format` for a `Vec3`. §9 rule 4 is the authority on the first half, and it reaches the integers, the floats, `bool`, `err`, pointers, slices — `str` among them — arrays, `Vec/Mask`, and every `enum` and `union enum` whose own module declares no `format`. What the supplied `format` writes, one clause per shape:

Shape	{ } writes
an integer, a float, a <code>bool</code>	what its <code>push_*</code> writes (the push table above)
an <code>err</code>	its qualified name (§7), or <code>ok</code>
a pointer	its address in hex, <code>nil</code> as <code>nil</code>
a <code>[]u8</code> or <code>[]const u8</code>	its bytes, as written — this is <code>str</code> (Aliases, above), and it is the one slice that is text rather than a list
any other slice, or an array	[, its elements under this same rule separated by , ,]
a <code>Vec[T, N]</code>	its <code>N</code> lanes in that form; a <code>Mask[T, N]</code> its lanes as <code>true/false</code>
an <code>enum</code>	the member's name, <code>Int</code> for <code>Kind.Int</code>
a <code>union enum</code>	the member's name, then <code>:</code> and the payload under this rule where it has one

A verb on a type whose module declares a `format` expands instead to one call to that function with the builder the expansion is already pushing into, which is why the expansion is still closed: every verb becomes `push` calls, or one call to that type's `format`.

Verbs: `{}` default, `{x}` hex, `{b}` binary, `{.N}` float precision. A literal brace is escaped by doubling: `{{ prints { and }}` prints `}`.

`{}`, `{x}` and `{b}` accept every integer type and print at the value's own width: `u8 255` is `ff` under `{x}`, `i16 -1` is `-1` under `{}`. Under `{x}` and `{b}` a signed value prints its **two's-complement bits at its own width**, no sign and no leading zeros — `i16 -1` is `ffff`, `i8 5` is `101` under `{b}` — so `{}` expands to the push of the argument's own type, and `{x}/{b}` to `push_hex_u32` or `push_bin_u32` for a width up to 32 bits and the `u64` form above it, `usize` and `isize` at the target's width. The widths of the `push_*` family are an implementation detail of `e.str`; where an expansion reaches a wider push, the widening is internal to the formatter intrinsic and is not a source-level conversion, so §6's list of implicit operations stays complete. A narrow float is under the same exemption: the formatter widens an `f16` or `bf16` to `f32` internally, so `{}` on one prints through `push_f32` and `{.N}` through `push_f32_fixed`, and there is no narrow-float push.

Pointers and nil

`nil` is a valid pointer value, and it is also the zero value of a slice — `ptr nil, len 0` (§7) — so `nil` is the spelling of an empty slice in expression position: `let xs: []u8 = nil, ret (nil, e)`. In debug builds every dereference is null-checked (§11). There is no optional keyword; where `nil` is not enough, `union enum u8 { Some: T, None }` (Tagged unions, above) is the optional.

5. Declarations

Every top-level declaration begins at **column 0** with its keyword. This is enforced by the formatter and assumed by tooling (§14).

```
use e.mem
type Vec3 = struct { ... }
const MAX_NODES: u32 = 4096
var g_tick: u64 = 0
error NotFound
fn dot(a: Vec3, b: Vec3) -> f32 { ... }
@import("kernel32", "GetTickCount64")
extern fn get_tick_count64() -> u64
```

Attributes

One or more attributes occupy **their own contiguous lines at column 0, immediately above the declaration they modify**:

```
@gpu(256)
fn saxpy(n: u32, a: f32, x: []const f32, y: []f32) { ... }

@test
fn parse_handles_empty_input(a: *mem.Arena) -> err { ... }
```

Never `@gpu fn saxpy(...)` on one line. The declaration keyword has to stay at column 0 for §14 to hold — otherwise `grep "^fn saxpy"` misses exactly the functions that carry attributes, which are the interesting ones. `neper fmt` moves them.

Declaration attributes are `@gpu(X, Y, Z)` (§10), `@test`, `@packed` and `@align(N)` — legal only on a type `... = struct` or type `... = union` declaration, and `@align(N)` requires `N` a power of two at or above the type's natural alignment unless `@packed` is also present — `@import(LIB, SYM)` — legal only on an `extern` declaration — and `@cc(CONV)` — legal on an `extern` declaration, on a plain `fn` that is to be C-callable — such a `fn` stays an ordinary neper function at every neper call site, called there under the named convention — or on a type declaration whose right side is an `extern fn(...) -> R` type (External functions, below). `@nocheck` is a statement-level block (§11) and is the one attribute that does not take a line of its own.

An attribute may appear at most once on a declaration. Multiple legal attributes have no source-order semantics; `neper fmt` sorts them by attribute name as specified in `docs/tooling.md` §6. A blank line or comment inside that attribute block or between it and the declaration is a syntax error, so attachment never depends on a tool's trivia policy.

- `const` is compile-time evaluated and has no storage.
- `var` at module scope is mutable static storage, zero-initialised unless given a compile-time initialiser. Not permitted in the GPU profile.
- Inside functions: `let` binds immutably, `var` binds mutably. Both infer their type from the initialiser unless annotated, and **both require an initialiser**. An unused `let`, `var` or parameter is not an error: `args` in a `main` that ignores it is legal, and there is no discard spelling for a binding.
- A local or parameter name is declared once per **active lexical scope**. It may not reuse a parameter, a binding in an enclosing scope, a module-scope name of its own module, a `use` qualifier or a builtin type name (§2). It may be reused in disjoint sibling blocks after the earlier binding's scope has ended. This permits independently generated blocks to compose without a function-wide name allocator while preserving the stronger property a reader needs: at every source position, one spelling resolves to exactly one declaration. Two bindings in the same block, including two sequential `let (data, e) = ...` bindings, still require different names.

Local initialisers

There is no uninitialised local and no hidden memset. A local whose initial contents do not matter says so, with one of two spellings that are legal only as the whole initialiser of an annotated binding:

```
var buf: [4096]u8 = zero // every byte 0 – a visible memset
let v: Vec3 = zero // a definite constant; legal on let
var buf: [4096]u8 = undef // no store at all; var only
```

`= zero` writes the zero value of the type (§7 defines it for every type) and is legal on an annotated `let` or `var`. `= undef` is legal on an annotated `var` only: it emits nothing in release; a debug build fills the storage with `0xCD`, the same byte fresh arena memory receives (§11), so a read-before-write shows up as a recognisable value rather than as whatever the stack held. `undef` is banned inside `@gpu`, and on a `let`, which could never be written afterwards. A module-scope `var` is zero-initialised already; `= zero` is permitted there and `= undef` is not.

Every other initialiser is an ordinary expression, so what is on the page is what is stored.

Functions

```
fn name(p: T, q: U) -> R { ... }
fn name(p: T) -> (R, err) { ... }
fn name(p: T) { ... } // returns void
```

Parameters are immutable bindings. Aggregates larger than two machine words are passed by hidden reference; this is an ABI detail, not a semantic one — pass `*T` when you want the callee to mutate. What makes it an ABI detail is one rule, mirroring D14: **the storage of a by-value argument is not written by anyone for the duration of the call** — not by the callee, not by the caller through another pointer, not by another thread. Doing so is undefined behaviour. Under that rule the compiler may pass the caller's storage by address and never copy, with no alias analysis; `f(x, &x)` where `f` writes through its second parameter is the program's bug, not the compiler's. `Vec[T, N]` (§4) is the one exemption from the size rule: a vector is a register class, passed in vector registers at any width, never by hidden reference, and returned in them up to the per-target budget (Multiple return values, below); `Mask[T, N]` (§4), which has no size, is passed and returned as the `Vec[T, N]` it masks.

Function pointers exist on the CPU: `fn(i32) -> i32` is a type. They are banned in the GPU profile.

Multiple return values

`(A, B)` in a return signature is a **calling convention, not a type**. There is no tuple value in neper: nothing to store in a struct, nothing to pass as a parameter, nothing to name `.0`.

```
fn divmod(a: i64, b: i64) -> (i64, i64) { ... }

let (q, r) = divmod(17, 5)
```

Returned values are destructured at the call site. The convention is the one a single return uses, widened. Every returned value belongs to one of two **return classes**: the **integer class** — integers, pointers, `enums`, `err` and `bool` — and the **vector class** — floats of every width, `Vec[T, N]` (§4) and `Mask[T, N]`, which travels as the `Vec[T, N]` it masks — a same-width vector of all-ones and all-zeros lanes, in that vector's registers — whatever register the level holds a mask in between calls. Each class has a per-target budget, counted in values:

Target	Integer class	Vector class
x64 (System V and Windows)	2 (rax, rdx)	2, from xmm0 upward — ymm/zmm at the level's width
aarch64	2 (x0, x1)	2, from v0 upward
x86-32	2 (eax, edx)	1, from xmm0 upward

The values of each class are assigned to that class's registers **in declaration order**: the first vector-class value starts at `xmm0`, the next at the first free register after it, a scalar float taking one register and a `Vec[T, N]`, or the `Mask[T, N]` of one, as many consecutive registers as its width needs at the level. When either class's budget is exhausted the **whole set** goes to a caller-provided return slot whose address travels in a hidden first parameter; a set is never split between registers and the slot. An **aggregate** — a `struct`, `union`, `union enum`, `[N]T`, a slice or `str` — belongs to neither class and is never returned in registers: a set containing one goes entirely through the slot, and only the two scalar classes use registers. `(i64, f64, err)` is two integer-class values and one vector-class value — registers on every target; `(f32, Vec[f32, 4])` is two vector-class values, `xmm0` and `xmm1` on x64, registers on aarch64, and the slot on x86-32, whose vector budget is one; `Vec[u8, 64]` alone at x64-v1 is one vector-class value in `xmm0–xmm3`. So `(i64, i64)` and `(*T, err)` come back in registers, and `(List[T], err)` — 40 bytes — comes back through a slot the caller reserved on its own stack. The program cannot tell which: there is still no tuple value, no layout the program can name, and no equality or ordering rule to define. This convention holds between neper functions only; an `extern` function (§5) returns one value in the C convention.

Two things follow, and both are deliberate:

- **Anything worth storing gets a named struct.** `result.line` reads; `result.0` does not, and positional access is unanchored for search — the same reason §3 fixes naming and §14 fixes declaration position.
- **There is one way to spell a product type.** Structs and tuples competing for the same job is exactly the redundancy a small language cannot afford.

Every returned value must be bound or discarded with `_`. A call that returns anything cannot stand as a statement on its own — only a call returning `void` can — so an ignored `err` is written down, `let _ = io.print("x\n")`, where the reviewer sees it; `try` is the other spelling. `_` binds nothing and may take any position:

```
let (q, _) = divmod(17, 5)
let _ = os.close(f)
```

External functions

`extern` declares a function whose body lives in a library the linker resolves. The foreign name and the calling convention sit on attribute lines above it:

```
@import("kernel32", "WriteFile")
@cc(c)
extern fn write_file(h: *void, buf: *const u8, n: u32, written: *u32, overlapped: *void) -> i32

@import("c", "printf")
extern fn c_printf(fmt: *const u8, ...) -> i32
```

- `extern` is a keyword and a column-0 declaration like `fn`, so §14 holds: one `extern` per name per module, found by `grep "^extern fn"`. The neper-side name obeys §3 — `write_file`, never `WriteFile` — which is possible because the foreign name is a string, not an identifier.
- `@import(LIB, SYM)` is required. `LIB` is the library with no platform prefix or suffix: `"kernel32"` is `kernel32.dll`, `"vulkan"` is `libvulkan.so` or `libvulkan.dylib`. `SYM` is the exported symbol, verbatim, decoration included. A library whose name differs per platform is declared in per-target files (§6).
- `@cc(CONV)` names the calling convention and defaults to `c`, the target's C convention: System V on Linux and macOS x64, `win64` on Windows x64, `AAPCS64` on aarch64, `cdecl` on x86-32. `sysv` and `win64` name the two x64 conventions explicitly and are compile errors on any target other than x64; they exist only to force a non-native convention on x64, which is why `kernel32` above is `@cc(c)` — the platform's native C convention, correct on every Windows arch. `stdcall` is the Win32 API convention on x86-32 and means `c` elsewhere. The convention **is part of an** `extern fn` **pointer type**: two `extern fn(...) -> R` types under different conventions are distinct types with no conversion between them. A convention is named on a pointer type through a type declaration — an attribute line `@cc(CONV)` above type `Cb = extern fn(i32) -> i32` gives `Cb` that convention — and an `extern fn(...) -> R` type written without one is `c`.

- A trailing `...` declares a **C variadic** — the one runtime-variadic form in the language, legal only on `extern`; it is not a pack (§9). Each extra argument crosses as its own type under the table below, and C's default promotions are not applied for you: an `f32`, or an integer narrower than 32 bits, in that position is a compile error — write `f64(x)` or `i32(x)`.
- An `extern` function has no body and no `err`. `err` **may not appear anywhere in an `extern` signature**, and `try` on an `extern` call is a compile error. C reports failure through a return value and a thread-local code; the wrapper in `e.os` (below) is where that becomes an `err`.
- The name of an `extern` function in value position is a function pointer of type `extern fn(...) -> R`, the C-convention function-pointer type. A plain `fn` with a body takes that type, and becomes callable from C, when it carries a `@cc` line of its own — that is how a callback is handed to the OS. A `fn(...) -> R` pointer to a function without `@cc` uses the neper convention (§5) and does not cross. A `@cc` function may be generic: each instantiation is a separate C-callable function, which is how `os.thread_create[Ctx]` (below) builds one trampoline per `Ctx` without a cast. The signature rule below — every type crosses, no `err`, no multiple return — applies to a `@cc fn` exactly as to an `extern`. `@cc` changes call ABI but does not export a stable native symbol: its address is handed to foreign code as a callback, and its linker name remains neper's canonical module/instance name. Exporting a neper function by external symbol name is not in v1; a C-facing shared library therefore needs a future `@export` proposal rather than relying on decoration.
- An `extern` cannot take comptime parameters (`[...]`).
- **Banned inside `@gpu` (§10)**: the device has no OS and no import table. **Not evaluable at compile time (§9)**: a `const` initialiser whose evaluation reaches an `extern` call is a compile error naming the `const` and the call chain.

What crosses the boundary. The ABI is the target C compiler's, and every neper type either maps onto exactly one C type or does not cross at all:

neper	C	Crosses
<code>i8 i16 i32 i64, u8 u16 u32 u64</code>	<code>int8_t ... uint64_t</code>	yes
<code>isize, usize</code>	<code>intptr_t, size_t</code>	yes
<code>f32, f64</code>	<code>float, double</code>	yes
<code>f16, bf16</code>	—	no; pass the bits as <code>u16</code>
<code>bool</code>	<code>_Bool</code> — one byte, 0 or 1	yes
<code>void</code>	<code>void</code>	as a return type only
<code>*T, *const T</code>	<code>T*, const T*</code>	yes when <code>T</code> crosses, is <code>void</code> , or is one of the two pointee-only rows below (<code>Vec[T, N]</code> , <code>Atomic[T]</code>); <code>nil</code> is <code>NULL</code>
<code>*void</code>	<code>void*</code>	yes
<code>extern fn(...) -> R</code>	function pointer	yes
<code>fn(...) -> R</code>	—	no; neper convention
<code>enum T</code>	its backing integer type	yes
<code>struct</code>	struct, by value under the target's aggregate rules	yes when non-empty and every field crosses or is an <code>[N]T</code> of a crossing <code>T</code> ; empty structs do not cross; <code>@packed</code> and <code>@align(N)</code> require an explicitly matching C declaration
<code>union</code>	union	yes, under the same condition
<code>union enum</code>	—	no
<code>[N]T</code>	—	no by value (C passes arrays as pointers); crosses only as a struct field, as <code>T[N]</code> , when <code>N > 0</code> and <code>T</code> crosses
<code>[]T, []const T, str</code>	—	no; pass <code>*T</code> or <code>*const T</code> and a <code>usize</code> separately
<code>err</code>	—	no
<code>(A, B)</code> multiple return	—	no; an <code>extern</code> returns one value
<code>Vec[T, N]</code>	—	no by value; <code>*Vec[T, N]</code> crosses as <code>T*</code> — a pointer to <code>N</code> contiguous <code>T</code> , aligned to the vector's width
<code>Atomic[T]</code>	—	no by value; <code>*Atomic[T]</code> crosses as <code>T*</code>
type, comptime parameters	—	no

"Crosses" means the value is passed and returned exactly as the target C compiler passes and returns the mapped C type. §4's struct layout is C's, so a struct that crosses needs no marshalling. A signature naming a type that does not

cross is a compile error at the declaration, not at the call.

An `extern` call is a direct call through the import table (PE) or the PLT/GOT (ELF, Mach-O): no wrapper, no thunk, nothing saved beyond what the convention requires. §13 states which linking stage each form needs.

e.os

`e.os` is the host operating-system surface, and the only place ordinary `lib/e` modules touch the OS: `io`, `thread`, `time` and the startup code that builds the root arena (§8) call `os.*` and declare no `extern` of their own. It is written per target in per-target files (§6): `os.linux.e` over raw system calls, `os.windows.e` over `kernel32`, `os.macos.e` over `libSystem` — Apple's system-call numbers are not a stable interface, which is what moved Go onto `libSystem` in 1.12. The Vulkan and CUDA driver bindings the GPU runtime (§10) needs are `extern` declarations in the runtime's own modules, over the same mechanism. Those GPU-driver imports are the one documented exception: they are not host services available to ordinary library code.

Two things in `e.os` are compiler intrinsics because the language cannot express them: `os.syscall(n: usize, a0: usize, a1: usize, a2: usize, a3: usize, a4: usize, a5: usize) -> isize`, the raw system call on Linux and the reason a Linux executable can import nothing (six arguments, always; unused ones are 0), and `os.thread_start(stack: [u8, entry: fn(*void), ctx: *void) -> (Thread, err)`, the `clone` and stack switch beneath `thread_create` on Linux — a new thread running `entry(ctx)` on `stack`, which `thread_create` carves from a fresh `os.reserve`; `os.thread_join` releases that stack after the thread has exited, and a detached thread's stack is released by the thread's own exit path. On Windows and macOS `thread_create[Ctx]` is neper source: it instantiates a `@cc` trampoline per `Ctx` (External functions, above) and passes it to `CreateThread / pthread_create`. Everything else is neper source over `extern`. Both intrinsics are compiler-known (§14) — an intrinsic appears in no `.e` file — and exist on Linux alone, so on any other target the name is an unknown-name error like any other.

```
type File      = struct { raw: usize } // fd, or a Windows HANDLE
type Proc      = struct { raw: usize }
type Thread    = struct { raw: usize }
type Lib       = struct { raw: usize }
type Socket    = struct { raw: usize }
type Poller    = struct { raw: usize }
type Mapping   = struct { raw: usize, address: *u8, len: usize }
type Watch     = struct { raw: usize }
type Clock     = enum u8 { Wall, Monotonic }
type SeekWhence = enum u8 { Start, Current, End }
type EntryKind = enum u8 { File, Dir, Symlink, Other }
type DirEntry  = struct { name: str, kind: EntryKind }
type FileInfo  = struct { kind: EntryKind, size: u64 }
type OpenFlags = struct { read: bool, write: bool, create: bool, truncate: bool, append: bool }
type Handle    = struct { raw: usize } // any OS handle: a File's raw, a pipe end
type Stdio     = struct { stdin: File, stdout: File, stderr: File, inherit: []const Handle } // what a child inherits
type SpawnOptions = struct { argv: []const str, env: []const str, inherit_env: bool, cwd: str, stdio: Stdio }
type SocketFamily = enum u8 { Ip4, Ip6 }
type SocketKind = enum u8 { Stream, Datagram }
type SocketShutdown = enum u8 { Read, Write, Both }
type SocketAddress = struct { family: SocketFamily, bytes: [16]u8, scope: u32, port: u16 }
type PollInterest = struct { readable: bool, writable: bool }
type PollEvent = struct { token: usize, readable: bool, writable: bool, closed: bool, failed: bool }
type WatchAction = enum u8 { Added, Removed, Modified, Renamed, Overflow }
type WatchEvent = struct { action: WatchAction, path: str, old_path: str }
type ErrorKind = enum u8 { NotFound, Denied, Exists, Interrupted, OutOfMemory, Timeout, WouldBlock, Unsupported, Invalid, Other }
type ErrorDetail = struct { kind: ErrorKind, native_code: i32, operation: str, subject: str }

error NotFound
error Denied
error Exists
error Interrupted
error OutOfMemory
error Failed
error Timeout
error WouldBlock
error Unsupported
```

Function	Notes
fn open(a: *mem.Arena, path: str, flags: OpenFlags) -> (File, err)	a holds the null-terminated (UTF-16 on Windows) path for the call, under mark/reset
fn read(f: File, buf: []u8) -> (usize, err)	short reads are ordinary; 0 at end of file
fn write(f: File, buf: []const u8) -> (usize, err)	short writes are ordinary
fn seek(f: File, off: i64, whence: SeekWhence) -> (u64, err)	sets and returns the absolute byte offset; SeekWhence is .Start, .Current or .End
fn close(f: File) -> err	
fn stdin() -> File, fn stdout() -> File, fn stderr() -> File	
fn readdir(a: *mem.Arena, path: str) -> ([]DirEntry, err)	names are allocated in a
fn stat(a: *mem.Arena, path: str) -> (FileInfo, err)	follows a final symlink
fn lstat(a: *mem.Arena, path: str) -> (FileInfo, err)	classifies a final symlink itself
fn mkdir(a: *mem.Arena, path: str) -> err	creates one directory; its parent must exist
fn remove_file(a: *mem.Arena, path: str) -> err	removes a file or symlink, never a directory
fn remove_dir(a: *mem.Arena, path: str) -> err	removes an empty directory
fn rename(a: *mem.Arena, src: str, dst: str) -> err	one host-filesystem rename operation
fn read_link(a: *mem.Arena, path: str) -> (str, err)	target bytes are allocated in a
fn pipe() -> (File, File, err)	a read end and a write end; what neper test (§13) uses for each test process's captured streams
fn spawn(a: *mem.Arena, argv: []const str, stdio: Stdio) -> (Proc, err)	argv[0] is the program; the child's three standard streams are the Files in stdio — os.stdin(), os.stdout(), os.stderr() to inherit, a pipe end to capture. stdio.inherit is the extra handles the child inherits beyond those three. Each keeps its raw value in the child — an inheritable handle survives CreateProcess with its value, and on POSIX spawn leaves the descriptor at its own number rather than dup2-ing it — so an ordinary parent may format raw into an argv entry of its choosing. Compiler-reserved test metadata uses a lower startup layer not exposed by this function (§13).
fn spawn_with_options(a: *mem.Arena, options: SpawnOptions) -> (Proc, err)	cwd == "" inherits the current directory; inherit_env chooses whether env overlays the parent environment or replaces it
fn wait(p: Proc) -> (i32, err)	the exit code, or 128 + signal on a signal
fn kill(p: Proc) -> err	terminates the child; wait still returns
fn exit(code: i32)	does not return; runs no defer
fn args(a: *mem.Arena) -> ([]str, err)	what main receives
fn env(a: *mem.Arena, name: str) -> (str, err)	NotFound when unset
fn random(buf: []u8) -> err	fills all bytes from the platform cryptographic random source, retrying interruptions; a zero-length buffer succeeds
fn page_size() -> usize	
fn reserve(n: usize) -> (*u8, err)	address space, inaccessible until committed; n rounds up to pages
fn commit(p: *u8, n: usize) -> err	readable and writable; the OS zero-fills
fn release(p: *u8, n: usize) -> err	returns the range reserve gave
fn clock(c: Clock) -> (i64, err)	nanoseconds; Wall since the Unix epoch, Monotonic from an arbitrary origin
fn thread_create[Ctx: type](entry: fn(*Ctx), ctx: *Ctx, stack: usize) -> (Thread, err)	what thread.spawn (§8) is; stack is the thread's stack size in bytes, written at every call because neper has no default parameters — thread.DEFAULT_STACK (1 MiB, §8) when nothing else is called for
fn thread_join(t: Thread) -> err	
fn thread_detach(t: Thread) -> err	the thread is never joined; its stack is released by its own exit path (above)
fn wait_u32(p: *Atomic[u32], expected: u32, timeout_ns: i64) -> err	returns when *p != expected, is woken spuriously, or times out; negative timeout means infinite and zero means poll once

Function	Notes
<code>fn wake_one_u32(p: *Atomic[u32]), fn wake_all_u32(p: *Atomic[u32])</code>	make one or all current waiters runnable; no count is promised
<code>fn socket_open(family: SocketFamily, kind: SocketKind) -> (Socket, err)</code>	creates a close-on-exec socket
<code>fn socket_set_nonblocking(s: Socket, enabled: bool) -> err</code>	controls whether an operation may return WouldBlock
<code>fn socket_bind(s: Socket, address: SocketAddress) -> err, fn socket_listen(s: Socket, backlog: u32) -> err</code>	server primitives
<code>fn socket_accept(s: Socket) -> (Socket, SocketAddress, err), fn socket_connect(s: Socket, address: SocketAddress) -> err</code>	accepted sockets inherit close-on-exec and nonblocking state
<code>fn socket_receive(s: Socket, dst: []u8) -> (usize, err), fn socket_send(s: Socket, src: []const u8) -> (usize, err)</code>	short transfers are ordinary; receive returns zero only for orderly stream shutdown
<code>fn socket_receive_from(s: Socket, dst: []u8) -> (usize, SocketAddress, err), fn socket_send_to(s: Socket, dst: SocketAddress, src: []const u8) -> (usize, err)</code>	datagram transfers preserve the peer address
<code>fn socket_shutdown(s: Socket, how: SocketShutdown) -> err, fn socket_close(s: Socket) -> err</code>	<code>socket_close</code> consumes the socket
<code>fn socket_resolve(a: *mem.Arena, host: str, port: u16, family: SocketFamily) -> ([]SocketAddress, err)</code>	results are allocated in a and retain platform order
<code>fn file_handle(f: File) -> Handle, fn socket_handle(s: Socket) -> Handle</code>	non-owning values for polling or explicit inheritance
<code>fn poller_open(a: *mem.Arena) -> (Poller, err)</code>	creates the platform readiness/completion poller
<code>fn poller_register(p: Poller, handle: Handle, token: usize, interest: PollInterest) -> err, fn poller_modify(p: Poller, handle: Handle, token: usize, interest: PollInterest) -> err, fn poller_unregister(p: Poller, handle: Handle) -> err</code>	one registration per handle; tokens are returned unchanged
<code>fn poller_wait(p: Poller, events: []PollEvent, timeout_ns: i64) -> (usize, err)</code>	fills a prefix of events; negative timeout means infinite and zero means poll once
<code>fn poller_wake(p: Poller) -> err, fn poller_close(p: Poller) -> err</code>	wake interrupts a current or next wait; close consumes the poller
<code>fn map_file(f: File, offset: u64, len: usize, writable: bool) -> (Mapping, err)</code>	maps a non-empty file range; internal page alignment does not change the exposed bytes
<code>fn mapping_bytes(m: Mapping) -> []const u8</code>	the exact requested range as a read-only view
<code>fn mapping_bytes_mut(m: Mapping) -> ([]u8, err)</code>	the exact requested range; returns Denied unless the mapping was opened writable
<code>fn mapping_flush(m: Mapping) -> err, fn mapping_close(m: Mapping) -> err</code>	flush requests host persistence; close consumes the mapping and invalidates derived pointers
<code>fn watch_open(a: *mem.Arena, path: str, recursive: bool) -> (Watch, err)</code>	opens the native directory-change source; Unsupported when recursive watching cannot be implemented faithfully
<code>fn watch_read(a: *mem.Arena, w: Watch, events: []WatchEvent) -> (usize, err)</code>	fills a prefix with root-relative paths; Overflow requires a rescan
<code>fn watch_close(w: Watch) -> err</code>	consumes the watch and wakes a blocked read
<code>fn dlopen(a: *mem.Arena, name: str) -> (Lib, err)</code>	name as in <code>@import</code> : no prefix, no suffix
<code>fn dlsym[F: type](a: *mem.Arena, l: Lib, sym: str) -> (F, err)</code>	F must be an extern fn type
<code>fn dlclose(l: Lib) -> err</code>	
<code>fn last_error_detail(operation: str, subject: str) -> ErrorDetail</code>	copies the classified <code>errno</code> / <code>GetLastError</code> after this thread's most recent failed <code>os.*</code> call and borrows the two caller strings
<code>fn error_message(a: *mem.Arena, detail: ErrorDetail) -> (str, err)</code>	renders the platform message for <code>native_code</code> into a

Every failing call returns one of the nine errors above after mapping the platform code; `Failed` is the catch-all, and `last_error_detail` gives an explicit portable classification plus the raw code when the mapping is not enough. This list is the v1 surface. A future standard module that needs more host functionality first adds a reviewed primitive here and a per-target implementation; it does not declare its own platform `extern`. Direct optional API bindings, including signals before such a primitive exists, belong in a package named for its actual external owner; Neper-owned additions extend `e.os`.

`OpenFlags` must request `read`, `write` or both. `truncate` and `append` require `write` and are mutually exclusive. `create` creates a missing file and otherwise opens the existing one; newly created files use user read/write permissions subject to the process umask on POSIX and the ordinary inherited ACL on Windows. `append` makes each individual `write` append atomically as the platform defines it. An invalid flag combination returns `Failed` before opening anything and sets the detail code to the platform's invalid-argument code.

Paths are UTF-8. On Windows an invalid UTF-8 path returns `Failed`; valid input is converted losslessly to UTF-16. On POSIX every path byte except zero is accepted, so a directory entry that is not valid UTF-8 is returned with its original bytes in `str`. `readdir` excludes `.` and `..`, preserves the platform enumeration order, does not follow symlinks when choosing `EntryKind`, and returns `Symlink` for a symlink or `Other` for an entry that disappears during classification.

`spawn` inherits the parent's environment and working directory, searches `PATH` when `argv[0]` contains no separator, and rejects an empty `argv`. Each `str` is one argument—Windows quoting and POSIX `argv` construction are implementation details and never reparsed by a shell. Only the three standard streams and handles named in `stdio.inherit` cross; all other library-created handles are close-on-exec/non- inheritable. Duplicate inherited handles are harmless and transmitted once.

`spawn_with_options` applies the same argument, handle and `PATH` rules. Its `env` entries are `NAME=VALUE`. When `inherit_env` is true they overlay the parent environment, with the last duplicate winning; when false they are the complete child environment. An empty `cwd` inherits the parent working directory.

`SocketAddress` stores an IPv4 address in the first four bytes and zeros the remaining twelve; IPv6 uses all sixteen bytes and `scope`. Ports are host-order values. `socket_resolve` performs no connection and retains the platform result order. Pollers store handles, interests and numeric tokens, never callbacks. A handle must be unregistered before its owning file or socket is closed. A wake is level-like: at least one current or subsequent `poller_wait` returns, and redundant wakes may coalesce.

`File`, `Proc`, `Thread`, `Lib`, `Socket`, `Poller`, `Mapping` and `Watch` are logically linear handles even though their raw bits can be copied. `close`, successful `wait`, `thread_join`, `thread_detach`, `dlclose`, `socket_close`, `poller_close`, `mapping_close` and `watch_close` consume all copies; another operation through any consumed copy is undefined behavior, because the OS may already have reused the raw value and neper keeps no hidden handle registry. `kill` after successful `wait` and a second `wait` are the same programmer error. Dropping a live process does not wait for it. On POSIX a signal termination is reported as `128 + signal`; an ordinary exit code with the same number is intentionally indistinguishable. Windows exceptions map to `128 +` the low seven bits of the exception code and leave the full code in the error detail.

Error detail state is per OS thread. A failed `os.*` call sets it before returning; a successful call leaves it unchanged. `last_error_detail` must be called before the next failing OS operation on that thread and copies the state into an ordinary value. A wrapper preserves the detail from the primitive that determines its returned `err`; it does not perform cleanup first. `operation` and `subject` are explicit borrowed labels, so neither exceptions nor a process-global message payload are introduced. `error_message` is the only locale-dependent rendering step.

`reserve`, `commit` and `release` accept page-aligned addresses and page-rounded lengths after the documented rounding of `reserve`; zero length is `Failed`. `commit` may cover an uncommitted subrange of one reservation. `release` must name the exact base and rounded length returned by `reserve`; partial, overlapping or foreign ranges return `Failed`. Arithmetic is checked without wrapping.

The caller keeps a thread context alive until `join` or until a detached thread has finished. Joining twice, joining a detached thread, or detaching twice is undefined behavior. A trap in an entry terminates the process under §11; `join` therefore never converts a language trap into an error.

6. Statements and expressions

```

if cond { } else if cond { } else { }

while cond { }

for i in 0..n { }
for v in slice { }
for i, v in slice { }
for v in it { } // anything with a `next` (§9)

switch k {
case .Int:
    ...
case .Float:
    ...
default:
    ...
}

break
continue
ret // a void function or a kernel
ret expr
defer stmt // runs when the enclosing block exits, in reverse order
defer { ... } // the block form; the same rule

```

- `ret` with no expression is the return of a void function or a kernel (§10); `ret expr` returns a value from every other function. Either in the other's position is a compile error. Reaching the closing brace is an implicit `ret` only for a void function or kernel. Every reachable path in any other function must execute a value-returning `ret`; failure to prove that is a compile error.
- The condition of `if` and `while`, and each operand of `&&`, `||` and `!`, is exactly `bool`. There is no truthiness conversion.
- `switch` cases do not fall through. Cases must be compile-time constants.
- A **case body** is the statements between `case X:` (or `default:`) and the next `case`, `default` or the closing `}` — an **implicit block**: it has its own scope, a `defer` in it runs at its exit, as `v` is bound for exactly that block, it takes no braces and never falls through. `neper fmt` puts a case body on its own lines, indented one level; a body never shares the `case` line.
- A `switch` on an `enum` or a tagged union (§4) must be **exhaustive** unless a `default` case is present; a missing member is a compile error naming it. `case .Member as x:` binds a tagged union's payload for that case's block.
- A `switch` on any other subject — an integer, an `err`, a `bool` — need not be exhaustive: without a `default`, a value no case names runs nothing. A case may list several values, `case 1, 2:`, `case ok, os.NotFound:`. Exhaustiveness is required only for an `enum` and a union `enum` subject.
- Every case value has the subject's type, using the usual contextual typing for an untyped literal. A value may occur in at most one case, and a `switch` has at most one `default`; duplicates are compile errors. `break` exits the innermost loop or `switch`. `continue` targets the innermost enclosing loop, including from a `switch` nested inside it. There are no labels.
- `for i in a..b` runs `i` from `a` up to but excluding `b`, in the bounds' type (§3); a range with `b <= a` is empty, never reversed and never an error. `for v in s` and `for i, v in s` walk a slice or an array: `v` is a **let copy** of the element for that iteration, never a reference, so writing an element is spelled `s[i] = ...` through the index form; `i` is `usize`. An array is iterated as the slice of itself.
- `for v in it` over any other subject looks for `I`'s `next` — `fn <i>_next(it: *I) -> (T, bool)` in `I`'s module (§9): the loop calls it, stops on `false`, and binds `v` to the `T`. The subject must be a `var` or a `*I`, since `next` mutates it. This is what makes `e.data.iter`, a tree walk and a directory walk ordinary `for` loops rather than three different shapes. A `for` over a **comptime** slice is a different thing again — it is unrolled (§9).
- `defer stmt` and `defer { ... }` are both legal — a single statement, or a block — and either runs when the **enclosing block** exits — by reaching its end, by `break` or `continue` leaving a loop body, or by `ret` (a `try` that returns included) leaving every open block up to the function's. Deferred statements run in reverse order of registration, an inner block's before an outer's, after the `ret` expression has been evaluated; a `defer` in a loop body runs once per iteration. A `trap` (§11) and `os.exit` (§5) run none. A deferred call cannot `try` — there is nothing to return from — so a fallible one is written `defer let _ = f()` (§7). It is the arena idiom's companion: acquire, `defer` the release, forget it.

- Registering a `defer` evaluates the receiver and every argument of a deferred call immediately, from left to right, and saves their values; only the call itself runs at exit. A deferred block captures places rather than snapshots and evaluates its statements at exit. This distinction makes `defer close(f)` retain the current handle while `defer { inspect(x) }` observes `x` as it stands at block exit. A `defer let _ = f(...)` discard is a deferred call for this rule and captures its arguments immediately. Any other deferred single statement behaves as a one- statement block.
- Control may not leave a deferred body: `ret` and `try` are illegal anywhere inside it, and `break` or `continue` may target only a loop or switch lexically inside that body. A nested `defer` is legal and runs when its deferred body's own scope exits.
- There is no `goto`, no labelled `break` in v1, and no expression-level `if`.

Operators

Arithmetic	<code>+ - * / %</code>	(overflow of <code>+ - * checked in debug, wraps in release; integer division by zero and <code>MIN / -1</code> trap in every mode – §11)</code>
Unary erand)	<code>-</code>	(negation; <code>-MIN</code> is an overflow check like <code>+ - *</code> ; a compile error on an unsigned op
Wrapping	<code>+% -% *%</code>	(explicitly wrapping, all build modes)
Bitwise	<code>& ^ ~ << >></code>	(a shift count <code>>=</code> the width traps in debug, is masked in release)
Compare	<code>== != < <= > >=</code>	
Logical	<code>&& !</code>	(short-circuit)
Assign	<code>= += -= *= /= %= += -= *= &= = ^= <<= >>=</code>	
Access	<code>. [] [a..b] & *</code>	(field, index, slice, address-of, dereference)

Precedence and associativity, tightest first. It is C's, with two changes: every bitwise operator binds tighter than every comparison, so `a & b == c` is `(a & b) == c`, and comparisons do not chain, so `a < b < c` is a compile error.

Level	Operators	Associativity
1	<code>a.b, a[i], a[lo..hi], f(...), f[...]</code>	left
2	prefix <code>- ! ~ & *</code>	right
3	<code>* / % *%</code>	left
4	<code>+ - +% -%</code>	left
5	<code><< >></code>	left
6	<code>&</code>	left
7	<code>^</code>	left
8	<code> </code>	left
9	<code>== != < <= > >=</code>	none — two in a row is a compile error
10	<code>&&</code>	left
11	<code> </code>	left

So `-a % b` is `(-a) % b`, `x << 1 + y` is `x << (1 + y)`, and `a || b && c` is `a || (b && c)`. Assignment is a statement, not an expression: `a = b = c` does not parse. `try` is not an operator; it prefixes one call expression and takes its result (§7). `neper fmt` neither adds nor removes parentheses, so what is on the page is what parses.

Integer `/` truncates toward zero and `%` takes the sign of the dividend, as in C, so `a == (a / b) * b + a % b` always holds; `%` is defined on integers only — on a float operand it is a compile error, and there is no `fmod` in the language. `>>` is arithmetic on a signed operand and logical on an unsigned one. Unary `-` on an unsigned operand is a compile error: negate through a signed type, `-i64(x)`, or write `0 -% x` for the wrapping complement. The right operand of `<<` and `>>` is **independent of the left**: it must be an unsigned integer type, and an untyped literal count becomes `u32` — the one binary operand a literal does not take from the other side (§3). The count is never converted, only checked against the left operand's width: a count `>=` the width traps in debug and is masked in release (§11). Float `/` follows IEEE 754 in every mode — `x / 0.0` is an infinity, `0.0 / 0.0` is NaN — and is never a check (§11, Floating point).

No operator overloading — `+` on strings does not exist and will not (§4).

`==` and `!=` apply to primitives, pointers and enums only. On slices they would silently compare pointer and length rather than contents, which is a trap; use `str.eq` or `mem.eq[T]`. Structs, unions and arrays are likewise compared by an explicit function, never by operator.

Implicit operations

This is the complete list of implicit operations in the language. §4, §10, §14 and D18 refer here rather than restating it:

Operation	Where
<code>a.b</code> reaches through a pointer: <code>p.x</code> where <code>p</code> is <code>*Vec3</code> or <code>*const Vec3</code>	any field access
<code>[]T</code> converts to <code>[]const T</code>	any use site
<code>*T</code> converts to <code>*const T</code>	any use site
<code>gpu.Buf[T]</code> is mapped onto a <code>[]T</code> , <code>[]const T</code> or <code>[]Atomic[T]</code> kernel parameter	<code>gpu.launch</code> only (§10)
a <code>for</code> over a <code>var</code> iterator passes the place as <code>*I</code> to its generated next call	compiler-generated iterator call only (§6, §9); no source expression is rewritten

Each removes a capability or names a boundary; none changes a value's bits or its type's width. Everything else — every widening, every narrowing, every integer-to-float — is a cast written on the page (§4), which is what §14 invariant 6 promises.

Operator types. Except for shifts, both operands of a binary arithmetic, wrapping, bitwise or comparison operator have exactly the same type after contextual typing of an untyped literal; the language never inserts a conversion. `+` `-` `*` `/` accept any integer or float type, `%` and bitwise operators accept integers only, and wrapping operators accept integers only. Unary `-` accepts signed integers and floats; `~` accepts integers. Arithmetic returns the operand type. `==` and `!=` accept integers, floats, `bool`, `err`, pointers of the same type after the one-way const conversion, and values of the same enum type. Ordering accepts integers, floats and values of the same enum type; enum ordering is backing-value ordering. Pointers and function pointers have equality but no ordering. Every comparison returns `bool`. Floating comparisons are IEEE ordered comparisons: any ordering comparison with NaN is false, `==` with NaN is false, and `!=` with NaN is true.

Evaluation order. Neper evaluates subexpressions left to right. For a call it evaluates the callee then arguments left to right; for a binary operator it evaluates the left operand before the right; for assignment it evaluates and saves the target place before the right-hand side; aggregate literal elements and fields are evaluated in source order; and a multiple-return expression is evaluated once before its results are assigned left to right. `&&` and `||` are the only operators that may skip their right operand. A trap or early return preserves all effects already performed and performs none to its right.

Conditional compilation

There is no preprocessor. Target-dependent code uses `when`, which type-checks all branches and emits one:

```
when target.arch == .X64 {
    ...
} else {
    ...
}
```

Both branches must parse and type-check, so dead configurations cannot rot.

`target` is a builtin comptime namespace: it needs no `use` and cannot be shadowed (§2). `target.arch` is a `target.Arch`, an enum with members `.X64`, `.Aarch64`, `.X86`, `.Srv`, `.Ptx`; `target.os` is a `target.Os` with members `.Windows`, `.Linux`, `.Macos`, and `.None` on the GPU targets — one member per `e.os` target (§5), so an Android host is deferred with its `os.android.e` (§17).

`when` is a **statement**, legal only inside a function body. It cannot wrap a module-scope declaration, so invariants 2 and 7 of §14 hold without exception. A `type`, `const` or `fn` that differs per target goes in a per-target file instead:

```
lib/e/sys.e // module e.sys on every target with no specific file
lib/e/sys.windows.e // module e.sys when target.os == .Windows
lib/e/sys.linux.e // module e.sys when target.os == .Linux
```

The rule: a file `<name>.<suffix>.e` whose suffix is an `Arch` or `Os` member in lowercase is compiled only for that target and is module `<name>` there; a plain `<name>.e` serves every target with no matching suffixed file. **At most one suffixed file may match a target:** two files matching one module on one target is a compile error. A module may also be split by architecture alone — `sys.x64.e`, `sys.aarch64.e` — but mixing arch and os suffixes for one module is a compile error at build time on any target matching both. Each variant is a complete module with its own declarations, so a glob still finds it

and §2 still holds.

7. Errors

`err` is a builtin distinct type. `ok` is its zero value. Error values are declared at module scope, one per line:

```
error NotFound
error Truncated
```

Error names and values

An error name is **module-scoped**, like every other declaration (§14 invariant 1): `error NotFound` in `e.os` and an `error NotFound` in a module of your own are two errors, and the module that declares one need know nothing about the other. Inside its module an error is named bare; from any other module it is qualified through the `use` qualifier, `os.NotFound`, exactly as a function is (§14 invariant 4).

The value of an error is the **32-bit FNV-1a hash of its fully qualified name** — the module name from §2 plus the declared name, `e.os.NotFound`; this is the one place FNV-1a is used, every other hash in the toolchain being `xxHash64` (§12). Nothing is numbered. The value is a function of the source text alone, so it is a compile-time constant, legal as a `switch` case (§6); it needs no global pass; it does not change when another module adds an error; and it is the same on every thread and in every incremental build (§12, §15). `ok` is `0`. A name whose hash is `0` is a compile error at the declaration, and the fix is to rename it.

Two distinct qualified names with the same hash are a collision, and it is caught at link time. Every `.em` Interface carries the module's **error table**, value to qualified name (§12). `neper` merges the tables of every module in the program before the executable is written — under its own linker or `--linker=system` alike (§13) — and rejects the link when one value maps to two names, naming both. The merged table goes into the executable. It is what `main`'s failure line (§13), the trap protocol (§11) and `neper test` read, and it is the only name-to-value data the binary carries.

An `err` is **formattable**: `{}` (§4) prints the qualified name, `e.os.NotFound`, or `ok`. A value absent from the table — reachable only through `undef` (§5) or a union pun — prints as `err(0x<value in hex>)`, `err(0xCDCDCDCD)` for `undef` storage.

```
use e.os

let (entries, e) = os.readdir(a, path)
switch e {
case ok:
    ...
case os.NotFound:
    ... // qualified: the error belongs to e.os
default:
    ret e
}
```

Fallible functions declare `err` as their last return value (§5):

```
error NotFound

fn read(path: str, a: *mem.Arena) -> ([u8, err]) {
    var out: [u8] = zero
    ...
    ret (out, NotFound)
}
```

`try` propagates: it evaluates the call, returns early if the `err` is non-`ok`, and yields the remaining values otherwise. It is only legal in a function whose last return element is `err`. On that early return every other return slot holds the **zero value** of its type, so a caller that ignores the `err` sees a definite value, never stack garbage.

Zero values

The zero value is defined for every zeroable value type. It is what `= zero` writes (§5), what a module-scope `var` starts as, and what `try` returns beside a failure. `void`, type, function types, multiple-return lists, argument packs, `Field` and `Member` are not value types with a zero and may not appear in any of those positions.

Type	Zero value
integers, usize, isize	0
floats	+0.0
bool	false
err	ok
*T, *const T, function pointers	nil
[]T, []const T	nil — { ptr: nil, len: 0 }; nil is its spelling in expression position (§4)
[N]T	N zero values of T
struct	every field zero; padding bytes zero
union	every byte zero
union enum	tag 0 — the first member — with a zero payload
enum	the member whose backing value is 0
Vec[T, N], Atomic[T]	every lane, or the wrapped value, zero
Mask[T, N]	every lane false; register-only (§4), so = zero on a local and a try return slot are the only places it is written

A type has a zero value unless it contains, at any depth — a field, an element, a payload — an `enum` with no member at 0. For such a type = `zero`, a module-scope `var` without an initialiser, and `try` in a function returning it are compile errors naming the outermost type and the offending field. Every other type has one, which is why library types built from structs need no rule of their own. The comptime-only types `meta.Field` and `meta.Member` (§9) are outside the table and outside that rule: neither has a zero value and neither needs one, because neither is ever storage — not a field, an element, a local or a pointee — so no = `zero`, no module-scope `var` and no `try` return slot can name one.

```
let entries = try os.readdir(a, path)
```

Where `try` may appear

`try` is a **statement-level form**, not an operator inside an expression. It is legal in exactly three positions:

The spelling is deliberately the keyword `try`, never `?call()`. Both are normally one lexical token and their model-token cost depends on the tokenizer, while the word is searchable, announces control flow to a reviewer, and leaves `?` available if an optional-type proposal ever earns it. The benchmark in `docs/tooling.md` §9 measures the cost rather than inferring it from character count.

```
try str.push(&b, name) // whole statement; callee returns only `err`
let entries = try os.readdir(a, path) // the entire initialiser of a `let` or `var`
l.items = try mem.alloc[T](a, n) // the entire right-hand side of an assignment
```

Anywhere else is a compile error naming the position: inside an argument list (`f(try g())`), as an operand (`((try f()) + 1)`), inside a literal, an index or a condition, or after `ret`. Write the temporary:

```
let v = try g()
f(v)
```

This is the same stance the language takes everywhere else — assignment is a statement, there is no expression-level `if`, comparisons do not chain (§6). Control flow lives at statement level, and `try` is control flow: a `try` buried in an argument list is a return point in the middle of an expression, and the arguments to its right silently never evaluate. One line per return point costs a temporary and buys a reader who can see every exit from a function by scanning the left margin.

When the call returns more than one non-error result, the binding or assignment names all of them: `let (a, b) = try f()` and `(a, b) = try f()`. The parenthesized list is the existing multiple-return binding syntax, not a tuple value. A whole-statement `try f()` is legal only when `err` is the call's sole result.

Two positions have no enclosing function to return from, so `try` in them is a compile error with no alternative spelling:

- **Inside `defer`.** A deferred call runs *during* a return; there is nowhere to propagate to. A deferred call that returns an `err` is written `defer let _ = f()` — discarding it deliberately — or the callee returns `void`.
- **Lexically inside a `const` initialiser**, when **condition**, **[...] argument or comptime-only call expression** (§9). Those initiating expressions have no enclosing function to return from. A normal helper function reached dynamically by the

interpreter may contain `try`; if its error reaches the initiating expression, evaluation fails with a compile error naming that root and the call chain. This is the lexical/dynamic distinction used whenever this document says “comptime context.”

To handle rather than propagate, destructure:

```
let (entries, e) = os.readdir(a, path)
if e != ok {
    ...
}
```

There are no exceptions, no unwinding, and no error payloads. An `err` is a 32-bit value; carrying detail is the caller's business.

8. Memory

There is no global allocator and no `malloc`. Memory is either stack storage or comes from an arena you created.

```
type Arena = struct {
    base: *u8,
    cap:  usize,
    off:  usize,
}

type Stats = struct { used: usize, capacity: usize }

error Exhausted // the arena is full (§11)

fn arena_from(buf: []u8) -> Arena
fn alloc[T: type](a: *Arena, n: usize) -> ([T], err) // aligned to align_of[T]
fn mark(a: *Arena) -> usize
fn reset(a: *Arena, m: usize)
fn copy[T: type](dst: [T], src: [const T] // dst.len < src.len is a bounds check (§11)
fn eq[T: type](x: [const T], y: [const T] -> bool // element-wise value equality; padding is ignored
fn cast[P: type, Q: type](p: Q) -> P // intrinsic (§4); P and Q are pointer types; Q is inferred
fn bitcast[T: type, U: type](x: U) -> T // intrinsic (§4); U is inferred and its size equals size_of[T]
fn size_of[T: type]() -> usize // intrinsic, comptime: §4's layout
fn align_of[T: type]() -> usize // intrinsic, comptime
fn stats(a: *const Arena) -> Stats // derived from off and cap; allocates nothing
```

`alloc` returns `Exhausted` when `n` elements do not fit, the one `err` in §11's table, and `copy` returns nothing: its only failure is a bounds violation, which is a check. `copy` has `memmove` semantics — `dst` and `src` may overlap. `stats` derives both its fields from the arena as it stands — `used` is `off`, `capacity` is `cap` — and does nothing else, so “how much of this arena did that phase use” is an ordinary line in every build mode. An `Arena` carries no counters: a high-water mark across `reset` calls and a count of allocations would each cost a branch and a store in every `alloc`, and `alloc` being a bump and a bounds compare is what D3 buys. `size_of` and `align_of` are compile-time constants and the only way to name a type's size. `mem.cast[P](p)` explicitly fills the first comptime parameter and infers trailing `Q` from `p` under §9; both must be pointer types, and the intrinsic then applies §4's pointer-cast rules. `mem.bitcast[T](x)` similarly infers trailing `U` from `x`: `x` is a value of any type whose size equals `size_of[T]` — a mismatch is a compile error naming both sizes — and the result is `x`'s bytes read as a `T`, so a pun never needs a union (§4). It is legal only when both types are sized and neither contains a pointer, slice, function pointer, type, `Atomic`, or target address-space value at any depth. This prevents it from casting away `const`, inventing provenance, changing an address space or manufacturing a callable address. A pointer or slice operand in device code is `mem.cast` by another name and is banned with it.

Reading bytes as `bool` requires the result to be `0` or `1`; reading them as an enum or tagged-union tag requires a declared member. A violation is an `invalid` check: it traps in debug and is unreachable behaviour in release. `undef` may create such bytes, but reading the invalid value—not creating its storage—is the violation. A pointer produced by `mem.cast` retains the source address, provenance, address space and constness.

`mem.eq` compares values, not object representations. Integers, booleans, errors, enums and pointers use their ordinary `==`; floats use ordinary IEEE equality; arrays, vectors and structs recurse in declaration/index order; tagged unions compare the tag and then the live payload; slices compare their elements. Padding is never read. Bare unions, atomics, masks, function pointers and types without ordinary equality are rejected at instantiation.

The typical shape:

```
fn process(input: str, out: *mem.Arena) -> err {
  var scratch_buf: [64*1024]u8 = undef // stack; no store, poisoned in debug
  var scratch = mem.arena_from(scratch_buf[0..])
  ...
  ret ok
}
```

`mark/reset` give you scoped deallocation without per-object bookkeeping. A program's root arena is a block reserved from the OS at startup; its size is **64 MiB** unless the link option `--arena SIZE` (§13) raises or lowers it. The startup code obtains it through `e.os` (§5) before `main` runs — `os.reserve` for the address range, then `os.commit` for the whole of it, free under Linux overcommit, charged against the pagefile on Windows, which is why the size is a link option rather than a large default — and passes it to `main` as its first parameter (§13). That parameter is the only way to reach it.

An Arena is a cursor, not an independently copyable owner. Copying an Arena value is legal only to transfer it to a new binding after the old binding is no longer used; using two copies derived from the same value is a programmer error and has undefined behaviour. Library APIs therefore pass `*Arena` and container structs must not embed duplicate live cursors. `mark(a)` returns an offset valid only for that arena. `reset(a, m)` requires `m <= a.off`; a larger mark is a bounds check. Resetting to a mark obtained from a different arena is likewise invalid, although equal numeric offsets cannot be distinguished at runtime and remain the programmer's responsibility.

`alloc[T](a, n)` first checks multiplication and alignment arithmetic without wrapping. `n == 0` returns `{ ptr: a.base + a.off, len: 0 }` without advancing `off`; the pointer may be nil only for an empty nil-backed arena and may not be dereferenced. Invalid arena invariants (`off > cap`, a nil base with nonzero capacity, or a base that cannot satisfy its recorded alignment) are `invalid` checks. `reset` poisons only the released byte range in debug and leaves bytes unchanged in release.

Library values that claim the top of an arena, including `str.Builder`, follow the same single-live-copy rule. `done` consumes that logical value: any later operation through it or a copy is an `invalid` check in debug and undefined in release. The implementation stores enough debug-only generation state to diagnose this without adding release fields. A sink called by formatting may not allocate from or reset the builder's arena and may not recursively write to the same builder; violating either rule is an `invalid` check.

A growable container must not hand out slices across a `push`. `list.e`'s `push` allocates a bigger buffer when the old one is full and abandons the old one in the arena; a slice taken before that push still points at the abandoned buffer, which stays readable and is silently stale. No check can catch it, because the memory is still valid memory. Take the slice after the last push, or take it again. The debug fill on `reset` (§11) catches the other hazard, a slice that outlives `mem.reset`; this one is the container's contract, stated in its comment.

Because no function allocates except from an arena it was handed — `gpu.open`'s arena on the `.Cpu` backend included (§10) — save the driver staging block of `gpu.upload/gpu.write` on a driver backend (§10, D3), the compiler knows a call's full cost from its signature — which is what makes cross-module inlining across `.em` files cheap to reason about.

Threads and atomics

OS threads, and nothing else. No runtime, no scheduler, no green threads, no thread pool in the language. A scheduler would add latency and unpredictability to the one thing neper is supposed to be good at.

```
use e.thread

fn worker(ctx: *Ctx) {
  ...
}

var ctx = Ctx{ ... }
let h = try thread.spawn[Ctx](worker, &ctx, thread.DEFAULT_STACK)
try thread.join(h)
```

`thread.spawn[Ctx: type](entry: fn(*Ctx), ctx: *Ctx, stack: usize) -> (Thread, err)` is `os.thread_create` (§5) under its own name, `thread.join(t: Thread) -> err` is `os.thread_join`, and `thread.detach(t: Thread) -> err` is `os.thread_detach`: a detached thread is never joined, and its stack is released by its own exit path (§5). The stack size is written at every call, because neper has no default parameters; `const DEFAULT_STACK: usize = 1024*1024` in `e.thread` is the value to write when nothing else is called for.

One arena per thread. This falls out of the arena model above rather than being added to it: there is no global allocator, so there is nothing for threads to contend on. Arenas are not thread-safe and are not meant to be — a shared one is a bug.

Thread state travels in an explicit context pointer, not thread-local storage. Platform TLS (`__thread`, `__declspec(thread)`) costs a lookup on some platforms and hides state that the rest of the language keeps visible. A parameter is free and greppable.

Atomics are builtin types, lowering to single instructions (`lock xadd`, `ldaxr/stlxr`) with no abstraction to strip. `Atomic[T]` is legal for exactly `T` an integer type of §4 — `i8` through `u64`, `isize`, `usize` — or a pointer — `*T`, `*const T`, `*void` or a function pointer, all word-sized; on the CPU every one is native at its width, and §10 narrows the set to 32- and 64-bit integers in device code:

```
type Counter = struct {
    hits: Atomic[u64],
}

let n = atomic.add(&c.hits, 1, .Relaxed)
let v = atomic.load(&c.hits, .Acquire)
atomic.store(&c.hits, 0, .Release)
let (won, seen) = atomic.cas(&c.hits, old, new, .AcqRel, .Acquire)
atomic.fence(.SeqCst)
```

`atomic.init(v)` constructs an `Atomic[T]` holding `v` — `Counter{ hits: atomic.init(0u64) }` — and is comptime-evaluable, so it is legal in a struct literal, a `const` and a module-scope `var` initialiser; the zero value of an `Atomic[T]` is `0` (§7), so = zero needs no constructor.

Orderings are `.Relaxed`, `.Acquire`, `.Release`, `.AcqRel`, `.SeqCst`, with the acquire-release semantics of the C11/C++11 model — well understood, and a direct map onto every target's hardware model.

`e.atomic` **is the operation surface**, every function an intrinsic (§14) and comptime-generic on `T`, inferred from `p` — from `v`, for `init` — per §9. `T` is any type `Atomic[T]` admits; the arithmetic and bitwise operations take an integer `T` only, and `init`, `load`, `store`, `xchg` and `cas` take a pointer `T` as well:

```
type Ordering = enum u8 { Relaxed, Acquire, Release, AcqRel, SeqCst }

fn init[T: type](v: T) -> Atomic[T] // comptime-evaluable (above)
fn load[T: type](p: *Atomic[T], o: Ordering) -> T // o: Relaxed, Acquire or SeqCst
fn store[T: type](p: *Atomic[T], v: T, o: Ordering) // o: Relaxed, Release or SeqCst
fn xchg[T: type](p: *Atomic[T], v: T, o: Ordering) -> T // the previous value
fn cas[T: type](p: *Atomic[T], expected: T, desired: T, success: Ordering, failure: Ordering) -> (bool, T)
fn add[T: type](p: *Atomic[T], v: T, o: Ordering) -> T // the previous value; wraps, like +%
fn sub[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn and[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn or[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn xor[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn min[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn max[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn fence(o: Ordering)
```

`cas` is the strong compare-and-swap: it returns `true` and `expected` when the exchange happened, `false` and the value it found otherwise, and never fails spuriously. A `load` with a release ordering or a `store` with an acquire one is a compile error, as C11 has it. A CAS failure ordering may be `.Relaxed`, `.Acquire` or `.SeqCst`, may not be `.Release` or `.AcqRel`, and may not be stronger than its success ordering; an invalid constant pair is a compile error and an invalid runtime pair is an invalid check. The device forms are these with a scope (§10).

There is no compile-time protection against data races. With no ownership system (D3), racing is the programmer's responsibility. The compiler assumes non-atomic accesses are race-free and optimises accordingly; a race on a non-atomic location is undefined behaviour, exactly as in C. This costs nothing at runtime, which is the point.

Threads are a CPU-profile feature. Inside `@gpu` there are no threads, no atomics on host memory, and no function pointers to spawn with — GPU parallelism is the launch grid (§10).

9. Compile-time parameters

Anything in square brackets is evaluated at compile time and monomorphised per distinct instantiation.

```
fn max[T: type](a: T, b: T) -> T {
  if a > b { ret a }
  ret b
}

type Buf[T: type, N: usize] = struct {
  data: [N]T,
  len:  usize,
}

let m = max[i32](3, 9)
var b: Buf[f32, 16] = zero
```

A comptime argument may be inferred from the value arguments whose parameter types mention it. The inference is **unambiguous** when every such argument that is not an untyped literal (§3) agrees on one type; the literals are then typed by the parameter. If no argument fixes it, or two disagree, it is a compile error naming the parameter. `max(x, 9)` with `x: i32` infers `T = i32` and types `9` as `i32`; `max(3, 9)` is an error naming `T`; `max(3i32, 9)` is not.

Inference structurally unifies each declared parameter type with the corresponding typed argument: qualifiers, pointers, slices, arrays and instantiated generic types are traversed recursively, aliases are replaced by their canonical underlying type, and array lengths may infer integer comptime parameters. Every occurrence of one parameter must produce the same value. Explicit arguments fill parameters from the left and inference fills only omitted trailing parameters; holes are not allowed. Inference never uses a return context, a protocol result or an untyped literal as its sole evidence. After inference, every literal is typed and the ordinary call rules run.

A generic declaration is parsed and name-resolved once. Rules independent of a comptime parameter are type-checked at declaration; any operation whose validity or type depends on such a parameter is checked at each instantiation and an error reports both the operation and the instantiation site. There are no declared constraints: `max[T]` is a valid template, but `max[SomeStruct]` fails unless `>` is defined for that concrete type.

A bracket after a name **resolved to a function or a type** is a comptime argument list; after a name resolved to a value it is an index. The concrete parser always builds the same lossless `BracketPostfix` node and needs no symbol table; name resolution classifies that node from the symbol table, never from its contents, and a name is never both (§14 invariant 1). A generic function appears in a **call** — with its bracket, or with the bracket inferred as above — or, with its bracket written in full, in value position, where `f[i32]` is a function pointer to that one instance (which is how `os.thread_create[Ctx]` hands a `@cc` trampoline to the OS, §5); a generic type appears only with its bracket. There is no pointer to a generic as such — the inferred form is never a value — and `f[N]` with `f` a function is always an instantiation even when `N` is a `const` that could have been an index.

A comptime parameter has a **kind**: `type` (`T: type`), an integer type (`N: usize`), `str` (`FMT: str`), `bool`, a fixed array of an integer type (`IDX: [N]u8`, which `simd.shuffle` takes, §4), a comptime-only struct value (`FIELD: Field`, Compile-time introspection below), or `fn` (`κ: fn`). In a comptime position a function name is a value of kind `fn`, as a type name is a value of kind `type`. The parameter is bound at compile time and monomorphised per function; inside the body `κ` is called like any function, and its concrete signature is visible after instantiation. A call involving `κ` is checked then; incompatible arity, parameter or return use is an error at the instantiation site with the declaration site as a note. It is not a function pointer and forms none, which is why `gpu.launch[κ]` (§10) can take a kernel where a pointer to device code is banned — and why any function, a device helper included, may declare a `κ: fn` parameter of its own: it is monomorphisation, not indirection. `neper-0` (roadmap) omits the `fn` kind along with packs.

A type expression in a declaration may refer to earlier comptime parameters and to comptime fields of their values. It is evaluated at instantiation before the body is checked, which is why `FIELD.ty` can be the result of `meta.get`. It may not refer to a runtime parameter or a later comptime parameter. This is ordinary dependent specialization of a monomorphised declaration, not runtime dependent typing, and is available to user declarations under the same rule as the `e.meta` intrinsics.

A comptime expression whose type is `type` may be written wherever a type is written. It is evaluated at the instantiation like any other comptime expression, and what it yields is a type in every respect — a return type, a parameter type, a local's annotation, a `[...]` argument, a protocol receiver. Two forms occur, both in Compile-time introspection below: `FIELD.ty` as the return type of `meta.get`, and `f.ty` as the receiver of `f.ty.format(x, b)`.

`const` initialisers are evaluated by the same compile-time interpreter, under the rules of the next subsection — enough to build lookup tables at compile time.

Compile-time evaluation

One interpreter evaluates four things: every `[...]` argument, every `const` initialiser, every `when` condition (§6), and every call to a **comptime-only** core function — `meta.fields`, `meta.members`, `meta.type_name` (Compile-time introspection, below) — standing in an ordinary function body with every argument comptime-known, which is evaluated at the enclosing instantiation and its result folded in exactly as a `[...]` argument's is. Those four sites are what **comptime context** means throughout this document, and a comptime-only function called anywhere else — with an argument the compiler cannot know at that site — is a compile error naming the function and the argument. It runs neper, not a second language: any function is callable at compile time unless its evaluation reaches **runtime state**, which is exactly this list:

- a module-scope `var`, read or written;
- an `Atomic[T]` operation other than `atomic.init`, or `atomic.fence` (§8);
- `thread.*`, and every `extern` call (§5) — all of `e.os`, `os.syscall` included;
- the `gpu.*` builtins and host API (§10).

Everything else is legal: stack arrays, pointers, slices, union punning, `mem.arena_from` over a comptime array and `mem.alloc` from it, `try` inside a function the interpreter executes, `defer`, `switch`, generic instantiation, `unreachable()` (which fails the evaluation). Reaching runtime state is a compile error naming the `const` — or the comptime argument — and the call chain from it to the offending operation.

Memory. Interpreter memory is a byte-addressed arena owned by the compiler, one per evaluation. Every stack frame, array and arena buffer the evaluated code creates lives in it; `&x`, slicing and `mem.cast` yield addresses into it; and a load or a store reinterprets the bytes at that address in the **target's layout** — the field offsets, sizes and alignment of §4 for the target being compiled, and the target's endianness. `union` punning at compile time therefore gives the bytes the target would give: writing `.i` and reading `.f` on a `Value` produces the same bits at compile time as at runtime, and the C bootstrap and the self-hosted compiler must agree byte for byte here, which is what the M2 determinism harness (roadmap) checks. A comptime result is target-independent only where its type has no target-dependent size — `usize`, `isize` and pointers are the target-dependent ones — which is one reason `.em` files are per target (§12).

Results. A `const`'s final value must be **pointer-free**: no `*T`, no `[ ]T`, with one exception — a `[ ]const u8` that refers to **read-only data the compiler emits**: a string literal, or a compiler-synthesised name, which is what `meta.type_name[T]()` and a `Field`'s or `Member`'s `name` return (Compile-time introspection, below). Either has an address in the executable's read-only data. A value that violates this — a slice into interpreter memory, a pointer to a comptime stack frame — is a compile error naming the `const` and the field or element holding the pointer. Everything the value was built from is discarded when the evaluation ends; the value itself is serialised into the `.em` (§12) and folded into every use as an immediate or a read-only data reference.

Budget. One evaluation — one `const`, one comptime argument, one `when` condition, or one comptime-only call folded into a body — is bounded to **10,000,000 interpreter steps** (one NIR instruction each), a **call depth of 1024**, and **64 MiB of interpreter memory** (the arena above). Exceeding any of the three is a compile error naming the `const` and the call chain at the point the budget ran out. The budget is per evaluation, not per module, and there is no flag to raise it: a table that needs more than ten million steps is generated into source by a program, where the reviewer can see it. A build cannot hang on a `const`.

Caching. An evaluation's cache key contains the canonical typed root expression, the fully typed bits of every comptime argument, the target triple and data-layout version, build mode, language and `.em` format versions, and the body hashes (§12) of every function it executed. Read-only string values are keyed by contents, not their temporary interpreter address. A callee edit, argument change, target change or semantic-version change therefore misses the cache. Pointer equality during an evaluation compares (`allocation identity`, `offset`) pairs; allocation identities are local to that evaluation, deterministic in execution order and may never appear in a serialized result.

Protocols

A generic function often needs an operation on its type parameter — hash this key, compare these two, print that. neper has no interfaces, no traits and no closures, so the operation is found by **name, at compile time, in the module that declares the type**.

```
fn get[K: type, V: type](m: *Map[K, V], key: K) -> (V, bool) {
  var out: V = zero
  let h = K.hash(key)
  ...
  if K.eq(m.keys[i], key) {
    ret (m.vals[i], true)
  }
  ret (out, false)
}
```

`K.hash(key)` resolves, at the instantiation, to `fn <k>_hash` in the module that declares `K`, where `<k>` is `K`'s own name in snake_case: `Sensor.hash` is `fn sensor_hash`, `Vec3.format` is `fn vec3_format`. Conversion to snake case scans Unicode scalar values: a boundary precedes an uppercase letter following a lowercase letter or digit, and precedes the last uppercase letter of a run when the next letter is lowercase; ASCII capitals are lowercased, digits and existing underscores are retained, and adjacent underscores collapse to one. Thus `HTTP2Client` becomes `http2_client`. Instantiate `get[Sensor, f32]` and the call is a direct call to `sample.sensor_hash`.

Instantiate `get[str, f32]` and there is no such call, because `str` is the language's own alias of the structural type `[]const u8` (§4, D48), which has no declaring module: rule 4 below supplies the slice `hash`, over the bytes. An alias of a **named nominal type** keeps that type's canonical declaration, declaring module and protocol name; an alias of a primitive or structural type has none. Protocol lookup therefore never depends on which alias spelling appears at a call site.

For an instantiated nominal generic type, the protocol stem is the declaration's uninstantiated name and the matching protocol function carries the same leading type parameters. For type `Box[T: type] = struct {...}`, lookup for `Box[i32].hash` requires `fn box_hash[T: type](v: Box[T]) -> u64` and instantiates it with `i32`. A non-generic declaration with one concrete `Box[...]` parameter is not the protocol for the generic type; users call such a specialization by its module-qualified name.

The type's name is in the function's name because §14 invariant 1 admits no overloading: a module that declares `Sensor` and `Reading` would otherwise need two `fn hash` lines, and `grep -n "^fn hash"` would stop returning exactly one. Prefixing keeps one definition per name, keeps every protocol function greppable on its own, and lets one module carry protocols for as many types as it declares.

Five rules keep this from becoming a second way to call a function:

1. **`T.f(...)` is legal only where `T` is a comptime type parameter or a comptime type value.** On a type you can name you write the module qualifier — `str.eq(a, b)` — because you know the module. The receiver need not be a name: any expression whose type is `type` and whose value is comptime-known stands there, which is what `f.ty.format(...)` in the introspection example below is (§2). The rule governs what an author **writes**; a protocol call the compiler generates on the author's behalf — a `for`'s `next` (§6), a `format` verb's `format` (§4), a container's `hash`, `eq` or `cmp` on a concrete element type — appears on no line and is outside it. §14 invariant 4 is untouched: every call site whose module is knowable, and that someone wrote, names it.
2. **Type-level names win.** Enum members, `<Union>.Tag` and the builtin type operations of §4 — `T.trunc(x)`, and the cast `T(x)` — resolve before protocol lookup, so `Kind.Int` is the member, `Node.Tag` is the tag type and `T.trunc(x)` is the truncating cast whatever the declaring module happens to declare. A member and a protocol name can never collide in the first place: §3 spells members `PascalCase` and functions `snake_case`.
3. **The first parameter is the type, by value:** `fn sensor_hash(v: Sensor)`, never `*Sensor` or `*const Sensor`. A pointer form would need an implicit address-of at `T.hash(key)`, a sixth implicit operation §6's closed list does not have; and by value costs nothing, since §5 already passes an aggregate larger than two machine words by hidden reference. A declaration with the required protocol name but an incompatible signature is a hard error at the instantiation site; it never falls back, and the diagnostic prints the required and found signatures. `next` is the one exception, because its subject is the **iterator** and not an element type: it takes `*I`, which is why §6 requires a `for`'s subject to be a `var` or a `*I`.
4. **A supplied fallback stands wherever the type's own module declares none.** The compiler supplies `hash`, `eq` and `format` for the integers, floats, `bool`, `err`, pointers, slices, arrays, `Vec/Mask`, and every `enum` and `union enum`; it supplies `cmp` for the same set except pointers. For a type of one of those shapes the supplied one is used whenever its declaring module declares no `fn` of that name, and whenever it has no declaring module at all — a structural alias such as `str` has none (§4, D48) — while a `fn` declared in the type's own module always wins over it. A shape outside that list, a `struct` being the one that matters, has no fallback, and a missing `fn` there is rule 5's error. A slice's `eq` and `hash` are over its contents, which is what §6 refuses to give `==`; what the supplied `format` writes for each shape is §4's Formatting subsection, and this rule is the authority on which types have one. Integer, boolean, error and enum

fallbacks use ordinary operators and hash canonical little-endian value bytes with xxHash64 seed 0. Pointer equality and hashing use the address through the same hash; `cmp` is not supplied for pointers. Container float equality treats all NaNs equal and both zeros equal, hashing canonicalizes every NaN to one quiet NaN and `-0` to `+0`, and float `cmp` is IEEE `totalOrder` after that canonicalization. Arrays, slices, vectors and tagged unions recurse in index or declaration order, including the tag before the live payload. Every lookup that lands on a fallback is recorded as a lookup edge, so declaring the `fn` later is not silently ignored (§12, D36).

5. **A missing function is an error at the instantiation site**, not inside the generic body, and it names what is missing and where it goes: instantiating `map.get[Vec3, f32]` needs `fn vec3_hash(v: Vec3) -> u64` in module `shapes` (`src/shapes.e`); none is declared.

The names the standard library looks for, by convention rather than by rule:

Spelled, for a type <code>Sensor</code>	Looked for by
<code>fn sensor_hash(v: Sensor) -> u64</code>	<code>e.data.map</code> , hash-based containers
<code>fn sensor_eq(a: Sensor, b: Sensor) -> bool</code>	<code>e.data.map</code> , containers, <code>e.test</code>
<code>fn sensor_cmp(a: Sensor, b: Sensor) -> i32</code>	<code>e.data.sort</code> , ordered containers
<code>fn sensor_format(v: Sensor, b: *str.Builder) -> err</code>	<code>printf/format</code> on a user type (§4), <code>e.log</code>
<code>fn window_next(it: *Window) -> (T, bool)</code>	<code>for</code> (§6), <code>e.data.iter</code>
<code>fn window_next_err(it: *Window) -> (T, bool, err)</code>	fallible iterator algorithms; never implicit language <code>for</code>

This is not an interface system. Nothing is declared, nothing is implemented, no type is a subtype of anything, and no value carries a tag. It is name lookup that happens at compile time, and it costs nothing: the call is direct, monomorphised, inlinable, with no vtable and no dispatch. `grep -n "^fn sensor_hash"` returns the one line that will run, from anywhere in the tree.

`next` is deliberately infallible because a language `for` has no visible propagation site. I/O and parser iterators expose `next_err`; a generic library calls `I.next_err(it)`, and ordinary code calls the declaring module and destructures or uses statement-level `try`. The `bool` says whether a value was produced; on non-ok error it is false and the value is its zero value. End of stream is (`zero`, `false`, `ok`), so end and failure cannot be confused.

Compile-time introspection

`e.meta` enumerates a type's structure while compiling, and its surface splits in two. `fields`, `members` and `type_name` are **comptime-only**: they return comptime values, and calling one outside a comptime context (Compile-time evaluation, above) is a compile error. `get` and `set` are **comptime-parameterised**, not comptime-only: only their `FIELD` argument must be comptime, each compiles to an ordinary field load or store at the offset `FIELD` names, and each is therefore legal in any body, at runtime, exactly as `v.x` is.

```
type Field = struct {
    name: str,
    ty:   type,
    offset: usize,
    size:  usize,
}

type Member = struct {
    name: str,
    value: u64,
}

type TypeKind = enum u8 {
    Int, Float, Bool, Err, Pointer, Slice, Array, Struct, Enum, UnionEnum, Vec,
}

fn fields[T: type]() -> []const Field // struct, or a union enum's payloads
fn members[E: type]() -> []const Member // enum, or a union enum's tags
fn type_name[T: type]() -> str // "shapes.Vec3"
fn kind[T: type]() -> TypeKind
fn element_type[T: type]() -> type // array, slice or Vec; otherwise a compile error
fn array_len[T: type]() -> usize // array or Vec; otherwise a compile error
fn backing_type[E: type]() -> type // enum or union-enum tag; otherwise a compile error
fn get[FIELD: Field, T: type](v: *const T) -> FIELD.ty
fn set[FIELD: Field, T: type](v: *T, x: FIELD.ty)
```

`Field` and `Member` are **comptime-only types**, in the sense §4 gives register-only `Mask[T, N]`: each is legal as a comptime value and nowhere else — a comptime parameter, a comptime binding, an element of a comptime slice — and is never a struct field, an array element, a runtime local or the pointee of a pointer. `mem.size_of` and `mem.align_of` on either is a compile error, because a type-valued field has no layout to name; neither has a zero value (§7); and neither crosses `extern` (§5) or reaches device code (§10).

`Member.value` is a `u64` holding the member's backing value **read as two's complement**, so a member of an `enum i64` with a negative value and a member of an `enum u64` above 2^{63} both fit one field; cast it back through the enum's own backing type, obtained with `meta.backing_type[E]()`, where the sign matters.

`kind`, `element_type`, `array_len` and `backing_type` are comptime-only with the same rules as `fields`. They provide enough shape information for a format module to decode primitives, arrays, vectors, enums and structs recursively. Tagged unions are not generically constructible in v1 because associating a runtime tag with a different payload type requires dynamic type selection; a format decoder requires a type-owned `fn <t>_decode_<format>(...) -> (T, err)` protocol for that shape. Bare unions, pointers and function pointers likewise require a type-owned decoder or are rejected by the format module. The language does not pretend that output-only `format` is a decoder.

`fields` on a bare union is a compile error: its members overlap by design (§4) and there is no fact about which one is live. `meta.get[FIELD, T]` and `meta.set[FIELD, T]` require that `FIELD` be an element of `meta.fields[T]()`; a `Field` of any other type is a compile error naming both types, so a field of one struct is never read at its offset in another.

For a tagged union, `fields` returns one entry for each payload-bearing variant in declaration order; payloadless variants have no `Field`. Its `name` is the variant name, `ty` the payload type, and `offset/size` the selected target layout. `get` and `set` additionally require the value's live tag to match that field: the ordinary tag check of §11 applies. `members` returns every tag, including payloadless ones, in declaration order. Reflection therefore exposes layout but does not select a runtime payload type; generic code switches on the tag or uses a type-owned decoder.

A for whose subject is a comptime value is always unrolled. The interpreter unrolls it, and the loop variable is a distinct comptime value in each copy — which is what lets the body use it in a comptime position: a `[...]` argument, a type, a protocol receiver. There is no condition on the body, because the alternative is a `[]const Field` surviving to runtime, and that is the type table D53 forbids. The §9 step budget bounds the unrolling. Nothing survives to runtime but the bodies.

Together with protocols that is enough to write one function that serialises every struct in the program:

```
fn format_struct[T: type](v: *const T, b: *str.Builder) -> err {
  try str.push(b, meta.type_name[T]())
  try str.push(b, "{ ")
  for f in meta.fields[T]() {
    try str.push(b, f.name)
    try str.push(b, ": ")
    try f.ty.format(meta.get[f, T](v), b)
    try str.push(b, " ")
  }
  try str.push(b, "}")
  ret ok
}
```

Every rule that body needs is one of the above. `meta.type_name[T]()` and `meta.fields[T]()` are comptime-only calls with comptime-known arguments standing in an ordinary body — the fourth comptime context, folded at the instantiation. `f` is a **comptime binding from an unrolled** `for`, spelled `snake_case` like any other local binding (§3). `f.ty` is a comptime expression of type `type`, so it stands where a type stands and where a protocol receiver stands (above). `meta.get[f, T](v)` passes a comptime-only struct value as a comptime argument and compiles to one field load at `f.offset`, which is why it may run at runtime while `meta.fields[T]()` may not. And `f.ty.format(...)` is a protocol call whose receiver is a comptime type value, which is exactly what rule 1 admits and what §14 invariant 4 excepts.

`fmt.json`, `e.cli` filling a config struct from `argv`, `e.log` writing structured fields and an `x.<owner>.db.*` driver mapping a row are the same shape. Without this each of them needs hand-written marshalling per type, which is the boilerplate generated code gets wrong most often.

The **enumeration** is compile-time only, and that is the whole point: there is no type table in the binary, dead-code elimination still removes what nothing calls, and the loop above is gone by codegen — what the emitter sees is the straight-line pushes and field loads you would have written by hand. Runtime type inspection remains a non-goal (§1).

Argument packs

A trailing parameter declared `...` accepts a comptime-known list of arguments. The function is monomorphised per distinct argument shape, so there is no `va_list`, no runtime type information, and no boxing — the pack is gone by codegen.

```
fn printf[FMT: str](args: ...) -> err // e.io
fn format[FMT: str](a: *mem.Arena, args: ...) -> (str, err) // e.str
fn launch[K: fn](q: *gpu.Queue, g: gpu.Grid, args: ...) -> err // e.gpu
```

In **v1** a `...` parameter appears only in these three functions, and all three are compiler intrinsics (§14). No user code and no other core function can declare one: `...` in any other signature is a compile error (the C variadic on an `extern`, §5, is a different thing with a different rule). There is no pack API — no `args.len`, no iteration, no per-element type query — because a pack exists only for the compiler to expand: `printf` and `format` expand against the format string into straight-line pushes, and into a call to a type's own `format` where the argument has one (§4, Protocols above); `launch` checks the pack against the parameter list of the `@gpu` function `k` names (§10). Packs are comptime-only: there are no runtime variadics in neper, and a pack cannot be stored, forwarded at runtime, or inspected.

This is a language exception, and it is recorded as one (D19): three intrinsics behind a token, not a general mechanism the library could have written. It is the cheaper of the two honest choices — a comptime pack API with length, indexing and type dispatch is an interpreter feature with rules of its own — and the three cover what formatting and kernel launch need. Opening `...` to user code is deferred, not undecided: it needs that API, and nothing in `lib/e` or the self-hosted compiler needs it.

Together, comptime parameters, evaluation and these three intrinsic argument packs cover the use cases that this specification accepts in place of conventional generics, macros and `#define` constants. Argument packs themselves do not replace generics, and **v1** exposes no additional metaprogramming mechanism.

10. The GPU profile

A function marked `@gpu` is compiled for GPU targets **and** for the CPU. The CPU build is not a fallback for production — it is the debugger. Set a breakpoint, step a single work item, inspect memory, then run the same kernel on the device.

```
use e.gpu

@gpu(256) // workgroup size: 256 invocations along x
fn saxpy(n: u32, a: f32, x: []const f32, y: []f32) {
  let i = gpu.gid.x // u32
  if i >= n { ret } // the grid is rounded up to whole workgroups
  let k = usize(i) // indices are usize (§3)
  y[k] = a*x[k] + y[k] // a multiply, then an add; never fused (§11)
}
```

Platforms

Two GPU backends, both compute-only. There is no graphics pipeline — no vertex or fragment stages — and none is planned.

Backend	Runtime	Reaches
SPIR-V	Vulkan compute	AMD, NVIDIA, Intel, Qualcomm, ARM Mali — on Windows and Linux; on macOS through the supported second-tier MoltenVK path below
PTX	CUDA driver	NVIDIA only, with access to CUDA-side interop

The floors: **Vulkan 1.2** with the `bufferDeviceAddress` and `scalarBlockLayout` features, which is what gives a device slice a real address (Device types and slices, below); **PTX ISA 6.0** on `sm_50`, with `--cpu sm_50 | sm_70 | sm_80 | sm_90` (§13) selecting a level above it for a `ptx` module — standalone, or embedded by a host build, since `--cpu` is given once per target family (§13) — `sm_50` the default. Everything optional beyond a floor is a capability (Capabilities, below).

Apple GPUs. Apple ships no Vulkan. neper reaches Apple GPUs through **MoltenVK** — Vulkan translated onto Metal — and that path is *supported but second-tier*: it is exercised in CI, bugs in it are real bugs, but performance and feature parity with native backends are not promised, and Metal-only capabilities are not exposed. A native **Metal backend** (Metal Shading Language or AIR, plus a Metal runtime) is scheduled as **M5**, after the core CPU and GPU targets land. It

is sequenced there and not earlier because a GPU backend is the most expensive kind of target to add — an emitter *and* a runtime — and Apple is one vendor.

This is a decision, not an omission. Until M5, "runs on a Mac GPU" means "runs through MoltenVK".

Capabilities are per device, and they are checked at launch. `f64` and `f16` arithmetic, subgroup operations and 64-bit atomics are optional on real hardware — SPIR-V gates them behind capabilities, and consumer GPUs frequently lack `Float64` or run it at a fraction of `f32` speed. A kernel that needs a capability the device lacks fails at `gpu.launch` with `gpu.Unsupported`, naming the kernel and the capability. It cannot be a compile-time error, because the compiler does not know which device the binary will meet.

Not targeted, and not planned: DirectX/DXIL, WebGPU, ROCm/HIP interop (AMD is served through Vulkan), and OpenCL.

What runs where

One source tree, one program, one binary: the GPU module is embedded as data in the executable and loaded by the CPU side at runtime. Which GPU targets a build embeds is the `--gpu` option (§13, GPU targets in a build). Partitioning is explicit — neper never decides for you what runs where.

Code	Compiled for
Plain fn, reached only from CPU code	CPU
@gpu fn	GPU and CPU
Plain fn reached from a kernel	Both, by inference

The third row is what keeps `@gpu` from spreading. A helper like `dot(a, b)` needs no annotation; if a kernel calls it, it is compiled into the kernel-owning module's GPU output as well (§12) and must satisfy the restrictions below. `@gpu` marks kernel *entry points* — the functions `gpu.launch` can name — not everything the device executes. A helper that uses any `gpu.*` builtin — `gpu.gid`, `gpu.sid`, a barrier, a `gpu.atomic_*` or subgroup builtin — or names a shared type (Address spaces, below) is **device-only**: callable from a kernel or another device-only helper, and a compile error to call from plain CPU code, where those builtins have no value (D24).

Restrictions inside @gpu

Banned	Reason
Recursion	No call stack of unbounded depth
Function pointers	No indirect calls
Module-scope var	No mutable global state
try lexically in the @gpu entry function	A kernel entry point returns nothing, so there is nothing to propagate to; <code>err</code> is an ordinary value in device code, and reached plain helpers may return <code>(T, err)</code> and use <code>try</code> among themselves (Device types and slices, below)
Arbitrary pointer casts (<code>mem.cast</code>)	Address spaces are distinct
extern calls (§5)	The device has no OS and no import table
<code>thread.*</code> , <code>atomic.*</code> (§8)	GPU parallelism is the launch grid; host atomics have no scope — device atomics are <code>gpu.atomic_*</code> (below)
<code>&x</code> , <code>x[lo..hi]</code> on a private variable — a <code>let</code> , <code>var</code> or by-value parameter holding an array, a struct or a scalar in the invocation's own storage	A function-storage pointer cannot be re-based or passed in SPIR-V (Device types and slices, below). A slice parameter or local points at buffer or shared memory, so <code>x[lo..hi]</code> and <code>&x[k]</code> on it are allowed; a <code>shared var</code> is workgroup storage, not a local, and may be sliced and addressed (Shared memory, below)
<code>= undef</code> (§5)	Nothing on the device fills it; a <code>shared var</code> is the one storage in device code with unspecified contents (below)
A bare union (§4)	SPIR-V's logical addressing cannot reinterpret a local's bytes; <code>mem.bitcast[T]</code> (§8) is the pun, and a variant type is a union enum, laid out with non-overlapping payloads in device code (Device types and slices, below)
<code>usize</code> , <code>isize</code> in a kernel parameter or a <code>Buf[T]</code> element	Their width differs between <code>spv</code> and the CPU (below); as locals they are legal

Banned	Reason
<code>gpu.barrier()</code> or a subgroup builtin in divergent control flow	Undefined on the device; a barrier trap in the CPU build (below)

Violations are reported at the `@gpu` function that introduces them, with the call chain that reaches the offending construct.

Kernels and workgroups

A kernel is a function carrying `@gpu(X)`, `@gpu(X, Y)` or `@gpu(X, Y, Z)`. The integer literals are its **workgroup size** — invocations per workgroup along each axis; an omitted axis is 1. Bare `@gpu` on a function is a compile error naming it: the workgroup size is what `gpu.lid`, a `shared var` and `gpu.barrier()` are defined against, so it is on the page, not in a default. It compiles to `LocalSize` in SPIR-V and to the block size at launch in PTX. Options may follow the integers: `caps(...)` and `ftz` (Capabilities, below).

Each workgroup dimension is a positive untyped integer literal representable in `u32`; their product is computed without wrapping and may not exceed 1,024, the language-wide compile-time ceiling. The device may impose a smaller per-axis or product limit, checked at launch as `Unsupported`. Each option may occur at most once; `caps` contains no duplicate member. An unknown option, zero dimension, fourth dimension or overflow is a compile error at the attribute.

```
@gpu(256)
fn saxpy(n: u32, a: f32, x: []const f32, y: []f32) { ... }

@gpu(16, 16)
fn transpose(n: u32, src: []const f32, dst: []f32) { ... }
```

Inside a kernel and every helper it reaches, these are values, not calls:

```
gpu.grid // global invocation id, .x .y .z, each u32
gpu.lid  // id within the workgroup: 0 <= lid.x < X, and so on
gpu.wgid // workgroup id
gpu.sid  // id within the subgroup (Subgroups, below)
```

`gpu.grid`, `gpu.lid` and `gpu.wgid` have the type `gpu.Id` — type `Id = struct { x: u32, y: u32, z: u32 }` (Host side, below) — so an id is passed to a helper, or held in a local, under that name; `gpu.sid` is a `u32`.

`gpu.launch[K](q, g, ...)` takes a `gpu.Grid`, an **invocation count** per axis — `gpu.grid1(n)` is `n` invocations along `x` — and converts it into a workgroup count by dividing by `K`'s workgroup size and rounding **up**. The invocations in the rounded-up remainder run; their `gid` lies at or beyond the grid, and the kernel guards against them, as `saxpy`'s `if i >= n { ret }` does. A workgroup count beyond the device's limit (Vulkan guarantees 65,535 per axis — 16.7M invocations along one axis at 256) fails at launch with `gpu.TooLarge`; a workgroup size the device cannot run (128 invocations are guaranteed, 256 is near-universal, 1,024 is every desktop part) fails with `gpu.Unsupported`, naming the kernel and the limit.

A zero invocation count on any axis makes the launch an ordered no-op. Grid values, rounding and products are checked with non-wrapping arithmetic before submission; an axis above `u32` range or a rounded workgroup count above the backend limit is `TooLarge`. Consequently every observable `gid`, `lid` and `wgid` fits its `u32` type.

A kernel returns `void`, takes no comptime parameters, and each of its parameters is a device storage type or a device slice (below). `gpu.launch[K]` **is its only caller, on both profiles**: a direct call to an `@gpu` function from CPU code, from another kernel or from a helper is a compile error naming it, because outside a launch `gpu.grid`, a `shared var` and `gpu.barrier()` have no meaning. On the CPU the launch runs the kernel's CPU build (CPU execution model, below).

Device types and slices

Device storage types. A type is a device storage type when it is a fixed-width integer (`i8` through `u64`), a float, an `enum`, `err` (a `u32` in every layout, §7), a `Vec[T, N]` of those (§4), an `Atomic[T]` with `T` a 32- or 64-bit integer, or a `struct`, `[N]T` or `union enum` composed of device storage types. A bare `union` is not a device storage type and is banned in device code altogether (Restrictions, above). Not device storage types: `usize` and `isize` — their width differs between `spv` and the CPU (below), so a `struct` holding one would have two layouts; they are legal as locals inside a kernel, where every index and `.len` is one, and nowhere in a buffer or an argument block. Not device storage types either: `bool` and `Mask[T, N]` — register types inside a kernel with no storage form in SPIR-V; a buffer holds `u8` or `u32` — and every pointer and slice, function pointers, `*void` and `type`. `gpu.Buf[T]` requires a device-storage `T` that is neither `union` form — a `union enum` element, at any depth, is a compile error naming the element type, because its device layout (below) is not its CPU layout, and a by-value kernel parameter, which crosses in the argument block with the same layout, is under the same

rule — so device memory holds plain data and never a pointer: nothing in a buffer means something on only one side of the boundary.

“Device-executable type” includes device storage types plus register-only `bool`, `Mask`, local `usize/ isize`, and device/shared pointers and slices in the positions defined below. “Kernel argument type” is narrower: a device storage type by value or a device slice produced from `Buf[T]`. These terms are used independently; being legal in a register never makes a value legal in a buffer or argument block.

`err` in device code is an ordinary `u32`-backed value. A helper reached from a kernel may return `(T, err)`, and helpers use `try` between themselves as on the CPU. Only a **kernel entry point** returns nothing, so `try` anywhere in a kernel body — which would escape the kernel — is the compile error the Restrictions table names; a kernel reports failure by writing an `err`, or a flag, into a `Buf`.

Layout in a buffer is §4’s — the same size, alignment and field offsets as on the CPU — expressed to SPIR-V as explicit offsets under **scalar block layout** (Vulkan 1.2 `scalarBlockLayout`), not `std430`, whose `vec3` padding and array-stride rules would give one struct two layouts. An upload is therefore a `memcpy`. A `union enum` in device code is the exception, and the reason it is kept out of a `Buf[T]`: its payloads are laid out **without overlap** — the tag, then each member’s payload at its own offset — because SPIR-V cannot alias two typed members over one storage. The program cannot observe the difference (§4), and a kernel local, a helper parameter or a return value of `union enum` type is legal.

Slices. The SPIR-V floor is **Vulkan 1.2 with** `bufferDeviceAddress`: every module uses the `PhysicalStorageBuffer64` addressing model, so a pointer into device memory is a real 64-bit address, as on the CPU and in PTX. A `[]T` in device code is §4’s `{ ptr, len }` pair — a 64-bit device address and a `usize` — and supports what it supports on the CPU: `x[k]`, `x[lo..hi]`, `&x[k]`, `x.len`, for `v in x`, passing to a helper. `gpu.launch` maps a `Buf[T]` onto a `[]T`, `[]const T` or `[]Atomic[T]` parameter by passing that pair in the kernel’s argument block. Under logical addressing none of this is expressible — a pointer cannot be re-based or stored — so the floor is what buys one slice type for both profiles. A device below it (Vulkan 1.1, or 1.2 without `bufferDeviceAddress` or `scalarBlockLayout`) is not supported: `gpu.open` returns `gpu.Unsupported`.

usize on the device is 32 bits on the `spv` target and 64 bits on `ptx`. On a CPU it has the target pointer width: 32 bits on `x86`, 64 bits on `x64` and `aarch64`. Vulkan’s buffer ranges and dispatch limits are 32-bit, and 64-bit integers are an optional capability (`.Int64`, below) that the index type must not depend on. It is the one type whose width differs between a kernel’s CPU build and its SPIR-V build — §9 already lists `usize` as target-dependent — which is why it is not a device storage type: it appears in device code only as a local and as the `len` of a slice, and `gpu.launch` writes each slice’s `len` at the device’s width when it builds the argument block, so the slice header is the CPU’s `{ address, len }` pair with the `len` field at 32 bits on `spv`. A `Buf[T]` of more than $2^{32} - 1$ elements cannot be given to a SPIR-V device: `gpu.alloc` and `gpu.upload` return `gpu.TooLarge` there.

Address spaces. Device code has three: *device* memory (buffers), *shared* memory (the workgroup’s, below) and *private* memory (an invocation’s locals). A slice or pointer type names its space. `[]T` and `*T` are device memory; `[]shared T` and `*shared T`, with the `const` forms `[]const shared T` and `*const shared T`, are shared memory. They are distinct types with no conversion between them, because in SPIR-V the storage class is part of the pointer type. A helper that serves both is written once as a generic over the slice type and monomorphised per space (§9):

```
fn sum[S: type](xs: S) -> f32 { ... } // S = []const f32 or []const shared f32
```

Private memory has no slice or pointer type at all: inside device code, `&x` and `x[lo..hi]` on a function-local (private) variable — a `let`, `var` or by-value parameter holding an array, a struct or a scalar in the invocation’s own storage, not a `shared var` — are compile errors, because a function-storage pointer cannot be re-based or passed in SPIR-V. Index the local in place, or pass the array by value. A slice parameter or local is not private storage — it points at buffer or shared memory — so sub-slicing it and `&x[k]` on it are allowed, as the paragraph above says. `shared` in a type is legal only in code compiled for the device — a kernel or a helper it reaches — and makes that helper device-only (What runs where, above). On the CPU build the qualifier is erased: a `[]shared f32` is an ordinary slice into the workgroup’s storage.

Shared memory

`shared var` declares **workgroup-shared storage**: one instance per workgroup, visible to every invocation of it, lowered to the `Workgroup` storage class in SPIR-V and `.shared` in PTX.

```
@gpu(256)
fn block_sum(xs: []const f32, out: []f32) {
  shared var tile: [256]f32
  // precondition at launch: xs.len is a multiple of 256
  tile[usize(gpu.lid.x)] = xs[usize(gpu.gid.x)]
```

```

    gpu.barrier()
    ...
}

```

- `shared` is a keyword (§3). `shared var name: T` is a **statement**, legal only directly in the body of an `@gpu` function — not in a nested block or loop, not in a helper, never at module scope. Its **storage** lives from workgroup start to workgroup end wherever the statement sits; its **name** is in scope from that line to the end of the kernel body, like any other local — a use above the line is a compile error. `T` is a device storage type; in practice a `[N]T` array, a struct of them, or an `Atomic[T]`.
- It takes **no initialiser** — the one local declaration in the language without one (a module-scope `var` has a defined zero value; this has none). It belongs to the workgroup, not to an invocation, so no single invocation could initialise it. Its contents are unspecified until written; the protocol is write, `gpu.barrier()`, read. A debug CPU build fills it with `0xCD` at workgroup start (§11), so a read before the barrier is recognisable.
- `tile[lo..hi]` is a `[]shared f32` and `&tile[i]` a `*shared f32`; that is how a helper receives it.
- The total per kernel is recorded in its Interface entry (§12) with the workgroup size and the capability set, and checked at launch against the device's limit (16 KB guaranteed by Vulkan, 48 KB on every NVIDIA part): `gpu.TooLarge`.

Barriers, memory ordering and atomics

Builtin	Semantics
<code>gpu.barrier()</code>	A control barrier at Workgroup scope with AcquireRelease semantics over workgroup memory : every invocation of the workgroup arrives before any leaves, and every shared write before it is visible to every invocation after it. It says nothing about device memory. <code>OpControlBarrier(Workgroup, Workgroup, AcquireRelease WorkgroupMemory); bar.sync 0</code> in PTX.
<code>gpu.memory_barrier(.Workgroup)</code>	A memory barrier alone, AcquireRelease over workgroup and device memory at workgroup scope: this invocation's earlier writes, to buffers and to shared, become visible to the workgroup. No control synchronisation. <code>OpMemoryBarrier; membar.cta</code> .
<code>gpu.memory_barrier(.Device)</code>	The same at Device scope: this invocation's buffer writes become visible to every invocation of the launch that acquires after it. <code>membar.gl</code> .

Uniformity. `gpu.barrier()` must be reached by every invocation of the workgroup — the same barrier, the same number of times. It is legal only in **uniform control flow**: not under a condition on `gid`, `lid`, `sid` or loaded data, and not after an early `ret` that some invocations took. A violation is undefined behaviour on the device — a hang, on most hardware, which does not check because it cannot cheaply — and a trap of kind `barrier` in the CPU build (CPU execution model, below), which does.

Device atomics. `Atomic[T]` (§8) is a device storage type for `T` in `i32`, `u32`, `i64`, `u64`, so a buffer element, a slice parameter and a `shared var` can be atomic: `Buf[Atomic[u32]]` maps onto `[]Atomic[u32]`, and `shared var count: Atomic[u32]` is legal. Atomic operations in device code are the `gpu.atomic_*` builtins — §8's operations plus a **scope**:

```

gpu.atomic_add(p, v, .Relaxed, .Device) // p: *Atomic[T] or *shared Atomic[T]
gpu.atomic_load(p, .Acquire, .Workgroup)
gpu.atomic_store(p, v, .Release, .Device)
gpu.atomic_cas(p, old, new, .AcqRel, .Acquire, .Device)
gpu.atomic_sub atomic_min atomic_max atomic_and atomic_or atomic_xor atomic_xchg // (p, v, order, scope)

```

`gpu.Scope` is `.Workgroup` — the invocations of this workgroup — or `.Device` — the whole launch, and the host after a `gpu.sync`. Orderings are §8's, with the C11 meaning. The scope is the argument with no CPU counterpart, which is why these carry the `gpu.` qualifier instead of giving `atomic.*` a second signature per name (§14 invariant 1); in the CPU build they lower to §8's atomics, whose scope is the system. `atomic.*` itself is banned in device code (Restrictions, above). 64-bit atomics need `.Atomic64` (Capabilities, below). CAS failure ordering follows §8 exactly: no release semantics and no ordering stronger than success.

An acquire load or acquire part of an atomic operation at scope `S` that observes a value written by a release store or release part at scope `S` synchronizes with it; all ordinary memory operations sequenced before the release happen before those sequenced after the acquire. `memory_barrier` orders only the calling invocation and must be paired with an atomic release/acquire or another specified synchronization; it does not by itself make another invocation observe a write. `gpu.barrier` supplies both the control rendezvous and matching release/acquire for shared memory. Races on non-atomic device or shared memory are undefined as on the CPU.

Subgroups

A subgroup is the hardware's SIMD group of invocations — a warp, a wave — of a device-dependent size between 4 and 64. It is never a compile-time constant and never something a kernel may depend on for correctness. The set is the minimum a reduction, a scan or a compaction needs, and nothing that names a lane layout:

```
gpu.subgroup_size() -> u32 // uniform across the launch
gpu.sid // this invocation's index in its subgroup, u32
gpu.subgroup_add[T](v: T) -> T // reduction over the subgroup; also _min _max, and _and _or _xor for integer T
gpu.subgroup_all(b: bool) -> bool // also _any
gpu.subgroup_ballot(b: bool) -> u64 // bit i set when lane i passed true; adds .Int64 to the inferred set
gpu.subgroup_broadcast[T](v: T, lane: u32) -> T // lane must be uniform across the subgroup
gpu.subgroup_shuffle[T](v: T, lane: u32) -> T // lane per invocation
gpu.subgroup_elect() -> bool // true in exactly one active invocation
```

T is a 32- or 64-bit integer or float — comptime-generic per D5, inferred from v per §9 — except that `_and`, `_or` and `_xor` take an integer T only. The lane of `subgroup_broadcast` must be **uniform** across the subgroup — one value in every invocation, as SPIR-V's `OpGroupNonUniformBroadcast` requires — where `shuffle`'s is per invocation; a non-uniform lane is undefined on the device and a barrier trap in the CPU build. Every subgroup builtin needs uniform control flow within the subgroup, the rule barriers have at workgroup level, with the same consequence: undefined on the device, a barrier trap in the CPU build. The float `subgroup_add`, `subgroup_min` and `subgroup_max` reductions run in hardware order and are approximate builtins (§11, Floating point). They require `.Subgroup` (below): `GroupNonUniform` with `Arithmetic`, `Ballot` and `Shuffle` in SPIR-V; `shfl.sync` and `vote.sync` in PTX (`redux.sync` from `sm_80`).

A broadcast or shuffle lane must be less than `subgroup_size()` and name an active invocation. An invalid or inactive source lane is a bounds check: it traps in the CPU debug build and is undefined in release and on the device. Algorithms that operate on a partially active subgroup use `subgroup_ballot` to select an active source.

Capabilities

The compiler **infers each kernel's capability set** from the types and builtins the kernel and every helper it reaches use, records it in the kernel's Interface entry of the `.spv.em` or `.ptx.em` (§12), and `gpu.launch` compares it against the device (D26): a missing capability fails with `gpu.Unsupported`, naming the kernel and the capability. `gpu.has(dev, cap)` lets a program choose a kernel before launching one.

Used in a kernel	gpu.Cap	SPIR-V capability	Vulkan feature
i32 u32 f32, err, usize as a local (32-bit), enum up to 32 bits, bool in registers	—	Shader	none — the floor
Mask[T, N] (§4)	that of T	OpTypeVector of OpTypeBool, split as Vec is; registers only, never in a buffer	—
any []T, Buf[T], pointer into device memory	—	PhysicalStorageBufferAddresses	bufferDeviceAddress, scalarBlockLayout — the floor
i8 u8, enum u8 — any value in device code, in a register as in a buffer or in shared; <code>time.to_date</code> and <code>time.to_time</code> (§16), whose results hold u8 fields	.Int8	Int8, StorageBuffer8BitAccess	shaderInt8, storageBuffer8BitAccess
i16 u16, enum u16 — any value in device code, as above; <code>bf16</code> in storage	.Int16	Int16, StorageBuffer16BitAccess	shaderInt16, storageBuffer16BitAccess
f16 arithmetic	.Float16	Float16, plus .Int16 for storage	shaderFloat16
bf16 arithmetic	—	none: computed in f32 and rounded (§4); stored as 16 bits under .Int16	—
i64 u64; <code>time.to_date</code> and <code>time.to_time</code> (§16)	.Int64	Int64	shaderInt64
f64	.Float64	Float64	shaderFloat64
<code>gpu.atomic_*</code> on <code>Atomic[i64]</code> , <code>Atomic[u64]</code>	.Atomic64	Int64Atomics	shaderBufferInt64Atomics, shaderSharedInt64Atomics
<code>gpu.subgroup_*</code> , <code>gpu.sid</code>	.Subgroup	GroupNonUniform, GroupNonUniformArithmetic, GroupNonUniformBallot, GroupNonUniformShuffle	subgroupSupportedOperations includes those; subgroupSize at most 64

Used in a kernel	gpu.Cap	SPIR-V capability	Vulkan feature
gpu.subgroup_ballot	.Subgroup and .Int64	its u64 result	as the two rows above
@gpu(..., ftz)	.Ftz	DenormFlushToZero execution mode	shaderDenormFlushToZeroFloat32
Vec[T, N]	that of T	OpTypeVector of 2 to 4 lanes; N > 4 is split into N/4 four-lane vectors — Vec[f32, 8] is two vec4, Vec[u8, 16] four — the counterpart of §4's split on a narrow CPU; the buffer layout stays §4's	—

Reported, never required — the device either does it or not, and `gpu.has` says which: `.DenormPreserve`, the device keeps f32 denormals (`shaderDenormPreserveFloat32`; every desktop part), which §11's Floating point rule depends on.

On PTX the floor is `sm_50`, where every row above is available except `f16` arithmetic (`sm_53`) and native `bf16` (`sm_80`; computed in `f32` below it, as on every other target). The same `gpu.Cap` values report them.

`caps(...)` in the attribute is an upper bound, checked at compile time. `@gpu(256, caps(.Int64))` declares that the kernel needs `.Int64` and nothing else; a use that would infer a capability outside the list — a helper three calls down that touches an `f64` — is a compile error at the kernel with the call chain. It exists so that a kernel written for a mobile device cannot silently grow a desktop-only requirement. The inferred set is still what launch checks. `ftz` is the other option, defined under §11's Floating point. Omitting `caps` sets no upper bound. A listed capability need not actually be used; duplicates are errors, and `ftz` both enables the execution mode and infers `.Ftz`. Inference examines all statically reachable instructions, types, storage declarations and signatures after `when` selection and generic instantiation, whether or not a runtime branch is expected to execute.

Host side

Device memory is a **distinct type**, not a host slice. `gpu.Buf[T]` is a handle to device memory; a host pointer is meaningless on the device and the reverse, so the type system keeps them apart rather than trusting a convention.

```

type Backend = enum u8 { Cpu, Vulkan, Cuda }
type DeviceKind = enum u8 { Unknown, Cpu, Integrated, Discrete, Virtual, Other }
type DeviceKey = struct { backend: Backend, uuid: [16]u8 }
type DeviceInfo = struct {
    key: DeviceKey,
    key_valid: bool,
    index: u32,
    name: str,
    kind: DeviceKind,
    memory_bytes: u64,
    memory_known: bool,
    capabilities: []const Cap,
    supported: bool,
}
type Device = struct { state: *void } // usable from any thread
type Queue = struct { state: *void } // one thread at a time
type Buf[T: type] = struct { owner: u32, slot: u32, generation: u32, len: usize }
type Grid = struct { x: usize, y: usize, z: usize } // invocation counts
type Id = struct { x: u32, y: u32, z: u32 } // gpu.gid, gpu.lid, gpu.wgid (Kernels and workgroups, above)
type Cap = enum u8 { Int8, Int16, Int64, Float16, Float64, Atomic64, Subgroup, Ftz, DenormPreserve }
type Scope = enum u8 { Workgroup, Device }

error NoDevice // no currently visible device matches the requested index or key
error AmbiguousDevice // several visible devices match one key; never choose arbitrarily
error Unsupported // below the floor, or a capability, workgroup size or shared size the device lacks
error OutOfMemory // device memory, or the driver's; on .Cpu the open arena's mem.Exhausted instead (open, below)
error TooLarge // a length or grid beyond a device limit; a download destination too small; a write past the end o
f its buffer
error Lost // the device or driver failed; every later call on it that returns an err returns Lost
error WrongDevice // a queue and buffer/device do not share an owner
error InvalidHandle // released, closed, joined or otherwise consumed handle

```

`Device.state` and `Queue.state` point into the bookkeeping block allocated by `open`; callers never dereference or replace them. A `Buf` is a value handle: `owner` identifies the device, `slot` indexes its handle table, `generation` rejects stale copies,

and `len` is the element count. These fields are fixed because the language has no private-field mechanism; their representation is public, while their contents remain module-owned invariants.

Function	Semantics
<code>fn devices(a: *mem.Arena, b: Backend, limit: usize) -> ([]const DeviceInfo, err)</code>	Enumerates a bounded snapshot of visible devices of <code>b</code> , including devices below Neper's floor with supported <code>== false</code> . Records, names and capability slices are allocated from <code>a</code> . No visible devices returns an empty slice and <code>ok</code> ; a backend not embedded in the build returns <code>Unsupported</code> . More than <code>limit</code> records is <code>TooLarge</code> , not a silently truncated success. See Device discovery and selection below.
<code>fn open(a: *mem.Arena, b: Backend, index: u32) -> (*Device, err)</code>	Opens device <code>index</code> of backend <code>b</code> . <code>open</code> takes one bookkeeping block from <code>a</code> — for the device, its queues and its buffer handles — and the <code>Device</code> guards that block with its own lock, so it is the one piece of arena memory in <code>lib/e</code> that several threads touch, and a stays the caller's. On a driver backend (<code>.Vulkan</code> , <code>.Cuda</code>) that block is all <code>a</code> is used for: <code>Buf[T]</code> storage and staging are device and driver memory, and <code>OutOfMemory</code> is their failure. <code>.Cpu</code> always exists at index <code>0</code> and runs the CPU build (below); it has no driver and no staging, so it allocates every <code>Buf[T]</code> from <code>a</code> — once, at <code>alloc</code> or <code>upload</code> ; the only memory source it has, and named on the page as D3 requires — and <code>upload</code> and <code>write</code> memcopy straight into that storage. Exhaustion surfaces as <code>a's err</code> , <code>mem.Exhausted</code> , from <code>alloc</code> or <code>upload</code> . A backend the build did not embed — one absent from <code>--gpu</code> (§13) — returns <code>Unsupported</code> ; <code>.Cpu</code> needs no device module and is never absent.
<code>fn open_id(a: *mem.Arena, key: DeviceKey) -> (*Device, err)</code>	Opens the exact currently visible backend-scoped key, revalidating identity while opening. No match is <code>NoDevice</code> ; multiple matches are <code>AmbiguousDevice</code> ; an unavailable compiled backend or a matching device below the floor is <code>Unsupported</code> . Never substitutes an index, name, other backend or CPU. Allocation, cleanup and ownership rules are those of <code>open</code> .
<code>fn info(a: *mem.Arena, dev: *Device) -> (DeviceInfo, err)</code>	Copies the selected device's opening-time descriptor into <code>a</code> , including owned copies of its name and capability slice. No device-memory allocation or queue synchronization. A closed/stale device is <code>InvalidHandle</code> ; a lost device is <code>Lost</code> . The captured index is informational, not a current locator.
<code>fn close(dev: *Device) -> err</code>	Atomically begins closing, rejects new operations, waits for every queue, releases every buffer and queue, then the device. A repeated close is <code>InvalidHandle</code> ; a driver failure is <code>Lost</code> .
<code>fn has(dev: *Device, c: Cap) -> bool</code>	Capability query; returns <code>false</code> after closing begins.
<code>fn queue(dev: *Device) -> (*Queue, err)</code>	A new in-order stream: a hardware queue where the device has a spare one, otherwise a separate command stream on a shared one. The ordering guarantees are the same either way.
<code>fn alloc[T: type](q: *Queue, n: usize) -> (Buf[T], err)</code>	<code>n</code> elements of device memory, contents unspecified.
<code>fn upload[T: type](q: *Queue, src: []const T) -> (Buf[T], err)</code>	Atomically <code>alloc</code> then <code>write</code> ; if staging or submission fails, it reclaims the allocation before returning the error and no handle escapes.
<code>fn len[T: type](b: Buf[T]) -> usize</code>	The element count <code>alloc</code> or <code>upload</code> gave it; use through a released or stale handle is an <code>invalid</code> check.
<code>fn write[T: type](q: *Queue, dst: Buf[T], off: usize, src: []const T) -> err</code>	A synchronous copy of <code>src</code> into driver-owned staging memory, then an in-order transfer to <code>dst[off..]</code> enqueued on <code>q</code> . Returns when <code>src</code> has been read. It checks <code>off <= len</code> and <code>src.len <= len-off</code> , without overflowing; failure is <code>TooLarge</code> .
<code>fn launch[K: fn](q: *Queue, g: Grid, args: ...) -> err</code>	Enqueues kernel <code>K</code> over <code>g</code> with <code>args</code> ; returns once queued.
<code>fn download[T: type](q: *Queue, src: Buf[T], dst: []T) -> err</code>	Validates ownership, liveness and destination length before waiting; then waits for every earlier submission on <code>q</code> and copies <code>gpu.len(src)</code> elements into <code>dst</code> . A small destination is <code>TooLarge</code> without blocking.
<code>fn sync(q: *Queue) -> err</code>	Blocks until every submission on <code>q</code> has completed.

Function	Semantics
<code>fn release[T: type](q: *Queue, b: Buf[T]) -> err</code>	Consumes the logical handle and enqueues release behind earlier submissions. A stale copy is <code>InvalidHandle</code> , a queue from another device is <code>WrongDevice</code> , and device loss is <code>Lost</code> .
<code>fn grid1(x: usize) -> Grid, fn grid2(x: usize, y: usize) -> Grid, fn grid3(x: usize, y: usize, z: usize) -> Grid</code>	Invocation counts; an omitted axis is 1.

The kernel is a comptime parameter. `gpu.launch[saxpy](q, g, ...)` names the kernel in the bracket, as `printf[FMT]` names its format string and `thread.spawn[Ctx]` its context type, and that is what makes the compile-time check of the trailing pack possible: the compiler can see `saxpy`'s parameter list. `κ: fn` declares a comptime parameter of kind `fn`: in a comptime position a function name is a value of kind `fn`, the way a type name is a value of kind `type` (§9). It is bound at compile time and monomorphised per kernel; it is not a function pointer, so host code never forms a pointer to device code. `launch` additionally requires that `κ` names an `@gpu` function; a plain function, a helper reached from a kernel, or a function pointer in that position is a compile error at the call.

Everything after the grid is the argument pack (§9), matched positionally against `κ`'s parameters at compile time:

Argument	Parameter	Rule
<code>Buf[T]</code>	<code>[]T</code> , <code>[]const T</code> , or <code>[]Atomic[T]</code> when <code>T</code> is <code>Atomic[T]</code>	The pair { device address, len } is passed — the one implicit operation of the launch, listed in §6
a value of a device storage type	the same type	By value, in the argument block; no conversion, as everywhere
an untyped literal	an integer or float device-storage parameter	Takes the parameter's type (§3); literals never become <code>bool</code> , <code>err</code> or an enum without its ordinary contextual member syntax
anything else — a host slice or pointer	—	Compile error naming the parameter

Arity, types and address spaces are all checked at the call site; nothing about a launch is discovered on the device.

Device discovery and selection

This is an extension of the planned M3 `e.gpu` surface, not an implemented feature or a new implicit dispatch rule. M2.5 freezes its identity, allocation, error and tooling contracts; M3 supplies CPU/Vulkan execution evidence and M4 adds CUDA.

Indices are temporary, keys are exact selectors. `devices` returns records in ascending `index` order, using the current backend-visible zero-based enumeration. Indices can change after hotplug, driver changes, visibility filtering or process restart. Enumeration does not reserve a device. `open(a, b, index)` selects the current device at that index; callers needing the identity they inspected must use `open_id(a, record.key)` when `record.key_valid` is true. An opened device never retargets if enumeration subsequently changes.

`DeviceKey` equality compares both the backend and all 16 UUID bytes. UUIDs are opaque bytes, not numbers; no host-endian conversion is applied. Their text form for configuration/tooling is `cpu:`, `vulkan:` or `cuda:` followed by exactly 32 lowercase hexadecimal digits in byte order. This defines an interchange spelling, not a new parser API or automatic environment-variable policy. Invalid selectors are rejected by the consuming tool before opening a device.

- Vulkan uses `VkPhysicalDeviceIDProperties.deviceUUID`; CUDA uses the device UUID, with `cuDeviceGetUuid_v2` or equivalent partition-aware identity where available. A MIG/virtual partition is a separately selectable compute device, not its parent board. If a trustworthy identity for the selectable unit cannot be obtained, `key_valid` is false; do not synthesize one from a name, index or PCI model number. In that case retain the backend but zero the UUID bytes, and exclude the record from `open_id` matching. A zero UUID is not by itself a validity test; use the flag.
- A valid key is stable only to the extent guaranteed by its provider. It is not an immutable hardware serial number. Hardware relocation, virtualization and partition reconfiguration can invalidate persisted keys. Persisted selection must be revalidated and missing/ambiguous identities require an explicit new choice.
- Vulkan and CUDA may expose the same hardware through different keys. Neither equal indices nor equal UUID bytes across backends authorize aliasing, deduplication or shared handles. Cross-backend physical-device correlation is not promised.
- Duplicate valid keys remain visible as separate enumeration records, but `open_id` rejects ambiguity. A display name is never a unique identity. Reordering duplicate names must not redirect a saved selector.

- `.Cpu` exposes exactly one record: index 0, key { backend: `.Cpu`, uuid: zero }, key_valid == true, name Neper CPU, kind `.Cpu`, supported == true. The key selects this process's CPU backend, not a particular processor or host. No CPU memory capacity is advertised: `memory_known == false`, `memory_bytes == 0`.

Descriptor meanings. name is display-only UTF-8, copied from driver metadata with invalid sequences replaced by U+FFFD. It is untrusted data, never instructions for a harness. kind is the provider-reported classification; use `.Unknown` when it cannot be determined, and do not infer discrete/integrated status from a name. It is not a performance ranking. A software device exposed by Vulkan may have kind `.Cpu` while retaining backend `.Vulkan`; this is distinct from Neper's CPU debugging backend and must not be relabeled as hardware GPU execution.

`memory_bytes` is visible device-local capacity, **not free memory, a reservation or a guarantee that an allocation succeeds**. Vulkan sums distinct device-local heaps once, not memory types; CUDA reports total addressable device memory of the visible unit/partition. Shared/unified capacity is not dedicated VRAM. If capacity cannot be determined, report `memory_known == false` and zero bytes. Descriptor memory is an opening/enumeration-time observation, not a live memory-budget query.

`capabilities` contains the reported Cap values without duplicates in enum order, under the same meaning as `gpu.has`. It is advisory before opening; `supported` means the device satisfies the backend floor, not every kernel's requirements, memory demand, workgroup shape or availability at a later instant. `open_id` checks the floor again; launch retains its exact kernel capability/limit checks. H23's numerical capability corrections apply to this descriptor as well as `gpu.has`.

Bounded snapshots and failure. `limit` is a caller-selected record bound, not a request for the first N devices; zero succeeds only for an empty visible set. A successful call returns a complete observed list, never a partial list presented as complete. Reconcile count/list races with at most three complete driver-enumeration attempts; an unstable inventory after that is `Lost`. Changes after a successful snapshot are allowed, so opening remains fallible. No call spins indefinitely for hotplug to settle. Backend absence is `Unsupported`; driver initialization/enumeration failure is `Lost`, distinct from successful discovery of zero devices. These are API-level results once program startup succeeds; this extension does not bypass §13's platform loader/linker requirements or promise recovery from startup failure.

On failure, `devices` returns (nil, error) and `info` returns a zero descriptor; caller-arena exhaustion is `mem.Exhausted`. Failed calls roll back their temporary caller-arena allocations; the caller must not allocate concurrently from that arena. Successful records/names/slices remain valid until that arena is reset or destroyed, independent of device closure. They hold no open-device handles or reservations. Internal discovery driver resources are released before return; no hidden persistent inventory allocation, logical-device creation or kernel launch is implied. Driver discovery/initialization may still perform driver-owned work.

`open_id` resolves one native device handle and verifies its identity through opening; removal or identity change fails rather than reopening a replacement at the old ordinal. This protects against ordinary reordering/hotplug, not a malicious driver forging hardware identity. Capability and descriptor collection for `info` belongs to the device bookkeeping allocation already charged to `open/open_id`. Failed opens return a nil device and release partial driver resources/bookkeeping; they do not transfer a half-open resource or reserve the failed ordinal for retry.

Multiple devices stay explicit. Several `Devices` may be open simultaneously; each queue and buffer belongs to one logical open-device identity. Even two opens of the same physical GPU do not share handles. Cross-device use is `WrongDevice` for fallible calls; non-fallible operations retain their documented check behavior. There is no automatic work splitting, migration, peer copy, shared allocation or cross-device completion-token interoperability. To move data, explicitly download through the source queue, then upload to the destination queue. Waiting on one device does not synchronize another. Failure/loss does not trigger transparent replay elsewhere; dependent application work must handle the failure explicitly.

Selection policy is application-owned: enumerate, filter by kind/capability/capacity, choose a candidate, and open its exact key. An explicit saved key takes precedence only when the application says so; Neper does not invent a fastest-device heuristic or read an ambient GPU-selection variable. CPU fallback remains explicit, and the CPU backend remains a debugger rather than an optimized production fallback.

Example selecting a supported discrete Vulkan device with `Float64` by its key; the first matching enumeration entry is this application's policy, not a promise that it is the fastest GPU. It does not fall back to a different device on an open failure and does not assert that this capability alone makes every kernel legal:

```
use e.gpu
use e.mem
```

```

fn open_discrete_f64(a: *mem.Arena) -> (*gpu.Device, err) {
  let candidates = try gpu.devices(a, .Vulkan, 64)
  for candidate in candidates {
    if !candidate.supported || !candidate.key_valid { continue }
    if candidate.kind != .Discrete { continue }
    for capability in candidate.capabilities {
      if capability == .Float64 {
        ret gpu.open_id(a, candidate.key)
      }
    }
  }
  ret (nil, gpu.NoDevice)
}

```

The descriptor storage in this example lives in `a`; repeated selection should use a separate short-lived discovery arena and copy the value-only `DeviceKey` into the opening call. Never reset an arena that also owns an open device's bookkeeping. Returning `NoDevice` for no policy match above is application policy; the raw `devices` API returns an empty successful list when nothing is visible.

Provider references: [Vulkan device identity](#) and [CUDA device discovery/UUIDs](#). Their provider limits constrain identity stability; Neper does not strengthen them into an unconditional cross-reboot, cross-backend or cross-machine identity promise.

Queues and synchronisation

`gpu.launch` **is asynchronous on Vulkan and CUDA**. It returns once the work is queued, so the CPU continues while the device runs. The `.Cpu` backend is the explicit synchronous exception: it executes on the calling thread and returns after the grid.

```

fn run(q: *gpu.Queue, x: []const f32, y: []f32, other: []f32) -> err {
  let dx = try gpu.upload[f32](q, x) // x has been copied out when this returns
  defer let _ = gpu.release(q, dx) // ordered behind everything queued below
  let dy = try gpu.upload[f32](q, y)
  defer let _ = gpu.release(q, dy)

  try gpu.launch[saxpy](q, gpu.grid1(x.len), u32(x.len), 2.0, dx, dy)

  cpu_side_work(other) // runs while the GPU is busy

  try gpu.sync(q) // wait for the queue to drain
  try gpu.download[f32](q, dy, y)
  ret ok
}

```

Call	Semantics
<code>gpu.launch</code>	Driver backends queue work and return immediately; <code>.Cpu</code> completes the grid before returning
<code>gpu.upload</code> , <code>gpu.write</code>	Copies the source into staging now , queues the transfer, returns; the source is free
<code>gpu.download</code>	Implicitly synchronises the queue, then copies back
<code>gpu.sync(q)</code>	Blocks until the queue drains
<code>gpu.release</code>	Queued behind prior submissions; the memory goes away when they have finished

A queue is an in-order stream. Everything submitted to one `Queue` — transfers, launches, releases — completes in submission order, and `gpu.sync(q)` and `gpu.download` wait for exactly that queue's earlier submissions and nothing else. `defer let _ = gpu.release(q, dx)` after a launch is therefore safe by construction: the release is queued behind the launch that reads `dx`. Two queues are unordered with respect to each other; whether they run concurrently is the device's business. A `Buf[T]` belongs to the device, not to a queue, and may be used from any queue of its device — with only what `gpu.sync` gives as ordering between them, so a buffer written on one queue and read on another without a `sync` between is a race, which is undefined behaviour under D14.

Every device, queue and buffer carries an opaque owner ID and generation. All host calls validate them before accessing driver state. A fallible call given a queue or buffer from another device returns `wrongDevice`; a released buffer, closed device, destroyed queue or stale copied generation returns `InvalidHandle`. Non-fallible `has` and `len` follow the special rules below. Handles are logically linear even though the language can copy their bits: `release` and `close` consume all copies. `NoDevice` is returned by `open` and `open_id`; `AmbiguousDevice` is returned by `open_id` for duplicate matching keys.

Unsupported covers a missing compiled backend, floor, capability or workgroup feature; TooLarge covers validated sizes and grids; OutOfMemory covers allocation and staging; a driver reset or irrecoverable submit, wait or transfer failure marks the device lost and returns `Lost` thereafter.

Upload's cost is on the page. `upload` and `write` are synchronous so that the double-buffer pattern — upload, refill the host buffer, upload again — is correct without a lifetime rule the compiler cannot check. The price is one host memcopy of `src.len * mem.size_of[T]()` bytes into driver-owned, host-visible staging memory, held until the queued transfer completes; on an integrated GPU, whose device memory is host-visible, that one copy is the whole transfer. On a driver backend that staging block is **allocated from the driver on every call, outside every arena**, and released when the transfer completes: it is the one allocation in `lib/e` that is not visible as an arena parameter (D3 records the exception), and `OutOfMemory` is its failure. On `.Cpu` there is no staging: `upload` and `write` memcopy straight into the `Buf`'s storage, allocated once from the arena passed to `gpu.open` (Host side, above), and `mem.Exhausted` is `alloc`'s and `upload`'s failure. A program that cannot afford the copy keeps its data on the device between launches, which is what `Buf[T]` is for.

Threads. A live `*gpu.Device` may be used from any thread concurrently — `queue` and `has` — because the runtime guards the bookkeeping block it took at `open` (Host side, above) with its own lock; nothing else in `lib/e` synchronises arena memory. Each `*gpu.Queue` is used from one thread at a time: two threads submitting to one queue without their own synchronisation is undefined behaviour under D14. The intended shape is one queue per CPU thread, each driving its own stream. `close` is an exclusive transition: it acquires the device lock, marks the device closing, waits for operations that entered before the mark, then drains and destroys the device. New fallible calls return `InvalidHandle`, `has` returns `false`, and `len` through a stale buffer is an `invalid` check. Concurrent `close` calls serialize and all but the first return `InvalidHandle`.

`download` syncing implicitly is the one convenience here, and it is worth it: a download that returned stale data because you forgot to sync is a silent wrong answer, which is the failure mode this language works hardest to avoid. Use `gpu.sync` explicitly when you want to wait without copying.

Nothing crosses the boundary implicitly. The device cannot allocate from a host arena, perform I/O, or call back into CPU code. There is no unified address space and no shared virtual memory in v1.

CPU execution model

The CPU build of a kernel is what `gpu.open(a, .Cpu, 0)` runs. It is the debugger and the test vehicle (§13), and its semantics are the device's — not a serial approximation in which every missing barrier and every shared-memory race, the dominant class of kernel bug, quietly disappears.

The workgroup is the unit. A `launch` on the CPU backend runs the grid on the calling thread, one workgroup after another in workgroup-id order, and returns when the last has finished; `upload`, `write` and `download` are memcopies and `sync` is a no-op. A workgroup never splits across threads; a later option may spread workgroups over several host threads, and nothing a kernel can observe distinguishes the two.

One host thread runs a whole workgroup by barrier loop-fission, the technique of `pocl` and Intel's CPU OpenCL. The compiler cuts the kernel body at every `gpu.barrier()` (and every subgroup builtin, which is a barrier for its subgroup) into regions, and emits the workgroup as a sequence of loops: region 0 for every invocation in `lid` order, then region 1 for every invocation, and so on. A local that lives across a barrier is given one slot per invocation, an array indexed by `lid` in the workgroup's frame; a `shared var` is one storage per workgroup, in that same frame, filled with `0xCD` in debug before region 0. This is exactly the ordering a barrier guarantees, so a kernel that is correct on the device is correct here, and a kernel that reads `shared` before the barrier that publishes it reads `0xCD` here rather than whatever another invocation happened to have written.

"Cuts at every barrier" is a control-flow transformation, not a textual split. The compiler builds a resumable state machine per invocation with a program-counter slot; it repeatedly runs every active invocation until its next barrier, verifies that all active invocations reached the same static barrier and occurrence count, performs the barrier, then resumes them. Loops may therefore execute a barrier repeatedly. If one invocation exits, reaches a different barrier or needs another iteration while its peers do not, the CPU build traps `barrier`; the same program is undefined on a device. Subgroups use the same machine over each active subgroup.

Stepping. A breakpoint in a kernel stops at one invocation's iteration of one region. `gpu.gid`, `gpu.lid`, `gpu.wgid` and `gpu.sid` are ordinary locals in that frame, so any debugger reading the §13 subset shows which invocation is stopped; the workgroup's `shared vars` are locals of the enclosing workgroup frame; a local that lives across a barrier appears as an array indexed by `lid` in a standard debugger, and `neper dap` (§13, M4) renders the current invocation's element. "Step a

single work item" means: step through this invocation's iteration, then the next invocation's, in `lid` order, with the workgroup's state in view throughout.

Divergence is a checked trap. A debug CPU build records, per invocation, the barrier it reached — by source position and by arrival count. When any invocation of a workgroup reaches a different barrier than the others, or returns before reaching one the others reached, the build traps with kind `barrier` (§11): `trap[barrier]: invocation (7,0,0) of workgroup (3,0,0) returned before the barrier at kern.e:41:5 that invocation (0,0,0) reached`. That is the bug the device would turn into a hang. The same check covers a subgroup builtin in divergent code.

What is identical, stated exactly. Integer and float results of a kernel on the CPU backend and on a device are bit-identical under §11's Floating point rule, with these exceptions and no others: the approximate builtins listed there; the width of `usize` (32 bits on `spv`, above), which matters only to a kernel that overflows it; denormal `f32` results on a device without `.DenormPreserve`; and the order in which atomics from different invocations are applied, which the device does not fix either. Debug checks (§11) fire in the CPU build and not on the device, which carries no checks — so a kernel that would trap on the CPU is a kernel whose device result is unspecified, and the CPU build is where it is found. Roadmap M3's criterion is written in these terms.

Subgroups on the CPU are 32 consecutive invocations in `lid` order, and `gpu.subgroup_size()` reports 32. A workgroup whose size is not a multiple of 32 has a partial last subgroup, exactly as on a device: `subgroup_size()` still reports 32, the absent lanes are inactive, and an inactive lane reads as `false` in `ballot`, `all` and `any` and contributes nothing to a reduction. A kernel that is correct only at that size is wrong on some device; the CPU build does not try to hide that.

The CPU backend is not a performance path. It exists so that a kernel can be stepped, checked and tested on the machine that compiled it, which is one of the better arguments for compiling `@gpu` functions both ways (§13).

11. Build modes and safety

Check	kind	debug	release
Slice/array bounds	bounds	trap	off
Null dereference	null	trap	off
Tagged-union payload access against the wrong tag (§4)	tag	trap	off
Integer overflow (+ - *, unary -)	overflow	trap	wrap
Integer division by zero (/ % on integers; float / is IEEE, below)	divide	trap	trap
Integer division overflow (MIN / -1, MIN % -1)	divide	trap	trap
Narrowing cast to a value not representable (§4)	narrow	trap	integer: truncate — the C result; float-to-integer: saturate, NaN to 0
Shift by a count >= the width (§6)	shift	trap	count masked to width - 1
Integer-to-enum cast naming no member (§4)	enum	trap	trap
Invalid value representation, arena/builder state or consumed debug-tracked handle where a section names an invalid check	invalid	trap	undefined behavior
<code>simd.load_aligned/store_aligned</code> at an address not aligned to the vector width (§4)	align	trap	off
<code>unreachable()</code> reached	unreachable	trap	trap
Divergent control flow at <code>gpu.barrier()</code> or a subgroup builtin, or a non-uniform <code>lane</code> argument to a subgroup operation — the CPU build of a kernel only (§10)	barrier	trap	off
Arena exhaustion (<code>mem.Exhausted</code> , §8)	—	err	err

Semantics are identical up to the point a check would fire, so a program that never trips a check behaves identically in both modes. The rows divide in two. The **arithmetic rows** — `overflow`, `divide`, `narrow`, `shift`, `enum` — have a defined release result on every target: `wrap`, `trap`, `truncate/saturate`, `mask` or `trap`, and nothing in those rows is undefined behaviour. Division by zero and `MIN / -1` trap in every mode because x64 raises `#DE` for them regardless, and on a target

that does not (aarch64 yields 0) the check is one compare — a release behaviour that varied by target would break the sentence above. The **memory/state rows** — `bounds`, `null`, `tag`, `align`, `barrier`, `invalid` — remove their check in release. The operation then does what the hardware does: an out-of-range write, a `nil` dereference, a wrong-member read or a misaligned vector access in a release build is **undefined behaviour**, as in C, and the debug build is where it is found. Arena exhaustion is an ordinary error in every mode, not a check — running out of memory is a normal condition.

`@nocheck { ... }` disables the debug-only rows for a block in debug builds, for the rare hot loop that needs it during development. It cannot disable the rows that trap in release.

Trap protocol

A check that fires **traps**, and a trap is one thing in every build mode: the runtime writes one record to `stderr`, then a symbolised backtrace, and exits with code **134** (`128 + SIGABRT`, what every crash pipeline already classifies as an abort). No unwinding runs; no defer runs.

When the compiler-generated test root installs a control handle, the runtime also writes the same trap as one authenticated framed JSON object to that handle before writing `stderr`. The parent creates a random 128-bit nonce through `os.random`; the startup stub receives it and the handle through reserved OS startup metadata that is removed before constructing `neper args`. A frame is that nonce, a `u32` little-endian byte length, then exactly that many UTF-8 JSON bytes. The parent rejects a wrong nonce, oversized length, invalid JSON or partial frame as a crashed test. Source code cannot read the installed nonce or handle through a language API, so neither user `stderr` nor a guessed raw handle can impersonate a control record.

```
src/lex.e:88:14: trap[bounds]: index 7 out of bounds for len 5
src/lex.e:88:14: trap[overflow]: i32 + overflows: 2147483647 + 1
src/lex.e:88:14: trap[narrow]: 300 does not fit u8
src/lex.e:88:14: trap[tag]: Node.BinOp read while tag is Lit
src/lex.e:88:14: trap[unreachable]: token kind not handled
  at lex.next_token (src/lex.e:88)
  at parse.parse_expr (src/parse.e:412)
  at main.main (src/main.e:19)
```

Each frame is `module.function` under §2's naming rule, so `src/main.e` is module `main` and its entry point is `main.main`.

The record is `file:line:col: trap[<kind>]: <values>` — the same shape as a compiler diagnostic (§13), with `trap[kind]` in the place of `error[code]`, so the tooling of §14 parses one format. `<kind>` is the column of the table above. The values are the operands the check saw: the index and the length, the operands and the operator, the value and the target type, the member read and the tag found. The backtrace comes from the unwind info and the `.nepersym` line-and-symbol section (`.nepsym` in COFF/PE, whose section-name field is limited to eight bytes) §13 (Debug information) emits in every build mode, under the own linker and `--linker=system` alike, which is why it costs nothing to promise it in release. Under `neper test` the same record is captured into the failing test's JSON object rather than the summary (§13).

`unreachable()` is the **one always-on builtin**: a call expression, legal in any function, retained in every build mode, whose only effect is a trap of kind `unreachable` with the optional `str` literal argument as its values — `unreachable("token kind not handled")`. The compiler treats the point after it as dead. It exists because a `switch default` that cannot happen, or a violated invariant, has to be expressible, and because release elides every other check, so a library could not build one from the constructs it has. Assertion helpers stay libraries (§13): `test.assert` returns `test.Failed` rather than trap, and `unreachable()` is for the path that cannot happen. Inside `@gpu` the CPU build carries the message; the device build lowers `unreachable()` to the backend's trap instruction.

Debug fills

Two fills exist in debug builds and neither exists in release. `mem.reset(a, m)` fills `[m, a.off)` with `0xDD` before lowering `off`; `mem.alloc` fills the memory it hands out with `0xCD`, the same byte = `undef` storage receives (§5). A slice that outlives the `mem.reset` it should not have survived therefore reads back as `0xDD 0xDD ...`, a read-before-write as `0xCD 0xCD ...`, and both are recognisable in a debugger and in a failing test rather than being whatever the previous user left. The cost is one `memset` per `reset` and per `alloc`; it buys detection of the one lifetime hazard the arena model creates (§8) with no per-object bookkeeping, which is why it fits under D3. These are fills, not checks: nothing traps, and `@nocheck` does not remove them.

Floating point

No implicit contraction, in any build mode, on any target. Every `f32` and `f64` operation on the page is one IEEE 754 operation, correctly rounded to its type: `a*x + y` is a multiply and then an add, two roundings, in a debug build, a release build, the CPU build of a kernel and the device build alike. The optimiser does not contract, reassociate, factor, drop a comparison a NaN could change, or assume the sign of a zero. There is no fast-math flag and there will not be one: a CPU and a GPU that agree bit for bit is what makes the CPU build a debugger (§10) and `neper test` a test of a kernel.

Every floating operation that produces NaN returns one canonical positive quiet NaN for its width (zero payload except the quiet bit); signaling NaN inputs are quieted to that value. Loads, stores and `mem.bitcast` preserve NaN bits until an arithmetic or math operation consumes them. This canonicalization, and signed-zero behavior, are part of code generation on every backend. Scalar comparisons are the ordered rules of §6; protocol container equality and total ordering are the distinct rules of §9.

Float division is under the same rule and is never a check: `x / 0.0` is $\pm\text{inf}$, `0.0 / 0.0` is NaN, in every build mode on every target, exactly as IEEE 754 says — §11's `divide` rows are integer rows.

`math.fma(a, b, c)` — with its lane-wise form `simd.fma` (§4) — is the **only fused multiply-add** — one rounding, on every target: `vfmadd` at `x64-v3`, `fmadd` on `aarch64`, the fused instruction on every GPU, and on `x64-v1/v2`, which have none, a correctly rounded software sequence, slow and exact. A generator that wants the FMA writes it, and the reviewer sees it. Vector arithmetic (§4) is under the same rule lane by lane, and `simd.reduce_add` sums in a fixed tree so that it, too, is bit-identical everywhere.

Target	How the rule is kept
CPU emitters	<code>A *</code> feeding <code>a +</code> never selects a fused instruction; <code>math.fma</code> selects one. <code>min/max</code> are IEEE 754-2019 minimum/maximum, NaN-propagating, emitted as compare-and-select where the hardware's instruction is not
SPIR-V	Every arithmetic result carries <code>NoContraction</code> ; no <code>FastMath</code> modes. <code>OpFDiv</code> and <code>sqr</code> t — which Vulkan bounds only to 2.5 ULP and to <code>inversesqr</code> t's precision — are emitted as correctly rounded sequences over <code>Fma</code> with <code>NoContraction</code> , for <code>f32</code> and <code>f64</code>
PTX	<code>add.rn</code> , <code>mul.rn</code> , <code>div.rn</code> , <code>sqr</code> t.rn, and <code>fma.rn</code> for <code>math.fma</code> alone — the <code>.rn</code> forms, which <code>ptxas</code> does not contract — never <code>.approx</code> or <code>.full</code>

For `math.min` and `math.max`, one NaN operand produces the canonical NaN rather than selecting the number; `min(-0, +0)` is `-0` and `max(-0, +0)` is `+0`, independent of operand order.

Denormals are preserved. The runtime never sets FTZ or DAZ in `MXCSR` or `FZ` in `FPCR`; a SPIR-V module carries no denormal execution mode and a PTX module no `.ftz`, so a device preserves `f32` denormals exactly when it does so by default — which `gpu.has(dev, .DenormPreserve)` (§10) reports, and every desktop part does. `@gpu(N, ftz)` is the opt-in for a kernel that would rather flush: it sets `DenormFlushToZero` on SPIR-V (the launch-checked capability `.Ftz`), `.ftz` on every PTX instruction, and FTZ and DAZ around the CPU build's run of that kernel, so the three still agree. `f16` and `bf16` follow §4 — computed in `f32` under this rule, then rounded — which is why their results are deterministic across targets (D25). `ftz` applies to `f32` operations and therefore to the widened computation of `f16` and `bf16`; it does not flush `f64`, whose denormals remain preserved.

Approximate builtins, the complete list: the transcendental family of `e.math` — `sin` `cos` `tan` `asin` `acos` `atan` `atan2` `exp` `exp2` `log` `log2` `log10` `pow` — and `math.rsqr`t, which use the hardware's approximation on a device and a software one on the CPU, each within a stated bound — on a device the Vulkan specification's precision table applies; on the CPU every `e.math` transcendental is within 2 ULP; `math.rsqr`t is within 2 ULP on both; the per-function bounds are recorded in `lib/e/math.e` when it is implemented — and not bit-identical to the other; and the float `gpu.subgroup_add`, `_min` and `_max` reductions, which apply in hardware order. Everything else in `e.math` — `sqr`t, `fma`, `abs`, `min`, `max`, `floor`, `ceil`, `round`, `trunc`, `copysign` — is exact and identical everywhere. A kernel that uses no approximate builtin is **bit-identical** between its CPU build and every device, subject only to the exceptions §10's CPU execution model lists, and roadmap M3 is done when that holds.

The special-value contract is fixed even for approximate functions. `sqr`t(`x`) is NaN for finite `x < 0` except that `sqr`t(`-0`) == `-0`; `log*` of a negative value is NaN and of either zero is `-inf`; `exp*` overflow is `+inf` and underflow rounds normally; `sin`, `cos` and `tan` of infinity are NaN; `asin` and `acos` outside `[-1, 1]` are NaN; `atan` accepts infinities; and `atan2` follows IEEE quadrants and signed zeros. `pow` follows IEEE 754 `pow`: an exponent of zero returns one, a negative finite base with a non-integer exponent is NaN, and signed zero and infinity follow exponent sign and odd-integer parity. ULP bounds apply only to finite, mathematically defined results; these NaN, infinity and signed-zero outcomes are exact requirements.

Every `e.math` function is **comptime-generic over the float type**, one per name (§14): `math.sqrt[T: type](x: T) -> T` for `T` in `f16`, `bf16`, `f32`, `f64`, inferred from its argument per §9, so `math.sqrt(dot(v, v))` on an `f32` is `sqrt[f32]`. `math.fma` is an intrinsic (§14) because the emitter must select the fused instruction for it and for nothing else.

The functions with more than one argument are `atan2(y, x)`, `pow(x, y)`, `fma(a, b, c)`, `min(a, b)`, `max(a, b)` and `copysign(x, y)`, every parameter and the result `T`. `round` rounds **ties to even** — `RoundEven` on SPIR-V, the round-to-nearest-even instruction on every CPU level — so the device and the host agree on `round(2.5)`, which is `2.0`.

12. Compiled modules (.em)

Each module compiles to one file per target triple, named `<module>.<target>.em`:

```
math.x64-windows.em  math.aarch64-linux.em  math.x86-windows.em
math.spv.em          math.ptx.em
```

A dotted module name keeps its dots, and the files are written to `.neper/<mode>/<module>.<target>.em` under the project root (§2) — `.neper/debug/e.mem.x64-windows.em`, `.neper/release/e.mem.x64-windows.em` — so a debug and a release `.em` of one module coexist by directory. A module's `.spv.em` or `.ptx.em` holds only its device-reachable functions (§10, D24) and is produced by a host build for each GPU target `--gpu` lists (§13, GPU targets in a build).

The final segments `windows`, `linux`, `macos`, `x64`, `x86`, `aarch64`, `spv` and `ptx`, and compound suffixes formed from them, are reserved for source-target and artifact selection. A user module or ordinary source basename whose final segment is one of them is a compile error; this prevents `thing.windows.e` from being interpreted both as a module and as the Windows variant of `thing`.

Contents

Part	Purpose
Header	magic NEPM, format version, target triple, flags, build mode
Strings	deduplicated UTF-8 strings referenced by numeric index from every other section; index 0 is the empty string
Interface	exported declarations — every module-scope declaration, the language having no visibility mechanism (§5) — in a compact binary form, each with its signature hash ; for a <code>@gpu</code> kernel, its workgroup size, shared byte total and inferred capability set (§10); the module's error table — value to qualified name (§7); and the interface hash over the whole section
Deps	fine-grained edges (Incremental rebuilds, below): signature edges for foreign declarations, value edges for foreign constants, body edges for functions inlined, instantiated, comptime-executed or device-compiled, and lookup edges for every protocol name (§9) examined
NIR	typed IR for exported and inline-eligible functions and for every generic template, each with its body hash
Code	machine code (or SPIR-V/PTX) with relocations: the module's own functions, plus the monomorphised instances and device-compiled helpers it emitted under module-local linkage
Debug	the standard-format debug sections of §13 — line tables, and the locals-and-types subset in DWARF or CodeView — plus the source hash; from M4, the neper-format side table beside them

The file is little-endian regardless of target. Its fixed 32-byte header is: bytes `0..3` magic NEPM; `u16` format version; `u16` header size; `u32` target-triple string-table index; `u32` flags; `u8` build mode; three reserved zero bytes; `u32` section count; `u32` section-directory offset; and `u32` whole-file CRC32C with that field zeroed. The directory has one 24-byte entry per section: `u32` kind, `u32` flags, `u64` offset, `u64` length. Sections are ordered by kind, eight-byte aligned, non-overlapping and contained in the file; unknown optional kinds are skipped and an unknown required kind rejects the file. Strings are UTF-8 encoded as `u32` byte length followed by bytes, with no terminator. Integers in Interface, Deps and NIR use fixed-width little-endian fields; lists begin with `u32` counts. Every NIR opcode has a versioned numeric ID and length-prefixed operands, so an unknown opcode rejects the module rather than being misparsed. The compiler validates all counts, offsets, lengths and UTF-8 before allocating from them. A version mismatch is a cache miss, never a source diagnostic; the module is rebuilt.

Carrying **both** NIR and machine code is deliberate: NIR enables cross-module inlining and re-emission for a different CPU feature level within the same target triple and data layout; it does not retarget a target-dependent `.em` to another OS, architecture or pointer width. Machine code makes linking a `memcpy` plus relocation fixups.

Incremental rebuilds

Two hashes per declaration. Every declaration carries a *signature hash* and a *body hash*. The signature hash covers what a reference needs: the name, the kind, the full type — parameters and their types, return types, comptime parameters, struct fields in order, enum members and backing type, an error's qualified name — and the attributes (§5). The body hash covers the signature hash plus the declaration's NIR: a function's body, a generic template, a *const*'s evaluated value. Both are **xxHash64 with seed 0** over the compiler's canonical little-endian serialization — as is the content hash the own linker uses to find candidate duplicate instances (§13); FNV-1a 32-bit is the error-value hash alone (§7) — and are independent of thread count (§15). The interface hash over every exported signature hash survives as a cheap first check, not as the rule.

Edges, not module hashes. While compiling module A the compiler records, per foreign symbol or protocol lookup, the exact observation it depended on:

- a **signature edge** (B, sym, signature-hash) for every symbol of B that A calls, takes the address of, names as a type or uses as an error;
- a **value edge** (B, sym, body-hash) for every foreign *const* that A reads, because its evaluated value is compiled into A;
- a **body edge** (B, sym, body-hash) for every function of B whose NIR was compiled *into* A: inlined at a call site, instantiated with A's comptime arguments (§9), executed by the interpreter for one of A's *consts* or comptime arguments (§9), or compiled for the device because a kernel in A reaches it (§10, D24). Body edges are transitive: a body edge to `push[T]` carries body edges to everything `push[T]`'s expansion pulled in;
- a **lookup edge** (B, name, observed-signature-or-absent) for every protocol name (§9) that A looked up in B, including an incompatible declaration and every lookup that fell back to a compiler-supplied *hash*, *eq*, *cmp* or *format*, which records the fallback it took. Declaring that name in B later changes the edge, so a protocol added after an instantiation invalidates the instantiating module instead of being silently ignored.

A module is recompiled when its own source hash changes, when any recorded edge no longer matches the current hash of its target, or when a protocol lookup produces a different presence or signature. Nothing else recompiles it. Modules compile in parallel; the edges of one build are the *Deps* of the next. Every edge follows a *use*, and the *use* graph is acyclic (§2), so no module's edges lead back to itself and the recompilation set of an edit is finite and ordered.

What body isolation means. Editing a function body leaves a dependent untouched only when the dependent holds a signature edge and no body edge to it: the callee was called, not inlined; it is not generic; it did not run at compile time; it is not reachable from one of the dependent's kernels. For everything else the dependent is recompiled, which is the correct outcome, because the old body is compiled into it. The earlier rule — "editing a body never disturbs dependents" — would have shipped the stale copy silently, the failure mode this language works hardest to eliminate, in the build system itself.

Inlining cap. Cross-module inlining is limited to callees whose NIR body is at most **40 instructions**. A larger callee is called through its exported symbol however hot the site, and a call takes only a signature edge. The cap keeps body edges confined to leaf helpers — accessors, arithmetic, a bounds test — so a body edit in ordinary code recompiles nothing but its own module. Within one module there is no cap: the module is one unit of recompilation anyway. Later optimiser heuristics (roadmap M4) may inline less than the cap across a *.em* boundary, never more.

Where instances go. A monomorphised instance of another module's generic, and a plain function compiled for the device because this module's kernel reaches it, are emitted into *this* module's *.em* — the instantiating or kernel-owning module — with **module-local linkage**: not exported, not in the Interface, not the target of anyone's edge. Two modules that both instantiate `push[i32]` each carry a copy; the determinism rule (§15) makes the two copies byte-identical, and the own linker (§13) groups copies by content hash and verifies full identity before folding, so the executable holds one `push[i32]`. Under `--linker=system` each copy stays a local symbol in its own object and duplicates cost bytes only. Ownership needs no coordination between modules, which is what keeps their compilation parallel.

The own linker never treats a 64-bit hash match as identity. It first groups candidates by hash, then compares their canonical NIR type identity, relocation graph and emitted bytes; only byte-for-byte and relocation-for-relocation equal instances fold. Hash collisions therefore affect speed, not correctness.

Canonical serialization fixes field and attribute order, integer widths, UTF-8 string encoding, float bit patterns, and NIR opcode and operand numbering. It contains no addresses, map iteration order or thread-dependent identifiers. A change to that serialization increments the *.em* format version and the data-layout version used by the comptime cache.

Incremental equals clean. An incremental rebuild must produce an executable byte-identical to a clean build of the same sources — §15's requirement across thread counts, applied across time. The M2 harness (roadmap) checks it, and

it is the property the edge rule exists to deliver.

13. Toolchain

One binary, named `neper`, does everything — compile, format, index, disassemble. There is no separate build system and no package manager.

```
neper build <file.e> [--target ARCH-OS|spv|ptx] [--gpu spv,ptx|none] [--cpu LEVEL]...
    [--release] [--g] [--linker=own|system] [--arena SIZE]
    [--libpath DIR] [--link LIB] [-j N] [-o PATH]
neper run <file.e> [the build options] [-- ARGS...]
neper fmt [--check] [FILE.e|-]
neper test [--filter PAT] [-j N] [--timeout SECS] [--release] [--test-arena SIZE] [FILE.e]
neper tokens [--path VIRTUAL.e] <FILE.e|->
neper parse [--path VIRTUAL.e] <FILE.e|->
neper index [--all] [--path VIRTUAL.e] [FILE.e|-]
neper dis <file.em> [--symbol NAME] # disassemble a compiled module
neper dap # neper's own debug adapter (M4); lldb-dap and codelldb work from neper-0
neper info # versions, targets and machine-readable capabilities

--json # every finite command: versioned JSONL from docs/tooling.md; not dap
--absolute-paths # machine output: opt in to absolute paths; not dap
--language-version V # select an advertised MAJOR.MINOR source version
```

`neper run <file.e>` takes a **program root** — a file that declares `fn main` (§2) — builds it and runs it, with everything after `--` as its args. `neper build <file.e>` takes a program root and writes the executable, named by `-o PATH` or, by default, by the root module's name in the working directory (`hello, hello.exe`); or it takes any other `.e` file and writes its `.em` and those of the modules it reaches, to `.neper/<mode>/` under the project root (§12). A file not under a source root is module `<filename>`, and `e.*` comes from the toolchain's own `lib/` (§2), so `neper run examples/hello.e` builds and runs module `hello` from any working directory, with no package around it. `--arena SIZE` sizes the root arena (§8; the default is 64 MiB) — `SIZE` is an integer with an optional `K`, `M` or `G` suffix, in binary units — `--target` defaults to the full host triple, `--linker` selects the linker (Linking, below), and `-j N` the worker count (§15). `--cpu LEVEL` selects the instruction level (Target CPU levels, below) and is given **at most once per target family** — `x64-*`, `aarch64-*` or `x86-*` for the host arch, `sm_*` for a `ptx` module, embedded or standalone — so a host build with `--gpu ptx` writes `--cpu x64-v4 --cpu sm_80`; a family given no level keeps its default, and a level whose family the build does not target — `sm_80` under `--gpu spv`, `aarch64-v8.2` on an `x64` build — is an error.

`-o PATH` names the executable for a program root. For `--target spv|ptx` it names the one root device `.em`; dependencies remain in `.neper/<mode>`. For a non-root host module it must name an existing directory, into which that module's `.em` is copied after the normal cache write; a file path is an option error. `run` accepts `-o` for the temporary/persistent executable but never for captured program output.

A host target is `x64-windows`, `x64-linux`, `x64-macos`, `aarch64-windows`, `aarch64-linux`, `aarch64-macos`, `x86-windows` or `x86-linux`; unsupported architecture/OS combinations are option errors. The OS selects per-target source files and ABI, so it is never inferred from the build machine after `--target` is written. `neper run` requires that target to equal the host triple; otherwise it reports an option error after building nothing. `--cpu native` is accepted only for a host-family target that equals the host architecture and selects detected host features; it is invalid for cross-compilation and never applies to SPIR-V or PTX.

`--libpath DIR` adds an ordered library search directory and may repeat. `--link LIB` adds a static archive or import library by path or logical name and may repeat; logical names search explicit directories, then the platform defaults. These options imply `--linker=system` when the own linker cannot consume the named format. `@import` names search the same directories at link time and use the platform mapping (`name.dll`, `libname.so`, `libname.dylib`) unless the declaration supplies a filename containing a separator or suffix. Search order and every resolved input are recorded in `.neper/<mode>/build-manifest.json`, whose reproducible format is fixed by `docs/tooling.md` §7. No arbitrary linker-argument escape hatch exists in v1.

GPU targets in a build. `--gpu` lists the GPU targets a host build embeds — `spv`, `ptx` or `spv,ptx` — and defaults to `spv`; `--gpu none` is a host-only build, and an `@gpu` declaration in the root's reachable module graph is a compile error naming the kernel. Unreachable files outside that graph do not affect it. A host build compiles every kernel-owning module — and only the device-reachable functions in it and in the modules those reach (§10, D24) — once per listed GPU target into

`<module>.<gpu-target>.em` (§12), and embeds those Code sections in the executable as the data §10 loads at runtime. `--target spv|ptx` alone builds only device modules, for inspection with `neper dis`, and produces no executable. The device half of a module never sees `use e.os` or any host-only declaration: device compilation includes only the device-reachable functions (D24, D37), so `target.os` is `.None` there and a host-only name is simply absent.

`neper dis` prints each function of a `.em` as address, bytes and mnemonic — machine code for a CPU target, SPIR-V or PTX text for a device one — with a source line marker, from `.nepersym/.nepsym` (Debug information, below), wherever the line changes. `--symbol NAME` limits the output to one function; the file name carries the target (§12), so there is no `--target` option. `NAME` is the canonical linkage name printed by unfiltered output: `module.function` for ordinary functions and `module.function[canonical-arguments]#content-hash` for module-local generic or device instances. A non-unique source name is an error listing the canonical matches.

`neper fmt`, `neper test` **and** `neper index` operate on everything under the project's roots (§2) — every `.e` file, for the first and the third; every module, for `neper test`. A `FILE.e` argument names one file instead, which is how a file outside every root — `examples/hello.e`, `examples/sample.e` — is formatted, indexed, or has its module's `@test` functions run (Testing, below); `neper index --all` also covers the toolchain's `lib/.fmt`, `tokens`, `parse`, and single-file `index accept - as stdin` under the exact `--path` and output rules in `docs/tooling.md` §4 and §6.

The versioned JSONL envelope, source identifiers, byte-precise spans, diagnostic and fix records, lossless token/syntax trees, complete symbol/reference index, capability query, test ordering, build manifest and generated-source maps are normative in `tooling.md`. In particular, `index` covers locals, members and references as well as module-scope definitions, carries `module` explicitly, reports compiler-generated protocol resolutions, and uses reproducible root-relative paths unless `--absolute-paths` is requested. A harness never has to parse human output or duplicate name resolution.

Successful commands exit 0; source, option and link errors exit 1; internal compiler failures exit 2; and `neper test` exits 1 when any test fails, crashes or times out. Warnings are disabled in v1: every emitted diagnostic is either a note attached to an error or an error that makes the command fail.

Diagnostics

```
src/parse.e:112:9: error[E-TYPE-0003]: cannot pass `i64` where `i32` is expected
```

A code is `E-<CATEGORY>-<NNNN>` from the stable, checked registry `diagnostics.md`. Categories describe semantics and never depend on this document's section order; a code is never reused. `--json` uses the versioned diagnostic, related-location and structured-fix records of `docs/tooling.md` §3. A runtime trap (§11) is a structured payload of a run or test record rather than overloading a compiler diagnostic.

Diagnostics are sorted independently of worker completion: reproducible source identifier, then byte offset, diagnostic code, and message. A primary error precedes its notes, whose order is call-chain or source order as applicable. Command-line and link diagnostics without a file sort before source diagnostics in argument order. This order is identical in text and JSON modes.

Program entry

Every executable has exactly one `main`, in the root module — the program root named to `neper run` or `neper build` (§2, above) — with exactly this signature:

```
use e.mem

fn main(a: *mem.Arena, args: []str) -> err {
    ...
    ret ok
}
```

It mirrors `@test` (Testing, below): one signature, no variants. A function named `main` in the root module with any other signature is a compile error naming the expected one; in any other module `main` is an ordinary function (§2).

- `a` is the **root arena** (§8): the block the startup code reserved and committed before `main` ran, sized by `--arena` (above). There is no other way to reach it. A program that wants a second arena carves one from this one or from a stack buffer.
- `args` are the command-line arguments, `args[0]` the program as it was invoked, each `str` allocated in the root arena by the startup code — what `os.args` (§5) returns.

- Returning `ok` exits the process with code 0. Returning any other value writes one line, `error: <qualified name> — error: e.os.NotFound` — to `stderr` from the error table (§7), and exits with code 1. `main`'s own defers have already run, as on any `ret`.
- A failed check exits through the trap protocol (§11) with code 134 instead, and `os.exit(code)` (§5) exits with the code given and runs nothing.

Nothing runs before `main` except the startup code that builds the arena and the argument slice: no static initialisers, no constructors, nothing a library can hook (Testing, below, is the consequence).

If reserving or committing the root arena, decoding arguments, or allocating their slice fails, startup writes `startup:`

`<operation>: <platform code>` directly through the platform's raw `stderr` facility and exits 1. It cannot return an `err` or run a `defer` because `main` has not begun. Failure to write the diagnostic still exits 1.

Testing

Testing is built in, because in `neper` a library could not do it. There are no macros, no reflection, and no static initialisers — nothing runs before `main` — so a library has no way to discover tests except a hand-maintained registration table. That means writing every test name twice, and its failure mode is the worst one available: a test that is written, compiles, and silently never runs.

```
use e.mem
use e.test
use lex

@test
fn parse_handles_empty_input(a: *mem.Arena) -> err {
  let toks = try lex.run("", a)
  if toks.len != 0 {
    ret test.Failed
  }
  ret ok
}
```

A test is a function marked `@test` taking a single arena and returning `err`. One signature, no variants. Returning non-`ok` is failure. `e.test` is the module a test imports: it declares `error Failed` for the plain-failure case and holds the assertion helpers, which are ordinary functions — these four, and no more in `e.test`:

Function	Returns Failed when
<code>fn assert(cond: bool, msg: str) -> err</code>	<code>cond</code> is false
<code>fn eq[T: type](a: T, b: T, msg: str) -> err</code>	<code>T.eq(a, b)</code> is false; <code>T</code> is a type with an <code>eq</code> (§9), which includes every type <code>==</code> takes (§6)
<code>fn near(a: f64, b: f64, abs: f64, rel: f64, msg: str) -> err</code>	the distance between <code>a</code> and <code>b</code> exceeds both <code>abs</code> and <code>rel</code> times the larger magnitude
<code>fn fail(msg: str) -> err</code>	always

Each returns `ok` otherwise and, on failure, records `msg` in the test's JSON object (Report, below); none traps, so `try test.assert(toks.len == 0, "empty input lexes to nothing")` is the idiom. `unreachable()` (§11) stays for the path that cannot happen. Golden-file comparison is deferred (§17). How `msg` reaches the record: `e.test` holds a module-scope `var current: *Record` — type `Record = struct { ... }`, the running test's record — which the synthesised root (Execution, below) sets before each test; `assert`, `eq`, `near` and `fail` append `msg` through it, and the root writes it out as the test's end record.

`near` requires finite, non-negative `abs` and `rel`; invalid tolerances return `Failed`. Equal infinities pass, unequal infinities fail, and any NaN fails. For finite operands it first accepts `abs(a-b) <= abs` when the subtraction is finite; otherwise, or if that test fails, it sets `m = max(abs(a), abs(b))` and accepts when `m != 0` and `abs(a/m - b/m) <= rel`. Outside a synthesised test root `current` is `nil`: helpers still return `ok` or `Failed` normally and simply have no report record to append to. The hidden pointer exists only in test builds, is initialized and cleared by compiler-generated calls, and is the sole explicit exception to the library no-hidden-state rule. Assertion helpers may be called only by the thread executing the test function; calling them from a spawned worker is undefined behavior. Workers return results to that thread explicitly before it joins them.

The compiler owns exactly three things: discovery, isolation, reporting.

- *Discovery* is free — every declaration is already in the symbol table, so `@test` is one bit. No registration, one definition site.
- *Isolation* is a process and fresh arena per test (Execution, below). Leak-free by construction, and "did this test exceed its memory budget" becomes an ordinary check rather than a tooling project.
- *Reporting* is a summary by default and `--json` for machines — one object per test (Report, below). This matters more here than usual: a structured pass/fail stream is what the tooling in §14 consumes.

`@test` functions are excluded entirely from non-test builds.

Execution. In a debug build the primary way a buggy test fails is a trap (§11), which ends a process. So `neper test` never runs tests in its own process:

- **The test process's entry point is synthesised.** `neper test` generates a program root per module and invokes it once per test: a `main` (Program entry, above) that receives, through `args`, the module name and exact test name. The inherited control handle and nonce are reserved startup metadata, not program arguments. It calls only that test, writing a `begin` record before the call and, after it, one JSON record — the test's name, outcome, error, message and `duration_ms` — as its `end` record to the control handle. Every control record uses the trap protocol's length-prefixed JSON frame: the `begin` object is `{"begin": "<name>"}` and the `end` object is the test's result. The generated root imports the module under test under a compiler-internal qualifier containing a byte no source identifier can spell, so it cannot collide with a source alias or declaration — generated code is not subject to source-level invariants — so a module named `main`, or one that declares an ordinary `fn main` of its own (§2), is tested like any other.
- **One child process per test**, built in **debug** mode by default; `--release` is accepted and runs the same tests against release semantics (§11) — wrap, truncate, elided memory checks — with nothing else changed. The parent — `neper` itself — spawns it through `os.spawn` (§5) with its `stdout` and `stderr` on `os.pipe` ends and the write end of a control pipe in `Stdio.inherit`, its raw number in the child's `args` (§5); reads them, kills it with `os.kill` on a timeout, and writes the report; nothing the runner needs lies outside `e.os`. The process runs exactly the named test and exits.
- **Tests are scheduled in deterministic module/source order** but up to `-j N` child processes run at once (default: one per logical core, §15). Every test starts with fresh module-scope variables; no test can affect another's globals.
- **Each test gets a fresh arena of 16 MiB** by default — `--test-arena SIZE` changes it, `SIZE` spelled as `--arena` spells it (above) — carved from the test process's root arena (Program entry, above); process exit then reclaims it.
- **stdout and stderr are captured per test.** They are dedicated pipes for that process. After it exits the parent drains both pipes to EOF before finalizing the record, so control-pipe delivery order cannot misattribute bytes.
- **A crash is an outcome, not the end of the run.** In test mode the trap runtime writes its machine-readable trap object to the inherited control pipe and the human rendering to `stderr`. User `stderr` can therefore never impersonate a trap. An abnormal exit without a trap object is still `crashed`; later tests already have independent processes and need no restart protocol.
- **Every test has a timeout**, `--timeout SECS`, default **60 seconds**. When it expires the parent kills the test process, records outcome `timeout`, and records the outcome and continues scheduling the remaining independent tests. The timeout uses `Clock.Monotonic`; `SECS` is a finite decimal in $(0, 86400]$, measured to the platform clock's millisecond ceiling.
- `--filter PAT` **selects tests** whose `module.test_name` contains `PAT`, with `*` as a wildcard for any run of characters: `e.str.*`, `*empty*`. Matching is case-sensitive over UTF-8 bytes; `\*` names a literal asterisk and `\\` a literal backslash. Any other escape is an option error. Zero matches is a successful run with a zero-test summary.
- **A `FILE.e` operand selects one module.** `neper test examples/sample.e` runs the `@test` functions of that file's module and nothing else, under every rule above — its own child process, a fresh arena per test, source order. The named file is a module whether or not it lies under a source root (§2), so this is how the `@test` functions of a file outside every root are run at all; with no operand, every module under the project's roots runs.
- **A test that spawns threads should join them before it returns.** Any remaining threads are terminated with that test's process after output pipes are drained; their incomplete work is not reported as a separate failure. Kernels run through the CPU backend (§10) inside the same process, so a `barrier` trap is a `crashed` outcome like any other.

Report. `--json` buffers completed children and emits one `record:"test"` object per test in deterministic module/source order, never worker-completion order, then one `record:"test_summary"` and the command `result` object. The common envelope and summary counters are `docs/tooling.md §7`; each test record carries:

Field	Value
name, module, file, line	the @test function's declaration
outcome	"passed", "failed" (returned a non-ok err), "crashed", "timeout"
error	the qualified name of the returned err ("e.test.Failed"), "ok", or JSON null when the test did not return
message	the msg of the test.assert that failed, when one did; JSON null otherwise
duration_ms	monotonic elapsed time of the process, rounded up to milliseconds
timeout_s	the limit that applied
stdout, stderr	arbitrary captured bytes; valid UTF-8 is emitted as a JSON string, otherwise an object { "encoding": "base64", "data": "..."} }
trap	the nullable trapPayload of docs/tooling.md's schema: kind, byte-precise nullable span, string values, and backtrace; for an abnormal exit that is not a language trap, kind is "exit" and values carries the code or signal

The summary object carries the counts per outcome and the total duration. The parent's exit code is 0 when every test passed and 1 otherwise; a crashed test process never becomes the parent's exit code.

Everything else stays a library, and can, because none of it needs discovery: assertion helpers, property generators, fakes, golden-file comparison. They are ordinary neper functions that a test calls. The line is drawn here deliberately — fixtures, mocking and parameterised cases are all reasonable requests, and none of them belong inside a compiler.

Known limitation: neper test runs host-native tests only. Testing an aarch64 or x86-32 build from an x64 host needs an emulator, which is out of scope; those targets require an explicit external runner. GPU kernels are tested through their CPU build (§10), which is one of the better arguments for compiling @gpu functions both ways.

Linking

neper links its own executables. The compiler already holds the code and relocations in memory, so the default path writes no object files and spawns no process: NIR goes to machine code goes to a finished executable, with the module-local instances of §12 folded by xxHash64 content hash (§12) on the way.

Whichever linker runs, neper holds every module's Interface, so it is neper that merges the error tables (§7) into the executable and rejects a hash collision before any linker is involved.

This is the single largest serial section of a build — everything else parallelises (§15) — so it caps the speedup no matter how many cores are available. A system linker costs milliseconds just to spawn, then writes objects to disk only to read them back, re-parse symbol tables, and redo relocations we already computed. A full relink runs 100ms–1s; doing it ourselves is realistically sub-10ms.

The work is staged by difficulty, because the two cases are not remotely equal:

Case	Difficulty
Static pure-neper executable: a freestanding ELF, or a PE whose import table names kernel32.dll alone	Easy — a header, program headers, code, and one fixed import table. M2
Dynamic imports by name from any .dll/.so/.dylib	Moderate — general import tables, PLT/GOT, load commands, and the PDB writer. Covers libc, Vulkan and the CUDA driver, and therefore most real programs. M4
Static archives (.a/.lib)	Hard — COMDAT dedup, weak symbols, section GC. Never

The easy case includes a PE with a fixed import table because without one it would deliver nothing on Windows: a freestanding executable can do I/O only on Linux, over raw system calls, and hello.e, the self-hosted compiler and lib/e all need kernel32 on Windows, which is one of the two M0 platforms. kernel32.dll is the one library os.windows.e names — files, processes, memory, threads, the clock and LoadLibrary/GetProcAddress for os.dlopen are all in it — so e.io, e.thread and e.time on Windows are implemented over dynamic imports by construction, and a fixed table of that one DLL is a few hundred bytes the linker writes the same way every time. The general case, any library by name, waits for M4.

The third case is where an own linker stops being worth it, so --linker=system **stays permanently** as the escape hatch for static archives and exotic platform requirements. It is also what M0 and M1 use while the own linker does not exist yet, behind a seam thin enough to swap: one function that writes objects and spawns a process, with nothing about object

files leaking into the backend.

Which stage a program needs follows from its `extern` declarations (§5) and from the target's `e.os`. A program whose only OS surface is `e.os` is the easy case on Linux, where `os.linux.e` issues raw system calls and imports nothing, and on Windows, where every import is `kernel32`. Any other `@import` — the program's own, or `e.os` on macOS (`libSystem`), which is every executable for that target — needs the dynamic-import stage; the GPU runtime's Vulkan and CUDA driver bindings (§5) are such imports, so a program that opens a `.Vulkan` or `.Cuda` device links with `--linker=system` until M4 (roadmap). An `@import` whose `LIB` exists only as a static archive needs `--linker=system`.

Platform obligations we are taking on: PE import tables and base relocations, SEH `.pdata/.xdata`, ELF `PT_*` headers and `DT_NEEDED`, Mach-O load commands and chained fixups, **ad-hoc code signing on Apple Silicon**, which is mandatory rather than optional — an unsigned aarch64 binary will not execute — and, on Windows, **the PDB**: the MSF container with the DBI, module, symbol and line streams, and no type stream. That scope is what symbolication, breakpoints and stepping need; the type stream that would carry the locals-and-types subset (Debug information, below) into a PDB is not scheduled, so a Windows build that wants locals in the debugger links with `--linker=system`, whose PDB carries the CodeView records the compiler emits. The PDB writer lands with the dynamic-import stage at M4. **Until then, Windows symbolication by any external tool — WinDbg, Visual Studio, ETW, a debugger — requires `--linker=system`**. The trap protocol's own backtrace (§11) does not, under either linker: it reads the `.nepersym` section (`.nepersym` on COFF/PE; Debug information, below), which the compiler emits into every object file it writes and the own linker writes directly, and which the runtime finds through the executable's own section table.

Target CPU levels

Which instructions the emitter may use is a build option, not a property of the language. Nothing in the front end or in NIR is ISA-aware; instruction selection, register allocation and scheduling are the emitter's job alone.

For x86-64 we adopt the industry-standard microarchitecture levels verbatim rather than inventing our own feature flags:

--cpu	Requires	Notes
x64-v1	SSE2	The 2003 baseline. Maximum compatibility. No narrow-float support: <code>f16/bf16</code> are widened in software.
x64-v2	SSE4.2, POPCNT, CMPXCHG16B	
x64-v3	AVX2, FMA, BMI1/2, LZCNT, MOVBE, F16C	Default. F16C converts <code>f16↔f32</code> in hardware; arithmetic still widens to <code>f32</code> .
x64-v4	AVX-512F/BW/CD/DQ/VL	Opt-in only — see below. Narrow arithmetic still widens to <code>f32</code> here: AVX-512 FP16/BF16 are not part of the level, and the native instructions are selected only under <code>native</code> on a machine that has them (Sapphire Rapids, Zen 4+).

Other targets:

--cpu	Meaning
x86-v1	i686 baseline with SSE2. Default for x86-32, and its only level
aarch64-v8.0	ARMv8.0-A, NEON (mandatory in the base ISA). <code>f16</code> converts natively, computes via <code>f32</code> . 2011-era; opt-in
aarch64-v8.2	Adds native FP16 arithmetic and dot product. Default for aarch64: Apple Silicon and every recent ARM server qualify
aarch64-v8.6	Adds native BF16 arithmetic
sm_50	PTX ISA 6.0, the floor (§10). Default for <code>ptx</code> ; <code>f16</code> and <code>bf16</code> arithmetic widen to <code>f32</code>
sm_70	Adds <code>f16</code> arithmetic (available from <code>sm_53</code>) and independent thread scheduling
sm_80	Adds <code>redux.sync</code> for the integer subgroup reductions and native <code>bf16</code> arithmetic
sm_90	Hopper; the profile selects no instruction beyond <code>sm_80</code> 's, and the level exists so a build can name the part
native	Detect and target the building machine. Never the default — it produces binaries that fault elsewhere.

x64-v3 is the default, deliberately, and x64-v4 is not. AVX-512 is absent from Alder Lake and later Intel consumer parts, present on Zen 4/5, and on some earlier Intel server parts it triggers frequency throttling that costs more than the wider vectors gain. `v3` is close to universal on hardware from the last decade and carries the instructions that actually matter.

Register allocation is affected as much as instruction selection: `v4` exposes 32 vector registers rather than 16, which changes allocation decisions well beyond the choice of mnemonic.

Instructions we will not select implicitly. `pdep/pext` are microcoded and extremely slow on AMD before Zen 3. They are available as explicit `simd` intrinsics and are never chosen by the emitter on your behalf.

Runtime dispatch — one binary carrying several versions of a hot function, selected by CPUID at startup — is deferred past M4. When it lands it will be explicit (an `@cpu("x64-v4")` attribute on the function, the level as a string so the token is a legal literal), never automatic. Until then `@cpu` is not an attribute (§5).

ARM SVE is out of scope. Its vector length is not known at compile time, which does not map onto fixed-width types like `Vec[f32, 8]`. aarch64 targets NEON's fixed 128-bit vectors; SVE is a separate problem for later.

On the GPU, SPIR-V capabilities and subgroup operations are the equivalent knob, declared per kernel.

Debug information

Debug information goes out in the standard formats, in a fixed subset, on the default path — because the readers already exist and neper's own would not for three milestones. Neper's own compact form is an optimisation layered on top at M4, not the route to a working debugger.

Emitted always, in standard formats, in every build mode:

What	Format	Why it is not optional
Line tables	<code>.debug_line</code> (ELF/Mach-O); on Windows, CodeView line records — into the PDB under <code>--linker=system</code> , and under the own linker from the M4 PDB writer (Linking, above); the trap protocol reads <code>.nepersym</code> (<code>.nepsym</code> on COFF/PE) under both	The cheapest part of DWARF — a small state machine, no DIE tree
Symbol tables	ELF <code>.symtab</code> , Mach-O <code>LC_SYMTAB</code> , COFF symbols	Symbolication for every external tool
Unwind info	<code>.eh_frame</code> , or <code>.pdata/.xdata</code> on Windows	Correct stack walks, required in release too

This trio is what perf, VTune, Superluminal, Instruments, ETW and every crash-dump pipeline actually read. For a language whose whole pitch is hardware utilisation, users live in a profiler, and a profiler that cannot symbolise is worthless.

The trap protocol's data source is `.nepersym` (`.nepsym` on COFF/PE). PE's `IMAGE_SECTION_HEADER.Name` is eight bytes, so `.nepersym` cannot be represented in a linked PE image; `.nepsym` is the fixed platform spelling, not linker truncation. The compiler emits the line and symbol tables a second time, in its own compact form, as a named non-loaded section `.nepersym` (or `.nepsym`) in every object file it writes under `--linker=system`. The compiler also emits a linker-retention directive (`/include` on COFF, `SHF_GNU_RETAIN` or a generated linker script on ELF, `no_dead_strip` on Mach-O). Each contribution is a length-prefixed table with relocatable symbol references; the final link concatenates contributions, applies relocations and emits a sorted address index. Duplicate entries with the same address, canonical name and line table fold. The own linker performs the same merge directly from `.ems`. The runtime locates it through the executable's own section table — PE, ELF or Mach-O — so §11's symbolised backtrace has one source in every build mode, whichever linker produced the binary and whether or not a PDB exists.

The merged `.nepersym/.nepsym` begins with NEPS, u16 version, u16 reserved zero and u32 entry count. Each fixed-size entry contains u64 `start`, u64 `end`, and u32 indices for canonical function name, file and line-program; entries sort by `(start,end,name)` and ranges are half-open. A following UTF-8 string table and delta-encoded line programs use the same bounded length encoding as `.em`. The runtime binary-searches the address table, applies the executable load bias, and prints an address with `? fields` when no entry covers it. Malformed or absent data degrades to raw addresses and never prevents the original trap.

Emitted in every debug build, from neper-0 (roadmap) onward, as the default path: locals and types in a fixed subset of DWARF (ELF, Mach-O) and CodeView (PE). The subset is the tags in this table and nothing beyond them:

Tag	Carries
<code>compile_unit</code>	one per module: name, producer, language, line-table offset
<code>subprogram</code>	one per function: name, linkage name, address range, frame base
<code>formal_parameter, variable</code>	name, type, location
<code>base_type</code>	every primitive of §4, including f16, bf16, err (as u32) and bool
<code>structure_type, member</code>	struct, and the <code>{ptr, len}</code> of a slice, the tag-and-payload of a union enum, the wrapped value of <code>Atomic[T]</code>

Tag	Carries
<code>union_type</code>	bare union
<code>enumeration_type</code>	enum, and the Tag of a union enum
<code>pointer_type</code> , <code>const_type</code>	pointers and pointee const qualification
<code>subroutine_type</code>	the parameter and return signature referenced by every function pointer
<code>array_type</code>	<code>[N]T</code> and <code>Vec[T, N]</code>
<code>inlined_subroutine</code>	one per inlined call site — release builds with <code>--g</code> only, below

Every location is one `DW_OP_fbreg` offset from the frame base (or the CodeView equivalent, `S_REGREL32` against the frame pointer). A debug build gives every named local a frame slot for its whole scope and writes it back at every assignment, so one location covers the scope and there are no location lists, no `.debug_loc`, no `.debug_ranges`; one abbreviation table, written once, covers every module. That is hundreds of lines of emitter, not thousands, and it is read by `lldb`, `gdb`, `WinDbg` and Visual Studio, and through them by `lldb-dap` and `code1ldb` in VS Code and Zed, from the first milestone that has locals to show. The full generality of DWARF — location lists, expression programs, type units, DWARF 5 string offsets — is where its cost lives, and none of it is emitted. A release build carries the trio only; `--g` adds the subset to a release build, and there every inlined call site is recorded as an `inlined_subroutine` so a backtrace still names the function on the page.

Debug builds do not inline. The inliner is off in a debug build, in every module and across every `.em` boundary, so every frame in a backtrace is a real call, a breakpoint on a function is hit whenever it runs, and a step never enters a body that is not on the page. The other passes — constant folding, DCE, register allocation — stay on; none of them moves a local out of its frame slot.

Neper's own format, at M4, as an optimisation. The `.em` already carries a typed NIR and a full type table, so a side table of locals, scopes and locations over structures that are serialised anyway is nearly free to write, loads in microseconds, and knows things the standard formats cannot express — that a `[]const u8` is a string, which arena a slice points into, the poison bytes of §11, the work item a CPU-built kernel is stepping (§10). It goes into the `Debug` section beside the standard subset (§12), never instead of it, and it is read by `neper dap` and the debug engine beneath it (Editor integration, below). It is sequenced after the standard path because it is worthless without that engine, and the engine is the expensive part.

Editor integration

VS Code and Zed are both DAP clients: neither reads DWARF or PDB directly. Both already ship adapters over `lldb` — `lldb-dap` and `code1ldb` — and `lldb` reads the subset above. From `neper-0`, then, editor debugging costs **zero** adapters: install the extension, point it at the executable, set a breakpoint, inspect locals. On Windows the split is this:

symbolication — backtraces, profilers — works through the own PDB from M4, which carries module, symbol and line streams and no type stream (Linking, above); **locals in a debugger** need types, so on Windows they go through the CodeView records in the object files plus `--linker=system's` PDB for as long as the PDB type stream stays deferred (§17).

`neper dap` is neper's own adapter, at M4, over neper's own debug engine, and it exists for what a generic debugger cannot do: render a slice as its bytes and a tagged union as its live member, show the arena a pointer belongs to, and step one work item of a CPU-built kernel with the workgroup's shared state visible (§10). The adapter itself is a thin JSON-RPC server; the substance is the engine beneath it — process control (`ptrace` on Linux, Mach exception ports and the debugger entitlement on macOS, `DebugActiveProcess` on Windows), `int3` breakpoint patching, single-step, stack walking, and rendering variables from location records — which is why it is sequenced after the standard path exists, and not before.

Editor *language* smarts — highlighting, go-to-definition, hover — are a separate concern travelling over LSP, not DAP. `neper index` (§14) is the seed a language server would grow from; none is scheduled.

Standard library

The surfaces of `lib/e` this document does not write — `e.data.sort`, `e.data.map`, the rest of `e.thread` beyond `spawn`, `join`, `detach` and `DEFAULT_STACK` (§8) — are specified by their `lib/e` source and indexed by `neper index` (§14). This document is authoritative only where it writes a signature; where it does not, the source is.

14. Invariants for tooling and generated code

These are language rules, not style advice. They exist so that a search, a glob or a line-anchored edit is *reliable* rather than probabilistic.

1. **One definition per name per module namespace.** No overloading, ever. The sole parallel namespaces are types and qualifier/value names, selected by syntax as §2 states; locals may not exploit that separation. `grep -n "^fn dot"` returns exactly one line in the module that defines it.
2. **Every top-level declaration starts at column 0** with one of `use type const var fn error extern`. The only other thing at column 0 is an attribute line (§5), which sits above its declaration rather than in front of it — precisely so that the keyword stays first on its own line. A comment line may also start at column 0: a comment begins with `//`, never with a keyword, so `grep "^fn"` is unaffected. The parser relies on this too: an unclosed bracket may not cross one of these keywords (§3), so a dropped `)` is caught at the bracket that opened it rather than cascading through the file.
3. **File path is module name.** A glob finds a module without opening it.
4. **Every cross-module reference written by the author is qualified.** A call site someone wrote names its module. The invariant governs what is written in source, so there are exactly two exceptions and neither appears as an unqualified call on any line. The first is a protocol call on a comptime type parameter or a comptime type value (§9) — `T.hash(v)`, `f.ty.format(x, b)` — legal only in a generic or comptime body, where no concrete module exists yet to name. The second is the calls the compiler generates on the author's behalf: `for`'s `next` (§6), a `format` verb's `format` (§4), and a container's `hash`, `eq` or `cmp` on a concrete element type — nothing is written at those sites, so there is nothing to qualify. What any of them resolves to is still one `grep "^fn sensor_hash"` away, and the type is in the name.
5. **No macros, no preprocessor, no textual code generation.** What you read is what compiles.
6. **No implicit conversions.** Every cast is written down and greppable; the five implicit operations that exist are listed once, in §6.
7. **Order-independent module scope.** An insertion can go anywhere; no forward declarations to keep in sync.
8. **One canonical layout**, enforced by `neper fmt --check`. Diffs stay minimal and edits are textually predictable. `docs/tooling.md` §6 fixes the complete contract, including whitespace, wrapping, blank lines, attributes, imports, comments and stdin behavior. It deliberately preserves semantically meaningful author choices such as parentheses and numeric base, so it does not overclaim one spelling per syntax tree.
9. `neper index` emits versioned JSONL of every definition and reference — with explicit kind, name, qualified name, module, signature, attributes, byte-precise spans and resolved target — so tools do not parse source or duplicate protocol lookup. `docs/tooling.md` §5 is the closed schema. An intrinsic (below) has kind `intrinsic` and a null span because it is declared in no `.e` file.
10. **Casing is enforced**, by the table in §3, which is canonical: `snake_case` for functions and values (variables, parameters, fields), for modules, and for the comptime binding an unrolled `for` introduces (§9), which is a local binding and not a parameter; `PascalCase` for types, enum members, errors and type-kind comptime parameters; `SCREAMING_SNAKE` for constants and value-kind comptime parameters. A name's kind is legible without a lookup, and a search pattern never has to guess at casing.
11. **Tokens and syntax are public data.** `neper tokens` and `neper parse` expose the grammar's lossless, versioned result with original-byte spans. A formatter, editor or generation harness never needs a private compiler API.
12. **Generated code keeps provenance.** An optional `.e.map.json` maps generated spans to generator inputs without changing language semantics or introducing a preprocessor (`docs/tooling.md` §8).

Intrinsics. A closed set of core functions has no `neper` body and no declaration — an intrinsic appears in no `.e` file; the compiler knows each under its qualified name: `mem.cast`, `mem.bitcast`, `mem.size_of` and `mem.align_of` (§4, §8), `meta.*` (§9), `atomic.*` (§8), `os.syscall` and `os.thread_start` (§5), `math.fma` (§11), `simd.*` (§4), the `gpu.*` builtins (§10), and the three pack functions `io.printf`, `str.format` and `gpu.launch` (§9) — the only functions in the language with a pack parameter; the `...` on an `extern` (§5) is a C variadic, not a pack. `unreachable()` (§11) is the one builtin that is a keyword rather than a core function, which is why it alone needs no qualifier. A module whose whole surface is intrinsics — `e.simd` is the only one — still has its file under `lib/e`, so `use` resolves it as it resolves any module; the file declares the module's types and nothing else. They are called like any other function, so invariants 1, 4 and 6 hold at every call site, and `neper index` lists each with kind `intrinsic` and no file or line, so a tool never searches `lib/e` for a body that is not there.

15. Compile speed and parallelism

Single-thread budget

- Lex → parse → resolve/typecheck → NIR → regalloc → emit. No pass manager, no separate optimisation pipeline.
- Optimisations limited to inlining, constant folding, DCE and a good linear-scan register allocator. This is roughly -O1 and it is deliberate. Inlining is off in debug builds (§13) and capped at 40 NIR instructions per callee across a .em boundary (§12).
- All compiler data structures are arena-allocated and never freed; the process exits instead.
- Strings interned once at lex time; identifiers compare as integers thereafter.
- No LLVM, and no external linker on the default path once the own linker lands (M2 for the easy case, M4 for dynamic imports; --linker=system remains for static archives, §13).

Parallelism

Scaling across cores is a design constraint on the compiler's architecture, not an optimisation applied afterwards. Several earlier decisions exist partly to serve it.

Phase 1 — lex and parse — is embarrassingly parallel. Because neper has no preprocessor, no textual includes, no macros and no order-dependent module scope (§2, §14), *every source file can be parsed without knowing anything about any other file*. There is no coordination to do: hand N files to N threads. This is structurally unavailable to C and C++, where headers make parsing context-dependent, and to languages whose macros must run before the parse completes.

Phase 2 — typecheck, NIR, codegen — parallelises at function granularity. Interfaces are built in `use` order, which exists because the `use` graph is a DAG (§2): a module's interface needs only those of the modules it uses, and a cycle would have no first module to build. Once interfaces exist, each function body is independent. Function-level rather than module-level work items matter on high-core-count machines, where a project may have fewer modules than the machine has threads.

Work-stealing, not static partitioning. A fixed thread pool with per-thread Chase-Lev dequeues. This is not just a load-balancing nicety: on hybrid CPUs — Intel P-cores and E-cores, Apple's performance and efficiency cores — an equal static split strands the whole build behind the slowest E-core. Work-stealing absorbs a 2–3× per-core speed difference with no special casing.

Per-thread arenas, and no shared allocator. This is where §8 pays off. Allocator contention is the classic reason compilers stop scaling; neper has no global allocator to contend on. Each worker owns its arena, and per-thread state is padded to cache-line boundaries to avoid false sharing.

Two genuinely shared structures: the string intern table and the comptime instantiation cache (§9). Both are sharded by hash prefix so contention is spread, and they are the only places in the compiler where threads meet on the hot path.

Hyperthreading: default to logical cores, but measure. SMT helps the branchy, pointer-chasing, memory-latency-bound work in the front end, and helps much less in register allocation and emission, which are throughput-bound. The default is one worker per logical core; `-j N` overrides it. We are not going to claim a number before there is something to benchmark.

Determinism is a hard requirement. Output must be byte-identical regardless of thread count or scheduling. No thread-order-dependent identifiers, no codegen that depends on hash iteration order, and diagnostics collected per module then sorted into source order before printing. A build that varies with `-j` is a build you cannot cache, diff, or reproduce.

Known serial points, listed because Amdahl's law is unforgiving and these are what will actually cap the speedup:

Serial section	Mitigation
Linking	The reason neper links its own executables (§13)
Interface construction for a dependency chain	Parse-only phase 1 shortens it; deep chains still serialise
Final executable write	Unavoidable

NUMA-aware thread pinning and per-node arenas are deferred; they matter only on large multi-socket machines.

16. Standard library: time

The coordinated next-contract standard-library changes in `stdlib-hardening.md` and D84 apply during M2.5 migration of delivered CPU surfaces. Their exact declarations are in `module-apis.md`: composable buffered I/O, common cancellation/deadline control, lossless JSON/data editing, handle-relative filesystem operations and bounded process supervision. H07 replaces legacy temporal error-detail transport in that revised checked profile. Until its versioned implementation lands, the archived M2 compiler retains its original ABI; do not treat these signatures as proof of current availability. Later crypto, networking, test-support and image/codecs retain their independent delivery gates. Ordinary parameter syntax and the §5 shadowing rules are unchanged. The module plan and actual toolchain capabilities have distinct authorities (SL11).

`e.time` allocates nothing, anywhere. Every type is a plain struct on the stack.

```
type Timestamp = struct { nanos: i64 } // wall clock, UTC, Unix epoch
type Instant   = struct { nanos: i64 } // monotonic, arbitrary epoch
type Duration  = struct { nanos: i64 }
type Date      = struct { year: i32, month: u8, day: u8 }
type Time      = struct { hour: u8, minute: u8, second: u8, nanos: u32 }
type Timer     = struct { started: Instant }
```

The three clock types are `i64` underneath, `Date` and `Time` are civil fields, `Timer` wraps one `Instant`, and all six are **distinct types**. Because there are no implicit conversions (§4), the compiler rejects adding two timestamps, passing an `Instant` where a `Timestamp` belongs, or mixing a duration with a point in time. The strictness already paid for elsewhere buys this for free.

`Instant` **and** `Timestamp` **are separate on purpose**. Elapsed time is measured with the monotonic clock; a wall clock can jump backwards under NTP, a DST change or a manual set, and measuring a duration with one is a classic production bug. Making them different types turns that into a compile error.

```
fn now() -> (Timestamp, err) // os.clock(.Wall)
fn monotonic() -> (Instant, err) // os.clock(.Monotonic)
fn timer_start() -> (Timer, err) // Timer{ started: try monotonic() }
fn timer_elapsed(t: Timer) -> Duration // since(t.started)
fn since(start: Instant) -> Duration // monotonic() - start; a clock that read once reads again

let start = try time.monotonic()
...
try io.printf["took {}us\n"](time.as_micros(time.since(start)))
```

Both direct clock reads and `timer_start` are fallible because `os.clock` (§5) is: the wrapper does not swallow an `err` the OS returned. `since` is not, because a clock that has already been read on this thread does not start failing; should the read fail regardless, `since` traps through `unreachable()` (§11) rather than return a value it did not measure. `timer_elapsed` has exactly the same post-first-read rule as `since`.

The rest of the surface, every function pure and allocation-free:

Function	Semantics
<code>fn timestamp_add(t: Timestamp, d: Duration) -> Timestamp</code> , <code>fn instant_add(t: Instant, d: Duration) -> Instant</code>	an ordinary <code>+</code> on the nanos: an overflow check in debug, wraps in release (§11)
<code>fn timestamp_diff(a: Timestamp, b: Timestamp) -> Duration</code> , <code>fn instant_diff(a: Instant, b: Instant) -> Duration</code>	<code>a - b</code>
<code>fn duration_add(a: Duration, b: Duration) -> Duration</code> , <code>fn duration_sub(a: Duration, b: Duration) -> Duration</code> , <code>fn duration_neg(d: Duration) -> Duration</code> , <code>fn duration_scale(d: Duration, n: i64) -> Duration</code>	duration with duration: <code>+</code> , <code>-</code> , unary <code>-</code> and <code>*</code> on the nanos, each an overflow check in debug and wrapping in release (§11)
<code>fn timestamp_cmp(a: Timestamp, b: Timestamp) -> i32</code>	<code>-1, 0, 1</code> ; likewise <code>instant_cmp</code> and <code>duration_cmp</code>
<code>fn days(n: i64) -> Duration</code> , <code>hours</code> , <code>minutes</code> , <code>seconds</code> , <code>millis</code> , <code>micros</code> , <code>nanos</code>	constructors
<code>fn as_days(d: Duration) -> i64</code> , <code>as_hours</code> , <code>as_minutes</code> , <code>as_seconds</code> , <code>as_millis</code> , <code>as_micros</code> , <code>as_nanos</code>	truncating toward zero
<code>fn to_date(t: Timestamp) -> Date</code> , <code>fn to_time(t: Timestamp) -> Time</code>	civil fields in UTC

Function	Semantics
<code>fn to_date_at(t: Timestamp, offset_minutes: i32) -> Date, to_time_at</code>	at an explicit zone offset
<code>fn from_civil(d: Date, t: Time) -> (Timestamp, err)</code>	the inverse; an invalid date is <code>time.Invalid</code>
<code>fn format_iso8601(t: Timestamp, buf: [u8]) -> str</code>	2026-09-04T12:00:00.000000000Z, 30 bytes; <code>buf.len < 30</code> is a bounds check
<code>fn parse_iso8601(s: str) -> (Timestamp, err)</code>	that form, or without the fraction; else <code>time.Invalid</code>

`e.time` declares one error, `Invalid`. The comparators and the arithmetic are spelled `<t>_<op>` — `timestamp_cmp`, `instant_add`, `duration_scale` — because that is §9's protocol convention (D52) and not merely a way around having no overloading (§14): the three clock types are distinct on purpose, and under this spelling `timestamp_cmp`, `instant_cmp` and `duration_cmp` are exactly what `e.data.sort` and every ordered container find when they look up `fn <t>_cmp` in the module that declares the type, where a bare `cmp` would resolve to nothing and leave a `Timestamp` unsortable. The constructors, accessors and conversions — `days`, `as_millis`, `to_date`, `format_iso8601` — carry no prefix: each exists once, over one type, and nothing looks one up by protocol.

`Date` is valid for years 1677 through 2262, months 1 through 12 and a day present in that proleptic-Gregorian month. `Time` requires hour 0–23, minute and second 0–59, and nanos below 1,000,000,000; leap seconds are not accepted. `offset_minutes` is in `[-1439, 1439]`. `from_civil` rejects a valid civil value whose nanosecond timestamp does not fit `i64`. Duration constructors and arithmetic use the ordinary debug- checked/release-wrapping integer rules stated in the table. ISO parsing accepts exactly `YYYY-MM-DDTHH:MM:SSZ` or that form with `.` followed by one through nine decimal digits, padded on the right to nanoseconds; no whitespace, lowercase `t/z`, offset or leap second is accepted. Years remain four digits because the representable `Timestamp` range lies inside 1677–2262.

Formatting needs no arena. A formatted date has a bounded length, so it goes in a caller-supplied stack buffer and the function returns a slice into it. Strings need an arena because their length is unbounded; dates never do.

```
var buf: [32]u8 = undef
let s = time.format_iso8601(now, buf[0..])
```

Civil conversion is pure arithmetic — `days-from-civil`, no lookup tables — so `to_date`, `to_time` and their inverses are usable inside `@gpu` — requiring `.Int8` for their `u8` fields and `.Int64` for the arithmetic (§10, Capabilities); `from_civil`'s `err` is an ordinary value in device code (§10). Reading a clock is an OS call — `time.now` and `time.monotonic` are `os.clock(.Wall)` and `os.clock(.Monotonic)` (§5) — so they are not.

UTC only

`e.time` handles UTC and nothing else. Time zones are an explicit offset in minutes:

```
let local = time.to_date_at(now, 120) // UTC+02:00
```

The IANA time zone database is megabytes of data plus file I/O plus a parser. None of that can live in a freestanding core library that allocates nothing, so a real `tz` database is a separate library for programs that need one.

Leap seconds are ignored, following Unix convention.

Accepted cost: "what is the local time in Europe/Sofia" is not answerable from `e.time` alone. That is the right trade for a systems language and the wrong one for business software, and it is a deliberate choice rather than an oversight.

17. Open questions

No unresolved semantic design question remains as of 2026-09-04. The concrete grammar is `docs/grammar.ebnf`; the public token, syntax, index, diagnostic, format, manifest, source-map and capability contracts are `docs/tooling.md`; stable codes are allocated in `docs/diagnostics.md`; the module architecture is `docs/modules.md`; the exact proposed toolchain APIs are `docs/module-apis.md`; and the normalized, machine-readable library plan is `docs/modules.json`. These companions are normative parts of this specification, not implementation notes.

Before compiler implementation begins, the documentation phase has four mechanical exit criteria: every code example parses under the grammar, every rejected example names a registered diagnostic, the `tests/conformance/` corpus described by `docs/tooling.md` §9 exists, and the initial multi-tokenizer generated-code benchmark is recorded. Those are

verification deliverables with fixed formats, not choices an implementer is allowed to make. Character count alone is never used as evidence of LLM token efficiency. `docs/general-purpose-verification.md` defines the broader workload corpus, model-generation metrics and release gates without adding language semantics.

Deliberately deferred rather than undecided, each with its rationale in place: runtime CPU dispatch and its `@cpu("...")` attribute (§13), ARM SVE (§13), NUMA-aware thread pinning (§15), auto-vectorisation (§4), static-archive linking (§13), cross-target test execution (§13), golden-file comparison in `e.test` (§13), a native Metal backend (§10, scheduled as M5), an argument-pack API for user code (§9), spreading a CPU-backend launch over several host threads (§10), a PDB type stream (§13), and an Android host — an `.Android` member of `target.Os` with its `os.android.e` (§5, §6), which the Vulkan backend would then reach (§10).

Not targeted at all, so that nobody reads their absence as an oversight: RISC-V, WebAssembly, 32-bit ARM, any big-endian ISA, iOS, DirectX, WebGPU, graphics pipelines, and Vulkan 1.1 devices (§10, D22: below the floor, not deferred). Each CPU ISA is a hand-written emitter and each GPU backend is an emitter plus a runtime; the list stays short on purpose.

Module architecture

docs/modules.md | 17305 bytes | sha256 4aa0a467c3321834

neper — module architecture

Status: normative next-contract plan, not an implementation-status report. `docs/spec.md` controls language semantics and signatures already fixed there. `modules.json` is the machine-readable authority for names, delivery tiers, layers, status, milestones, dependencies and blockers. `module-apis.md` fixes the exact proposed public surface of every toolchain module. All three views change together.

The adopted [standard-library hardening](#) (D84) defines composition, migration and executable acceptance. Delivered CPU surfaces migrate in M2.5; later modules remain proposals until their own tests and implementation exist. The catalogue currently has 128 modules: 33 core, 77 extended, 18 experimental.

1. Namespace policy

Prefix	Meaning	Distribution
<code>e.*</code>	Stable facilities owned by the language and guaranteed by the applicable toolchain version	Toolchain
<code>algo.*</code>	Pure algorithms over caller-owned data	Toolchain
<code>text.*</code>	Unicode and text processing	Toolchain
<code>crypto.*</code>	Pure cryptographic primitives; entropy is caller-supplied	Toolchain
<code>fmt.*</code>	Interchange-format parsers and writers	Toolchain
<code>gfx.*</code>	Pure geometry/paint/image values and GPU scene rendering	Toolchain
<code>ui.*</code>	Declarative, GPU-rendered application framework	Toolchain
<code>x.<owner>.<package>.*</code>	Optional, platform, vendor or externally versioned package	Separate source package

Namespaces describe ownership and domain, not whether code is implemented with an intrinsic. `e.*` availability begins at the module's delivery milestone; it does not put every module into the bootstrap.

Imports use spec §2's default final-segment qualifier or an explicit alias. Final segments are not globally reserved:

```
use e.data.map
use x.acme.collections.map as acme_map
```

External packages always include an owner. Each vendored package root contains a canonical `neper-package.json`. The legacy pre-pacman form records exact source and dependency pins; the M6 form in [pacman.md](#) records language/exports and dependency constraints, while source identity and resolved pins belong to registry/lock metadata. This migration must version the closed schema explicitly; the two forms are not interchangeable just because earlier drafts both called them version 1. Neither is a second compiler resolver.

2. Delivery tiers

Delivery tier is independent of dependency layer. The ordered `tiers` arrays in `modules.json` are authoritative, contain every toolchain module exactly once and express product commitment rather than import legality:

Tier	Compatibility and delivery contract
core	Required for the first stable CPU release; numbered milestones and explicitly documented cross-cutting gates both apply
extended	Toolchain-owned and stable once delivered, but independently deliverable and not a first-stable gate
experimental	An API proposal available for evaluation with no compatibility promise

Core is deliberately small: language foundations; ordinary collections; hashing, randomness, UUIDs and bit sets; basic UTF-8; shared cancellation; host I/O/process/thread/time services; testing; and JSON, CSV and INI. Extended contains specialized containers and algorithms, Unicode, cryptography, GPU support, application services, networking and advanced formats. Pure geometry/paint/image values and PNG/JPEG/WebP codecs form one extended stabilization cohort (`image-codec-conformance`); GPU rendering and UI do not gate it. Experimental contains workload-dependent collection conveniences, GPU tensor composition, and the declarative GPU UI proposal. Promotion changes `modules.json`, adds the applicable conformance workload and records a compatibility decision. Experimental modules never satisfy a dependency of a core or extended module. This conservative rule protects public-type stability transitively, not only import legality.

`text.utf8` and `e.cancel` are core additions delivered through M2.5's cross-cutting library gate, with `schedule:"later"`, `milestone:null` rather than inventing an unvalidated Mn value. Their gate is specified in `stdlib-hardening.md`; this does not change the preserved M2 completion criteria or imply they are already available.

The catalogue in `module-apis.md` remains dependency-layer ordered so a reader can understand implementation direction. Product UI and generated documentation present the tier order from `modules.json`: core, extended, then experimental.

3. Dependency layers

Layer	Role
0	Language foundations and compiler intrinsics
1	Pure byte, string and path foundations
2	Containers and pure algorithm/text/crypto domains
3	Host platform boundary
4	Host services
5	Language-owned runtimes
6	Application facilities, networking and interchange formats

`modules.json` lists every direct edge. A module may use only a layer allowed by its layer's `may_depend_on`, and a same-layer dependency is legal only when listed. This prevents the cycles that a loose “same row or lower” diagram cannot catch.

The JSON arrays carry presentation order, not identity or priority. Consumers key layers by `id` and modules/packages by `name`; they reject duplicate keys, unknown dependency names, disallowed layer edges, dependency cycles, or any mismatch between module names and the level-three headings in the `api_catalog` file. A `blocked_by` value is either a toolchain module name or a stable kebab-case design identifier; only `direct_dependencies` defines graph edges. A package reservation is never inserted into the toolchain module graph.

Global constraints:

- `e.os` is the only ordinary toolchain module declaring host-platform `externs`. Reviewed GPU-driver loading in `e.gpu` and explicit `x.*` bindings are the two exceptions.
- Every allocation has a documented source: caller arena, retained borrowed arena, caller-provided storage, bounded private scratch or an explicit driver exception. Classify allocation-free calls separately. No general global allocator or pre-main initializer is introduced; compiler-owned test-report state and existing legacy platform-detail state are explicit exceptions, not invisible guarantees.
- Pure modules never read files, environment, clocks, entropy or network state.
- Portable `err` values remain the default propagation surface. Host modules that expose native detail identify its capture time and ownership. The old `last_error_detail` path is thread-local and temporal despite returning a struct; H07 replaces it in the revised checked API. Primary errors, cleanup failures and partial progress must remain distinguishable; do not imply the migration is done.
- Mutating slice/container operations use `_in_place`, except stateful resource operations whose receiver already makes mutation explicit.

- A CI source check compares column-zero `use` declarations with the exact edges in `modules.json`; the compiler independently enforces an acyclic import graph.
- `python scripts/check_module_plan.py` checks the current plan/catalogue's graph, tiers, naming and import-qualifier consistency. It supplements, not replaces, the M2.5 real parser/resolver and executable API tests.

4. Canonical grouping

Layer 0 — foundations

- `e.mem`: arenas, layout and casts.
- `e.meta`: compile-time type shape.
- `e.math`: scalar floating operations.
- `e.simd`: vectors and masks.
- `e.atomic`: atomic operations and ordering.

Layer 1 — pure primitives

- `e.bytes`: endian and bit operations.
- `e.str`: byte strings, ASCII search/edit/classification, builders, parsing and formatting.
- `e.path`: platform-explicit path manipulation with no filesystem access.

Layer 2 — containers and pure domains

Containers are one module per data structure rather than the former `e.data` grab bag:

- `e.data.list`, `e.data.deque`, `e.data.stack`, `e.data.queue`, `e.data.linked`, `e.data.ring`, `e.data.heap`, `e.data.tree`.
- `e.data.map`, `e.data.sort`, `e.data.iter`, `e.data.disjoint_set`, `e.data.graph`, `e.data.slot_map`.

Pure algorithm domains are:

- `algo.rand`, `algo.uuid`, `algo.hash`, `algo.graph`, `algo.stat`, `algo.bitset`, `algo.complex`, `algo.decimal`, `algo.bignum`, `algo.deflate`.
- `algo.linalg.matrix`, `algo.linalg.tensor`.
- `text.encoding`, `text.utf8`, `text.unicode`, `text.normalize`, `text.collate`, `text.regex`.
- `crypto.hash`, `crypto.mac`, `crypto.kdf`, `crypto.aead`, `crypto.sign`, `crypto.kx`, `crypto.random`; `crypto.x509` composes certificates and validation over the format layer.
- `gfx.geometry`, `gfx.paint`, `gfx.image`; `ui.style` and `ui.layout` are pure value and constraint engines despite their application-facing names.
- `text.shape`, `text.layout`: deterministic font shaping and visual text layout over caller-provided font data.

This separation keeps non-cryptographic table/checksum hashes out of the security namespace, decomposes the former `algo.text`, and prevents matrices and tensors from becoming unrelated top-level buckets.

Layer 3 — platform boundary

- `e.os`: the reviewed minimum for files, processes, VM, clocks, threads, wait/wake, sockets, polling, dynamic loading, environment and entropy. Higher modules add a primitive here before using a platform facility; they never declare their own host extern.

Layer 4 — host services

- `e.cancel`, `e.io`, `text.io`, `e.fs`, `e.fs.mmap`, `e.fs.watch`, `e.proc`, `e.thread`, `e.sync`, `e.channel`, `e.concurrent.queue`, `e.concurrent.map`, `e.task`, `e.time`, `e.time.calendar`, `e.tz`.

Channels are separate from locks because they allocate and impose a higher-level concurrency protocol. `e.task` is a bounded, cooperative-cancellation pool rather than compiler-generated async syntax. Calendar arithmetic is pure despite living at the host-service layer beside the clocks it commonly consumes. `e.tz` carries the toolchain-pinned IANA database and also accepts explicit compatible data. Timers live in `e.time`, not `e.debug`.

Layer 5 — language-owned runtimes

- `e.gpu`: compiler-owned GPU model and host runtime, including bounded device discovery, backend-scoped exact-ID selection, device descriptors and explicit per-device queues/buffers (spec §10; CPU/Vulkan M3, CUDA M4).
- `e.asset`: immutable, linker-generated lookup over manifest-declared executable assets.
- `e.test`: assertion functions; discovery and isolation stay in the compiler.
- `e.test.support`: explicit fake clocks, scripted I/O and cooperative schedule drivers, not hooks into ambient OS state or a general deterministic thread runtime.
- `e.test.coverage` and `e.test.fuzz`: compiler-instrumented coverage plus deterministic corpus mutation and minimization.
- `e.debug`: backtrace capture and symbolization only.

Layer 6 — application and protocols

- `e.metrics`, `e.log`, `e.cli`, `e.async`, `e.async.io`, `text.locale`, `text.template`.
- `e.net`, `e.net.tls`, `e.net.http`, `e.net.ws`.
- `e.db`: generic SQL connections, transactions, prepared statements and streaming row readers; concrete drivers remain owner-qualified packages.
- `e.gpu.tensor`: explicit GPU tensor operations over pure `algo.linalg.tensor` views.
- `gfx.scene`: renderer-neutral display lists, retained scenes and GPU composition.
- `ui.asset`: deterministic scale/theme/locale variant selection, fonts and bounded decoded-image/GPU texture caching over `e.asset`.
- `ui.window`, `ui.input`, `ui.widget`, `ui.animation`, `ui.accessibility`, `ui.testing`, `ui.app`: the experimental declarative GPU application framework.
- `fmt.json`, `fmt.csv`, `fmt.ini`, `fmt.uri`, `fmt.mime`, `fmt.asn1`, `fmt.pem`, `fmt.multipart`, `fmt.mail`, `fmt.quoted_printable`, `fmt.gzip`, `fmt.zstd`, `fmt.bzip2`, `fmt.lzw`, `fmt.zlib`, `fmt.zip`, `fmt.tar`, `fmt.yaml`, `fmt.xml`, `fmt.html`, `fmt.html.template`, `fmt.png`, `fmt.jpeg`, `fmt.webp`, `fmt.bson`, `fmt.msgpack`, `fmt.protobuf`.

HTTP/1.1 is the initial `e.net.http` surface. HTTP/2 requires a separate future proposal because HPACK and multiplexed connection state are not an incremental flag. `e.async` is an explicit readiness/completion loop, not futures or compiler-generated coroutines; `e.async.io` composes typed, cancellation-aware operations over it. Streaming response bodies and bounded SSE parsing are part of `e.net.http`; they need no new namespace or HTTP/2 implementation. `e.path` owns pure bounded glob matching; `fmt.json` owns lossless numbers, Pointer and transactional in-memory Patch. `e.proc` owns bounded child supervision, and `e.fs` exposes handle-relative root operations over reviewed `e.os` primitives. Exact semantics and delivery tests are in `stdlib-hardening.md`; none permits automatic retries of partial side effects.

`fmt.html` is distinct from `fmt.xml`: it implements HTML error recovery and the WHATWG tree-construction algorithm into a bounded, arena-owned tree. Like every format module it receives bytes or a reader and never opens a file or fetches a resource itself.

`fmt.html.template` adds context-sensitive escaping over `text.template`; it is separate from parsing because template execution generates a stream rather than an HTML tree. Image codecs decode into caller-owned `gfx.image` pixels. Certificate validation is split the same way: `fmt.asn1` and `fmt.pem` own encodings, while `crypto.x509` owns chain and identity policy used by `e.net.tls`.

The UI family is specified in `ui-framework.md`. Declarative widgets are immutable frame-arena descriptions, not runtime objects. Reconciliation stores persistent elements and state behind generation-checked identifiers; layout produces retained render nodes, and `gfx.scene` compiles them into explicit GPU work. Native windows, input, clipboard, IME and accessibility enter only through reviewed `e.os` primitives. There is no garbage collector, global widget registry, reflection-based property system or hidden allocation.

5. Optional package reservations

Optional package names identify their actual external owner; `x.neper.*` is forbidden because toolchain-owned facilities use the domain namespaces above:

- `x.khronos.{vulkan,opengl}`, `x.google.skia`, `x.nvidia.cuda.{cublas,cudnn}`, `x.apple.metal`.

- `x.sqlite.sqlite`, `x.oracle.mysql`, `x.postgresql.libpq`: concrete database drivers implementing `e.db`, named for the SQLite project, Oracle-owned MySQL, and PostgreSQL's official `libpq` client library respectively.

A package has no promised declarations until its own versioned specification pins its upstream ABI/data version and enumerates the complete surface. Compiler backends such as ROCm, OpenCL or WebGPU are not library package reservations. URI, MIME, locale, TLS, compression and archives are toolchain modules rather than pseudo-external packages; their standard/data snapshot is tied to the toolchain version and exposed where reproducibility requires it.

6. Protocol dependencies

Protocol	Consumers
<code><t>_hash</code> , <code><t>_eq</code>	<code>e.data.map</code> , equality assertions and document nodes
<code><t>_cmp</code>	sorting, heaps, ordered trees and collation
<code><t>_format</code>	formatted output, logging and writers
<code><i>_next</code>	in-memory containers and iterator adapters
<code><i>_next_err</code>	filesystem, network and streaming-format readers
<code>e.meta</code> shape queries	structural encoders/decoders and CLI configuration

These are direct monomorphized calls fixed by spec §9, not runtime interfaces, registration or RTTI.

7. API ownership

- `surface:"spec"` means `spec.md` already fixes the declarations' semantics; `surface:"partial"` means the spec fixes only the subset it names; and `surface:"planned"` means `module-apis.md` is the exact proposal to freeze before implementation of that milestone. `surface:"source"` means an implementation exists and the same revision verifies its complete public declarations against `module-apis.md`.
- `schedule:"scheduled"` requires a non-null `milestone`; every other schedule has `milestone:null`. A blocker describes a prerequisite and does not alter that normalized delivery state.
- `module-apis.md` lists every proposed public type, error, constant and function for layers 0–6. A name absent there is not planned.
- `spec` surfaces change only through `decisions.md`. `planned` surfaces are frozen before implementation of their milestone begins.
- Optional `x.*` packages own separate specifications because an exact binding cannot exist before an upstream version is selected.
- Removing or renaming a module or declaration is an ecosystem compatibility change. Adding a declaration also requires protocol, import-qualifier and local/parameter shadowing impact analysis. No blanket non-breaking claim follows from addition.
- Installed availability is a separate compiler-derived versioned inventory, with per-symbol/target implementation, stability, contract and test evidence. Never infer availability from `surface` or `milestone` alone (H11/H18, SL11).

Module API catalogue

docs/module-apis.md | 164529 bytes | sha256 c583bb4fdc983955

neper module API catalogue

Status: exact public-surface proposal for the toolchain modules in `modules.md`. Delivery commitment and presentation order come from `modules.json`'s core, extended and experimental tiers; this file remains dependency-layer ordered. This is the next-contract design, not an installed-toolchain availability report. The coordinated changes in `stdlib-hardening.md` migrate delivered CPU APIs during M2.5; later facilities retain their stated delivery gates. Semantic details in `spec.md` remain authoritative; this file fixes names, public value shapes and signatures. A module implements no additional public declaration unless this file is amended.

Notation follows neper source except where a compiler intrinsic's signature uses the explicit dependent metavariables documented with that intrinsic. `type X = struct { state: *void }` is the uniform one-word handle for arena-allocated implementation state. Such a value is physically copyable, but only one copy may remain live for operations that consume or mutate the state; copying never creates a second owner. Callback context is always explicit; there are no closures. Read-only slices are `[]const T`.

Qualified names in signatures resolve through that module's `direct_dependencies` in `modules.json`. They use the dependency's final segment except `linalg_tensor`, the explicit alias for `algo.linalg.tensor`; `ui.widget` additionally uses `layout` for `text.layout` and `ui_layout` for `ui.layout`; `e.async.io` uses `cancel_api` for `e.cancel` because it declares `cancel`. Imports themselves are not public declarations. The catalogue's declaration fragments intentionally omit function bodies and are not standalone modules.

The extraction contract is intentionally mechanical: each module occurs exactly once as a level-three code heading, and the first and only neper fence before the next level-three heading contains all of that module's public declarations in canonical order. Prose after the fence adds constraints but no declaration. A harness locates a surface by exact heading; it never scans the whole document for a short name.

1. Language foundations

e.mem

```
type Arena = struct { base: *u8, cap: usize, off: usize }
type Stats = struct { used: usize, capacity: usize }
error Exhausted

fn arena_from(buf: []u8) -> Arena
fn alloc[T: type](a: *Arena, n: usize) -> ([T, err)
fn mark(a: *Arena) -> usize
fn reset(a: *Arena, m: usize)
fn copy[T: type](dst: [T, src: []const T)
fn eq[T: type](x: []const T, y: []const T) -> bool
fn cast[P: type, Q: type](p: Q) -> P
fn bitcast[T: type, U: type](x: U) -> T
fn size_of[T: type]() -> usize
fn align_of[T: type]() -> usize
fn stats(a: *const Arena) -> Stats
```

For `cast`, `P` and `Q` must be pointer types; callers spell `P` and inference fills the trailing `Q` from `p`. For `bitcast`, callers spell `T` and inference fills `U` from `x`; the representation restrictions are those in `spec §4` and `§8`.

e.meta

```
type Field = struct { name: str, ty: type, offset: usize, size: usize }
type Member = struct { name: str, value: u64 }
type TypeKind = enum u8 { Int, Float, Bool, Err, Pointer, Slice, Array, Struct, Enum, UnionEnum, Vec }

fn fields[T: type]() -> []const Field
fn members[E: type]() -> []const Member
fn type_name[T: type]() -> str
fn kind[T: type]() -> TypeKind
```

```

fn element_type[T: type]() -> type
fn array_len[T: type]() -> usize
fn backing_type[E: type]() -> type
fn get[FIELD: Field, T: type](v: *const T) -> FIELD.ty
fn set[FIELD: Field, T: type](v: *T, x: FIELD.ty)

```

Field and Member are comptime-only types exactly as specified by spec §9.

e.math

Every function is generic over F, which must be f16, bf16, f32 or f64.

```

fn sqrt[F: type](x: F) -> F
fn rsqrt[F: type](x: F) -> F
fn fma[F: type](a: F, b: F, c: F) -> F
fn abs[F: type](x: F) -> F
fn min[F: type](a: F, b: F) -> F
fn max[F: type](a: F, b: F) -> F
fn floor[F: type](x: F) -> F
fn ceil[F: type](x: F) -> F
fn round[F: type](x: F) -> F
fn trunc[F: type](x: F) -> F
fn copysign[F: type](x: F, y: F) -> F
fn sin[F: type](x: F) -> F
fn cos[F: type](x: F) -> F
fn tan[F: type](x: F) -> F
fn asin[F: type](x: F) -> F
fn acos[F: type](x: F) -> F
fn atan[F: type](x: F) -> F
fn atan2[F: type](y: F, x: F) -> F
fn exp[F: type](x: F) -> F
fn exp2[F: type](x: F) -> F
fn log[F: type](x: F) -> F
fn log2[F: type](x: F) -> F
fn log10[F: type](x: F) -> F
fn pow[F: type](x: F, y: F) -> F

```

e.simd

V is a legal Vec[T,N]; M is its Mask[T,N]; I is an integer vector with the same lane count where required.

```

fn splat[V: type](x: T) -> V
fn load[V: type](s: []const T, off: usize) -> V
fn store[V: type](s: []T, off: usize, v: V)
fn load_aligned[V: type](s: []const T, off: usize) -> V
fn store_aligned[V: type](s: []T, off: usize, v: V)
fn load_masked[V: type](s: []const T, off: usize, m: M) -> V
fn store_masked[V: type](s: []T, off: usize, m: M, v: V)
fn gather[V: type](s: []const T, indices: Vec[u32, N]) -> V
fn shuffle[V: type, IDX: [N]u8](a: V, b: V) -> V
fn cmp_eq[V: type](a: V, b: V) -> M
fn cmp_ne[V: type](a: V, b: V) -> M
fn cmp_lt[V: type](a: V, b: V) -> M
fn cmp_le[V: type](a: V, b: V) -> M
fn cmp_gt[V: type](a: V, b: V) -> M
fn cmp_ge[V: type](a: V, b: V) -> M
fn select[V: type](m: M, yes: V, no: V) -> V
fn any[V: type](m: M) -> bool
fn all[V: type](m: M) -> bool
fn bits[V: type](m: M) -> u64
fn mask[V: type](packed: u64) -> M
fn reduce_add[V: type](v: V) -> T
fn reduce_min[V: type](v: V) -> T
fn reduce_max[V: type](v: V) -> T
fn convert[V: type, W: type](v: V) -> W
fn fma[V: type](a: V, b: V, c: V) -> V
fn pdep(x: u64, bit_mask: u64) -> u64
fn pext(x: u64, bit_mask: u64) -> u64

```

Here V = Vec[T,N], M = Mask[T,N], and w is another vector with N lanes. T, N and M in these signatures are dependent names inferred from V, matching the notation already used by spec §4; they add no runtime reflection.

e.atomic

```

type Ordering = enum u8 { Relaxed, Acquire, Release, AcqRel, SeqCst }

fn init[T: type](v: T) -> Atomic[T]
fn load[T: type](p: *Atomic[T], o: Ordering) -> T
fn store[T: type](p: *Atomic[T], v: T, o: Ordering)
fn xchg[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn cas[T: type](p: *Atomic[T], expected: T, desired: T, success: Ordering, failure: Ordering) -> (bool, T)
fn add[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn sub[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn and[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn or[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn xor[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn min[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn max[T: type](p: *Atomic[T], v: T, o: Ordering) -> T
fn fence(o: Ordering)

```

2. Pure foundations

e.bytes

```

type Endian = enum u8 { Little, Big }
type Base64Alphabet = enum u8 { Standard, Url }
type Base32Alphabet = enum u8 { Standard, Hex }
type Base85Alphabet = enum u8 { Ascii85, Z85 }
type Reader = struct { data: []const u8, off: usize }
type Writer = struct { data: []u8, off: usize }
error End
error Invalid
error TooLarge

fn reader(data: []const u8) -> Reader
fn writer(data: []u8) -> Writer
fn remaining_reader(r: *const Reader) -> usize
fn remaining_writer(w: *const Writer) -> usize
fn skip(r: *Reader, n: usize) -> err
fn read[T: type](r: *Reader, endian: Endian) -> (T, err)
fn write[T: type](w: *Writer, v: T, endian: Endian) -> err
fn read_bytes(r: *Reader, n: usize) -> ([]const u8, err)
fn write_bytes(w: *Writer, src: []const u8) -> err
fn load[T: type](src: []const u8, off: usize, endian: Endian) -> (T, err)
fn store[T: type](dst: []u8, off: usize, v: T, endian: Endian) -> err
fn reverse_in_place(data: []u8)
fn rotate_left[T: type](v: T, n: u32) -> T
fn rotate_right[T: type](v: T, n: u32) -> T
fn count_ones[T: type](v: T) -> u32
fn leading_zeros[T: type](v: T) -> u32
fn trailing_zeros[T: type](v: T) -> u32
fn base64_encoded_len(n: usize, padded: bool) -> (usize, err)
fn base64_encode(dst: []u8, src: []const u8, alphabet: Base64Alphabet, padded: bool) -> (str, err)
fn base64_decode(dst: []u8, src: str, alphabet: Base64Alphabet) -> ([]u8, err)
fn base32_encoded_len(n: usize, padded: bool) -> (usize, err)
fn base32_encode(dst: []u8, src: []const u8, alphabet: Base32Alphabet, padded: bool) -> (str, err)
fn base32_decode(dst: []u8, src: str, alphabet: Base32Alphabet) -> ([]u8, err)
fn base85_encoded_len(n: usize, alphabet: Base85Alphabet) -> (usize, err)
fn base85_encode(dst: []u8, src: []const u8, alphabet: Base85Alphabet) -> (str, err)
fn base85_decode(dst: []u8, src: str, alphabet: Base85Alphabet) -> ([]u8, err)

```

Generic numeric operations accept integer and float primitives only; bit operations accept unsigned integers only. Base32 accepts the RFC 4648 standard and extended-hex alphabets with optional canonical padding. Base85 supports unframed ASCII85 and Z85; Z85 rejects input outside its four-byte or five-character quantum.

e.str

```

type Sink = struct { ctx: *void, write: fn(ctx: *void, bytes: []const u8) -> err }
type Builder = struct { arena: *mem.Arena, start: usize, len: usize, reserved: usize, sink: Sink, flushing: bool }
type Split = struct { source: str, separator: str, off: usize, finished: bool }
error NotOnTop

```

```

error BadNumber
error InvalidSeparator

fn builder(a: *mem.Arena, cap: usize) -> (Builder, err)
fn builder_to(a: *mem.Arena, cap: usize, sink: Sink) -> (Builder, err)
fn done(b: *Builder) -> str
fn concat(a: *mem.Arena, x: str, y: str) -> (str, err)
fn join(a: *mem.Arena, parts: []const str, sep: str) -> (str, err)
fn eq(x: str, y: str) -> bool
fn format[FMT: str](a: *mem.Arena, args: ...) -> (str, err)
fn parse_i64(s: str) -> (i64, err)
fn parse_u64(s: str) -> (u64, err)
fn parse_f32(s: str) -> (f32, err)
fn parse_f64(s: str) -> (f64, err)
fn parse_i64_radix(s: str, radix: u8) -> (i64, err)
fn parse_u64_radix(s: str, radix: u8) -> (u64, err)
fn compare(x: str, y: str) -> i32
fn compare_ascii_fold(x: str, y: str) -> i32
fn starts_with(s: str, prefix: str) -> bool
fn ends_with(s: str, suffix: str) -> bool
fn contains(s: str, needle: str) -> bool
fn find(s: str, needle: str) -> (usize, bool)
fn find_from(s: str, needle: str, start: usize) -> (usize, bool)
fn rfind(s: str, needle: str) -> (usize, bool)
fn count(s: str, needle: str) -> usize
fn trim(s: str) -> str
fn trim_start(s: str) -> str
fn trim_end(s: str) -> str
fn trim_bytes(s: str, bytes: str) -> str
fn split_once(s: str, separator: str) -> (str, str, bool)
fn split(s: str, separator: str) -> (Split, err)
fn split_next(it: *Split) -> (str, bool)
fn lines(s: str) -> Split
fn replace(a: *mem.Arena, s: str, needle: str, replacement: str) -> (str, err)
fn repeat(a: *mem.Arena, s: str, repeat_count: usize) -> (str, err)
fn ascii_lower_in_place(s: []u8)
fn ascii_upper_in_place(s: []u8)
fn is_ascii_space(b: u8) -> bool
fn is_ascii_digit(b: u8) -> bool
fn is_ascii_alpha(b: u8) -> bool
fn is_ascii_alnum(b: u8) -> bool
fn push(b: *Builder, s: str) -> err
fn push_byte(b: *Builder, v: u8) -> err
fn push_bool(b: *Builder, v: bool) -> err
fn push_err(b: *Builder, v: err) -> err
fn push_i8(b: *Builder, v: i8) -> err
fn push_i16(b: *Builder, v: i16) -> err
fn push_i32(b: *Builder, v: i32) -> err
fn push_i64(b: *Builder, v: i64) -> err
fn push_isize(b: *Builder, v: isize) -> err
fn push_u8(b: *Builder, v: u8) -> err
fn push_u16(b: *Builder, v: u16) -> err
fn push_u32(b: *Builder, v: u32) -> err
fn push_u64(b: *Builder, v: u64) -> err
fn push_usize(b: *Builder, v: usize) -> err
fn push_hex_u32(b: *Builder, v: u32) -> err
fn push_hex_u64(b: *Builder, v: u64) -> err
fn push_bin_u32(b: *Builder, v: u32) -> err
fn push_bin_u64(b: *Builder, v: u64) -> err
fn push_f32(b: *Builder, v: f32) -> err
fn push_f64(b: *Builder, v: f64) -> err
fn push_f32_fixed(b: *Builder, v: f32, precision: u8) -> err
fn push_f64_fixed(b: *Builder, v: f64, precision: u8) -> err

```

All search indices and slices are byte offsets. The trim functions without an explicit byte set remove ASCII whitespace only; Unicode whitespace and case mapping belong to `text.unicode`. An empty `needle` is found at the requested valid starting offset, `rfind` returns `s.len`, and `count` returns `s.len + 1`; replacement follows the same non-overlapping boundary rule. `split` returns `InvalidSeparator` for an empty separator and otherwise preserves empty fields. `lines` recognizes LF and CRLF, removes the terminator and does not synthesize a final empty line. Radices are in 2..36.

e.path

Paths are pure strings and always name the convention being applied; host behavior is never inferred inside this module.

```
type Style = enum u8 { Posix, Windows }
type Parts = struct { root: str, dir: str, base: str, stem: str, ext: str }
error Invalid

fn separator(style: Style) -> u8
fn is_absolute(path: str, style: Style) -> bool
fn split(path: str, style: Style) -> Parts
fn basename(path: str, style: Style) -> str
fn dirname(path: str, style: Style) -> str
fn extension(path: str, style: Style) -> str
fn stem(path: str, style: Style) -> str
fn join(a: *mem.Arena, parts: []const str, style: Style) -> (str, err)
fn normalize(a: *mem.Arena, path: str, style: Style) -> (str, err)
fn relative(a: *mem.Arena, base: str, target_path: str, style: Style) -> (str, err)
fn replace_extension(a: *mem.Arena, path: str, ext: str, style: Style) -> (str, err)
type GlobOptions = struct { state: *const void }
type GlobOptions = struct { style: Style, case_sensitive: bool, max_pattern_bytes: usize, max_steps: usize }
error TooLarge

fn glob(a: *mem.Arena, pattern: str, options: GlobOptions) -> (Glob, err)
fn glob_match(pattern: *const Glob, path: str) -> (bool, err)
```

Globs are pure matching over relative paths, not filesystem traversal. * and ? match within a component; a whole ** component matches zero or more components. Bracket classes support ASCII ranges and ! negation; malformed patterns are Invalid. Pattern separators are /; input separators follow the explicit Style. Matching is byte-exact unless ASCII case folding is requested, never host-locale dependent. Literal dotfiles are not implicitly excluded. Parent/absolute paths are Invalid; matching proves no filesystem containment. Work-limit exhaustion is TooLarge, not false. Ignore-file precedence/negation is application policy, not an implicit glob rule.

3. Containers and traversal

e.data.list

```
type List[T: type] = struct { items: []T, len: usize, arena: *mem.Arena }
type Iter[T: type] = struct { items: []const T, index: usize }

fn init[T: type](a: *mem.Arena, capacity: usize) -> (List[T], err)
fn from_slice[T: type](a: *mem.Arena, src: []const T) -> (List[T], err)
fn slice[T: type](l: *List[T]) -> []T
fn slice_const[T: type](l: *const List[T]) -> []const T
fn reserve[T: type](l: *List[T], capacity: usize) -> err
fn push[T: type](l: *List[T], v: T) -> err
fn pop[T: type](l: *List[T]) -> (T, bool)
fn insert[T: type](l: *List[T], index: usize, v: T) -> err
fn remove[T: type](l: *List[T], index: usize) -> T
fn clear[T: type](l: *List[T])
fn iter[T: type](l: *const List[T]) -> Iter[T]
fn iter_next[T: type](it: *Iter[T]) -> (T, bool)
```

e.data.deque

```
type Deque[T: type] = struct { items: []T, head: usize, len: usize, arena: *mem.Arena }
type Iter[T: type] = struct { deque: *const Deque[T], index: usize }

fn init[T: type](a: *mem.Arena, capacity: usize) -> (Deque[T], err)
fn len[T: type](d: *const Deque[T]) -> usize
fn reserve[T: type](d: *Deque[T], capacity: usize) -> err
fn push_front[T: type](d: *Deque[T], v: T) -> err
fn push_back[T: type](d: *Deque[T], v: T) -> err
fn pop_front[T: type](d: *Deque[T]) -> (T, bool)
fn pop_back[T: type](d: *Deque[T]) -> (T, bool)
fn get[T: type](d: *const Deque[T], index: usize) -> T
fn clear[T: type](d: *Deque[T])
fn iter[T: type](d: *const Deque[T]) -> Iter[T]
```

```
fn iter_next[T: type](it: *Iter[T]) -> (T, bool)
```

e.data.stack

```
type Stack[T: type] = struct { items: list.List[T] }
type Iter[T: type] = struct { stack: *const Stack[T], remaining: usize }

fn init[T: type](a: *mem.Arena, capacity: usize) -> (Stack[T], err)
fn len[T: type](s: *const Stack[T]) -> usize
fn reserve[T: type](s: *Stack[T], capacity: usize) -> err
fn push[T: type](s: *Stack[T], value: T) -> err
fn peek[T: type](s: *const Stack[T]) -> (T, bool)
fn pop[T: type](s: *Stack[T]) -> (T, bool)
fn clear[T: type](s: *Stack[T])
fn iter[T: type](s: *const Stack[T]) -> Iter[T]
fn iter_next[T: type](it: *Iter[T]) -> (T, bool)
```

Iteration is LIFO order and does not mutate the stack.

e.data.queue

```
type Queue[T: type] = struct { items: deque.Deque[T] }
type Iter[T: type] = struct { queue: *const Queue[T], index: usize }

fn init[T: type](a: *mem.Arena, capacity: usize) -> (Queue[T], err)
fn len[T: type](q: *const Queue[T]) -> usize
fn reserve[T: type](q: *Queue[T], capacity: usize) -> err
fn enqueue[T: type](q: *Queue[T], value: T) -> err
fn peek[T: type](q: *const Queue[T]) -> (T, bool)
fn dequeue[T: type](q: *Queue[T]) -> (T, bool)
fn clear[T: type](q: *Queue[T])
fn iter[T: type](q: *const Queue[T]) -> Iter[T]
fn iter_next[T: type](it: *Iter[T]) -> (T, bool)
```

Iteration is FIFO order and does not mutate the queue.

e.data.linked

```
type NodeId = u32
const NONE: NodeId = 4294967295
type Node[T: type] = struct { value: T, previous: NodeId, next: NodeId, live: bool }
type List[T: type] = struct { nodes: list.List[Node[T]], first: NodeId, last: NodeId, len: usize }
type Iter[T: type] = struct { list: *const List[T], next: NodeId }
error InvalidNode
error TooLarge

fn init[T: type](a: *mem.Arena, capacity: usize) -> (List[T], err)
fn len[T: type](l: *const List[T]) -> usize
fn first[T: type](l: *const List[T]) -> (NodeId, bool)
fn last[T: type](l: *const List[T]) -> (NodeId, bool)
fn node[T: type](l: *const List[T], id: NodeId) -> (*const Node[T], err)
fn push_front[T: type](l: *List[T], value: T) -> (NodeId, err)
fn push_back[T: type](l: *List[T], value: T) -> (NodeId, err)
fn insert_before[T: type](l: *List[T], at: NodeId, value: T) -> (NodeId, err)
fn insert_after[T: type](l: *List[T], at: NodeId, value: T) -> (NodeId, err)
fn remove[T: type](l: *List[T], id: NodeId) -> (T, err)
fn clear[T: type](l: *List[T])
fn iter[T: type](l: *const List[T]) -> Iter[T]
fn iter_next[T: type](it: *Iter[T]) -> (T, bool)
```

Node identifiers remain stable across insertions and removals because removed slots are not reused. A removed identifier and every identifier issued before `clear` returns `InvalidNode`. Insertion returns `TooLarge` before a node identifier would overflow. The nodes and list borrow the arena for their lifetime.

e.data.ring

```
type Ring[T: type] = struct { items: []T, head: usize, len: usize }
type Iter[T: type] = struct { ring: *const Ring[T], index: usize }

fn init[T: type](storage: []T) -> Ring[T]
fn len[T: type](r: *const Ring[T]) -> usize
fn capacity[T: type](r: *const Ring[T]) -> usize
```

```

fn push[T: type](r: *Ring[T], v: T) -> bool
fn push_overwrite[T: type](r: *Ring[T], v: T) -> (T, bool)
fn pop[T: type](r: *Ring[T]) -> (T, bool)
fn peek[T: type](r: *const Ring[T]) -> (T, bool)
fn clear[T: type](r: *Ring[T])
fn iter[T: type](r: *const Ring[T]) -> Iter[T]
fn iter_next[T: type](it: *Iter[T]) -> (T, bool)

```

e.data.map

```

type Map[K: type, V: type] = struct { state: *void }
type Set[K: type] = struct { map: Map[K, bool] }
type Iter[K: type, V: type] = struct { state: *const void, slot: usize }
type SetIter[K: type] = struct { inner: Iter[K, bool] }

fn init[K: type, V: type](a: *mem.Arena, capacity: usize) -> (Map[K, V], err)
fn len[K: type, V: type](m: *const Map[K, V]) -> usize
fn reserve[K: type, V: type](m: *Map[K, V], capacity: usize) -> err
fn put[K: type, V: type](m: *Map[K, V], key: K, value: V) -> (bool, err)
fn get[K: type, V: type](m: *const Map[K, V], key: K) -> (V, bool)
fn get_ptr[K: type, V: type](m: *Map[K, V], key: K) -> (*V, bool)
fn remove[K: type, V: type](m: *Map[K, V], key: K) -> (V, bool)
fn clear[K: type, V: type](m: *Map[K, V])
fn set_init[K: type](a: *mem.Arena, capacity: usize) -> (Set[K], err)
fn set_add[K: type](s: *Set[K], key: K) -> (bool, err)
fn set_has[K: type](s: *const Set[K], key: K) -> bool
fn set_remove[K: type](s: *Set[K], key: K) -> bool
fn iter[K: type, V: type](m: *const Map[K, V]) -> Iter[K, V]
fn iter_next[K: type, V: type](it: *Iter[K, V]) -> (K, V, bool)
fn set_iter[K: type](s: *const Set[K]) -> SetIter[K]
fn set_iter_next[K: type](it: *SetIter[K]) -> (K, bool)

```

e.data.sort

```

fn in_place[T: type](items: []T)
fn in_place_by[T: type, Ctx: type](items: []T, ctx: *Ctx, cmp: fn(*Ctx, T, T) -> i32)
fn stable_in_place[T: type](a: *mem.Arena, items: []T) -> err
fn stable_in_place_by[T: type, Ctx: type](a: *mem.Arena, items: []T, ctx: *Ctx, cmp: fn(*Ctx, T, T) -> i32) -> err
fn radix_u32_in_place(a: *mem.Arena, items: []u32) -> err
fn radix_u64_in_place(a: *mem.Arena, items: []u64) -> err
fn is_sorted[T: type](items: []const T) -> bool

```

e.data.heap

```

type Heap[T: type] = struct { items: list.List[T] }
type HeapBy[T: type, Ctx: type] = struct { items: list.List[T], ctx: *Ctx, cmp: fn(*Ctx, T, T) -> i32 }
type Iter[T: type] = struct { items: []const T, index: usize }

fn init[T: type](a: *mem.Arena, capacity: usize) -> (Heap[T], err)
fn from_slice[T: type](a: *mem.Arena, source: []const T) -> (Heap[T], err)
fn len[T: type](h: *const Heap[T]) -> usize
fn push[T: type](h: *Heap[T], v: T) -> err
fn peek[T: type](h: *const Heap[T]) -> (T, bool)
fn pop[T: type](h: *Heap[T]) -> (T, bool)
fn clear[T: type](h: *Heap[T])
fn init_by[T: type, Ctx: type](a: *mem.Arena, capacity: usize, ctx: *Ctx, cmp: fn(*Ctx, T, T) -> i32) -> (HeapBy[T, Ctx], err)
fn from_slice_by[T: type, Ctx: type](a: *mem.Arena, source: []const T, ctx: *Ctx, cmp: fn(*Ctx, T, T) -> i32) -> (HeapBy[T, Ctx], err)
fn len_by[T: type, Ctx: type](h: *const HeapBy[T, Ctx]) -> usize
fn push_by[T: type, Ctx: type](h: *HeapBy[T, Ctx], value: T) -> err
fn peek_by[T: type, Ctx: type](h: *const HeapBy[T, Ctx]) -> (T, bool)
fn pop_by[T: type, Ctx: type](h: *HeapBy[T, Ctx]) -> (T, bool)
fn clear_by[T: type, Ctx: type](h: *HeapBy[T, Ctx])
fn heapify_in_place[T: type](items: []T)
fn heapify_in_place_by[T: type, Ctx: type](items: []T, ctx: *Ctx, cmp: fn(*Ctx, T, T) -> i32)
fn iter[T: type](h: *const Heap[T]) -> Iter[T]
fn iter_by[T: type, Ctx: type](h: *const HeapBy[T, Ctx]) -> Iter[T]
fn iter_next[T: type](it: *Iter[T]) -> (T, bool)

```

The default heap is a min-heap under `T.cmp`; `HeapBy` uses `cmp`. Bulk construction is linear time. Iteration exposes internal heap order, not sorted order.

e.data.tree

```

type Map[K: type, V: type] = struct { state: *void }
type Set[K: type] = struct { map: Map[K, bool] }
type Iter[K: type, V: type] = struct { state: *const void }
type SetIter[K: type] = struct { inner: Iter[K, bool] }

fn init[K: type, V: type](a: *mem.Arena) -> Map[K, V]
fn len[K: type, V: type](m: *const Map[K, V]) -> usize
fn put[K: type, V: type](m: *Map[K, V], key: K, value: V) -> (bool, err)
fn get[K: type, V: type](m: *const Map[K, V], key: K) -> (V, bool)
fn lower_bound[K: type, V: type](m: *const Map[K, V], key: K) -> (K, V, bool)
fn upper_bound[K: type, V: type](m: *const Map[K, V], key: K) -> (K, V, bool)
fn remove[K: type, V: type](m: *Map[K, V], key: K) -> (V, bool)
fn clear[K: type, V: type](m: *Map[K, V])
fn set_init[K: type](a: *mem.Arena) -> Set[K]
fn set_add[K: type](s: *Set[K], key: K) -> (bool, err)
fn set_has[K: type](s: *const Set[K], key: K) -> bool
fn set_remove[K: type](s: *Set[K], key: K) -> bool
fn iter[K: type, V: type](m: *const Map[K, V]) -> Iter[K, V]
fn iter_from[K: type, V: type](m: *const Map[K, V], key: K) -> Iter[K, V]
fn iter_next[K: type, V: type](it: *Iter[K, V]) -> (K, V, bool)
fn set_iter[K: type](s: *const Set[K]) -> SetIter[K]
fn set_iter_next[K: type](it: *SetIter[K]) -> (K, bool)

```

Tree iteration is ascending by key. `iter_from` begins at the first key not less than `key`.

e.data.disjoint_set

```

type DisjointSet = struct { parent: []u32, rank: []u8, sets: usize }
error TooLarge
error TooSmall

fn init(parent: []u32, rank: []u8, count: usize) -> (DisjointSet, err)
fn len(s: *const DisjointSet) -> usize
fn set_count(s: *const DisjointSet) -> usize
fn find(s: *DisjointSet, value: u32) -> u32
fn same(s: *DisjointSet, a: u32, b: u32) -> bool
fn union(s: *DisjointSet, a: u32, b: u32) -> bool
fn reset(s: *DisjointSet)

```

The caller supplies storage. `count` must fit `u32` and both slices. `find` performs path compression and `union` uses union by rank; indices outside `count` follow the ordinary bounds-trap rule.

e.data.graph

```

type NodeId = u32
const NONE: NodeId = 4294967295
type Edge[E: type] = struct { from: NodeId, to: NodeId, value: E }
type Builder[E: type] = struct { node_count: usize, edges: list.List[Edge[E]] }
type Graph[E: type] = struct { node_count: usize, offsets: []const usize, edges: []const Edge[E] }
type Neighbors[E: type] = struct { edges: []const Edge[E], index: usize }
type Nodes = struct { next: NodeId, end: NodeId }
error InvalidNode
error TooLarge

fn builder[E: type](a: *mem.Arena, initial_nodes: usize, edge_capacity: usize) -> (Builder[E], err)
fn add_directed[E: type](b: *Builder[E], from: NodeId, to: NodeId, value: E) -> err
fn add_undirected[E: type](b: *Builder[E], a: NodeId, b_node: NodeId, value: E) -> err
fn finish[E: type](a: *mem.Arena, b: *const Builder[E]) -> (Graph[E], err)
fn node_count[E: type](g: *const Graph[E]) -> usize
fn edge_count[E: type](g: *const Graph[E]) -> usize
fn nodes[E: type](g: *const Graph[E]) -> Nodes
fn nodes_next(it: *Nodes) -> (NodeId, bool)
fn neighbors[E: type](g: *const Graph[E], node: NodeId) -> (Neighbors[E], err)
fn neighbors_next[E: type](it: *Neighbors[E]) -> (Edge[E], bool)

```

`finish` constructs immutable compressed-sparse-row adjacency, preserving insertion order among edges from the same node. An undirected edge is stored as two directed edges; it reserves both first and leaves the builder unchanged on error. Nodes are dense `0..node_count`; node counts that do not fit `u32` return `TooLarge`. Graphs own no resources beyond their caller arena.

e.data.slot_map

```

type Key = struct { slot: u32, generation: u32 }
type SlotMap[T: type] = struct { state: *void }
type Iter[T: type] = struct { state: *const void, slot: u32 }
error Full
error TooLarge

fn init[T: type](a: *mem.Arena, initial_capacity: usize) -> (SlotMap[T], err)
fn len[T: type](m: *const SlotMap[T]) -> usize
fn capacity[T: type](m: *const SlotMap[T]) -> usize
fn insert[T: type](m: *SlotMap[T], value: T) -> (Key, err)
fn get[T: type](m: *const SlotMap[T], key: Key) -> (*const T, bool)
fn get_mut[T: type](m: *SlotMap[T], key: Key) -> (*T, bool)
fn remove[T: type](m: *SlotMap[T], key: Key) -> (T, bool)
fn clear[T: type](m: *SlotMap[T])
fn iter[T: type](m: *const SlotMap[T]) -> Iter[T]
fn iter_next[T: type](it: *Iter[T]) -> (Key, T, bool)

```

Capacity is fixed and caller-funded. Removal increments the slot generation before reuse; a generation never wraps, and a slot whose next generation would wrap is retired permanently. `clear` invalidates every key. This is the standard facility for independently retired arena values and stable externally stored handles.

e.data.iter

```

type Map[I: type, T: type, U: type] = struct { it: I, f: fn(T) -> U }
type MapCtx[I: type, T: type, U: type, Ctx: type] = struct { it: I, ctx: *Ctx, f: fn(*Ctx, T) -> U }
type Filter[I: type, T: type] = struct { it: I, pred: fn(T) -> bool }
type FilterCtx[I: type, T: type, Ctx: type] = struct { it: I, ctx: *Ctx, pred: fn(*Ctx, T) -> bool }
type Take[T: type, I: type] = struct { it: I, remaining: usize }
type Skip[T: type, I: type] = struct { it: I, remaining: usize }
type Enumerate[T: type, I: type] = struct { it: I, index: usize }
type Chain[T: type, A: type, B: type] = struct { left: A, right: B, left_done: bool }
type Zip[X: type, Y: type, A: type, B: type] = struct { left: A, right: B }

fn map[I: type, T: type, U: type](it: I, f: fn(T) -> U) -> Map[I, T, U]
fn map_next[I: type, T: type, U: type](it: *Map[I, T, U]) -> (U, bool)
fn map_ctx[I: type, T: type, U: type, Ctx: type](it: I, ctx: *Ctx, f: fn(*Ctx, T) -> U) -> MapCtx[I, T, U, Ctx]
fn map_ctx_next[I: type, T: type, U: type, Ctx: type](it: *MapCtx[I, T, U, Ctx]) -> (U, bool)
fn filter[I: type, T: type](it: I, pred: fn(T) -> bool) -> Filter[I, T]
fn filter_next[I: type, T: type](it: *Filter[I, T]) -> (T, bool)
fn filter_ctx[I: type, T: type, Ctx: type](it: I, ctx: *Ctx, pred: fn(*Ctx, T) -> bool) -> FilterCtx[I, T, Ctx]
fn filter_ctx_next[I: type, T: type, Ctx: type](it: *FilterCtx[I, T, Ctx]) -> (T, bool)
fn take[T: type, I: type](it: I, n: usize) -> Take[T, I]
fn take_next[T: type, I: type](it: *Take[T, I]) -> (T, bool)
fn zip[X: type, Y: type, A: type, B: type](left: A, right: B) -> Zip[X, Y, A, B]
fn zip_next[X: type, Y: type, A: type, B: type](it: *Zip[X, Y, A, B]) -> (X, Y, bool)
fn reduce[I: type, T: type, U: type](it: *I, initial: U, f: fn(U, T) -> U) -> U
fn reduce_ctx[I: type, T: type, U: type, Ctx: type](it: *I, initial: U, ctx: *Ctx, f: fn(*Ctx, U, T) -> U) -> U
fn skip[T: type, I: type](it: I, n: usize) -> Skip[T, I]
fn skip_next[T: type, I: type](it: *Skip[T, I]) -> (T, bool)
fn enumerate[T: type, I: type](it: I) -> Enumerate[T, I]
fn enumerate_next[T: type, I: type](it: *Enumerate[T, I]) -> (usize, T, bool)
fn chain[T: type, A: type, B: type](left: A, right: B) -> Chain[T, A, B]
fn chain_next[T: type, A: type, B: type](it: *Chain[T, A, B]) -> (T, bool)
fn find[I: type, T: type](it: *I, pred: fn(T) -> bool) -> (T, bool)
fn position[I: type, T: type](it: *I, pred: fn(T) -> bool) -> (usize, bool)
fn any[I: type, T: type](it: *I, pred: fn(T) -> bool) -> bool
fn all[I: type, T: type](it: *I, pred: fn(T) -> bool) -> bool
fn count[I: type, T: type](it: *I) -> usize
fn min[I: type, T: type](it: *I) -> (T, bool)
fn max[I: type, T: type](it: *I) -> (T, bool)
fn collect[I: type, T: type](a: *mem.Arena, it: *I) -> (list.List[T], err)
fn partition[I: type, T: type](a: *mem.Arena, it: *I, pred: fn(T) -> bool) -> (list.List[T], list.List[T], err)
type TryMap[I: type, T: type, U: type, Ctx: type] = struct { it: *I, ctx: *Ctx, f: fn(*Ctx, T) -> (U, err), ended: bool }
type TryFilter[I: type, T: type, Ctx: type] = struct { it: *I, ctx: *Ctx, pred: fn(*Ctx, T) -> (bool, err), ended: bool }

fn try_map[I: type, T: type, U: type, Ctx: type](it: *I, ctx: *Ctx, f: fn(*Ctx, T) -> (U, err)) -> TryMap[I, T, U, Ctx]

```

```

fn try_map_next_err[I: type, T: type, U: type, Ctx: type](it: *TryMap[I, T, U, Ctx]) -> (U, bool, err)
fn try_filter[I: type, T: type, Ctx: type](it: *I, ctx: *Ctx, pred: fn(*Ctx, T) -> (bool, err)) -> TryFilter[I, T, Ctx]
fn try_filter_next_err[I: type, T: type, Ctx: type](it: *TryFilter[I, T, Ctx]) -> (T, bool, err)
fn try_collect[T: type, I: type](a: *mem.Arena, it: *I, limit: usize) -> (list.List[T], err)
fn try_reduce[I: type, T: type, U: type, Ctx: type](it: *I, initial: U, ctx: *Ctx, f: fn(*Ctx, U, T) -> (U, err)) -> (U, err)

```

`take[T](it, n)` and `zip[X, Y](left, right)` write the item type(s) explicitly; their trailing iterator types are structurally inferred. Other adapters infer their item types from the callback signatures. No return-context inference is required.

Every collection iterator borrows its source until exhaustion. Any mutation of that source invalidates all of its live iterators; using an invalid iterator is a debug `invalid` trap and release undefined behavior. Iteration never allocates or mutates the collection. Lists, dequeues, rings and queues use logical element order; stacks use LIFO order; hash maps use deterministic table-slot order; trees use ascending key order; heaps expose heap storage order. Set iterators expose only keys.

Try adapters require `l.next_err` and borrow the source; they never close it. Source errors and callback errors propagate once and put the adapter in terminal state; later `next_err` returns no item and `ok`. Failure's value is not a valid yielded item. `try_collect` returns a zero result and rolls back its allocations on error/limit, but cannot undo consumed input. The source and callbacks must not allocate from that result arena during collection. `try_reduce` returns its last fully committed accumulator alongside an error. Ordinary `for` never silently consumes `next_err`.

4. Pure algorithms, text and cryptography

`algo.rand`

```

type Pcg64 = struct { state: u64, stream: u64 }
type Xoshiro256 = struct { s0: u64, s1: u64, s2: u64, s3: u64 }
type Mt19937 = struct { state: [624]u32, index: u32 }

fn pcg64(seed: u64, stream: u64) -> Pcg64
fn pcg64_next(r: *Pcg64) -> u64
fn pcg64_bounded(r: *Pcg64, upper: u64) -> u64
fn pcg64_f64(r: *Pcg64) -> f64
fn xoshiro256(seed: [4]u64) -> Xoshiro256
fn xoshiro256_next(r: *Xoshiro256) -> u64
fn xoshiro256_bounded(r: *Xoshiro256, upper: u64) -> u64
fn xoshiro256_f64(r: *Xoshiro256) -> f64
fn mt19937(seed: u32) -> Mt19937
fn mt19937_next(r: *Mt19937) -> u32

```

`bounded(..., 0)` returns zero; otherwise it is unbiased rejection sampling.

`algo.uuid`

```

type Uuid = struct { bytes: [16]u8 }
error Invalid

fn v4(random: [16]u8) -> Uuid
fn v7(unix_millis: u64, random: [10]u8) -> (Uuid, err)
fn parse(s: str) -> (Uuid, err)
fn format(uuid: Uuid, dst: [16]u8) -> (str, err)
fn version(uuid: Uuid) -> u8
fn variant(uuid: Uuid) -> u8
fn uuid_eq(a: Uuid, b: Uuid) -> bool
fn uuid_cmp(a: Uuid, b: Uuid) -> i32
fn uuid_hash(uuid: Uuid) -> u64
fn uuid_format(uuid: Uuid, b: *str.Builder) -> err

```

`format` writes the 36-byte lowercase hyphenated form. Entropy and time are supplied by the caller; this module never reads `e.os`.

`algo.hash`

```

type XxHash64 = struct { seed: u64, total: u64, v1: u64, v2: u64, v3: u64, v4: u64, buffer: [32]u8, buffered: u8 }
type Crc32 = struct { value: u32 }

```

```

fn fnv1a32(data: []const u8) -> u32
fn fnv1a64(data: []const u8) -> u64
fn xxhash64(data: []const u8, seed: u64) -> u64
fn xxhash64_init(seed: u64) -> XxHash64
fn xxhash64_update(h: *XxHash64, data: []const u8)
fn xxhash64_done(h: *const XxHash64) -> u64
fn crc32(data: []const u8) -> u32
fn crc32_init() -> Crc32
fn crc32_update(h: *Crc32, data: []const u8)
fn crc32_done(h: *const Crc32) -> u32
fn adler32(data: []const u8) -> u32

```

algo.deflate

```

type Encoder = struct { state: *void }
type Decoder = struct { state: *void }
type Level = enum u8 { Fast, Balanced, Best }
type Status = enum u8 { NeedInput, NeedOutput, Finished }
error Invalid
error TooLarge

fn encoder(storage: []u8, level: Level) -> (Encoder, err)
fn decoder(storage: []u8, window_limit: usize) -> (Decoder, err)
fn encode(e: *Encoder, input: []const u8, output: []u8, finish: bool) -> (usize, usize, Status, err)
fn decode(d: *Decoder, input: []const u8, output: []u8, finish: bool) -> (usize, usize, Status, err)
fn encoder_storage(level: Level) -> usize
fn decoder_storage(window_limit: usize) -> (usize, err)

```

The calls return consumed input and written output. State uses caller storage and performs no allocation. This module implements raw RFC 1951 DEFLATE only; gzip and ZIP framing belong to `fmt.gzip` and `fmt.zip`.

algo.graph

```

type Traversal = struct { order: []const graph.NodeId, parent: []const graph.NodeId }
type Components = struct { component: []const u32, count: u32 }
type Paths = struct { distance: []const f64, previous: []const graph.NodeId }
error Cycle
error InvalidWeight
error TooLarge

fn bfs[E: type](a: *mem.Arena, g: *const graph.Graph[E], start: graph.NodeId) -> (Traversal, err)
fn dfs[E: type](a: *mem.Arena, g: *const graph.Graph[E], start: graph.NodeId) -> (Traversal, err)
fn topological[E: type](a: *mem.Arena, g: *const graph.Graph[E]) -> ([]const graph.NodeId, err)
fn weak_components[E: type](a: *mem.Arena, g: *const graph.Graph[E]) -> (Components, err)
fn strong_components[E: type](a: *mem.Arena, g: *const graph.Graph[E]) -> (Components, err)
fn dijkstra[E: type, Ctx: type](a: *mem.Arena, g: *const graph.Graph[E], start: graph.NodeId, ctx: *Ctx, weight: fn(*Ctx, graph.Edge[E]) -> f64) -> (Paths, err)

```

Traversal order is deterministic from node order and each adjacency list's insertion order. An unreachable node has parent `graph.NONE`; the start is its own parent. Topological sorting returns the lexicographically smallest available node first and returns `Cycle` without a partial order. Component identifiers are assigned by the smallest node in each component. Dijkstra rejects negative, NaN and infinite weights as `InvalidWeight`; unreachable distance is positive infinity and its predecessor is `graph.NONE`. All returned slices are arena-owned.

algo.stat

```

type Moments = struct { count: u64, mean: f64, m2: f64, min: f64, max: f64 }
type Regression = struct { count: u64, mean_x: f64, mean_y: f64, m2_x: f64, m2_y: f64, cov: f64 }

fn moments() -> Moments
fn moments_add(s: *Moments, x: f64)
fn moments_merge(dst: *Moments, src: *const Moments)
fn variance_population(s: *const Moments) -> (f64, bool)
fn variance_sample(s: *const Moments) -> (f64, bool)
fn standard_deviation_population(s: *const Moments) -> (f64, bool)
fn standard_deviation_sample(s: *const Moments) -> (f64, bool)
fn regression() -> Regression
fn regression_add(s: *Regression, x: f64, y: f64)
fn regression_slope(s: *const Regression) -> (f64, bool)
fn regression_intercept(s: *const Regression) -> (f64, bool)

```

```
fn correlation(s: *const Regression) -> (f64, bool)
```

algo.bitset

```
type BitSet = struct { words: []u64, len: usize }
error TooSmall

fn init(storage: []u64, bit_count: usize) -> (BitSet, err)
fn len(s: *const BitSet) -> usize
fn clear_all(s: *BitSet)
fn fill_all(s: *BitSet)
fn get(s: *const BitSet, index: usize) -> bool
fn set(s: *BitSet, index: usize)
fn unset(s: *BitSet, index: usize)
fn toggle(s: *BitSet, index: usize)
fn count(s: *const BitSet) -> usize
fn first_set(s: *const BitSet) -> (usize, bool)
fn next_set(s: *const BitSet, after: usize) -> (usize, bool)
fn union_in_place(dst: *BitSet, src: *const BitSet)
fn intersect_in_place(dst: *BitSet, src: *const BitSet)
fn difference_in_place(dst: *BitSet, src: *const BitSet)
fn complement_in_place(s: *BitSet)
fn is_subset(a: *const BitSet, b: *const BitSet) -> bool
fn eq(a: *const BitSet, b: *const BitSet) -> bool
```

Bits at indices `len..storage.len*64` are always zero. Operations requiring two sets require equal logical lengths; a mismatch is a debug bounds trap and release undefined behavior, like incompatible slice bounds in other pure primitives.

algo.complex

```
type Complex[F: type] = struct { re: F, im: F }

fn make[F: type](re: F, im: F) -> Complex[F]
fn add[F: type](a: Complex[F], b: Complex[F]) -> Complex[F]
fn sub[F: type](a: Complex[F], b: Complex[F]) -> Complex[F]
fn mul[F: type](a: Complex[F], b: Complex[F]) -> Complex[F]
fn div[F: type](a: Complex[F], b: Complex[F]) -> Complex[F]
fn neg[F: type](z: Complex[F]) -> Complex[F]
fn conj[F: type](z: Complex[F]) -> Complex[F]
fn abs[F: type](z: Complex[F]) -> F
fn arg[F: type](z: Complex[F]) -> F
fn exp[F: type](z: Complex[F]) -> Complex[F]
fn log[F: type](z: Complex[F]) -> Complex[F]
fn sqrt[F: type](z: Complex[F]) -> Complex[F]
fn pow[F: type](z: Complex[F], w: Complex[F]) -> Complex[F]
fn sin[F: type](z: Complex[F]) -> Complex[F]
fn cos[F: type](z: Complex[F]) -> Complex[F]
fn tan[F: type](z: Complex[F]) -> Complex[F]
```

F is f32 or f64. Branch cuts and signed-zero behavior follow C99 Annex G; operations inherit e.math's NaN canonicalization and no-contraction rules.

algo.decimal

```
type Coefficient = struct { low: u64, high: i64 }
type Decimal = struct { coefficient: Coefficient, scale: u8 }
type Rounding = enum u8 { ToEven, AwayFromZero, TowardZero, Floor, Ceiling }
error Invalid
error Overflow
error Inexact

fn make(coefficient: Coefficient, scale: u8) -> (Decimal, err)
fn normalize(value: Decimal) -> Decimal
fn add(a: Decimal, b: Decimal) -> (Decimal, err)
fn sub(a: Decimal, b: Decimal) -> (Decimal, err)
fn mul(a: Decimal, b: Decimal) -> (Decimal, err)
fn div(a: Decimal, b: Decimal, scale: u8, rounding: Rounding) -> (Decimal, err)
fn quantize(value: Decimal, scale: u8, rounding: Rounding) -> (Decimal, err)
fn compare(a: Decimal, b: Decimal) -> i32
fn parse(source: str) -> (Decimal, err)
fn format(value: Decimal, b: *str.Builder) -> err
```

```
fn to_i64(value: Decimal, rounding: Rounding) -> (i64, err)
fn from_i64(value: i64, scale: u8) -> (Decimal, err)
```

scale is 0..38; value is coefficient * 10^{-scale}. Arithmetic never silently rounds: only operations carrying a Rounding argument may discard decimal digits. Parsing is locale-free and consumes the complete ordinary or scientific decimal.

algo.bignum

```
type Sign = enum u8 { Zero, Positive, Negative }
type Int = struct { sign: Sign, limbs: []u32, arena: *mem.Arena }
type Rat = struct { num: Int, den: Int }
error DivideByZero
error Invalid

fn int_zero(a: *mem.Arena) -> Int
fn int_from_i64(a: *mem.Arena, v: i64) -> (Int, err)
fn int_parse(a: *mem.Arena, s: str, radix: u8) -> (Int, err)
fn int_format(v: Int, b: *str.Builder, radix: u8) -> err
fn int_cmp(a: Int, b: Int) -> i32
fn int_add(a: *mem.Arena, x: Int, y: Int) -> (Int, err)
fn int_sub(a: *mem.Arena, x: Int, y: Int) -> (Int, err)
fn int_mul(a: *mem.Arena, x: Int, y: Int) -> (Int, err)
fn int_divmod(a: *mem.Arena, x: Int, y: Int) -> (Int, Int, err)
fn int_gcd(a: *mem.Arena, x: Int, y: Int) -> (Int, err)
fn rat_make(a: *mem.Arena, num: Int, den: Int) -> (Rat, err)
fn rat_add(a: *mem.Arena, x: Rat, y: Rat) -> (Rat, err)
fn rat_sub(a: *mem.Arena, x: Rat, y: Rat) -> (Rat, err)
fn rat_mul(a: *mem.Arena, x: Rat, y: Rat) -> (Rat, err)
fn rat_div(a: *mem.Arena, x: Rat, y: Rat) -> (Rat, err)
fn rat_cmp(a: Rat, b: Rat) -> i32
fn rat_format(v: Rat, b: *str.Builder) -> err
```

algo.linalg.matrix

```
type Matrix[T: type] = struct { data: []T, rows: usize, cols: usize, stride: usize }
type ConstMatrix[T: type] = struct { data: []const T, rows: usize, cols: usize, stride: usize }
error Shape
error Singular

fn view[T: type](data: []T, rows: usize, cols: usize, stride: usize) -> (Matrix[T], err)
fn view_const[T: type](data: []const T, rows: usize, cols: usize, stride: usize) -> (ConstMatrix[T], err)
fn get[T: type](m: Matrix[T], row: usize, col: usize) -> T
fn set[T: type](m: Matrix[T], row: usize, col: usize, v: T)
fn transpose[T: type](m: Matrix[T]) -> Matrix[T]
fn fill[T: type](m: Matrix[T], v: T)
fn copy[T: type](dst: Matrix[T], src: ConstMatrix[T]) -> err
fn add[T: type](dst: Matrix[T], a: ConstMatrix[T], b: ConstMatrix[T]) -> err
fn multiply[T: type](dst: Matrix[T], a: ConstMatrix[T], b: ConstMatrix[T]) -> err
fn determinant_f64(a: *mem.Arena, m: ConstMatrix[f64]) -> (f64, err)
fn inverse_f64(a: *mem.Arena, dst: Matrix[f64], src: ConstMatrix[f64]) -> err
```

algo.linalg.tensor

```
type Tensor[T: type] = struct { data: []T, shape: []const usize, stride: []const usize }
type ConstTensor[T: type] = struct { data: []const T, shape: []const usize, stride: []const usize }
error Shape

fn view[T: type](data: []T, shape: []const usize, stride: []const usize) -> (Tensor[T], err)
fn contiguous[T: type](a: *mem.Arena, data: []T, shape: []const usize) -> (Tensor[T], err)
fn as_const[T: type](tensor: Tensor[T]) -> ConstTensor[T]
fn elements[T: type](t: Tensor[T]) -> usize
fn offset[T: type](t: Tensor[T], index: []const usize) -> usize
fn reshape[T: type](a: *mem.Arena, t: Tensor[T], shape: []const usize) -> (Tensor[T], err)
fn fill[T: type](tensor: Tensor[T], value: T)
fn copy[T: type](dst: Tensor[T], src: ConstTensor[T]) -> err
fn add[T: type](dst: Tensor[T], x: ConstTensor[T], y: ConstTensor[T]) -> err
```

text.encoding

```
type Encoding = enum u8 { Utf8, Utf16Le, Utf16Be, Utf32Le, Utf32Be }
type InvalidPolicy = enum u8 { Reject, Replace }
type Decoder = struct { encoding: Encoding, policy: InvalidPolicy, pending: [4]u8, pending_len: u8, bom_seen: bool }
```

```

}
type Encoder = struct { encoding: Encoding, emit_bom: bool, started: bool }
error Invalid
error Incomplete
error TooSmall

fn detect_bom(src: []const u8) -> (Encoding, usize, bool)
fn decoder(encoding: Encoding, policy: InvalidPolicy, consume_bom: bool) -> Decoder
fn encoder(encoding: Encoding, emit_bom: bool) -> Encoder
fn decode(d: *Decoder, src: []const u8, dst_utf8: []u8, final: bool) -> (usize, usize, err)
fn encode(e: *Encoder, src_utf8: str, dst: []u8, final: bool) -> (usize, usize, err)
fn decoded_len(encoding: Encoding, src: []const u8, policy: InvalidPolicy) -> (usize, err)
fn encoded_len(encoding: Encoding, src_utf8: str, emit_bom: bool) -> (usize, err)
fn to_utf8(a: *mem.Arena, encoding: Encoding, src: []const u8, policy: InvalidPolicy) -> (str, err)
fn from_utf8(a: *mem.Arena, encoding: Encoding, src: str, emit_bom: bool) -> ([]u8, err)

```

The streaming calls return consumed source bytes followed by written destination bytes. `TooSmall` is resumable and consumes only complete scalar values. `final` reports a trailing partial sequence as `Incomplete`; `Replace` emits `U+FFFD` for each maximal invalid subsequence. This toolchain module intentionally excludes locale and legacy code pages.

text.utf8

```

type Decode = struct { scalar: u32, width: u8 }
type Iterator = struct { data: str, off: usize }
error Invalid
error TooSmall

fn validate(s: str) -> bool
fn decode(s: str, off: usize) -> (Decode, err)
fn encode(scalar: u32, dst: []u8) -> (u8, err)
fn count(s: str) -> (usize, err)
fn byte_offset(s: str, scalar_index: usize) -> (usize, err)
fn iterator(s: str) -> Iterator
fn iterator_next(it: *Iterator) -> (u32, bool)
fn iterator_next_err(it: *Iterator) -> (u32, bool, err)

```

text.unicode

```

type Category = enum u8 { Lu, Ll, Lt, Lm, Lo, Mn, Mc, Me, Nd, Nl, No, Pc, Pd, Ps, Pe, Pi, Pf, Po, Sm, Sc, Sk, So, Zs, Zl, Zp, Cc, Cf, Cs, Co, Cn }
type Graphemes = struct { text: str, off: usize }

fn category(scalar: u32) -> Category
fn combining_class(scalar: u32) -> u8
fn is_whitespace(scalar: u32) -> bool
fn is_alphabetic(scalar: u32) -> bool
fn is_numeric(scalar: u32) -> bool
fn to_lower_simple(scalar: u32) -> u32
fn to_upper_simple(scalar: u32) -> u32
fn casefold(a: *mem.Arena, s: str) -> (str, err)
fn graphemes(s: str) -> Graphemes
fn graphemes_next(it: *Graphemes) -> (str, bool)

```

text.normalize

```

type Form = enum u8 { Nfc, Nfd, Nfkc, Nfkd }
error Invalid

fn is_normalized(s: str, form: Form) -> (bool, err)
fn normalize(a: *mem.Arena, s: str, form: Form) -> (str, err)

```

text.collate

```

type Options = struct { case_sensitive: bool, numeric: bool }

fn codepoint_cmp(a: str, b: str) -> i32
fn natural_cmp(a: str, b: str, options: Options) -> i32

```

Locale-aware collation is exposed by `text.locale`; this module remains the locale-independent Unicode collation mechanism beneath it.

text.locale

```

type Database = struct { state: *void }
type Locale = struct { state: *const void }
type NumberOptions = struct { minimum_fraction: u8, maximum_fraction: u8, grouping: bool, sign_always: bool }
type CurrencyOptions = struct { code: str, accounting: bool }
type DateStyle = enum u8 { Short, Medium, Long, Full }
error InvalidData
error NotFound
error Invalid

fn load(a: *mem.Arena, source: []const u8) -> (Database, err)
fn builtin(a: *mem.Arena) -> (Database, err)
fn version(db: *const Database) -> str
fn locale(db: *const Database, tag: str) -> (Locale, err)
fn canonical_tag(a: *mem.Arena, tag: str) -> (str, err)
fn format_i64(a: *mem.Arena, selected_locale: Locale, value: i64, options: NumberOptions) -> (str, err)
fn format_f64(a: *mem.Arena, selected_locale: Locale, value: f64, options: NumberOptions) -> (str, err)
fn parse_f64(selected_locale: Locale, value: str) -> (f64, err)
fn format_currency(a: *mem.Arena, selected_locale: Locale, value: decimal.Decimal, options: CurrencyOptions) -> (str, err)
fn format_date(a: *mem.Arena, selected_locale: Locale, value: calendar.DateTime, style: DateStyle) -> (str, err)
fn compare(selected_locale: Locale, a: str, b: str) -> i32
fn lower(a: *mem.Arena, selected_locale: Locale, value: str) -> (str, err)
fn upper(a: *mem.Arena, selected_locale: Locale, value: str) -> (str, err)

```

builtin uses the CLDR release pinned to the toolchain and reported by version; load accepts explicit compatible data. Parsing consumes the whole input. Currency codes are caller-supplied ISO 4217 identifiers. No process-global locale exists.

text.template

```

type Template = struct { state: *void }
type Options = struct { max_bytes: usize, max_nodes: usize, max_depth: u16 }
type Value = union enum u8 { Null, Bool: bool, I64: i64, U64: u64, F64: f64, Text: str, Bytes: []const u8 }
type Binding = struct { name: str, value: Value }
error InvalidTemplate
error MissingValue
error TooDeep
error TooLarge

fn parse(a: *mem.Arena, source: str, options: Options) -> (Template, err)
fn execute(template: *const Template, writer: *io.Writer, bindings: []const Binding) -> err
fn validate[T: type](template: *const Template) -> err
fn execute_typed[T: type](template: *const Template, writer: *io.Writer, value: *const T) -> err

```

Templates provide deterministic interpolation, conditionals and bounded iteration over explicit values or compile-time-inspected structs. The core engine performs no contextual escaping; specialized output modules such as fmt.html.template own it.

text.regex

```

type Regex = struct { state: *void }
type Match = struct { start: usize, end: usize }
type Captures = struct { whole: Match, groups: []const Match }
type Options = struct { case_insensitive: bool, multiline: bool, dot_matches_newline: bool }
error InvalidPattern
error TooComplex

fn compile(a: *mem.Arena, pattern: str, options: Options) -> (Regex, err)
fn is_match(r: *const Regex, text: str) -> bool
fn find(r: *const Regex, text: str, from: usize) -> (Match, bool)
fn captures(a: *mem.Arena, r: *const Regex, text: str, from: usize) -> (Captures, bool, err)
fn replace_all(a: *mem.Arena, r: *const Regex, text: str, replacement: str) -> (str, err)

```

The accepted syntax is regular only: no backreferences, recursion or lookbehind.

gfx.geometry

```

type Point = struct { x: f32, y: f32 }
type Size = struct { width: f32, height: f32 }

```

```

type Rect = struct { x: f32, y: f32, width: f32, height: f32 }
type Insets = struct { left: f32, top: f32, right: f32, bottom: f32 }
type Radius = struct { x: f32, y: f32 }
type RRect = struct { rect: Rect, top_left: Radius, top_right: Radius, bottom_right: Radius, bottom_left: Radius }
type Transform = struct { m00: f32, m01: f32, m02: f32, m10: f32, m11: f32, m12: f32 }
type PathVerb = enum u8 { Move, Line, Quad, Cubic, Close }
type Path = struct { verbs: []const PathVerb, points: []const Point }
type PathBuilder = struct { state: *void }
error Invalid
error TooLarge

fn rect(x: f32, y: f32, width: f32, height: f32) -> Rect
fn contains(r: Rect, p: Point) -> bool
fn intersect(a: Rect, b: Rect) -> Rect
fn union_rect(a: Rect, b: Rect) -> Rect
fn transform_identity() -> Transform
fn transform_translate(x: f32, y: f32) -> Transform
fn transform_scale(x: f32, y: f32) -> Transform
fn transform_rotate(radians: f32) -> Transform
fn transform_multiply(a: Transform, b: Transform) -> Transform
fn transform_point(t: Transform, p: Point) -> Point
fn path_builder(a: *mem.Arena, max_verbs: usize, max_points: usize) -> (PathBuilder, err)
fn move_to(b: *PathBuilder, p: Point) -> err
fn line_to(b: *PathBuilder, p: Point) -> err
fn quad_to(b: *PathBuilder, control: Point, end: Point) -> err
fn cubic_to(b: *PathBuilder, first: Point, second: Point, end: Point) -> err
fn close_path(b: *PathBuilder) -> err
fn finish(b: *PathBuilder) -> Path

```

Coordinates are logical pixels. Values must be finite; rectangles and sizes have non-negative dimensions. Paths and transforms allocate nothing after builder creation and are independent of any rendering backend.

gfx.paint

```

type Color = struct { red: f32, green: f32, blue: f32, alpha: f32 }
type Blend = enum u8 { SourceOver, Source, DestinationOver, Multiply, Screen, Overlay, Darken, Lighten }
type StrokeCap = enum u8 { Butt, Round, Square }
type StrokeJoin = enum u8 { Miter, Round, Bevel }
type Stroke = struct { width: f32, cap: StrokeCap, join: StrokeJoin, miter_limit: f32 }
type Stop = struct { offset: f32, color: Color }
type Brush = union enum u8 { Solid: Color, Linear: LinearGradient, Radial: RadialGradient }
type LinearGradient = struct { start: geometry.Point, end: geometry.Point, stops: []const Stop }
type RadialGradient = struct { center: geometry.Point, radius: f32, stops: []const Stop }
error Invalid

fn rgba(red: f32, green: f32, blue: f32, alpha: f32) -> Color
fn srgb8(red: u8, green: u8, blue: u8, alpha: u8) -> Color
fn premultiply(color: Color) -> Color
fn validate(bush: *const Brush) -> err

```

Colors are linear-light floating-point RGBA; srgb8 performs the defined sRGB transfer. Gradient stops are borrowed, ordered and bounded to 0..1.

gfx.image

```

type Format = enum u8 { R8, Rgba8, Bgra8, Rgba16Float }
type Alpha = enum u8 { Opaque, Straight, Premultiplied }
type Info = struct { width: u32, height: u32, format: Format, alpha: Alpha, frames: u32 }
type Image = struct { pixels: []u8, width: u32, height: u32, stride: usize, format: Format, alpha: Alpha }
type ConstImage = struct { pixels: []const u8, width: u32, height: u32, stride: usize, format: Format, alpha: Alpha }
error Invalid
error TooLarge

fn required_bytes(width: u32, height: u32, format: Format, stride: usize) -> (usize, err)
fn make(pixels: []u8, width: u32, height: u32, stride: usize, format: Format, alpha: Alpha) -> (Image, err)
fn make_const(pixels: []const u8, width: u32, height: u32, stride: usize, format: Format, alpha: Alpha) -> (ConstImage, err)
fn allocate(a: *mem.Arena, width: u32, height: u32, format: Format, alpha: Alpha) -> (Image, err)
fn clear(image: Image, color: paint.Color)
fn copy(dst: Image, src: ConstImage, dst_origin: geometry.Point) -> err

```

Images are pixel views, not codecs or GPU resources. Encoders and decoders belong in `fmt.*`; upload and caching belong in `gfx.scene`.

`text.shape`

```
type FontId = u32
type Direction = enum u8 { LeftToRight, RightToLeft }
type Font = struct { id: FontId, data: []const u8, face_index: u32 }
type Feature = struct { tag: u32, value: u32, start: usize, end: usize }
type Glyph = struct { id: u32, cluster: usize, advance_x: f32, advance_y: f32, offset_x: f32, offset_y: f32 }
type Run = struct { font: FontId, direction: Direction, script: u32, language: str, glyphs: []const Glyph }
type Options = struct { direction: Direction, script: u32, language: str, features: []const Feature }
error InvalidFont
error InvalidText
error Unsupported
error TooLarge

fn validate_font(font: Font) -> err
fn shape(a: *mem.Arena, font: Font, source: str, options: Options) -> (Run, err)
```

Shaping is deterministic over caller-provided OpenType font bytes and the toolchain's pinned Unicode tables. It performs substitutions and positioning but no line breaking, font discovery, fallback, rasterization or hidden file access.

`text.layout`

```
type Align = enum u8 { Start, End, Center, Justify }
type Wrap = enum u8 { None, Word, Character }
type FontChoice = struct { font: shape.Font, size: f32 }
type Style = struct { fonts: []const FontChoice, language: str, line_height: f32 }
type GlyphRun = struct { run: shape.Run, origin: geometry.Point, size: f32 }
type Line = struct { runs: []const GlyphRun, bounds: geometry.Rect, baseline: f32, start: usize, end: usize }
type Layout = struct { source: str, lines: []const Line, bounds: geometry.Rect }
type Options = struct { width: f32, max_lines: u32, align: Align, wrap: Wrap, ellipsis: str }
error MissingGlyph
error Invalid
error TooLarge

fn layout(a: *mem.Arena, source: str, style: Style, options: Options) -> (Layout, err)
fn hit_test(value: *const Layout, point: geometry.Point) -> usize
fn caret(value: *const Layout, byte_offset: usize) -> geometry.Rect
fn selection(a: *mem.Arena, value: *const Layout, start: usize, end: usize) -> ([]geometry.Rect, err)
```

The module performs Unicode bidi resolution, line breaking, fallback and visual placement. Byte offsets always identify UTF-8 boundaries in source.

`ui.style`

```
type Length = union enum u8 { Auto, Px: f32, Percent: f32, Flex: f32 }
type EdgeLengths = struct { left: Length, top: Length, right: Length, bottom: Length }
type Display = enum u8 { Flex, Grid, Stack, None }
type Position = enum u8 { Flow, Absolute }
type Overflow = enum u8 { Visible, Clip, Scroll }
type Style = struct { display: Display, position: Position, width: Length, height: Length, min_width: Length, min_height: Length, max_width: Length, max_height: Length, margin: EdgeLengths, padding: EdgeLengths, background: paint.Brush, opacity: f32, overflow: Overflow }
error Invalid

fn defaults() -> Style
fn validate(value: *const Style) -> err
```

Styles are ordinary immutable values. There is no selector engine, cascading global sheet or reflective property lookup in version 1.

`ui.layout`

```
type Axis = enum u8 { Horizontal, Vertical }
type MainAlign = enum u8 { Start, End, Center, SpaceBetween, SpaceAround, SpaceEvenly }
type CrossAlign = enum u8 { Start, End, Center, Stretch, Baseline }
type Constraints = struct { min_width: f32, max_width: f32, min_height: f32, max_height: f32 }
type Flex = struct { axis: Axis, main: MainAlign, cross: CrossAlign, gap: f32 }
type GridTrack = union enum u8 { Px: f32, Flex: f32, Auto }
```

```

type Grid = struct { columns: []const GridTrack, rows: []const GridTrack, column_gap: f32, row_gap: f32 }
type Child = struct { desired: geometry.Size, flex: f32 }
type Result = struct { size: geometry.Size, children: []const geometry.Rect }
error Invalid
error Overflow

fn constrain(value: geometry.Size, limits: Constraints) -> geometry.Size
fn flex(a: *mem.Arena, spec: Flex, limits: Constraints, children: []const Child) -> (Result, err)
fn grid(a: *mem.Arena, spec: Grid, limits: Constraints, children: []const Child) -> (Result, err)

```

Layout is a deterministic pure constraint solver. Scroll state, widget measurement and render-tree traversal remain in `ui.widget`.

crypto.hash

```

type Sha256 = struct { h: [8]u32, block: [64]u8, block_len: u8, total: u64 }
type Sha512 = struct { h: [8]u64, block: [128]u8, block_len: u8, total_hi: u64, total_lo: u64 }
type Sha3_256 = struct { lanes: [25]u64, block: [136]u8, block_len: u8 }
type Sha3_512 = struct { lanes: [25]u64, block: [72]u8, block_len: u8 }

fn sha256(data: []const u8) -> [32]u8
fn sha512(data: []const u8) -> [64]u8
fn sha3_256(data: []const u8) -> [32]u8
fn sha3_512(data: []const u8) -> [64]u8
fn sha256_init() -> Sha256
fn sha256_update(h: *Sha256, data: []const u8)
fn sha256_done(h: *Sha256) -> [32]u8
fn sha512_init() -> Sha512
fn sha512_update(h: *Sha512, data: []const u8)
fn sha512_done(h: *Sha512) -> [64]u8
fn sha3_256_init() -> Sha3_256
fn sha3_256_update(h: *Sha3_256, data: []const u8)
fn sha3_256_done(h: *Sha3_256) -> [32]u8
fn sha3_512_init() -> Sha3_512
fn sha3_512_update(h: *Sha3_512, data: []const u8)
fn sha3_512_done(h: *Sha3_512) -> [64]u8
fn legacy_sha1(data: []const u8) -> [20]u8
fn legacy_md5(data: []const u8) -> [16]u8
fn equal_constant_time(a: []const u8, b: []const u8) -> bool

```

crypto.mac

```

type HmacSha256 = struct { inner: hash.Sha256, outer: hash.Sha256 }
type HmacSha512 = struct { inner: hash.Sha512, outer: hash.Sha512 }

fn hmac_sha256(key: []const u8, message: []const u8) -> [32]u8
fn hmac_sha512(key: []const u8, message: []const u8) -> [64]u8
fn verify_sha256(key: []const u8, message: []const u8, tag: [32]u8) -> bool
fn verify_sha512(key: []const u8, message: []const u8, tag: [64]u8) -> bool
fn sha256_init(key: []const u8) -> HmacSha256
fn sha256_update(state: *HmacSha256, bytes: []const u8)
fn sha256_done(state: *HmacSha256) -> [32]u8
fn sha512_init(key: []const u8) -> HmacSha512
fn sha512_update(state: *HmacSha512, bytes: []const u8)
fn sha512_done(state: *HmacSha512) -> [64]u8

```

HMAC follows RFC 2104; verification compares fixed-length tags in constant time. Inputs are borrowed only for the call; bounded internal stack state is declared. No key generation, entropy read, truncation or secret logging is implicit. Streaming state lets HKDF process multiple input segments without concatenating unbounded caller data. `done` consumes the message state; `reinitialize` before reuse.

crypto.kdf

error TooLarge

```

fn hkdf_sha256_extract(salt: []const u8, input_key: []const u8) -> [32]u8
fn hkdf_sha256_expand(dst: []u8, prk: [32]u8, info: []const u8) -> err
fn hkdf_sha512_extract(salt: []const u8, input_key: []const u8) -> [64]u8
fn hkdf_sha512_expand(dst: []u8, prk: [64]u8, info: []const u8) -> err

```

HKDF follows RFC 5869. Expand rejects output longer than 255 hash blocks before writing dst; salt omission has the RFC-defined zero-salt behavior. Output aliases with key/info inputs are forbidden unless separately proven safe. HKDF is not a password-storage KDF; no password-hashing security claim or custom construction is introduced. Release requires published independent vectors and boundary tests.

crypto.aead

```
error InvalidKey
error InvalidNonce
error TooSmall
error Authentication

fn aes128_gcm_seal(dst: []u8, key: [16]u8, nonce: [12]u8, aad: []const u8, plain: []const u8) -> (usize, err)
fn aes128_gcm_open(dst: []u8, key: [16]u8, nonce: [12]u8, aad: []const u8, sealed: []const u8) -> (usize, err)
fn aes256_gcm_seal(dst: []u8, key: [32]u8, nonce: [12]u8, aad: []const u8, plain: []const u8) -> (usize, err)
fn aes256_gcm_open(dst: []u8, key: [32]u8, nonce: [12]u8, aad: []const u8, sealed: []const u8) -> (usize, err)
fn chacha20_poly1305_seal(dst: []u8, key: [32]u8, nonce: [12]u8, aad: []const u8, plain: []const u8) -> (usize, err)
fn chacha20_poly1305_open(dst: []u8, key: [32]u8, nonce: [12]u8, aad: []const u8, sealed: []const u8) -> (usize, err)
```

The sealed representation is ciphertext followed by the 16-byte authentication tag.

crypto.sign

```
type Ed25519PublicKey = struct { bytes: [32]u8 }
type Ed25519SecretKey = struct { bytes: [32]u8 }
type Ed25519Signature = struct { bytes: [64]u8 }
error InvalidKey
error InvalidSignature

fn ed25519_public_from_secret(secret: Ed25519SecretKey) -> (Ed25519PublicKey, err)
fn ed25519_sign(secret: Ed25519SecretKey, message: []const u8) -> (Ed25519Signature, err)
fn ed25519_verify(public: Ed25519PublicKey, message: []const u8, signature: Ed25519Signature) -> bool
```

Ed25519SecretKey.bytes is the 32-byte seed form. Verification rejects non-canonical encodings and small-order public keys.

crypto.kx

```
type X25519PublicKey = struct { bytes: [32]u8 }
type X25519SecretKey = struct { bytes: [32]u8 }
type X25519SharedKey = struct { bytes: [32]u8 }
error InvalidKey

fn x25519_public_from_secret(secret: X25519SecretKey) -> X25519PublicKey
fn x25519_exchange(secret: X25519SecretKey, peer: X25519PublicKey) -> (X25519SharedKey, err)
```

The scalar is clamped by the operation. An all-zero shared secret is InvalidKey.

crypto.random

```
type ChaCha20 = struct { key: [32]u8, nonce: [12]u8, counter: u32, block: [64]u8, used: u8 }
error Exhausted

fn chacha20_init(key: [32]u8, nonce: [12]u8, counter: u32) -> ChaCha20
fn chacha20_fill(r: *ChaCha20, dst: []u8) -> err
fn chacha20_next_u64(r: *ChaCha20) -> (u64, err)
fn chacha20_bounded(r: *ChaCha20, upper: u64) -> (u64, err)
```

The counter never wraps; a request that would do so returns Exhausted before reusing a block. chacha20_bounded(..., 0) returns zero and otherwise uses rejection sampling.

crypto.x509

```
type PublicKey = union enum u8 { Ed25519: sign.Ed25519PublicKey }
type Certificate = struct { der: []const u8, subject: str, issuer: str, dns_names: []const str, not_before: time.Instant, not_after: time.Instant, public_key: PublicKey, is_ca: bool }
type Pool = struct { certificates: []const Certificate }
type VerifyOptions = struct { roots: Pool, intermediates: Pool, dns_name: str, now: time.Instant, usage: KeyUsage,
```

```

    max_depth: u16 }
type KeyUsage = enum u8 { ServerAuth, ClientAuth, CodeSigning, EmailProtection, Any }
type Chain = struct { certificates: []const Certificate }
error InvalidCertificate
error UnknownAuthority
error Expired
error NameMismatch
error InvalidUsage
error TooDeep

fn parse(a: *mem.Arena, der: []const u8) -> (Certificate, err)
fn parse_pem(a: *mem.Arena, source: str) -> ([]const Certificate, err)
fn pool(a: *mem.Arena, certificates: []const Certificate) -> Pool
fn verify(a: *mem.Arena, leaf: Certificate, options: VerifyOptions) -> (Chain, err)
fn verify_signature(certificate: Certificate, issuer: Certificate) -> err

```

Parsing and verification use the DER and PEM modules and the algorithm set pinned to the toolchain. Verification receives roots and time explicitly; it never consults a host trust store, clock or network revocation service implicitly.

5. Platform and host services

e.os

```

type File = struct { raw: usize }
type Proc = struct { raw: usize }
type Thread = struct { raw: usize }
type Lib = struct { raw: usize }
type Handle = struct { raw: usize }
type Socket = struct { raw: usize }
type Poller = struct { raw: usize }
type Mapping = struct { raw: usize, address: *u8, len: usize }
type Watch = struct { raw: usize }
type WatchAction = enum u8 { Added, Removed, Modified, Renamed, Overflow }
type WatchEvent = struct { action: WatchAction, path: str, old_path: str }
type Clock = enum u8 { Wall, Monotonic }
type SeekWhence = enum u8 { Start, Current, End }
type EntryKind = enum u8 { File, Dir, Symlink, Other }
type DirEntry = struct { name: str, kind: EntryKind }
type FileInfo = struct { kind: EntryKind, size: u64 }
type OpenFlags = struct { read: bool, write: bool, create: bool, truncate: bool, append: bool }
type Stdio = struct { stdin: File, stdout: File, stderr: File, inherit: []const Handle }
type SpawnOptions = struct { argv: []const str, env: []const str, inherit_env: bool, cwd: str, stdio: Stdio }
type SocketFamily = enum u8 { Ip4, Ip6 }
type SocketKind = enum u8 { Stream, Datagram }
type SocketShutdown = enum u8 { Read, Write, Both }
type SocketAddress = struct { family: SocketFamily, bytes: [16]u8, scope: u32, port: u16 }
type PollInterest = struct { readable: bool, writable: bool }
type PollEvent = struct { token: usize, readable: bool, writable: bool, closed: bool, failed: bool }
type ErrorKind = enum u8 { NotFound, Denied, Exists, Interrupted, OutOfMemory, Timeout, WouldBlock, Unsupported, Invalid, Other }
type ErrorDetail = struct { kind: ErrorKind, native_code: i32, operation: str, subject: str }
error NotFound
error Denied
error Exists
error Interrupted
error OutOfMemory
error Failed
error Timeout
error WouldBlock
error Unsupported

fn open(a: *mem.Arena, path: str, flags: OpenFlags) -> (File, err)
fn read(f: File, buf: []u8) -> (usize, err)
fn write(f: File, buf: []const u8) -> (usize, err)
fn seek(f: File, off: i64, whence: SeekWhence) -> (u64, err)
fn close(f: File) -> err
fn stdin() -> File
fn stdout() -> File
fn stderr() -> File

```

```

fn readdir(a: *mem.Arena, path: str) -> ([DirEntry, err])
fn stat(a: *mem.Arena, path: str) -> (FileInfo, err)
fn lstat(a: *mem.Arena, path: str) -> (FileInfo, err)
fn mkdir(a: *mem.Arena, path: str) -> err
fn remove_file(a: *mem.Arena, path: str) -> err
fn remove_dir(a: *mem.Arena, path: str) -> err
fn rename(a: *mem.Arena, src: str, dst: str) -> err
fn read_link(a: *mem.Arena, path: str) -> (str, err)
fn pipe() -> (File, File, err)
fn spawn(a: *mem.Arena, argv: []const str, stdio: Stdio) -> (Proc, err)
fn spawn_with_options(a: *mem.Arena, options: SpawnOptions) -> (Proc, err)
fn wait(p: Proc) -> (i32, err)
fn kill(p: Proc) -> err
fn exit(code: i32)
fn args(a: *mem.Arena) -> ([str, err])
fn env(a: *mem.Arena, name: str) -> (str, err)
fn random(buf: []u8) -> err
fn page_size() -> usize
fn reserve(n: usize) -> (*u8, err)
fn commit(p: *u8, n: usize) -> err
fn release(p: *u8, n: usize) -> err
fn clock(c: Clock) -> (i64, err)
fn thread_create[Ctx: type](entry: fn(*Ctx), ctx: *Ctx, stack: usize) -> (Thread, err)
fn thread_join(t: Thread) -> err
fn thread_detach(t: Thread) -> err
fn wait_u32(p: *Atomic[u32], expected: u32, timeout_ns: i64) -> err
fn wake_one_u32(p: *Atomic[u32])
fn wake_all_u32(p: *Atomic[u32])
fn socket_open(family: SocketFamily, kind: SocketKind) -> (Socket, err)
fn socket_set_nonblocking(s: Socket, enabled: bool) -> err
fn socket_bind(s: Socket, address: SocketAddress) -> err
fn socket_listen(s: Socket, backlog: u32) -> err
fn socket_accept(s: Socket) -> (Socket, SocketAddress, err)
fn socket_connect(s: Socket, address: SocketAddress) -> err
fn socket_receive(s: Socket, dst: []u8) -> (usize, err)
fn socket_send(s: Socket, src: []const u8) -> (usize, err)
fn socket_receive_from(s: Socket, dst: []u8) -> (usize, SocketAddress, err)
fn socket_send_to(s: Socket, dst: SocketAddress, src: []const u8) -> (usize, err)
fn socket_shutdown(s: Socket, how: SocketShutdown) -> err
fn socket_close(s: Socket) -> err
fn socket_resolve(a: *mem.Arena, host: str, port: u16, family: SocketFamily) -> ([SocketAddress, err])
fn file_handle(f: File) -> Handle
fn socket_handle(s: Socket) -> Handle
fn poller_open(a: *mem.Arena) -> (Poller, err)
fn poller_register(p: Poller, handle: Handle, token: usize, interest: PollInterest) -> err
fn poller_modify(p: Poller, handle: Handle, token: usize, interest: PollInterest) -> err
fn poller_unregister(p: Poller, handle: Handle) -> err
fn poller_wait(p: Poller, events: []PollEvent, timeout_ns: i64) -> (usize, err)
fn poller_wake(p: Poller) -> err
fn poller_close(p: Poller) -> err
fn map_file(f: File, offset: u64, len: usize, writable: bool) -> (Mapping, err)
fn mapping_bytes(m: Mapping) -> []const u8
fn mapping_bytes_mut(m: Mapping) -> ([u8, err])
fn mapping_flush(m: Mapping) -> err
fn mapping_close(m: Mapping) -> err
fn watch_open(a: *mem.Arena, path: str, recursive: bool) -> (Watch, err)
fn watch_read(a: *mem.Arena, w: Watch, events: []WatchEvent) -> (usize, err)
fn watch_close(w: Watch) -> err
fn dlopen(a: *mem.Arena, name: str) -> (Lib, err)
fn dlsym[F: type](a: *mem.Arena, l: Lib, sym: str) -> (F, err)
fn dlclose(l: Lib) -> err
fn last_error_detail(operation: str, subject: str) -> ErrorDetail
fn error_message(a: *mem.Arena, detail: ErrorDetail) -> (str, err)
type Dir = struct { raw: usize }
type FileLock = struct { raw: usize }
type ProcGroup = struct { raw: usize }
type ResolvePolicy = enum u8 { NoSymlinks, Beneath }

fn dir_open(a: *mem.Arena, path: str) -> (Dir, err)
fn dir_close(dir: Dir) -> err
fn open_at(a: *mem.Arena, dir: Dir, relative_path: str, flags: OpenFlags, policy: ResolvePolicy) -> (File, err)
fn remove_at(a: *mem.Arena, dir: Dir, relative_path: str, directory: bool) -> err

```

```

fn rename_at(a: *mem.Arena, src_dir: Dir, src_path: str, dst_dir: Dir, dst_path: str, overwrite: bool, durable: bool) -> err
fn file_lock(file: File, exclusive: bool, timeout_ns: i64) -> (FileLock, err)
fn file_unlock(lock: FileLock) -> err
fn proc_group_spawn(a: *mem.Arena, options: SpawnOptions) -> (ProcGroup, Proc, err)
fn proc_group_terminate(group: ProcGroup, force: bool) -> err
fn proc_group_close(group: ProcGroup) -> err

```

All path strings use the host convention. `SpawnOptions.cwd == ""` inherits the current directory; `inherit_env` controls whether `env` overlays the parent environment (`true`) or is the complete child environment (`false`). Environment entries are `NAME=VALUE`. `wait_u32` waits indefinitely when `timeout_ns < 0` and polls once when it is zero. In `SocketAddress`, IPv4 uses the first four bytes and zeros the remaining twelve. Pollers retain handles, interests and numeric tokens, never callbacks; callers unregister a handle before closing it.

Every failing `e.os` call records the native code and portable classification in thread-local runtime state. `last_error_detail` copies that state into an explicit value and must be called before another `e.os` operation on that thread. Higher-level APIs may expose a detail snapshot while ordinary callers retain cheap `err/try`. `operation` and `subject` are borrowed caller strings, never inferred global state; `error_message` is the only locale-dependent rendering operation in `e.os`.

These new primitives are the reviewed platform boundary for SL05/SL06, not permission for `e.fs/e.proc` to add externs. Directory-relative operations reject absolute paths, parent traversal and embedded NULs; `NoSymlinks` rejects every traversed link/reparse point. `Beneath` permits only traversal provably confined under the opened root, or returns `Unsupported`. No lexical-prefix or canonicalize-then-open safety claim. `remove_at/rename_at` traverse without following symlinks; removing a final symlink removes the link, not its target. Durable rename may succeed before persistence fails; report that partial effect. File locks are cooperative unless the platform explicitly guarantees more. Process groups promise supported descendant containment, not a security sandbox; unsupported strict containment fails before spawning. SL05 names platform delivery requirements and escape limitations.

`e.cancel`

```

type Token = struct { state: Atomic[u32] }
type Control = struct { token: *const Token, deadline: time.Instant, has_deadline: bool }
error Cancelled
error Timeout

fn token() -> Token
fn request(t: *Token)
fn requested(t: *const Token) -> bool
fn check(control: Control, now: time.Instant) -> err

```

`Token` is non-copyable after sharing and needs no allocation or worker pool. `request` is thread-safe and idempotent; `requested(nil)` is false. `Control` borrows its token until the operation acknowledges completion. `has_deadline=false` means no deadline; zero is a valid clock value, not a sentinel. `check` observes an explicit monotonic now: cancellation takes precedence when both conditions hold at that observation. Requesting cancellation does not join work or release its buffers. No ambient values, automatic timer, parent registry or hidden allocation is added. See `stdlib-hardening.md` SL03 for operation-level completion/partial-progress rules.

`e.io`

```

type Reader = struct { ctx: *void, read: fn(*void, [u8]) -> (usize, err) }
type Writer = struct { ctx: *void, write: fn(*void, [const u8]) -> (usize, err), flush: fn(*void) -> err }
type SliceReader = struct { data: [const u8], off: usize }
type SliceWriter = struct { data: [u8], off: usize }
type BufferedReader = struct { state: *void }
type BufferedWriter = struct { state: *void }
type Seeker = struct { ctx: *void, seek: fn(*void, i64, os.SeekWhence) -> (u64, err) }
type LimitedReader = struct { source: Reader, remaining: u64 }
type CountingWriter = struct { sink: Writer, count: u64 }
type TeeWriter = struct { left: Writer, right: Writer }
type MemoryWriter = struct { arena: *mem.Arena, start: usize, len: usize }
error End
error TooSmall
error NoProgress

fn reader(ctx: *void, read_fn: fn(*void, [u8]) -> (usize, err)) -> Reader
fn writer(ctx: *void, write_fn: fn(*void, [const u8]) -> (usize, err)) -> Writer
fn file_reader(file: *os.File) -> Reader

```

```

fn file_writer(file: *os.File) -> Writer
fn slice_reader(state: *SliceReader) -> Reader
fn slice_writer(state: *SliceWriter) -> Writer
fn buffered_reader(a: *mem.Arena, source: Reader, capacity: usize) -> (BufferedReader, err)
fn buffered_writer(a: *mem.Arena, sink: Writer, capacity: usize) -> (BufferedWriter, err)
fn file_seeker(file: *os.File) -> Seeker
fn limited_reader(state: *LimitedReader, source: Reader, limit: u64) -> Reader
fn counting_writer(state: *CountingWriter, sink: Writer) -> Writer
fn tee_writer(state: *TeeWriter, left: Writer, right: Writer) -> Writer
fn memory_writer(a: *mem.Arena, capacity: usize) -> (MemoryWriter, Writer, err)
fn memory_bytes(w: *const MemoryWriter) -> []const u8
fn read(r: *Reader, dst: []u8) -> (usize, err)
fn read_exact(r: *Reader, dst: []u8) -> err
fn read_all(a: *mem.Arena, r: *Reader, limit: usize) -> ([]u8, err)
fn read_until(a: *mem.Arena, r: *Reader, delimiter: u8, limit: usize) -> ([]u8, err)
fn write(w: *Writer, src: []const u8) -> (usize, err)
fn write_all(w: *Writer, src: []const u8) -> err
fn flush(w: *Writer) -> err
fn seek(s: *Seeker, off: i64, whence: os.SeekWhence) -> (u64, err)
fn copy(dst: *Writer, src: *Reader, scratch: []u8) -> (u64, err)
fn print(s: str) -> err
fn printf[FMT: str](args: ...) -> err
fn writer_with_flush(ctx: *void, write_fn: fn(*void, []const u8) -> (usize, err), flush_fn: fn(*void) -> err) -> Writer
fn buffered_source(buffer: *BufferedReader) -> Reader
fn buffered_sink(buffer: *BufferedWriter) -> Writer

```

limit is a hard maximum; crossing it returns `TooSmall` without retaining a partial result. A callback returning `(0, ok)` for a non-empty request returns `NoProgress`. `tee_writer` writes each input to the left sink before the right sink and stops on the first error; it therefore does not promise transactional duplication.

Buffered adapters borrow their wrapper and transitively its source/sink and arena. They never close the underlying stream. `writer` constructs an unbuffered sink whose flush callback is `nil` (flush succeeds without work); `writer_with_flush` supplies an explicit callback. `buffered_sink` flushes buffered bytes before forwarding flush. Partial writes retain only the unwritten suffix; a failed flush is not a rollback. Compression finish, protocol shutdown, and durable filesystem sync are distinct operations, never implied by generic flush. See `stdlib-hardening.md` SL02.

text.io

```

type Reader = struct { state: *void }
type Writer = struct { state: *void }
type Newline = enum u8 { Lf, CrLf, Native }
error End
error TooLarge
error Invalid

fn reader(a: *mem.Arena, source: io.Reader, source_encoding: encoding.Encoding, policy: encoding.InvalidPolicy, capacity: usize) -> (Reader, err)
fn reader_bom(a: *mem.Arena, source: io.Reader, fallback: encoding.Encoding, policy: encoding.InvalidPolicy, capacity: usize) -> (Reader, err)
fn read_line(a: *mem.Arena, r: *Reader, limit: usize) -> (str, err)
fn read_all(a: *mem.Arena, r: *Reader, limit: usize) -> (str, err)
fn writer(a: *mem.Arena, sink: io.Writer, source_encoding: encoding.Encoding, emit_bom: bool, newline: Newline, capacity: usize) -> (Writer, err)
fn write(w: *Writer, text: str) -> err
fn write_line(w: *Writer, text: str) -> err
fn flush(w: *Writer) -> err

```

Returned text is UTF-8. `read_line` accepts LF, CRLF, or a final unterminated line and excludes its terminator; a bare CR is content. `Newline.Native` is resolved when the writer is constructed. Reader and writer state is arena-owned and non-copyable.

e.fs

```

type EntryKind = enum u8 { File, Directory, Symlink, Other }
type Entry = struct { path: str, kind: EntryKind, size: u64 }
type Permissions = struct { owner_read: bool, owner_write: bool, owner_exec: bool, group_read: bool, group_write: bool, group_exec: bool, other_read: bool, other_write: bool, other_exec: bool }
type Metadata = struct { kind: EntryKind, size: u64, modified_ns: i64, accessed_ns: i64, created_ns: i64, permissions: Permissions, file_id: u64, link_count: u64 }

```

```

type Walk = struct { state: *void }
type WalkOptions = struct { recursive: bool, follow_symlinks: bool }
type ReplaceOptions = struct { overwrite: bool, durable: bool }
error NotFound
error Exists
error Denied
error Invalid
error Io

fn exists(a: *mem.Arena, path_text: str) -> (bool, err)
fn stat(a: *mem.Arena, path_text: str) -> (Entry, err)
fn metadata(a: *mem.Arena, path_text: str, follow_symlinks: bool) -> (Metadata, err)
fn set_permissions(a: *mem.Arena, path_text: str, permissions: Permissions) -> err
fn set_times(a: *mem.Arena, path_text: str, accessed_ns: i64, modified_ns: i64) -> err
fn make_dir(a: *mem.Arena, path_text: str) -> err
fn make_dirs(a: *mem.Arena, path_text: str) -> err
fn remove_file(a: *mem.Arena, path_text: str) -> err
fn remove_dir(a: *mem.Arena, path_text: str) -> err
fn copy_file(a: *mem.Arena, src: str, dst: str, scratch: []u8) -> err
fn move(a: *mem.Arena, src: str, dst: str) -> err
fn replace(a: *mem.Arena, src: str, dst: str, options: ReplaceOptions) -> err
fn symlink(a: *mem.Arena, target_path: str, link: str) -> err
fn read_link(a: *mem.Arena, path_text: str) -> (str, err)
fn canonical(a: *mem.Arena, path_text: str) -> (str, err)
fn current_dir(a: *mem.Arena) -> (str, err)
fn set_current_dir(a: *mem.Arena, path_text: str) -> err
fn executable_path(a: *mem.Arena) -> (str, err)
fn temp_dir(a: *mem.Arena) -> (str, err)
fn temp_file(a: *mem.Arena, dir: str, prefix: str) -> (str, os.File, err)
fn home_dir(a: *mem.Arena) -> (str, err)
fn read_file(a: *mem.Arena, path_text: str, limit: usize) -> ([]u8, err)
fn write_file(a: *mem.Arena, path_text: str, data: []const u8) -> err
fn walk(a: *mem.Arena, root_path: str, options: WalkOptions) -> (Walk, err)
fn walk_next_err(it: *Walk) -> (Entry, bool, err)
fn last_error_detail(operation: str, subject: str) -> os.ErrorDetail
type Root = struct { dir: os.Dir }

fn root(a: *mem.Arena, path_text: str) -> (Root, err)
fn root_close(r: *Root) -> err
fn open_at(a: *mem.Arena, r: *const Root, relative_path: str, flags: os.OpenFlags, policy: os.ResolvePolicy) -> (os.File, err)
fn remove_at(a: *mem.Arena, r: *const Root, relative_path: str, directory: bool) -> err
fn replace_at(a: *mem.Arena, src_root: *const Root, src_path: str, dst_root: *const Root, dst_path: str, options: ReplaceOptions) -> err
fn walk_close(it: *Walk) -> err

```

Unavailable creation times are -1; timestamps are Unix-epoch nanoseconds. Windows permissions map only the portable read-only/executable subset and unsupported changes return `os.Unsupported`. `temp_file` creates a new file atomically with exclusive access and never returns a predictable uncreated name. `replace` is atomic within one filesystem or returns `Invalid`; durable requests file and parent-directory persistence. `last_error_detail` snapshots the underlying failure without another host operation and is called before any cleanup that could replace it.

Root owns its directory handle and follows H01; its methods retain the SL06 handle-relative semantics rather than normalizing strings then using path APIs. `walk_close` releases an abandoned traversal and is required on early exit; exhaustion releases OS traversal handles. Path functions remain convenience APIs, not sandboxes. `last_error_detail` is a legacy M2 bridge pending H07, not the revised checked API's error-detail transport. H07 migration must preserve partial effects and cleanup errors.

e.fs.mmap

```

type Mapping = struct { raw: os.Mapping }
error Empty
error Invalid

fn open(a: *mem.Arena, path: str, writable: bool, offset: u64, len: usize) -> (Mapping, err)
fn bytes(m: Mapping) -> []u8
fn flush(m: Mapping) -> err
fn close(m: Mapping) -> err

```

Offset and length are byte values; the implementation performs page alignment without changing the returned slice. Zero length returns Empty. close consumes the mapping and invalidates every derived pointer or slice.

e.fs.watch

```
type Watch = struct { raw: os.Watch }
type Action = enum u8 { Added, Removed, Modified, Renamed, Overflow }
type Event = struct { action: Action, path: str, old_path: str }
error Closed
error Unsupported

fn open(a: *mem.Arena, path: str, recursive: bool) -> (Watch, err)
fn read(a: *mem.Arena, watch: Watch, events: []Event) -> (usize, err)
fn close(watch: Watch) -> err
```

Paths are relative to the watched root. Renamed supplies both paths when the host can pair them and otherwise appears as remove/add. Overflow means callers must rescan. Events may coalesce and no operation assumes one event per host change.

e.proc

```
type Command = struct { program: str, args: []const str, env: []const str, inherit_env: bool, cwd: str }
type Streams = struct { stdin: os.File, stdout: os.File, stderr: os.File }
type Child = struct { process: os.Proc, streams: Streams }
type Output = struct { status: i32, stdout: []u8, stderr: []u8 }
error TooLarge

fn spawn(a: *mem.Arena, command: Command, streams: Streams) -> (Child, err)
fn spawn_piped(a: *mem.Arena, command: Command) -> (Child, err)
fn wait(child: *Child) -> (i32, err)
fn kill(child: *Child) -> err
fn output(a: *mem.Arena, command: Command, limit: usize) -> (Output, err)
type Outcome = enum u8 { Exited, Cancelled, TimedOut, OutputLimit }
type RunOptions = struct { control: cancel.Control, stdout_limit: usize, stderr_limit: usize, terminate_grace: tim
e.Duration, contain_tree: bool }
type RunResult = struct { outcome: Outcome, status: i32, status_known: bool, stdout: []const u8, stderr: []const u
8, stdout_bytes: u64, stderr_bytes: u64, truncated: bool }

fn run(a: *mem.Arena, command: Command, options: RunOptions) -> (RunResult, err)
```

An empty cwd inherits the parent directory. inherit_env has the same overlay vs replacement meaning as os.SpawnOptions; environment entries are NAME=VALUE.

run drains stdout/stderr concurrently, enforces independent capture budgets, and returns only after reaping its child and closing its capture pipes. Exited includes nonzero status; cancellation/timeout/output limit are explicit outcomes, not API success of the child. Overflow retains bounded prefixes, initiates termination, and records truncation and observed byte totals. Infrastructure failure returns err and a valid possibly partial RunResult; callers must inspect err first. A control deadline governs running work; terminate_grace bounds cooperative shutdown before forced termination. A separate bounded drain follows, then inherited pipe writers are closed locally and truncation recorded. contain_tree requires a supported containment facility established before child code executes; unsupported containment is rejected before spawning. No shell is inserted. Child-only mode does not promise descendant cleanup. See SL05 for platform limits and test cases.

e.thread

```
type Thread = os.Thread
const DEFAULT_STACK: usize = 1048576

fn spawn[Ctx: type](entry: fn(*Ctx), ctx: *Ctx, stack: usize) -> (Thread, err)
fn join(thread: Thread) -> err
fn detach(thread: Thread) -> err
```

e.sync

```
type Mutex = struct { state: Atomic[u32] }
type RwLock = struct { state: Atomic[u32] }
type Condition = struct { state: Atomic[u32] }
type Semaphore = struct { state: Atomic[u32] }
type Event = struct { state: Atomic[u32], manual_reset: bool }
```

```

type Once = struct { state: Atomic[u32] }
type Barrier = struct { state: *void }
error Invalid

fn mutex() -> Mutex
fn mutex_lock(m: *Mutex)
fn mutex_try_lock(m: *Mutex) -> bool
fn mutex_lock_for(m: *Mutex, timeout: time.Duration) -> bool
fn mutex_unlock(m: *Mutex)
fn rwlock() -> RwLock
fn rwlock_read_lock(l: *RwLock)
fn rwlock_try_read_lock(l: *RwLock) -> bool
fn rwlock_read_lock_for(l: *RwLock, timeout: time.Duration) -> bool
fn rwlock_read_unlock(l: *RwLock)
fn rwlock_write_lock(l: *RwLock)
fn rwlock_try_write_lock(l: *RwLock) -> bool
fn rwlock_write_lock_for(l: *RwLock, timeout: time.Duration) -> bool
fn rwlock_write_unlock(l: *RwLock)
fn condition() -> Condition
fn condition_wait(c: *Condition, m: *Mutex)
fn condition_wait_for(c: *Condition, m: *Mutex, timeout: time.Duration) -> bool
fn condition_signal(c: *Condition)
fn condition_broadcast(c: *Condition)
fn semaphore(initial: u32) -> Semaphore
fn semaphore_wait(s: *Semaphore)
fn semaphore_try_wait(s: *Semaphore) -> bool
fn semaphore_wait_for(s: *Semaphore, timeout: time.Duration) -> bool
fn semaphore_post(s: *Semaphore, count: u32) -> err
fn event(manual_reset: bool, signaled: bool) -> Event
fn event_set(e: *Event)
fn event_reset(e: *Event)
fn event_wait(e: *Event)
fn event_wait_for(e: *Event, timeout: time.Duration) -> bool
fn once() -> Once
fn once_call[Ctx: type](o: *Once, ctx: *Ctx, f: fn(*Ctx) -> err) -> err
fn barrier(a: *mem.Arena, parties: u32) -> (Barrier, err)
fn barrier_wait(b: *Barrier) -> bool
fn barrier_close(b: *Barrier)

```

All waits recheck their state after `os.wait_u32`, so spurious wakes are invisible to callers. `semaphore_post` returns `Invalid` instead of wrapping the count. Timed waits return `false` only on timeout. A negative duration is invalid and zero polls once. `once_call` publishes completion only after `f` returns ok; an error permits a later caller to retry. Exactly one participant receives `true` from each barrier generation.

e.channel

```

type Channel[T: type] = struct { state: *void }
error Closed

fn init[T: type](a: *mem.Arena, cap: usize) -> (Channel[T], err)
fn send[T: type](c: *Channel[T], value: T) -> err
fn try_send[T: type](c: *Channel[T], value: T) -> (bool, err)
fn receive[T: type](c: *Channel[T]) -> (T, err)
fn try_receive[T: type](c: *Channel[T]) -> (T, bool, err)
fn close[T: type](c: *Channel[T]) -> err
fn len[T: type](c: *const Channel[T]) -> usize
fn capacity[T: type](c: *const Channel[T]) -> usize

```

A capacity of zero is invalid. Closing wakes all waiters; buffered values remain receivable before `Closed` is returned.

e.concurrent.queue

```

type Queue[T: type] = struct { state: *void }
error Closed
error Invalid

fn init[T: type](a: *mem.Arena, initial_capacity: usize) -> (Queue[T], err)
fn try_push[T: type](q: *Queue[T], value: T) -> (bool, err)
fn push[T: type](q: *Queue[T], value: T) -> err
fn push_for[T: type](q: *Queue[T], value: T, timeout: time.Duration) -> (bool, err)
fn try_pop[T: type](q: *Queue[T]) -> (T, bool, err)

```

```

fn pop[T: type](q: *Queue[T]) -> (T, err)
fn pop_for[T: type](q: *Queue[T], timeout: time.Duration) -> (T, bool, err)
fn close[T: type](q: *Queue[T]) -> err
fn len[T: type](q: *const Queue[T]) -> usize
fn capacity[T: type](q: *const Queue[T]) -> usize

```

This is a bounded multi-producer/multi-consumer FIFO. Closing wakes every waiter; buffered values remain readable before `Closed`. Timed calls return `false` only on timeout. Capacity is fixed and all state is allocated during `init`.

`e.concurrent.map`

```

type Map[K: type, V: type] = struct { state: *void }
error Closed
error Full
error Invalid

fn init[K: type, V: type](a: *mem.Arena, capacity: usize, shards: u16) -> (Map[K, V], err)
fn get[K: type, V: type](m: *const Map[K, V], key: K) -> (V, bool, err)
fn put[K: type, V: type](m: *Map[K, V], key: K, value: V) -> (bool, err)
fn remove[K: type, V: type](m: *Map[K, V], key: K) -> (V, bool, err)
fn len[K: type, V: type](m: *const Map[K, V]) -> (usize, err)
fn close[K: type, V: type](m: *Map[K, V]) -> err

```

The map is fixed-capacity and sharded. Individual operations are linearizable; `len` is a point-in-time sum and not a transaction with surrounding operations. User hash/equality protocol functions must be thread-safe. Iteration and borrowed value pointers are deliberately absent because concurrent mutation would invalidate them. `init` returns `Invalid` for zero capacity or shards; insertion of a new key into a full map returns `Full`, while replacing an existing key still succeeds.

`e.task`

```

type Pool = struct { state: *void }
type Task = struct { state: *void }
type Future[T: type] = struct { state: *void }
type Cancel = cancel.Token
type Status = enum u8 { Pending, Running, Succeeded, Failed, Cancelled }
type Options = struct { workers: u32, queue_capacity: usize, stack_size: usize }
error Cancelled
error Closed
error Invalid

fn pool(a: *mem.Arena, options: Options) -> (Pool, err)
fn submit[Ctx: type](p: *Pool, cancel_token: *Cancel, ctx: *Ctx, f: fn(*Ctx, *const Cancel) -> err) -> (Task, err)
fn future[T: type, Ctx: type](p: *Pool, cancel_token: *Cancel, ctx: *Ctx, f: fn(*Ctx, *const Cancel) -> (T, err)) -> (Future[T], err)
fn status(t: *const Task) -> Status
fn wait(t: *Task) -> err
fn wait_for(t: *Task, timeout: time.Duration) -> (bool, err)
fn future_get[T: type](f: *Future[T]) -> (T, err)
fn wait_any(tasks: []*Task, timeout: time.Duration) -> (usize, bool, err)
fn wait_all(tasks: []*Task) -> err
fn parallel_for[Ctx: type](p: *Pool, cancel_token: *Cancel, begin: usize, end: usize, grain: usize, ctx: *Ctx, body: fn(*Ctx, usize, *const Cancel) -> err) -> err
fn close(p: *Pool) -> err

```

The pool and all task records borrow their arena for the pool lifetime. Submission is bounded and never allocates secretly. Cancellation is cooperative: queued work may become `Cancelled`, while running work observes the shared `e.cancel` token; construct/request it through that module. The `Cancel` alias preserves a named task parameter type, not a second token system. `close` rejects new work, cancels queued work and joins every worker. `wait_all` returns the first error in input order, not completion order. `parallel_for` partitions `[begin..end)` into deterministic ranges no smaller than `grain` except the last; scheduling order is not observable and callers synchronize shared state explicitly.

`e.time`

```

type Timestamp = struct { nanos: i64 }
type Instant = struct { nanos: i64 }
type Duration = struct { nanos: i64 }
type Date = struct { year: i32, month: u8, day: u8 }
type Time = struct { hour: u8, minute: u8, second: u8, nanos: u32 }

```

```

type Timer = struct { started: Instant }
error Invalid

fn now() -> (Timestamp, err)
fn monotonic() -> (Instant, err)
fn timer_start() -> (Timer, err)
fn timer_elapsed(t: Timer) -> Duration
fn since(start: Instant) -> Duration
fn timestamp_add(t: Timestamp, d: Duration) -> Timestamp
fn timestamp_diff(a: Timestamp, b: Timestamp) -> Duration
fn timestamp_cmp(a: Timestamp, b: Timestamp) -> i32
fn instant_add(t: Instant, d: Duration) -> Instant
fn instant_diff(a: Instant, b: Instant) -> Duration
fn instant_cmp(a: Instant, b: Instant) -> i32
fn duration_add(a: Duration, b: Duration) -> Duration
fn duration_sub(a: Duration, b: Duration) -> Duration
fn duration_neg(d: Duration) -> Duration
fn duration_scale(d: Duration, n: i64) -> Duration
fn duration_cmp(a: Duration, b: Duration) -> i32
fn days(n: i64) -> Duration
fn hours(n: i64) -> Duration
fn minutes(n: i64) -> Duration
fn seconds(n: i64) -> Duration
fn millis(n: i64) -> Duration
fn micros(n: i64) -> Duration
fn nanos(n: i64) -> Duration
fn as_days(d: Duration) -> i64
fn as_hours(d: Duration) -> i64
fn as_minutes(d: Duration) -> i64
fn as_seconds(d: Duration) -> i64
fn as_millis(d: Duration) -> i64
fn as_micros(d: Duration) -> i64
fn as_nanos(d: Duration) -> i64
fn to_date(t: Timestamp) -> Date
fn to_time(t: Timestamp) -> Time
fn to_date_at(t: Timestamp, offset_minutes: i32) -> Date
fn to_time_at(t: Timestamp, offset_minutes: i32) -> Time
fn from_civil(d: Date, t: Time) -> (Timestamp, err)
fn format_iso8601(t: Timestamp, buf: [u8]) -> str
fn parse_iso8601(s: str) -> (Timestamp, err)

```

e.time.calendar

```

type DateTime = struct { date: time.Date, time: time.Time }
type Weekday = enum u8 { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday }
type IsoWeek = struct { year: i32, week: u8 }
type Components = struct { year: i32, month: u8, day: u8, hour: u8, minute: u8, second: u8, nanos: u32, weekday: Weekday, day_of_year: u16 }
error Invalid

fn is_leap_year(year: i32) -> bool
fn days_in_month(year: i32, month: u8) -> (u8, err)
fn valid_date(date: time.Date) -> bool
fn valid_time(value: time.Time) -> bool
fn weekday(date: time.Date) -> (Weekday, err)
fn day_of_year(date: time.Date) -> (u16, err)
fn iso_week(date: time.Date) -> (IsoWeek, err)
fn compare(a: DateTime, b: DateTime) -> i32
fn add_days(value: DateTime, days: i64) -> (DateTime, err)
fn add_months(value: DateTime, months: i64) -> (DateTime, err)
fn add_years(value: DateTime, years: i64) -> (DateTime, err)
fn difference_days(a: DateTime, b: DateTime) -> i64
fn components(value: DateTime) -> (Components, err)
fn from_components(value: Components) -> (DateTime, err)
fn format[PATTERN: str](value: DateTime, dst: [u8]) -> (str, err)
fn parse[PATTERN: str](source: str) -> (DateTime, err)

```

This is proleptic Gregorian calendar arithmetic with ISO-8601 weekdays and weeks. Adding months or years clamps the day to the last valid day of the target month. Patterns are compile-time strings over the closed verbs yyyy MM dd HH mm ss ssssssss; punctuation is literal and locale names are deliberately absent. Named zones and daylight-saving ambiguity belong to e.tz.

e.tz

```

type Database = struct { state: *void }
type Zone = struct { state: *const void }
type Local = struct { date: time.Date, time: time.Time }
type Offset = struct { seconds: i32, daylight: bool, abbreviation: str }
type Resolve = union enum u8 { Unique: time.Timestamp, Ambiguous: [2]time.Timestamp, Missing: [2]time.Timestamp }
error InvalidData
error NotFound
error TooLarge

fn load(a: *mem.Arena, source: []const u8) -> (Database, err)
fn builtin(a: *mem.Arena) -> (Database, err)
fn version(db: *const Database) -> str
fn zone(db: *const Database, name: str) -> (Zone, err)
fn offset_at(selected_zone: Zone, instant: time.Timestamp) -> Offset
fn to_local(selected_zone: Zone, instant: time.Timestamp) -> Local
fn resolve(selected_zone: Zone, local: Local) -> (Resolve, err)
fn names(db: *const Database) -> []const str

```

`builtin` exposes the IANA Time Zone Database release pinned by the toolchain and reported by `version`; therefore a toolchain upgrade is the only implicit data upgrade. `load` permits an explicitly supplied compatible database for reproducible historical work. `Ambiguous` timestamps are ascending. `Missing` contains the timestamps immediately before and after the local-time gap. Neither function consults mutable host time-zone state.

6. Language-owned runtimes

e.asset

```

type Attribute = struct { name: str, value: str }
type Asset = struct { name: str, media_type: str, bytes: []const u8, sha256: [32]u8, attributes: []const Attribute }

fn count() -> usize
fn at(index: usize) -> (Asset, bool)
fn get(name: str) -> (Asset, bool)
fn attribute(value: Asset, name: str) -> (str, bool)

```

The linker generates one immutable registry from the current target's `project.yaml` asset declarations. Entries are sorted by UTF-8 logical name; lookup is allocation-free and returns executable-backed slices valid for the process lifetime. Asset contents, names, media types, attributes and SHA-256 values are part of the executable and build identity. A build with no assets exposes an empty registry. This module never opens a file and provides no mutation, decompression or global cache.

e.gpu

```

type Backend = enum u8 { Cpu, Vulkan, Cuda }
type DeviceKind = enum u8 { Unknown, Cpu, Integrated, Discrete, Virtual, Other }
type DeviceKey = struct { backend: Backend, uuid: [16]u8 }
type DeviceInfo = struct {
    key: DeviceKey,
    key_valid: bool,
    index: u32,
    name: str,
    kind: DeviceKind,
    memory_bytes: u64,
    memory_known: bool,
    capabilities: []const Cap,
    supported: bool,
}
type Device = struct { state: *void }
type Queue = struct { state: *void }
type Buf[T: type] = struct { owner: u32, slot: u32, generation: u32, len: usize }
type Grid = struct { x: usize, y: usize, z: usize }
type Id = struct { x: u32, y: u32, z: u32 }
type Cap = enum u8 { Int8, Int16, Int64, Float16, Float64, Atomic64, Subgroup, Ftz, DenormPreserve }
type Scope = enum u8 { Workgroup, Device }

```

```

error NoDevice
error AmbiguousDevice
error Unsupported
error OutOfMemory
error TooLarge
error Lost
error WrongDevice
error InvalidHandle

fn open(a: *mem.Arena, backend: Backend, index: u32) -> (*Device, err)
fn devices(a: *mem.Arena, backend: Backend, limit: usize) -> ([]const DeviceInfo, err)
fn open_id(a: *mem.Arena, key: DeviceKey) -> (*Device, err)
fn info(a: *mem.Arena, device: *Device) -> (DeviceInfo, err)
fn close(device: *Device) -> err
fn has(device: *Device, capability: Cap) -> bool
fn queue(device: *Device) -> (*Queue, err)
fn alloc[T: type](q: *Queue, n: usize) -> (Buf[T], err)
fn upload[T: type](q: *Queue, src: []const T) -> (Buf[T], err)
fn len[T: type](buf: Buf[T]) -> usize
fn write[T: type](q: *Queue, dst: Buf[T], off: usize, src: []const T) -> err
fn launch[K: fn](q: *Queue, grid: Grid, args: ...) -> err
fn download[T: type](q: *Queue, src: Buf[T], dst: []T) -> err
fn sync(q: *Queue) -> err
fn release[T: type](q: *Queue, buf: Buf[T]) -> err
fn grid1(x: usize) -> Grid
fn grid2(x: usize, y: usize) -> Grid
fn grid3(x: usize, y: usize, z: usize) -> Grid

```

Discovery/selection is specified in [spec §10](#). `devices` is a bounded, caller-arena-owned snapshot, including copied names and capability lists; exceeding the limit fails rather than returning a partial success. Indices are temporary backend ordinals. `open_id` requires an exact backend-scoped UUID match, rejects duplicate matches with `AmbiguousDevice`, and never silently falls back. `key_valid == false` means stable-key selection is unavailable, not that a fabricated index/name hash may substitute. `info` copies the opening-time descriptor of the selected device. Memory is reported capacity, not free or reserved storage. Every queue/buffer belongs to one open device, including when two opens address the same GPU. These additions are planned for M3 CPU/Vulkan, with CUDA in M4; they do not claim implementation or an optimized production CPU fallback.

The device-only intrinsics are exactly `gid`, `lid`, `wgid`, `barrier`, `subgroup_size`, `subgroup_lane`, `subgroup_ballot`, `subgroup_any`, `subgroup_all`, `subgroup_broadcast`, `subgroup_add`, `subgroup_min`, `subgroup_max`, and the scoped atomic family specified by [spec §10](#).

`e.gpu.tensor`

```

type Tensor[T: type] = struct { data: gpu.Buf[T], shape: []const usize, stride: []const usize }
error Shape

fn upload[T: type](a: *mem.Arena, queue: *gpu.Queue, src: linalg_tensor.ConstTensor[T]) -> (Tensor[T], err)
fn download[T: type](queue: *gpu.Queue, src: Tensor[T], dst: linalg_tensor.Tensor[T]) -> err
fn add[T: type](queue: *gpu.Queue, dst: Tensor[T], x: Tensor[T], y: Tensor[T]) -> err
fn matmul[T: type](queue: *gpu.Queue, dst: Tensor[T], x: Tensor[T], y: Tensor[T]) -> err
fn release[T: type](queue: *gpu.Queue, tensor_view: Tensor[T]) -> err

```

The import of `algo.linalg.tensor` uses the deterministic alias `linalg_tensor`. Every operation is an explicit queue submission; this module never opens a device or allocates a host arena implicitly.

`e.test`

```

error Failed

fn assert(cond: bool, msg: str) -> err
fn eq[T: type](a: T, b: T, msg: str) -> err
fn near(a: f64, b: f64, abs: f64, rel: f64, msg: str) -> err
fn fail(msg: str) -> err

```

Discovery, process isolation and reporting belong to `neper test`, not this module.

`e.test.support`

```

type Clock = struct { instant: time.Instant, timestamp: time.Timestamp }
type ReadStep = struct { bytes: []const u8, failure: err }
type WriteStep = struct { max_bytes: usize, failure: err }
type ScriptedReader = struct { state: *void }
type ScriptedWriter = struct { state: *void }
type Schedule = struct { state: *void }

fn clock(instant: time.Instant, timestamp: time.Timestamp) -> Clock
fn advance(c: *Clock, elapsed: time.Duration) -> err
fn scripted_reader(a: *mem.Arena, steps: []const ReadStep) -> (ScriptedReader, err)
fn scripted_writer(a: *mem.Arena, steps: []const WriteStep, capacity: usize) -> (ScriptedWriter, err)
fn reader(script: *ScriptedReader) -> io.Reader
fn writer(script: *ScriptedWriter) -> io.Writer
fn captured(script: *const ScriptedWriter) -> []const u8
fn schedule(a: *mem.Arena, turns: []const u64) -> (Schedule, err)
fn checkpoint(s: *Schedule, participant: u64) -> (bool, err)
fn complete(s: *const Schedule) -> bool

```

These are caller-owned deterministic test doubles, never replacements for ambient OS state. Script steps are copied; input bytes remain borrowed until the scripted reader is released with its arena. Partial data plus an error is intentional. Script/capture exhaustion is explicit; no callback runs unboundedly. Clock advance rejects negative durations and overflow. Schedule is a single-threaded cooperative test driver: checkpoint advances only the prescribed participant, returning false for others; exhaustion returns test.Failed. It is not an OS-thread scheduler or race proof. Production APIs accept explicit clock//O/control inputs where applicable.

e.test.coverage

```

type Counter = struct { file_id: u32, region_id: u32, hits: u64 }
type Report = struct { counters: []const Counter }
error InvalidProfile
error TooLarge

fn snapshot(dst: []Counter) -> ([]Counter, err)
fn reset()
fn merge(a: *mem.Arena, profiles: []const Report) -> (Report, err)
fn write_json(writer: *io.Writer, report: *const Report) -> err

```

The compiler assigns stable file and region identifiers and owns instrumentation. This runtime only snapshots, resets, merges and serializes bounded counters.

e.test.fuzz

```

type Input = struct { bytes: []const u8, seed: u64 }
type Options = struct { max_input: usize, max_runs: u64, deadline: time.Instant }
type Result = struct { runs: u64, failing: Input, failed: bool }
type Target = fn(input: Input) -> err
error InvalidCorpus
error Limit

fn run(a: *mem.Arena, target_fn: Target, corpus: []const Input, options: Options) -> (Result, err)
fn minimize(a: *mem.Arena, target_fn: Target, failing: Input, deadline: time.Instant) -> (Input, err)

```

Mutation is deterministic from each explicit seed. Corpus persistence, subprocess isolation and reproduction commands belong to `neper test --fuzz`.

e.debug

```

type Frame = struct { address: usize, function: str, file: str, line: u32 }

fn backtrace(dst: []Frame) -> []Frame
fn symbolize(address: usize) -> Frame

```

Both functions allocate nothing. Missing symbol data yields empty strings and line zero while preserving the address.

7. Application facilities and networking

e.metrics

```

type Counter = struct { value: Atomic[u64] }
type Gauge = struct { value: Atomic[i64] }
type Histogram = struct { bounds: []const f64, counts: []Atomic[u64], sum_bits: Atomic[u64] }
type Snapshot = struct { count: u64, sum: f64, buckets: []const u64 }
error Invalid

fn counter() -> Counter
fn counter_add(c: *Counter, value: u64)
fn counter_get(c: *Counter) -> u64
fn gauge() -> Gauge
fn gauge_set(g: *Gauge, value: i64)
fn gauge_add(g: *Gauge, value: i64)
fn gauge_get(g: *Gauge) -> i64
fn histogram(a: *mem.Arena, bounds: []const f64) -> (Histogram, err)
fn histogram_observe(h: *Histogram, value: f64)
fn histogram_snapshot(a: *mem.Arena, h: *const Histogram) -> (Snapshot, err)

```

Bounds must be finite and strictly increasing. The final implicit bucket is positive infinity.

e.log

```

type Level = enum u8 { Trace, Debug, Info, Warn, Error }
type Value = union enum u8 { Bool: bool, I64: i64, U64: u64, F64: f64, String: str, Error: err }
type Field = struct { name: str, value: Value }
type Record = struct { timestamp: time.Timestamp, level: Level, message: str, fields: []const Field, frames: []const debug.Frame }
type Sink = struct { ctx: *void, write: fn(*void, *const Record) -> err }
type Logger = struct { sinks: []const Sink, minimum: Level }

fn logger(sinks: []const Sink, minimum: Level) -> Logger
fn enabled(l: *const Logger, level: Level) -> bool
fn write(l: *Logger, level: Level, message: str, fields: []const Field) -> err
fn write_with_backtrace(l: *Logger, level: Level, message: str, fields: []const Field, scratch: []debug.Frame) -> err
fn console_sink(file: *os.File) -> Sink
fn jsonl_sink(writer: *io.Writer) -> Sink

```

The caller owns every string and field until all synchronous sink calls return. There is no global logger.

e.cli

```

type ValueKind = enum u8 { Bool, I64, U64, F64, String }
type Option = struct { long: str, short: u8, kind: ValueKind, required: bool, repeated: bool, env: str, help: str }
type Command = struct { name: str, help: str, options: []const Option, subcommands: []const Command }
type Value = union enum u8 { Bool: bool, I64: i64, U64: u64, F64: f64, String: str }
type ParsedOption = struct { name: str, values: []const Value }
type Result = struct { command: str, options: []const ParsedOption, positionals: []const str }
error InvalidSpec
error InvalidArgument
error Missing
error Unknown

fn validate(command: *const Command) -> err
fn parse(a: *mem.Arena, command: *const Command, args: []const str) -> (Result, err)
fn parse_into[T: type](a: *mem.Arena, args: []const str) -> (T, err)
fn option(result: *const Result, name: str) -> (ParsedOption, bool)
fn help(a: *mem.Arena, command: *const Command, width: u16) -> (str, err)

```

parse_into uses e.meta field shape and field-owned parsing protocols; it does not introduce runtime reflection.

e.async

```

type Loop = struct { state: *void }
type Token = struct { value: usize }
type Interest = struct { readable: bool, writable: bool }
type Event = struct { token: Token, readable: bool, writable: bool, closed: bool, failed: bool }
error Unsupported
error Invalid

fn init(a: *mem.Arena) -> (Loop, err)
fn register(loop: *Loop, handle: os.Handle, token: Token, interest: Interest) -> err

```

```
fn modify(loop: *Loop, handle: os.Handle, token: Token, interest: Interest) -> err
fn unregister(loop: *Loop, handle: os.Handle) -> err
fn poll(loop: *Loop, events: []Event, timeout: time.Duration) -> (usize, err)
fn wake(loop: *Loop) -> err
fn close(loop: *Loop) -> err
```

This is readiness/completion polling only. It introduces no futures, coroutines, scheduler, callback registry or hidden state-machine transformation.

e.async.io

```
type Op[T: type] = struct { state: *void }
type AnyOp = struct { state: *void }
type State = enum u8 { Pending, Succeeded, Failed, Cancelled, TimedOut }
error Cancelled
error Timeout
error Closed

fn read(a: *mem.Arena, loop: *async.Loop, source: io.Reader, dst: []u8, control: cancel_api.Control) -> (Op[usize], err)
fn write(a: *mem.Arena, loop: *async.Loop, sink: io.Writer, src: []const u8, control: cancel_api.Control) -> (Op[usize], err)
fn accept(a: *mem.Arena, loop: *async.Loop, listener: os.Socket, control: cancel_api.Control) -> (Op[os.Socket], err)
fn connect(a: *mem.Arena, loop: *async.Loop, socket: os.Socket, address: os.SocketAddress, control: cancel_api.Control) -> (Op[bool], err)
fn state[T: type](op: *const Op[T]) -> State
fn erase[T: type](op: *Op[T]) -> AnyOp
fn take[T: type](op: *Op[T]) -> (T, err)
fn wait[T: type](loop: *async.Loop, op: *Op[T]) -> (T, err)
fn wait_any(loop: *async.Loop, ops: []AnyOp, timeout: time.Duration) -> (usize, bool, err)
fn cancel[T: type](op: *Op[T]) -> (bool, err)
type Progress = struct { bytes: u64, known: bool }
fn progress[T: type](op: *const Op[T]) -> Progress
```

Submission never blocks and every operation completes exactly once. Buffers, handles, contexts and cancellation tokens must outlive completion. Control uses e.cancel's explicit deadline flag. cancel returns whether it newly requested cancellation; it does not acknowledge completion. A completed operation wins a later cancellation request. Otherwise cancellation is cooperative and the terminal state reports Cancelled/TimedOut with observable partial effects preserved. No consumed bytes or transmitted bytes are silently reported as rolled back. wait drives the supplied loop; applications may instead poll it directly. take consumes a completed operation and returns its exact I/O error. erase is a non-owning typed conversion used only to build a heterogeneous wait_any slice; the original Op[T] must remain live.

Progress is a monotonic observation, not completion acknowledgement. Byte operations report exact completed bytes at terminal state even on failure/cancellation; operations without a byte metric report known=false. take remains mandatory to consume a terminal operation. An uncancellable blocking callback cannot be advertised as promptly cancellable: reject an incompatible submission or keep resources pinned until it ends.

e.net

```
type Socket = os.Socket
type Ip4 = struct { bytes: [4]u8 }
type Ip6 = struct { bytes: [16]u8, scope: u32 }
type Address = union enum u8 { Ip4: Ip4, Ip6: Ip6 }
type Endpoint = struct { address: Address, port: u16 }
type Family = enum u8 { Any, Ip4, Ip6 }
type Shutdown = enum u8 { Read, Write, Both }
error NotFound
error Refused
error Reset
error Timeout
error AddressInUse
error Unreachable
error Failed

fn parse_ip(s: str) -> (Address, err)
fn format_ip(address: Address, dst: []u8) -> (str, err)
fn resolve(a: *mem.Arena, host: str, port: u16, family: Family) -> ([]Endpoint, err)
fn tcp_connect(endpoint: Endpoint) -> (Socket, err)
```

```

fn tcp_listen(endpoint: Endpoint, backlog: u32) -> (Socket, err)
fn tcp_accept(listener: Socket) -> (Socket, Endpoint, err)
fn udp_bind(endpoint: Endpoint) -> (Socket, err)
fn receive(socket: Socket, dst: []u8) -> (usize, err)
fn send(socket: Socket, src: []const u8) -> (usize, err)
fn receive_from(socket: Socket, dst: []u8) -> (usize, Endpoint, err)
fn send_to(socket: Socket, dst: Endpoint, src: []const u8) -> (usize, err)
fn shutdown(socket: Socket, how: Shutdown) -> err
fn close(socket: Socket) -> err
fn reader(socket: *Socket) -> io.Reader
fn writer(socket: *Socket) -> io.Writer
fn resolve_with_control(a: *mem.Arena, host: str, port: u16, family: Family, control: cancel.Control) -> ([]Endpoint, err)
fn tcp_connect_with_control(endpoint: Endpoint, control: cancel.Control) -> (Socket, err)
fn receive_with_control(socket: Socket, dst: []u8, control: cancel.Control) -> (usize, err)
fn send_with_control(socket: Socket, src: []const u8, control: cancel.Control) -> (usize, err)

```

Controlled operations use `e.cancel` and preserve byte counts on failure. DNS may require bounded worker isolation; completion is not acknowledged while caller-owned buffers remain in use. Unsupported cancellation guarantees are reported explicitly, not implemented as a blocking call ignoring its deadline.

`e.net.tls`

```

type Version = enum u8 { Tls13 }
type ClientConfig = struct { server_name: str, trust_roots: []const u8, alpn: []const str, entropy: []const u8, now: time.Timestamp }
type ServerConfig = struct { certificate_chain: []const u8, private_key: []const u8, alpn: []const str, entropy: []const u8 }
type Stream = struct { state: *void }
error InvalidCertificate
error Handshake
error Protocol
error Closed
error Unsupported

fn client(a: *mem.Arena, source: io.Reader, sink: io.Writer, config: ClientConfig) -> (Stream, err)
fn server(a: *mem.Arena, source: io.Reader, sink: io.Writer, config: ServerConfig) -> (Stream, err)
fn handshake(stream: *Stream) -> err
fn reader(stream: *Stream) -> io.Reader
fn writer(stream: *Stream) -> io.Writer
fn protocol(stream: *const Stream) -> Version
fn negotiated_alpn(stream: *const Stream) -> str
fn close(stream: *Stream) -> err
fn handshake_with_control(stream: *Stream, control: cancel.Control) -> err

```

Version 1 implements TLS 1.3 only. Trust roots, certificate time and entropy are explicit inputs; the module never reads host trust state, clocks or randomness. Certificate and key inputs are bounded DER encodings. The cryptographic algorithms and validation profiles supported by a toolchain release are reported by `neper info` and tested against published protocol and malformed-peer vectors.

The release contract must name its cipher suites, signature/certificate algorithms, key schedule and entropy requirements; reporting merely TLS 1.3 is insufficient. HKDF/HMAC come from reviewed `crypto.kdf/crypto.mac` surfaces. SHA-384-based suites cannot be advertised until SHA-384/HKDF-SHA384 exists. The current Ed25519-only public signature surface does not establish compatibility with typical RSA/ECDSA certificate chains; those profiles require explicit reviewed additions or remain unsupported. Do not silently weaken certificate/hostname verification to improve connectivity. Entropy seeds require sufficient fresh caller entropy per independent handshake; copied/reused config bytes are not permission to repeat ephemeral randomness.

`e.net.http`

```

type Method = enum u8 { Get, Head, Post, Put, Patch, Delete, Options, Connect, Trace }
type Version = enum u8 { Http10, Http11 }
type Header = struct { name: str, value: str }
type Request = struct { method: Method, target: str, version: Version, headers: []const Header, body: []const u8 }
type Response = struct { version: Version, status: u16, reason: str, headers: []const Header, body: []const u8 }
type Limits = struct { start_line: usize, header_bytes: usize, header_count: usize, body_bytes: usize }
type Reader = struct { state: *void }
type Writer = struct { state: *void }
error Invalid

```

```
error TooLarge
error Unsupported
```

```
fn reader(a: *mem.Arena, source: io.Reader, limits: Limits) -> (Reader, err)
fn writer(sink: io.Writer) -> Writer
fn read_request(a: *mem.Arena, r: *Reader) -> (Request, err)
fn read_response(a: *mem.Arena, r: *Reader) -> (Response, err)
fn write_request(w: *Writer, req: *const Request) -> err
fn write_response(w: *Writer, response: *const Response) -> err
fn request(a: *mem.Arena, endpoint: net.Endpoint, req: *const Request, limits: Limits) -> (Response, err)
fn request_tls(a: *mem.Arena, endpoint: net.Endpoint, config: tls.ClientConfig, req: *const Request, limits: Limits) -> (Response, err)
fn header(headers: []const Header, name: str) -> (str, bool)
fn reason(status: u16) -> str
type ResponseHead = struct { version: Version, status: u16, reason: str, headers: []const Header }
type ResponseStream = struct { state: *void }
type SseEvent = struct { event: str, data: str, id: str, has_id: bool, retry_ms: u64, has_retry: bool }
type SseState = struct { id: str, has_id: bool, retry_ms: u64, has_retry: bool }
type SseReader = struct { state: *void }
type SseLimits = struct { line_bytes: usize, event_bytes: usize }

fn request_stream(a: *mem.Arena, endpoint: net.Endpoint, req: *const Request, limits: Limits, control: cancel.Control) -> (ResponseStream, err)
fn request_tls_stream(a: *mem.Arena, endpoint: net.Endpoint, config: tls.ClientConfig, req: *const Request, limits: Limits, control: cancel.Control) -> (ResponseStream, err)
fn response_head(stream: *const ResponseStream) -> ResponseHead
fn response_read(stream: *ResponseStream, dst: []u8) -> (usize, err)
fn response_close(stream: *ResponseStream) -> err
fn sse_reader(a: *mem.Arena, source: io.Reader, limits: SseLimits) -> (SseReader, err)
fn sse_next_err(it: *SseReader) -> (SseEvent, bool, err)
fn sse_state(it: *const SseReader) -> SseState
```

Chunked transfer encoding is supported. `request_tls` performs and verifies one TLS connection using the explicit configuration. Automatic redirects, cookies, compression and connection pooling remain outside the version-1 surface.

`ResponseStream` owns the connection; headers borrow its arena and `response_read` incrementally decodes framing without buffering a complete body. `Limits.body_bytes` is a total transfer cap (zero permits no body), not a mandatory allocation. Control applies through reads/close; closing early disposes the connection without draining an unbounded peer. No automatic replay, redirect or credential forwarding. Existing convenience request functions remain bounded full-body wrappers. SSE incrementally handles UTF-8, CR/LF/CRLF, comments, repeated data lines, id, event and decimal retry fields under the standard event-stream algorithm. Returned fields borrow reader storage until next call. The decoder ignores an initial UTF-8 BOM and replaces malformed UTF-8 with U+FFFD. An id containing NUL and a non-decimal retry field are ignored; a valid decimal retry exceeding u64 or an exceeded size limit fails `TooLarge`. EOF does not dispatch an unterminated event. The reader retains valid id/retry updates even in blocks without data; `sse_state` exposes that state, including after EOF. Its strings have the same borrow lifetime as event fields. Event id/retry fields report the effective retained values; `has_id` records whether a valid id was seen, including an empty id that resets it. Reconnection, persistence across readers and retry policy belong to the caller. An `io.Reader` adapter can wrap `response_read` for SSE; its lifetime and cancellation remain those of `ResponseStream`.

e.net.ws

```
type Opcode = enum u8 { Continuation, Text, Binary, Close, Ping, Pong }
type Frame = struct { final: bool, opcode: Opcode, payload: []const u8 }
type Connection = struct { state: *void }
error InvalidHandshake
error InvalidFrame
error TooLarge
error Closed

fn client_key(entropy: [16]u8, dst: []u8) -> (str, err)
fn client_upgrade(a: *mem.Arena, stream: io.Reader, sink: io.Writer, host: str, target_path: str, entropy: [16]u8) -> (Connection, err)
fn server_upgrade(a: *mem.Arena, stream: io.Reader, sink: io.Writer, request: *const http.Request) -> (Connection, err)
fn receive(a: *mem.Arena, connection: *Connection, limit: usize) -> (Frame, err)
fn send(connection: *Connection, frame: Frame, mask: [4]u8) -> err
fn ping(connection: *Connection, payload: []const u8, mask: [4]u8) -> err
fn close(connection: *Connection, code: u16, reason: str, mask: [4]u8) -> err
```

Client masking material and handshake entropy are caller-supplied. The module never reads OS randomness implicitly.

e.db

```

type Connection = struct { ctx: *void, driver: *const Driver }
type Statement = struct { ctx: *void, driver: *const Driver }
type Rows = struct { ctx: *void, driver: *const Driver }
type Transaction = struct { ctx: *void, driver: *const Driver }
type Value = union enum u8 { Null, Bool: bool, I64: i64, U64: u64, F64: f64, Text: str, Bytes: []const u8, Time: time.Instant }
type Parameter = struct { name: str, value: Value }
type Column = struct { name: str, kind: ValueKind, nullable: bool }
type ValueKind = enum u8 { Null, Bool, I64, U64, F64, Text, Bytes, Time }
type Driver = struct { close: fn(ctx: *void) -> err, prepare: fn(ctx: *void, sql: str) -> (Statement, err), execute: fn(ctx: *void, sql: str, params: []const Parameter) -> (u64, err), query: fn(ctx: *void, sql: str, params: []const Parameter) -> (Rows, err), begin: fn(ctx: *void) -> (Transaction, err), statement_close: fn(ctx: *void) -> err, statement_execute: fn(ctx: *void, params: []const Parameter) -> (u64, err), statement_query: fn(ctx: *void, params: []const Parameter) -> (Rows, err), rows_columns: fn(ctx: *void) -> []const Column, rows_next: fn(ctx: *void, dst: []Value) -> (bool, err), rows_close: fn(ctx: *void) -> err, transaction_execute: fn(ctx: *void, sql: str, params: []const Parameter) -> (u64, err), transaction_query: fn(ctx: *void, sql: str, params: []const Parameter) -> (Rows, err), transaction_commit: fn(ctx: *void) -> err, transaction_rollback: fn(ctx: *void) -> err }
error Closed
error InvalidQuery
error Constraint
error Busy
error Unsupported

fn close(connection: *Connection) -> err
fn prepare(connection: *Connection, sql: str) -> (Statement, err)
fn execute(connection: *Connection, sql: str, params: []const Parameter) -> (u64, err)
fn query(connection: *Connection, sql: str, params: []const Parameter) -> (Rows, err)
fn close_statement(statement: *Statement) -> err
fn execute_statement(statement: *Statement, params: []const Parameter) -> (u64, err)
fn query_statement(statement: *Statement, params: []const Parameter) -> (Rows, err)
fn columns(rows: *Rows) -> []const Column
fn reader_next_err(rows: *Rows, dst: []Value) -> (bool, err)
fn close_rows(rows: *Rows) -> err
fn begin(connection: *Connection) -> (Transaction, err)
fn execute_transaction(transaction: *Transaction, sql: str, params: []const Parameter) -> (u64, err)
fn query_transaction(transaction: *Transaction, sql: str, params: []const Parameter) -> (Rows, err)
fn commit(transaction: *Transaction) -> err
fn rollback(transaction: *Transaction) -> err

```

e.db defines the generic SQL connection, prepared-statement, transaction and streaming row-reader contract. Concrete database drivers are owner-qualified packages—initially x.sqlite.sqlite, x.oracle.mysql and x.postgresql.libpq; queries, parameters and row buffers are always explicit, and the module does not discover drivers or allocate hidden connection pools.

8. Declarative GPU UI

gfx.scene

```

type TextureId = struct { slot: u32, generation: u32 }
type SceneId = struct { slot: u32, generation: u32 }
type Clip = union enum u8 { Rect: geometry.Rect, Rounded: geometry.RRect, Path: geometry.Path }
type Command = union enum u8 { Save, Restore, Transform: geometry.Transform, Clip: Clip, FillRect: FillRect, FillPath: FillPath, StrokePath: StrokePath, Image: DrawImage, Text: DrawText, OpacityLayer: OpacityLayer }
type FillRect = struct { rect: geometry.Rect, brush: paint.Brush }
type FillPath = struct { path: geometry.Path, brush: paint.Brush }
type StrokePath = struct { path: geometry.Path, brush: paint.Brush, stroke: paint.Stroke }
type DrawImage = struct { texture: TextureId, source: geometry.Rect, destination: geometry.Rect, opacity: f32 }
type DrawText = struct { layout: *const layout.Layout, origin: geometry.Point, brush: paint.Brush }
type OpacityLayer = struct { bounds: geometry.Rect, opacity: f32 }
type DisplayList = struct { commands: []const Command }
type Builder = struct { state: *void }
type Renderer = struct { state: *void }
type Target = struct { state: *void }
error Invalid

```

```

error TooLarge
error OutOfMemory
error Lost

fn builder(a: *mem.Arena, max_commands: usize) -> (Builder, err)
fn push(b: *Builder, command: Command) -> err
fn finish(b: *Builder) -> DisplayList
fn renderer(a: *mem.Arena, device: *gpu.Device, queue: *gpu.Queue, max_scenes: u32, max_textures: u32) -> (Renderer, err)
fn upload_image(r: *Renderer, image_view: image.ConstImage) -> (TextureId, err)
fn update_image(r: *Renderer, texture: TextureId, image_view: image.ConstImage) -> err
fn release_image(r: *Renderer, texture: TextureId) -> err
fn compile(r: *Renderer, list: DisplayList) -> (SceneId, err)
fn render(r: *Renderer, scene: SceneId, render_target: Target, size: geometry.Size) -> err
fn release_scene(r: *Renderer, scene: SceneId) -> err
fn close(r: *Renderer) -> err

```

Display lists borrow their paths, gradients and text layouts until `compile` returns. A renderer owns bounded generation-checked GPU caches. Compilation may retain tessellation and glyph data but never application widget pointers. Rendering is explicit queue work followed by presentation through the target surface.

`ui.asset`

```

type Theme = enum u8 { Any, Light, Dark }
type Request = struct { scale: f32, locale: str, theme: Theme }
type ImageDecoder = struct { ctx: *void, decode: fn(*void, *mem.Arena, []const u8) -> (image.Image, err) }
type Cache = struct { state: *void }
error Missing
error InvalidVariant
error Decode
error Full

fn select(base: str, request: Request) -> (asset.Asset, err)
fn font(base: str, request: Request, face_index: u32) -> (shape.Font, err)
fn cache(a: *mem.Arena, renderer: *scene.Renderer, capacity: usize) -> (Cache, err)
fn texture(texture_cache: *Cache, scratch: *mem.Arena, base: str, request: Request, decoder: ImageDecoder) -> (scene.TextureId, err)
fn evict(texture_cache: *Cache, renderer: *scene.Renderer, base: str) -> err
fn clear(texture_cache: *Cache, renderer: *scene.Renderer) -> err
fn close(texture_cache: *Cache, renderer: *scene.Renderer) -> err

```

Variants share a manifest base attribute. Selection first filters by base, then chooses exact locale (falling back by removing subtags and finally to an empty locale), exact theme before Any, and the smallest scale not below the request or otherwise the largest scale. UTF-8 asset name breaks any remaining tie. The algorithm reads only `e.asset` metadata and is deterministic on every host.

`font` returns executable-backed OpenType bytes without copying. `texture` invokes the caller-supplied decoder into `scratch`, uploads before returning, and keys the bounded cache by selected asset SHA-256 plus decoder identity. The cache owns its texture entries but not the renderer; eviction and closure explicitly release them. No image codec, filesystem lookup or unbounded global cache is hidden here.

`ui.window`

```

type Id = struct { slot: u32, generation: u32 }
type Window = struct { state: *void, id: Id }
type Mode = enum u8 { Windowed, Maximized, Fullscreen }
type Cursor = enum u8 { Arrow, Text, Hand, Crosshair, ResizeHorizontal, ResizeVertical, Hidden }
type Options = struct { title: str, width: u32, height: u32, min_width: u32, min_height: u32, resizable: bool, transparent: bool, mode: Mode }
type Metrics = struct { logical_size: geometry.Size, framebuffer_width: u32, framebuffer_height: u32, scale: f32, focused: bool, visible: bool }
error Unsupported
error Invalid
error Closed

fn open(a: *mem.Arena, device: *gpu.Device, options: Options) -> (Window, err)
fn metrics(window: *const Window) -> (Metrics, err)
fn target(window: *const Window) -> (scene.Target, err)
fn title(window: *Window, value: str) -> err
fn cursor(window: *Window, value: Cursor) -> err

```

```
fn visible(window: *Window, value: bool) -> err
fn request_frame(window: *Window) -> err
fn clipboard_get(a: *mem.Arena, window: *Window) -> (str, err)
fn clipboard_set(window: *Window, value: str) -> err
fn close(window: *Window) -> err
```

Windows are logically linear handles backed only by reviewed e.os primitives. Coordinates exposed above the module are logical pixels; framebuffer dimensions are physical pixels. target is non-owning and becomes invalid when the window closes.

ui.input

```
type DeviceId = u32
type PointerId = u32
type Key = struct { physical: u32, logical: u32 }
type Modifiers = struct { shift: bool, control: bool, alt: bool, meta: bool, caps_lock: bool, num_lock: bool }
type PointerButton = enum u8 { Primary, Secondary, Middle, Back, Forward }
type PointerKind = enum u8 { Mouse, Touch, Pen }
type Pointer = struct { window: window.Id, device: DeviceId, pointer: PointerId, kind: PointerKind, position: geometry.Point, buttons: u32, changed: PointerButton }
type KeyEvent = struct { window: window.Id, key: Key, modifiers: Modifiers, repeat: bool }
type TextEvent = struct { window: window.Id, text: str }
type Composition = struct { window: window.Id, text: str, selection_start: usize, selection_end: usize }
type Event = union enum u8 { Frame: window.Id, Close: window.Id, Resize: window.Metrics, Focus: window.Id, Blur: window.Id, PointerDown: Pointer, PointerUp: Pointer, PointerMove: Pointer, Scroll: Pointer, KeyDown: KeyEvent, KeyUp: KeyEvent, Text: TextEvent, Composition: Composition }
type Queue = struct { state: *void }
error Closed
error TooLarge

fn queue(a: *mem.Arena, capacity: usize) -> (Queue, err)
fn poll(q: *Queue, timeout: time.Duration) -> (Event, bool, err)
fn capture(q: *Queue, window_value: window.Id, pointer: PointerId) -> err
fn release_capture(q: *Queue, window_value: window.Id, pointer: PointerId) -> err
fn composition_rect(q: *Queue, window_value: window.Id, rect: geometry.Rect) -> err
fn close(q: *Queue) -> err
```

Events preserve native ordering and borrow transient text until the next poll. Platform key codes are normalized into stable physical and Unicode-oriented logical values. Gesture recognition is built by widgets from pointer streams rather than being hidden in the OS boundary.

ui.widget

```
type Key = u64
type ElementId = struct { slot: u32, generation: u32 }
type StateId = struct { slot: u32, generation: u32 }
type Action = struct { ctx: *void, invoke: fn(*void, input.Event) -> err }
type Text = struct { value: str, style: layout.Style, color: paint.Color }
type Button = struct { action: Action, enabled: bool }
type Image = struct { texture: scene.TextureId, fit: Fit }
type Scroll = struct { axis: ui_layout.Axis, offset: f32 }
type Custom = struct { ctx: *void, measure: fn(*void, ui_layout.Constraints) -> geometry.Size, paint: fn(*void, *scene.Builder, geometry.Rect) -> err }
type Kind = union enum u8 { Box, Flex: ui_layout.Flex, Grid: ui_layout.Grid, Stack, Text: Text, Button: Button, Image: Image, Scroll: Scroll, Custom: Custom }
type Node = struct { key: Key, kind: Kind, style: style.Style, children: []const Node }
type Fit = enum u8 { Fill, Contain, Cover, None }
type BuildContext = struct { runtime: *Runtime, element: ElementId, frame: u64 }
type Runtime = struct { state: *void }
type Limits = struct { max_elements: usize, max_states: usize, state_bytes: usize, state_classes: u16, max_depth: u16, max_commands: usize }
error DuplicateKey
error InvalidTree
error TooDeep
error TooLarge
error StateType

fn runtime(a: *mem.Arena, renderer: *scene.Renderer, limits: Limits) -> (Runtime, err)
fn box(key: Key, value_style: style.Style, children: []const Node) -> Node
fn flex(key: Key, spec: ui_layout.Flex, value_style: style.Style, children: []const Node) -> Node
fn grid(key: Key, spec: ui_layout.Grid, value_style: style.Style, children: []const Node) -> Node
```

```

fn stack(key: Key, value_style: style.Style, children: []const Node) -> Node
fn text(key: Key, value: Text, value_style: style.Style) -> Node
fn button(key: Key, value: Button, value_style: style.Style, children: []const Node) -> Node
fn image(key: Key, value: Image, value_style: style.Style) -> Node
fn scroll(key: Key, value: Scroll, value_style: style.Style, children: []const Node) -> Node
fn state[T: type](ctx: *BuildContext, key: Key, initial: T) -> (*T, StateId, err)
fn invalidate(widget_runtime: *Runtime, element: ElementId)
fn reconcile(widget_runtime: *Runtime, frame_arena: *mem.Arena, root: Node, constraints: ui_layout.Constraints) ->
  (scene.SceneId, err)
fn dispatch(widget_runtime: *Runtime, event: input.Event) -> err
fn focus(widget_runtime: *Runtime, element: ElementId) -> err
fn close(widget_runtime: *Runtime) -> err

```

Node is the declarative syntax: ordinary literals and the allocation-free convenience constructors create an immutable tree in a resettable frame arena. Reconciliation matches siblings by nonzero key and kind, otherwise by position and kind. Persistent elements and typed state occupy bounded generation-checked slots owned by `Runtime`; widgets themselves are never retained. Removed state is destroyed logically at the reconciliation boundary and its size/alignment cell enters a bounded runtime free list. A reused `Action.ctx` must outlive the element that retains it; passing frame-arena context is `InvalidTree` in debug validation and undefined in release.

ui.animation

```

type Curve = enum u8 { Linear, EaseIn, EaseOut, EaseInOut }
type Controller = struct { start: time.Instant, duration: time.Duration, curve: Curve, repeating: bool, reverse: bool }

fn controller(now: time.Instant, duration: time.Duration, curve: Curve) -> Controller
fn value(c: *const Controller, now: time.Instant) -> f32
fn finished(c: *const Controller, now: time.Instant) -> bool
fn restart(c: *Controller, now: time.Instant)
fn request(runtime: *widget.Runtime, element: widget.ElementId)

```

Animation state is explicit. Sampling never reads a clock; the application supplies `now`, making animation deterministic in tests.

ui.accessibility

```

type Id = widget.ElementId
type Role = enum u8 { Application, Window, Group, Button, Checkbox, Radio, Text, TextField, Image, Link, List, ListItem, Table, Row, Cell, Slider, Progress, Scrollbar }
type State = struct { disabled: bool, focused: bool, selected: bool, checked: bool, expanded: bool, hidden: bool }
type Action = enum u8 { Focus, Press, Increment, Decrement, SetValue, Scroll }
type Node = struct { id: Id, role: Role, label: str, value: str, hint: str, state: State, bounds: geometry.Rect, actions: []const Action, children: []const Id }
type Tree = struct { root: Id, nodes: []const Node }
error Unsupported
error Invalid

fn build(a: *mem.Arena, runtime: *const widget.Runtime) -> (Tree, err)
fn publish(window_value: window.Id, tree: *const Tree) -> err
fn perform(runtime: *widget.Runtime, id: Id, action: Action, value: str) -> err

```

The semantics tree is separate from paint order but uses the same stable element identities. Publication crosses a reviewed `e.os` accessibility bridge and retains no caller strings after returning.

ui.testing

```

type Harness = struct { state: *void }
type Match = struct { element: widget.ElementId, count: usize }
error NotFound
error Ambiguous
error GoldenMismatch

fn harness(a: *mem.Arena, runtime: *widget.Runtime, width: u32, height: u32, scale: f32) -> (Harness, err)
fn pump(h: *Harness, root: widget.Node, now: time.Instant) -> err
fn send(h: *Harness, event: input.Event) -> err
fn by_key(h: *const Harness, key: widget.Key) -> Match
fn by_text(h: *const Harness, text: str) -> Match
fn snapshot(h: *Harness, a: *mem.Arena) -> (image.Image, err)
fn compare(actual: image.ConstImage, expected: image.ConstImage, tolerance: u8) -> err

```

```
fn close(h: *Harness) -> err
```

The harness uses the deterministic CPU rendering backend and a synthetic window. It does not require a display server and never sleeps; tests supply frame time.

```
ui.app
```

```
type App = struct { state: *void }
type Builder[Ctx: type] = struct { ctx: *Ctx, build: fn(*Ctx, *widget.BuildContext) -> (widget.Node, err) }
type Options = struct { window: window.Options, widget_limits: widget.Limits, frame_arena_bytes: usize, event_capacity: usize, backend: gpu.Backend }
error Closed
error Failed

fn init[Ctx: type](a: *mem.Arena, options: Options, builder: Builder[Ctx]) -> (App, err)
fn step(app: *App, timeout: time.Duration) -> (bool, err)
fn run(app: *App) -> err
fn stop(app: *App)
fn close(app: *App) -> err
```

step drains ordered input, rebuilds only invalidated subtrees, reconciles, lays out, compiles a scene and presents when a frame is due. It returns false after the last window closes or stop is called. run is exactly a loop over step; applications retain control by calling step themselves. All memory comes from the application arena plus a bounded frame arena reset after presentation.

All document decoders allocate retained strings and nodes in the arena passed to the call. Streaming readers retain only their documented scratch state. Every writer uses io.Writer; no format module opens files.

9. Interchange formats

```
fmt.json
```

```
type Number = struct { lexeme: str }
type Member = struct { key: str, value: Value }
type Value = union enum u8 { Null, Bool: bool, Number: Number, String: str, Array: []const Value, Object: []const Member }
type Event = union enum u8 { Null, Bool: bool, Number: Number, String: str, Key: str, BeginArray, EndArray, BeginObject, EndObject }
type Reader = struct { state: *void }
type Options = struct { allow_duplicate_keys: bool, max_depth: u16 }
error Invalid
error TooDeep
error DuplicateKey
error TooLarge

fn reader(a: *mem.Arena, source: io.Reader, options: Options) -> (Reader, err)
fn reader_next_err(r: *Reader) -> (Event, bool, err)
fn parse(a: *mem.Arena, source: str, options: Options) -> (Value, err)
fn write(writer: *io.Writer, value: *const Value) -> err
fn write_pretty(writer: *io.Writer, value: *const Value, indent: u8) -> err
fn encode[T: type](writer: *io.Writer, value: *const T) -> err
fn decode[T: type](a: *mem.Arena, source: str, options: Options) -> (T, err)
error InvalidPointer
error PatchFailed

fn number(source: str) -> (Number, err)
fn number_i64(value: Number) -> (i64, err)
fn number_u64(value: Number) -> (u64, err)
fn number_f64(value: Number) -> (f64, err)
fn number_from_i64(a: *mem.Arena, value: i64) -> (Number, err)
fn number_from_u64(a: *mem.Arena, value: u64) -> (Number, err)
fn number_from_f64(a: *mem.Arena, value: f64) -> (Number, err)
fn pointer(root: *const Value, path: str) -> (*const Value, err)
fn patch(a: *mem.Arena, root: *const Value, operations: *const Value, max_operations: usize, max_depth: u16) -> (Value, err)
```

Numbers preserve their validated JSON lexeme, including large integers, exponent spelling and negative zero. `parse` borrows number lexemes from source; streaming events borrow reader-owned storage until the next reader call. Writers revalidate public Number values and never silently round them through `f64`. `number_i64/u64` accept mathematically integral values exactly in range (including exponent forms); `number_f64` explicitly rounds to nearest ties-to-even and rejects nonfinite overflow. Typed integer encode/decode never takes a floating-point detour. Object order is preserved; duplicate keys are rejected unless Options explicitly allows them. These are semantic round trips, not whitespace-preserving source edits. Structural encoding/decoding still uses spec §9. See `stdlib-hardening.md` SL04.

`pointer` follows RFC 6901 JSON Pointer, including the empty root pointer and `~0/~1` escapes; URI-fragment notation is not accepted by this function. `patch` implements RFC 6902 `add/remove/replace/move/copy/test` into a new arena-owned tree. It never mutates root; all output strings and number lexemes are copied. Failure rolls back its own arena allocations and returns zero. Duplicate-key objects anywhere in a patch input are rejected. Numeric test equality is exact mathematical equality, not `f64` equality. Limits, invalid array indices, move-into-descendant and missing targets fail explicitly. Source-file edits additionally require H09 transactions.

`fmt.csv`

```
type Dialect = struct { delimiter: u8, quote: u8, crlf: bool, header: bool }
type Row = struct { fields: []const str }
type Reader = struct { state: *void }
error Invalid
error TooLarge

fn csv() -> Dialect
fn tsv() -> Dialect
fn reader(a: *mem.Arena, source: io.Reader, dialect: Dialect, field_limit: usize, row_limit: usize) -> (Reader, err)
fn reader_next_err(r: *Reader) -> (Row, bool, err)
fn write_row(writer: *io.Writer, row: Row, dialect: Dialect) -> err
fn decode_rows[T: type](a: *mem.Arena, source: io.Reader, dialect: Dialect) -> ([T], err)
fn encode_rows[T: type](writer: *io.Writer, rows: []const T, dialect: Dialect) -> err
```

`fmt.ini`

```
type Entry = struct { section: str, key: str, value: str }
type Document = struct { entries: []const Entry }
type Reader = struct { state: *void }
type Options = struct { case_sensitive: bool, allow_duplicate_keys: bool }
error Invalid
error DuplicateKey
error TooLarge

fn reader(a: *mem.Arena, source: io.Reader, options: Options) -> (Reader, err)
fn reader_next_err(r: *Reader) -> (Entry, bool, err)
fn parse(a: *mem.Arena, source: str, options: Options) -> (Document, err)
fn get(document: *const Document, section: str, key: str) -> (str, bool)
fn write(writer: *io.Writer, document: *const Document) -> err
fn decode[T: type](a: *mem.Arena, source: str, options: Options) -> (T, err)
fn encode[T: type](writer: *io.Writer, value: *const T) -> err
```

The accepted syntax is sections, `key=value`, `;/#` line comments, quoted values, and backslash escapes. It deliberately excludes interpolation and includes.

`fmt.uri`

```
type Uri = struct { scheme: str, authority: str, userinfo: str, host: str, port: str, path: str, query: str, fragment: str }
type EncodeSet = enum u8 { Path, PathSegment, Query, QueryComponent, Fragment, UserInfo }
error Invalid

fn parse(source: str) -> (Uri, err)
fn resolve(a: *mem.Arena, base: Uri, reference: Uri) -> (Uri, err)
fn normalize(a: *mem.Arena, value: Uri) -> (Uri, err)
fn format(a: *mem.Arena, value: Uri) -> (str, err)
fn percent_encode(a: *mem.Arena, source: []const u8, set: EncodeSet) -> (str, err)
fn percent_decode(a: *mem.Arena, source: str) -> ([u8], err)
fn query_get(query: str, name: str) -> (str, bool, err)
```

Parsing follows RFC 3986 and returns borrowed slices. Percent decoding never treats + as space; HTML form encoding is a separate concern.

fmt.mime

```
type Parameter = struct { name: str, value: str }
type MediaType = struct { major: str, subtype: str, parameters: []const Parameter }
type Header = struct { name: str, value: str }
error Invalid
error TooLarge

fn parse_media_type(a: *mem.Arena, source: str, limit: usize) -> (MediaType, err)
fn format_media_type(a: *mem.Arena, value: MediaType) -> (str, err)
fn extension_type(extension: str) -> (str, bool)
fn parse_headers(a: *mem.Arena, source: io.Reader, byte_limit: usize, count_limit: usize) -> ([]Header, err)
fn header(headers: []const Header, name: str) -> (str, bool)
```

Header names compare by ASCII case folding. Obsolete line folding is rejected.

fmt.asn1

```
type Class = enum u8 { Universal, Application, Context, Private }
type Tag = struct { class: Class, number: u32, constructed: bool }
type Value = struct { tag: Tag, content: []const u8, encoded: []const u8 }
type Reader = struct { data: []const u8, off: usize, depth: u16, max_depth: u16 }
error Invalid
error NonCanonical
error TooDeep
error TooLarge

fn reader(data: []const u8, max_depth: u16) -> Reader
fn reader_next_err(source_reader: *Reader) -> (Value, bool, err)
fn children(value: Value, max_depth: u16) -> (Reader, err)
fn decode[T: type](a: *mem.Arena, source: []const u8, max_depth: u16) -> (T, err)
fn encoded_len[T: type](value: *const T) -> (usize, err)
fn encode[T: type](dst: []u8, value: *const T) -> ([]u8, err)
```

The version-1 surface accepts and emits canonical DER only. Lengths, nesting and integer encodings are validated before typed decoding exposes a value.

fmt.pem

```
type Block = struct { label: str, headers: []const mime.Header, bytes: []const u8 }
error Invalid
error TooLarge

fn decode(a: *mem.Arena, source: str, byte_limit: usize) -> (Block, str, err)
fn encode(writer: *io.Writer, block: *const Block) -> err
```

decode returns the first strict PEM block and the unconsumed suffix. Base64 is decoded through e.bytes; encrypted legacy PEM headers are not interpreted.

fmt.multipart

```
type Part = struct { headers: []const mime.Header, body: io.Reader }
type Reader = struct { state: *void }
type Writer = struct { state: *void }
error InvalidBoundary
error Invalid
error TooLarge

fn reader(storage: []u8, source: io.Reader, boundary: str, part_limit: u32, byte_limit: u64) -> (Reader, err)
fn reader_next_err(source_reader: *Reader) -> (Part, bool, err)
fn writer(storage: []u8, sink: io.Writer, boundary: str) -> (Writer, err)
fn start_part(sink_writer: *Writer, headers: []const mime.Header) -> (io.Writer, err)
fn finish(sink_writer: *Writer) -> err
```

Bodies stream without implicit buffering. Boundaries are caller-supplied for writing, making output reproducible; nested multipart content uses another explicit reader.

`fmt.quoted_printable`

```

type Reader = struct { state: *void }
type Writer = struct { state: *void }
error Invalid

fn reader(storage: []u8, source: io.Reader) -> Reader
fn read(source_reader: *Reader, dst: []u8) -> (usize, err)
fn writer(storage: []u8, sink: io.Writer, line_limit: u8) -> (Writer, err)
fn write(sink_writer: *Writer, src: []const u8) -> (usize, err)
fn finish(sink_writer: *Writer) -> err

```

Decoding is strict RFC 2045. Encoding uses canonical uppercase hex escapes and caller-selected line limits.

`fmt.mail`

```

type Address = struct { name: str, address: str }
type Message = struct { headers: []const mime.Header, body: io.Reader }
type Date = struct { instant: i64, offset_minutes: i16 }
error InvalidAddress
error InvalidDate
error InvalidMessage
error TooLarge

fn parse_address(a: *mem.Arena, source: str) -> (Address, err)
fn parse_address_list(a: *mem.Arena, source: str) -> ([]const Address, err)
fn format_address(a: *mem.Arena, value: Address) -> (str, err)
fn parse_date(source: str) -> (Date, err)
fn read_message(a: *mem.Arena, source: io.Reader, header_limit: usize) -> (Message, err)
fn decode_header(a: *mem.Arena, source: str, output_limit: usize) -> (str, err)

```

The module parses Internet message headers and addresses without SMTP transport. MIME bodies are consumed through `fmt.mime`, `fmt.multipart` and `fmt.quoted_printable`; charset conversion is explicit through `text.encoding`.

`fmt.gzip`

```

type Reader = struct { state: *void }
type Writer = struct { state: *void }
error Invalid
error Checksum

fn reader(storage: []u8, source: io.Reader, output_limit: u64) -> (Reader, err)
fn read(r: *Reader, dst: []u8) -> (usize, err)
fn writer(storage: []u8, sink: io.Writer, level: deflate.Level) -> (Writer, err)
fn write(w: *Writer, src: []const u8) -> (usize, err)
fn finish(w: *Writer) -> err
fn storage_required(level: deflate.Level) -> usize

```

The reader validates RFC 1952 headers, trailer size and CRC32. The output limit is checked before exposing bytes. `finish` writes the final DEFLATE blocks and trailer.

`fmt.zstd`

```

type Reader = struct { state: *void }
type Writer = struct { state: *void }
type Level = enum u8 { Fast, Balanced, Best }
error Invalid
error Checksum
error Unsupported

fn reader(storage: []u8, source: io.Reader, output_limit: u64) -> (Reader, err)
fn read(r: *Reader, dst: []u8) -> (usize, err)
fn writer(storage: []u8, sink: io.Writer, level: Level) -> (Writer, err)
fn write(w: *Writer, src: []const u8) -> (usize, err)
fn finish(w: *Writer) -> err
fn reader_storage(window_limit: usize) -> (usize, err)
fn writer_storage(level: Level) -> usize

```

Version 1 supports standard frames without dictionaries. Window and decompressed output limits are mandatory; unsupported skippable or dictionary frames return `Unsupported`.

fmt.bzip2

```

type Reader = struct { state: *void }
error Invalid
error Checksum
error TooLarge

fn reader(storage: []u8, source: io.Reader, output_limit: u64) -> (Reader, err)
fn read(source_reader: *Reader, dst: []u8) -> (usize, err)
fn storage_required(block_limit: usize) -> (usize, err)

```

Version 1 provides bounded bzip2 decompression. Compression is deliberately omitted until a workload justifies its larger implementation and memory surface.

fmt.lzw

```

type Order = enum u8 { LeastSignificant, MostSignificant }
type Reader = struct { state: *void }
type Writer = struct { state: *void }
error Invalid
error TooLarge

fn reader(storage: []u8, source: io.Reader, order: Order, literal_width: u8, output_limit: u64) -> (Reader, err)
fn read(source_reader: *Reader, dst: []u8) -> (usize, err)
fn writer(storage: []u8, sink: io.Writer, order: Order, literal_width: u8) -> (Writer, err)
fn write(sink_writer: *Writer, src: []const u8) -> (usize, err)
fn finish(sink_writer: *Writer) -> err
fn storage_required(literal_width: u8) -> (usize, err)

```

Bit order and literal width are explicit so GIF- and TIFF-style streams cannot be silently confused.

fmt.zlib

```

type Reader = struct { state: *void }
type Writer = struct { state: *void }
error Invalid
error Checksum

fn reader(storage: []u8, source: io.Reader, output_limit: u64) -> (Reader, err)
fn read(source_reader: *Reader, dst: []u8) -> (usize, err)
fn writer(storage: []u8, sink: io.Writer, level: deflate.Level) -> (Writer, err)
fn write(sink_writer: *Writer, src: []const u8) -> (usize, err)
fn finish(sink_writer: *Writer) -> err
fn storage_required(level: deflate.Level) -> usize

```

The reader and writer implement RFC 1950 framing around algo.deflate and validate the Adler-32 trailer before successful completion.

fmt.zip

```

type Archive = struct { state: *void }
type Entry = struct { name: str, compressed_size: u64, size: u64, method: u16, crc32: u32, directory: bool }
type Limits = struct { entries: usize, name_bytes: usize, entry_bytes: u64, total_bytes: u64 }
error Invalid
error Unsupported
error Checksum
error TooLarge

fn open(a: *mem.Arena, source: io.Reader, seeker: io.Seeker, limits: Limits) -> (Archive, err)
fn entries(archive: *const Archive) -> []const Entry
fn entry_reader(storage: []u8, archive: *Archive, index: usize) -> (io.Reader, err)
fn extract(a: *mem.Arena, archive: *Archive, index: usize) -> ([]u8, err)

```

Version 1 reads stored and DEFLATE entries, ZIP64 sizes and UTF-8 names. It rejects encrypted entries, absolute paths and names containing a .. segment. CRC and all per-entry/aggregate limits are checked before successful extraction.

fmt.tar

```

type Reader = struct { state: *void }
type Entry = struct { name: str, size: u64, kind: u8, mode: u32, modified: i64, link: str }
error Invalid

```

```
error Unsupported
error TooLarge
```

```
fn reader(a: *mem.Arena, source: io.Reader, entry_limit: usize, byte_limit: u64) -> (Reader, err)
fn next(r: *Reader) -> (Entry, bool, err)
fn content(r: *Reader) -> io.Reader
fn skip(r: *Reader) -> err
```

The reader supports POSIX ustar and PAX path/size records. Each entry's content must be consumed or skipped before next. Absolute paths and .. segments are rejected.

fmt.yaml

```
type Pair = struct { key: Value, value: Value }
type Value = union enum u8 { Null, Bool: bool, Integer: i64, Float: f64, String: str, Sequence: []const Value, Mapping: []const Pair }
type Options = struct { max_depth: u16, allow_duplicate_keys: bool }
error Invalid
error TooDeep
error DuplicateKey
error Unsupported

fn parse(a: *mem.Arena, source: str, options: Options) -> (Value, err)
fn write(writer: *io.Writer, value: *const Value, indent: u8) -> err
fn decode[T: type](a: *mem.Arena, source: str, options: Options) -> (T, err)
fn encode[T: type](writer: *io.Writer, value: *const T) -> err
```

The subset is block mappings/sequences, flow collections, plain/single/double-quoted scalars, literal/folded blocks and null/boolean/integer/float core tags. Anchors, aliases, merge keys, directives, custom tags and multiple documents are Unsupported.

fmt.xml

```
type Attribute = struct { name: str, value: str }
type Event = union enum u8 { Start: StartElement, End: str, Text: str, Comment: str, Processing: Processing }
type StartElement = struct { name: str, attributes: []const Attribute, empty: bool }
type Processing = struct { target: str, data: str }
type Reader = struct { state: *void }
type Writer = struct { sink: io.Writer, depth: u16 }
type Options = struct { max_depth: u16, preserve_comments: bool }
error Invalid
error TooDeep
error Unsupported

fn reader(a: *mem.Arena, source: io.Reader, options: Options) -> (Reader, err)
fn reader_next_err(r: *Reader) -> (Event, bool, err)
fn writer(sink: io.Writer) -> Writer
fn start(w: *Writer, name: str, attributes: []const Attribute) -> err
fn text(w: *Writer, value: str) -> err
fn comment(w: *Writer, value: str) -> err
fn end(w: *Writer, name: str) -> err
```

XML 1.0 names, namespaces and entity escaping are supported. External entities and DTDs are always Unsupported; the module never performs hidden I/O.

fmt.html

```
type NodeId = u32
const NONE: NodeId = 4294967295
type NodeKind = enum u8 { Document, Doctype, Element, Text, Comment }
type Namespace = enum u8 { Html, Svg, MathML }
type Attribute = struct { namespace: Namespace, name: str, value: str }
type Node = struct { kind: NodeKind, namespace: Namespace, name: str, value: str, attributes: []const Attribute, parent: NodeId, first_child: NodeId, last_child: NodeId, previous_sibling: NodeId, next_sibling: NodeId }
type Document = struct { nodes: []const Node, root: NodeId }
type Children = struct { document: *const Document, next: NodeId }
type Options = struct { max_bytes: usize, max_nodes: usize, max_attributes: usize, max_depth: u16, preserve_comments: bool }
error InvalidEncoding
error TooDeep
error TooLarge
```

```

fn parse(a: *mem.Arena, source: str, options: Options) -> (Document, err)
fn parse_reader(a: *mem.Arena, source: io.Reader, options: Options) -> (Document, err)
fn node(document: *const Document, id: NodeId) -> *const Node
fn children(document: *const Document, parent: NodeId) -> Children
fn children_next(it: *Children) -> (NodeId, bool)
fn attribute(element: *const Node, name: str) -> (str, bool)
fn text_content(a: *mem.Arena, document: *const Document, root: NodeId) -> (str, err)
fn write(writer: *io.Writer, document: *const Document) -> err

```

The input is UTF-8 with an optional UTF-8 BOM; invalid UTF-8 returns `InvalidEncoding`. `parse_reader` reads no more than `max_bytes`, and `parse` rejects an input longer than that limit before allocating nodes. Ordinary malformed HTML is recovered rather than returned as an error. Tokenization, character-reference decoding, insertion modes, implied elements, foster parenting, raw-text elements and HTML/SVG/MathML namespace transitions follow the WHATWG HTML parsing algorithm as frozen by the toolchain version. The conformance snapshot and `html5lib` tree-construction fixtures used by a release are recorded in its build manifest.

`Document.nodes` is arena-owned and read-only. `NodeId` values are indices into that slice; `NONE` denotes a missing relation. The document node is root, has `namespace == .Html`, and has no name or value. Element names are ASCII-lowercase in the HTML namespace and preserve adjusted foreign names elsewhere. Text, comments, names and decoded attribute values are arena-owned. Attributes and children preserve source/tree-construction order. `attribute` uses ASCII-insensitive comparison for HTML elements and exact comparison for foreign elements. `text_content` concatenates descendant text nodes in tree order. `write` implements the matching HTML fragment serialization rules and performs no pretty-print rewrite.

The module does not open files, fetch subresources, execute scripts, apply CSS, construct a browser DOM or expose mutable tree operations. A caller loads a file through `e.fs` or supplies an `e.io.Reader`. Legacy encoding sniffing and conversion must occur before parsing and are outside the version-1 surface.

fmt.html.template

```

type Template = struct { inner: template.Template }
type Options = struct { max_bytes: usize, max_nodes: usize, max_depth: u16 }
error InvalidTemplate
error UnsafeContext
error MissingValue
error TooLarge

fn parse(a: *mem.Arena, source: str, options: Options) -> (Template, err)
fn execute(value: *const Template, writer: *io.Writer, bindings: []const template.Binding) -> err
fn validate[T: type](value: *const Template) -> err
fn execute_typed[T: type](value: *const Template, writer: *io.Writer, data: *const T) -> err

```

HTML templates track text, attribute, URI, CSS and script contexts and apply the matching escaping rules. Ambiguous or unsafe context transitions fail at parse time; trusted raw insertion is intentionally absent from version 1.

fmt.png

```

type DecodeOptions = struct { max_width: u32, max_height: u32, max_pixels: u64, verify_crc: bool }
type EncodeOptions = struct { compression: deflate.Level, interlace: bool }
error Invalid
error Unsupported
error Checksum
error TooLarge

fn inspect(source: io.Reader) -> (image.Info, err)
fn decode(a: *mem.Arena, source: io.Reader, options: DecodeOptions) -> (image.Image, err)
fn encode(writer: *io.Writer, value: image.ConstImage, options: EncodeOptions) -> err

```

PNG decoding supports the standard grayscale, RGB, indexed and alpha color types and rejects dimensions before pixel allocation. Encoding is deterministic for identical pixels and options.

fmt.jpeg

```

type DecodeOptions = struct { max_width: u32, max_height: u32, max_pixels: u64 }
type EncodeOptions = struct { quality: u8, progressive: bool }
error Invalid
error Unsupported

```

error TooLarge

```
fn inspect(source: io.Reader) -> (image.Info, err)
fn decode(a: *mem.Arena, source: io.Reader, options: DecodeOptions) -> (image.Image, err)
fn encode(writer: *io.Writer, value: image.ConstImage, options: EncodeOptions) -> err
```

Version 1 supports baseline and progressive Huffman JPEG with bounded dimensions. Arithmetic coding and embedded color-profile conversion return Unsupported.

fmt.webp

```
type DecodeOptions = struct { max_width: u32, max_height: u32, max_pixels: u64, first_frame_only: bool }
type EncodeOptions = struct { quality: f32, lossless: bool }
error Invalid
error Unsupported
error TooLarge

fn inspect(source: io.Reader) -> (image.Info, err)
fn decode(a: *mem.Arena, source: io.Reader, options: DecodeOptions) -> (image.Image, err)
fn encode(writer: *io.Writer, value: image.ConstImage, options: EncodeOptions) -> err
```

The module supports lossy and lossless WebP. Animation is inspectable; decoding more than the first frame remains Unsupported until a frame-sequence image type exists.

fmt.bson

```
type Element = struct { key: str, value: Value }
type Binary = struct { subtype: u8, data: []const u8 }
type Value = union enum u8 { Double: f64, String: str, Document: []const Element, Array: []const Value, Binary: Binary, Bool: bool, Null, I32: i32, I64: i64 }
error Invalid
error TooDeep
error TooLarge

fn parse(a: *mem.Arena, source: []const u8, max_depth: u16) -> (Value, err)
fn size(value: *const Value) -> (usize, err)
fn write(writer: *io.Writer, value: *const Value) -> err
fn decode[T: type](a: *mem.Arena, source: []const u8, max_depth: u16) -> (T, err)
fn encode[T: type](writer: *io.Writer, value: *const T) -> err
```

Unsupported BSON element tags are Invalid; JavaScript, regex and deprecated tags are intentionally absent from the value model.

fmt.msgpack

```
type Pair = struct { key: Value, value: Value }
type Ext = struct { kind: i8, data: []const u8 }
type Value = union enum u8 { Nil, Bool: bool, I64: i64, U64: u64, F64: f64, String: str, Binary: []const u8, Array: []const Value, Map: []const Pair, Ext: Ext }
type Reader = struct { state: *void }
error Invalid
error TooDeep
error TooLarge

fn reader(a: *mem.Arena, source: io.Reader, max_depth: u16) -> (Reader, err)
fn read_value(a: *mem.Arena, r: *Reader) -> (Value, err)
fn write(writer: *io.Writer, value: *const Value) -> err
fn decode[T: type](a: *mem.Arena, source: io.Reader, max_depth: u16) -> (T, err)
fn encode[T: type](writer: *io.Writer, value: *const T) -> err
```

fmt.protobuf

```
type WireType = enum u8 { Varint, Fixed64, Bytes, Fixed32 }
type Key = struct { number: u32, wire: WireType }
type Reader = struct { input: bytes.Reader }
error Invalid
error TooLarge

fn reader(source: []const u8) -> Reader
fn reader_next_err(r: *Reader) -> (Key, bool, err)
fn read_u64(r: *Reader) -> (u64, err)
```

```

fn read_i64(r: *Reader) -> (i64, err)
fn read_sint64(r: *Reader) -> (i64, err)
fn read_fixed32(r: *Reader) -> (u32, err)
fn read_fixed64(r: *Reader) -> (u64, err)
fn read_bytes(r: *Reader) -> ([]const u8, err)
fn skip(r: *Reader, wire: WireType) -> err
fn size_key(number: u32, wire: WireType) -> (usize, err)
fn size_varint(value: u64) -> usize
fn size_bytes(n: usize) -> (usize, err)
fn write_key(w: *io.Writer, number: u32, wire: WireType) -> err
fn write_u64(w: *io.Writer, value: u64) -> err
fn write_i64(w: *io.Writer, value: i64) -> err
fn write_sint64(w: *io.Writer, value: i64) -> err
fn write_fixed32(w: *io.Writer, value: u32) -> err
fn write_fixed64(w: *io.Writer, value: u64) -> err
fn write_bytes(w: *io.Writer, value: []const u8) -> err

```

These are protobuf wire-format primitives for generated message code. `reader_next_err` reads only a field key; the generated decoder calls the matching `read_*` or `skip` and enforces message-specific cardinality, required fields, recursion depth, packed encoding and oneof semantics. Field numbers are 1..536870911 excluding 19000..19999. Signed non-zigzag values use protobuf's ten-byte two's-complement varint; `sint64` is zigzag. Length-delimited results borrow source. The module does not invent field numbers from reflection and contains no descriptor runtime.

10. External packages

The `x.*` rows in `modules.md` are package reservations, not implicit toolchain modules. Each package publishes `docs/packages/<owner>/<package>.md` before implementation. That specification pins the upstream ABI/data version and lists every public declaration using the same format as this catalogue. `x.neper.*` is not a legal package namespace: Neper-owned facilities live in `e.*`, `algo.*`, `text.*`, `crypto.*`, `fmt.*`, `gfx.*` or `ui.*`. A remaining vendor reservation without a package specification promises zero functions and structures.

Roadmap

docs/roadmap.md | 65289 bytes | sha256 1c4ac20fcc8216e8

neper — roadmap

Sequenced so that each implementation milestone produces something runnable, the bootstrap compiler is discarded as early as possible, and no implementation starts against a moving language or harness contract.

The labels name deliverables, not a simple numeric order. The order is: S0, M0, `neper-0`, the M2 rewrite of the compiler in `neper-0`, M1 implemented inside that self-hosted compiler, the rest of M2 (`.em`, the pool and the own linker), the mandatory M2.5 language/compiler hardening gate, M3 and M4. M5 follows M4. Pacman P0/P1 may proceed after M2; P2/P3 and M6 completion follow M4 independently of M5. The later library waves remain deliberately unnumbered. M1 and M2 overlap by design; every other dependency is stated at its milestone.

M2.5 is planned in `post-m2-llm-hardening.md`. Its changes start only after M2 is ready and must be implemented, migrated and verified before M3 implementation starts. This scheduling decision does not change the language accepted by the bootstrap or the existing M2 completion contract.

The normative documents divide authority rather than duplicate it: `spec.md` defines language semantics, `grammar.ebnf` defines concrete syntax and the closed token/syntax-node registries, `tooling.md` and `schemas/neper-v1.schema.json` define machine interfaces, `diagnostics.md` owns stable diagnostic codes, `modules.json` owns the module/package plan, and `module-apis.md` fixes proposed toolchain APIs. The prose grouping in `modules.md` explains that machine plan. A change that crosses these boundaries updates every affected document and fixture in the same change.

The product commitment follows `modules.json`'s ordered delivery tiers. `core` gates the first stable CPU release. `extended` modules are stable only once independently delivered and do not delay that release. `experimental` APIs carry no compatibility promise. Dependency layers remain an orthogonal implementation constraint.

S0 — Specification, API and harness freeze

This is documentation and test-data work only. It precedes compiler implementation and turns the current design into an executable contract for humans, harnesses and models.

- Resolve every contradiction among the language specification, grammar, tooling protocol, diagnostic registry, module plan, API catalogue, package-manager design and general-purpose verification plan. Record intentional tradeoffs in `decisions.md`; do not leave behavior to an implementation choice.
- Validate `grammar.ebnf` mechanically and give every grammar production an accepted fixture and every stated restriction a rejected fixture with a registered primary diagnostic. Establish the normative `tests/conformance/{accept,reject,format,tokens,parse,tools}/` layout and version every expected result.
- Validate every tooling record and document against `schemas/neper-v1.schema.json`. Add sequence, key-order, sorting, lossless-source, recovery and cross-record tests for constraints JSON Schema cannot express.
- Freeze the one-heading/one-neper-fence API-extraction contract for all 128 toolchain modules. Check that `modules.json`, `modules.md` and `module-apis.md` contain the same names; that tiers are ordered, exhaustive and disjoint; that layers, direct dependencies, blockers, surfaces, schedules and milestones are valid; and that the graph is acyclic. Keep the `6 x.<owner>.*` entries classified as package reservations, never implicit modules.
- Design GP-01 through GP-14 before implementation: contracts, expected module graphs, failure injection, portability targets, resource budgets, deterministic artifacts and comparison implementations. A workload may expose a specification gap; it may not silently define new language behavior.
- Freeze the generated-code benchmark corpus and prompts. Record model, model version, tokenizer, compiler/formatter/schema versions, first-pass parse/type-check/ test rates, repair turns, model tokens, tokens per non-comment line and syntax node, unrelated formatted diff, hallucinated features/APIs, name/import collisions and cleanup/lifetime/race defects. Include equivalent C, C++, Rust, Go, Python, Java and C# tasks; never infer token efficiency from spelling length.
- Make documentation checks runnable as a single read-only validation command whose output is deterministic and suitable for CI and agent harnesses.

Done when: all normative documents and schemas validate; every registry and cross-document reference is closed and consistent; every proposed module API is mechanically extractable; the complete conformance and benchmark fixture manifests exist; the GP workloads have reviewable designs; and no unresolved item can change the grammar, semantics, diagnostic identity, machine protocol or public API beneath M0. Later specification changes remain possible, but require an explicit version or compatibility decision and updated fixtures.

M0 — Bootstrap compiler, walking skeleton

Throwaway C99. Enough language to print "hello, neper".

Implemented (2026-09-04). `bootstrap/neper.c` and `scripts/build-bootstrap.*` produce the native bootstrap on Windows and Linux. `tests/m0/run.*` exercise the cross-directory program root, both x64 argument paths, UTF-8 startup arguments, control flow, calls across register and stack arguments, slices, named-error propagation, deterministic rejection diagnostics, traps, and retained debug and compact-symbol metadata. M0 is complete and the incremental `neper-0` extension, including the first self-hosted compiler modules, is in progress.

- Lexer and parser generated or checked against grammar revision 1, including exact original-byte/scalar/UTF-16 position tracking, normalized line handling, the closed token registry and recovery barriers. The bootstrap need not expose tokens or parse, but it must not create a second concrete language
- Order-independent module resolution and type checking, with registered diagnostic codes and deterministic source ordering from the first rejected fixture
- NIR (typed SSA-lite IR), linear-scan register allocator
- x64 emitter, System V + Windows x64 ABIs
- Emit object files, link with the system linker
- Line tables, symbol tables and unwind info — enough for `lldb` breakpoints, stepping and stack traces, and enough for profilers to symbolise. The incremental `neper-0` implementation below now adds the specified locals-and-types subset (spec §13, D9)
- Subset: `fn`, `let/var`, integers, `if/while`, calls, structs by value, `use`, `err/try/ok`, pointer types and `[]T` slices over any element with `.len` and indexing (string literals as `[]const u8`; `main's *mem.Arena` and `[]str`), and an `e.mem` stub declaring `Arena` and `arena_from` — exactly what `hello.e` needs and no more
- `io.print` and the startup code over a fixed bootstrap intrinsic set — `os.write`, `os.stdout`, `os.stderr`, `os.exit`, `os.reserve`, `os.commit`, `os.args`; the general extern form and `e.os` land in M1
- The `main` entry signature (spec §13, D34): startup code that reserves the root arena, builds `args`, calls `main`, and maps its `err` to the exit code and the `error: <name> line`; the trap protocol's `stderr` record and exit code 134 (spec §11) — the backtrace reads the `.nepersym` line-and-symbol section (`.nepsym` on COFF/PE) the compiler emits into every object file (spec §13), which the system linker carries into the executable unchanged
- Program roots (spec §2, D41): `neper run <file.e>` names a file declaring `fn main`; a file outside every source root is module `<filename>`, so `examples/hello.e` is module `hello` with no package around it, and `e.*` is found in the toolchain's own `lib/` beside the binary

Done when: `neper run examples/hello.e` works on Windows and Linux, from any working directory; and the bootstrap passes the applicable S0 lexical, syntactic, module, type and trap fixtures with the expected primary codes and source spans.

neper-0 — the bootstrap subset

The throwaway C99 compiler (D4) implements exactly this subset and nothing beyond it, and the self-hosted compiler is written in it. `neper-0` is the language a compiler needs — `list.e` is `neper-0` code — and no more.

In progress (started 2026-09-04). The first executable increment implements typed integer range `for`, `break`, `continue`, integer local `++`, and lexical block scopes in the C99 bootstrap. `tests/neper0/run.*` cover signed ascending and empty ranges, early loop exits, sibling-scope name reuse, out-of-scope rejection, and invalid loop control on Windows and Linux. A second increment implements local fixed arrays with explicit or inferred literal lengths, `zero/undef` initialization, `.len`, element-size-aware reads and writes, mutable slice-element assignment, and bounds traps. The same suite covers adjacent narrow elements, signed loads, checked mutation, immutability, count mismatches, bounds failures, and non-overlapping slice/local frame slots. A third increment adds one-binding and `index/value` `for` iteration over arrays and slices, with iteration values held as immutable copies. A fourth increment adds checked `[lo..hi]`, `[lo..]`, and `[..hi]`

slicing with array-to-slice mutability propagation. The aggregate-foundation increment adds named struct declarations, deterministic natural layout and padding, source-ordered named-field literals including nested structs, `zero/undef` storage, field places, pointer-recursive layouts, chained pointer auto-dereference, `&/*`, mutation through `*T`, rejection through `*const T`, and the one-way mutable-to-const pointer conversion. A subsequent aggregate-value increment adds exact struct and array copies through bindings, assignments, nested literals, indexed places and by-value iteration; arrays may nest or hold structs; aggregates of at most two words use integer argument lanes, larger arguments use immutable hidden references, and aggregate returns use caller-owned slots on both x64 ABIs. The enum-and-variant increment adds explicit integer-backed enums with implicit or explicit checked values, bare-union shared storage, tagged union enum layout and literals, implicit `.tag` enums, debug tag checks on payload access, payload-copy as bindings, scalar cases and exhaustive non-fallthrough `switch`. It also enforces duplicate cases, `default`, enum zeroability, branch scopes, and `break/continue` targeting on both x64 ABIs. The cleanup increment adds both forms of lexical `defer`, immediate left-to-right argument capture for deferred calls, place capture for deferred blocks, reverse-order cleanup on normal and structured exits, per-iteration cleanup, deliberate discarded fallible calls, and return-value preservation. The iterator increment adds the compiler-generated `<type>_next(*Type) -> (T, bool)` lookup used by `for value in iterator`, requires a mutable variable or mutable pointer subject, and implements the two-result x64 ABI needed for scalar and aggregate yields. Direct source-level destructuring now supports `let/var` bindings, `_ discards`, and assignment to mutable locals for register- and caller-slot-returned result sets. The constant increment adds order-independent integer `const` declarations, dependency-cycle and overflow diagnostics, runtime constant references, and integer folding for array lengths. Generic functions now specialize `[T: type]` and `[N: usize]` parameters from explicit arguments or structural value-argument inference, cache duplicate instances, and substitute parameters through signatures, bodies, and array layouts. Generic aggregate types now share that canonical specialization cache, substitute field types and array lengths, compute concrete layouts, and support specialized aggregate literals. The fixed bootstrap host increment adds the complete `os.open/read/write/close`, standard-handle, directory, process, argument, virtual-memory and clock surface through a small C99 runtime object. Intrinsic signatures are compiler-owned, fallible calls use one deterministic caller-owned result layout on both x64 ABIs, host failures map to stable qualified `os.*` errors, and the Windows/Linux suite exercises success, failure and process exit paths. The first memory increment gives `mem.Arena` its specified 24-byte value layout and field places, and adds compiler-owned `mem.arena_from` and generic `mem.alloc[T]` bootstrap intrinsics. Allocation lowers every concrete element type to one runtime entry point with hidden size/alignment constants; zero-count, alignment, capacity, multiplication-overflow, cursor-stability and qualified `mem.Exhausted` behavior are covered on both x64 ABIs. Compound type arguments use structural specialization keys rather than colliding with their element types. The next memory increment adds `mem.mark`, source-located bounds-checked `mem.reset`, and allocation-free `mem.stats`; the suite verifies scoped rewind, alignment after a rewind, stable capacity reporting and the reset trap protocol on both hosts. The debug-info increment adds DWARF 4 DIEs on ELF and CodeView type and symbol records on COFF for functions, parameters, named locals, primitives, structures, bare and tagged unions, enums, pointers with pointee constness, slices, arenas, and arrays. Every source local has one stable frame slot and one `rbp`-based location for the full function; larger by-value parameters are copied out of their ABI-indirect input into those slots. Linux validation rejects malformed DIEs and location/range-list sections; Windows validation parses the COFF record streams, and DIA verification confirms the linked PDB reconstructs locals and recursive types. Generated functions whose local frame reaches one page now call a freestanding page probe before allocation on Windows and Linux, retaining Windows unwind metadata without adding a C-runtime dependency; both M0 suites execute a 16 KiB-frame regression and inspect the emitted probe call. The first self-hosted compiler checkpoint is built by the bootstrap on Windows and Linux. Its grammar revision 1 lexer now lives in `src/lex.e`; `src/main.e` imports and exercises that module. The lexer owns all 94 token kinds and scans keywords, comments, CR/LF/CRLF, original-byte spans, strict numeric and quoted/raw literal boundaries, invalid bytes and the complete longest-match punctuation set. Numeric lexing validates base digits, separator placement, exponents, and the closed integer/float suffix sets. Strings, raw strings and comments validate UTF-8 scalars and their context-specific control bytes; character literals decode to exactly one byte. Tokens carry half-open original-byte endpoints and normalized one-based start/end lines with both scalar and UTF-16 columns, including BOM, CRLF, BMP and astral-scalar fixtures. Invalid UTF-8 recovery consumes Unicode maximal subparts as one position unit; valid non-ASCII scalars in ASCII-only token positions remain one `Invalid` token rather than fragmenting by encoded byte. Every token owns its exact leading byte range, including the BOM and trailing trivia on EOF, so the stream partitions the original source without gaps. That range is now enumerable as exact- `span` BOM, maximal space-run and whole-comment trivia with scalar and UTF-16 columns. Comment recovery survives an invalid-byte token and keeps the rest of that physical line in comment trivia rather than reinterpreting it as code. Malformed quoted and raw-string tokens consume through their matching delimiter before recovery, while an unterminated ordinary quote stops before the next physical newline. The C99 bootstrap now loads transitive modules from the nearest project's `lib/` and `src/`, resolves default and explicit import qualifiers, canonicalizes cross-module functions, types, constants and errors,

rejects duplicate qualifiers and import cycles, and attributes diagnostics to their source file. `tests/selfhost/run.*` covers that module boundary, including a nested aliased module and an invalid imported module, on Windows and Linux. The parser foundation now lives in `src/parse.e` alongside the exact 54-kind grammar revision 1 registry in `src/syntax.e`. It owns the reusable token cursor and writes into caller-supplied node and child slices rather than embedding a fixed buffer in every tree. Top-level declarations and attributes own their exact token children; the `File` root interleaves those nodes with original separator and EOF tokens. Ranges are exclusive, delimiter errors produce `ErrorNodes`, and recovery resumes at top-level newline barriers. The first self-hosted `parse` command uses that lossless ordering and tests bounded-capacity failure. Function declarations now parse compile-time parameters, ordinary and variadic parameters, return specifications and their body into nested nodes; a malformed signature recovers at the next top-level newline barrier. Type declarations now parse generic parameters, aliases, struct and bare-union fields, enum members and tagged-union members into the corresponding frozen node kinds; empty enum bodies recover to the following declaration. Function blocks now own classified binding, assignment, call, `try`, `defer`, control-flow, return, `@nocheck` and `shared var` statement nodes. A malformed statement becomes a nested `ErrorNode`, resumes at the next block newline and preserves both its enclosing `Block/FnDecl` and following statements. Every statement now uses its grammar production; the legacy raw token-balancing fallback has been removed. Return values now produce primary, prefix, precedence-aware binary, call, field and bracket-postfix expression nodes, including grouped and member-shorthand primaries. Binding and assignment statements now retain their binding/target and initializer subtrees; call and `try` statements retain their call trees; `zero` and `undef` remain lossless initializer tokens; and failed statement parses roll back partial nodes before inserting an `ErrorNode`. Return statements now retain every value in a multiline or trailing-comma multiple-return list while preserving ordinary single-expression grouping. Bracket- postfix conformance now covers empty contents, indices, every open/closed range form, multi-index lists, soft newlines and trailing commas, and rejects mixtures of ranges and argument lists. Named, generic and fixed/inferred-array aggregate literals now retain a structured `NamedType` or `ArrayType` header and ordered `LiteralItem` children, including nested literals, named payloads, positional values and bare PascalCase members. Local type annotations now recursively retain named, pointer, slice, array and function-type nodes, including array-length expressions, parameters and return specifications. Function declarations now reuse the same trees for typed comptime parameters, ordinary parameters and single- or multiple-type returns. Signature conformance covers named and bare declaration variadics, bare function- type variadics, trailing commas and empty comptime arguments; it rejects named function-type variadics, one-item return-type lists and missing parameter commas. Type aliases, aggregate fields and union payloads now recurse through those nodes; enum discriminants retain expression nodes; and enums and tagged unions retain their backing type. `if/else if/else`, `while` and `when` now retain condition expressions and recursively parsed blocks; canonical-casing lookahead disambiguates PascalCase aggregate literals from lowercase and SCREAMING_SNAKE values followed by a block. Array literal items, including one-letter comptime values such as `N`, remain `NameExprs`; bare PascalCase members are recognized only in named aggregates. Block-introducing `if`, `while`, `when`, and `switch` expressions retain one-letter values at their outer depth while allowing aggregates in nested calls. `for` now retains iterable or bounded-range expressions and a recursively parsed body while preserving one- or two-name iterator bindings as source tokens; a one-letter comptime range endpoint before the body remains a `NameExpr`. `defer` now owns its structured simple statement or recursive block, `@nocheck` owns a validated directive block, `shared var` owns its type and optional initializer, and `break/continue` reject trailing syntax. `switch` now owns its subject and ordered `SwitchArm` children; arms retain multiple case expressions, optional captures, recursive statements and local error recovery. Both the colon and the final successful arm statement require their grammar-mandated physical newline; the closing brace cannot share the statement line. An empty arm requires a blank line so its leading and trailing separators are both explicit. Module `const/var` declarations now retain type and initializer trees, including the special variable initializer forms, and failed top-level parses transactionally discard partial nodes before recovery. Attributes now retain ordered expression arguments, including multiline lists, and recovery distinguishes a malformed attribute from a valid attribute on a malformed declaration. Attribute blocks now reject blank or comment-only gaps and cannot attach to `use`. Assignment parsing retains ordered multi-place targets, accepts multiline and trailing-comma forms plus chained explicit-dereference places, and rejects one-place pseudo-tuples, non-dereference unary or binary targets and storage-only `undef` values. Function types reject declaration-only named variadics. Soft newlines are now consumed only inside grammar-approved parentheses, brackets, literal bodies and type bodies; the lossless token stream retains them, block newlines stay hard, and parser recovery restores delimiter state after malformed nested syntax. Unterminated continuations now stop at declaration, switch-arm, containing-block and EOF recovery barriers; transactional rollback restores both syntax storage and error counts. Top-level nodes carry an internal root marker and are gathered from caller-owned node storage, so declaration count is no longer capped by an embedded 256-entry array. The postorder node stream now records parent adoption as well. Aggregate literals gather their direct children from that stream, removing their embedded 127-item ceiling in favor of the caller's explicit node and child capacities. Call/bracket arguments, generic type arguments, return-type/value lists, function-type parameters, tuple

assignments and attributes now use the same capacity-transparent path rather than embedded 32/64-entry child-ID arrays. Blocks, switch arms/cases, function signatures, type bodies and generic type declarations now use it as well. Every unbounded grammar list is therefore limited only by explicit caller node/child capacity; the remaining local child-ID arrays correspond to fixed-arity productions. The Windows and Linux self-host suites parse 261 top-level declarations and a 130-item aggregate in one source, crossing both former hidden limits. The parser's former 16-node/64-child validation shortcut is removed; CLI and test callers now declare their storage budgets explicitly. The remaining grammar productions and their conformance coverage are the next parser-front-end increments; this callout does not mark the `neper-0` milestone complete. The first arena-backed source loader now lives in `src/source.e`. The self-hosted CLI uses the fixed `e.os` file surface to grow a byte buffer through `mem.alloc[u8]`, preserve exact source bytes across growth, close every opened file, and drive `scan-file/parse-file`; cross-platform tests scan the compiler's own source, parse a loaded fixture and verify stable missing-file propagation. This is a source-loading checkpoint. The next discovery increment adds `src/project.e`: it finds the nearest ancestor containing `lib/` or `src/`, handles relative, POSIX and Windows path roots, and derives canonical dotted names for files below either source root while retaining the bare-name rule for an explicitly named file outside them. It now selects a matching OS or architecture variant ahead of the plain module, rejects simultaneous matches and handles nested module paths. `src/graph.e` completes the discovery pipeline by reading the parser's `UseDecl` nodes, loading every reachable module with project-root precedence and toolchain fallback, and rejecting missing or multiply rooted modules, duplicate local qualifiers, malformed reached sources and import cycles. The first semantic pass in `src/resolve.e` collects every module-scope type and value declaration before resolving references, keeps the type and qualifier/value namespaces separate, enforces builtin-name and duplicate rules, and validates qualified type and value members through local import aliases. It resolves the self-hosted compiler's own reachable source graph on both hosts, with the fixed compiler-owned `e.mem/e.os` names seeded explicitly rather than invented as source declarations. Its lexical scope pass now covers parameters, ordinary and tuple bindings, loop bindings, switch captures and `shared var`; active or module-level shadowing is rejected while names may be reused in disjoint sibling scopes. Running that pass over the compiler removed three pre-existing collisions. The same scope-aware pass now resolves unqualified expression and type names, including forward module declarations, generic type and value parameters, binding initializers, loop bodies, switch cases and captures, top-level initializers and implicit deferred-statement scopes. It rejects unknown names and types, use before binding and references after a lexical scope ends on both hosts. The first `src/check.e` increments now build caller-owned function, parameter, token, local and recursive type tables and check scalar literal context, bindings, lexical inference, exact-type arithmetic and comparisons, conditions, return statements and guaranteed returns through `if`, forward local and source-module-qualified direct calls, argument arity and types, numeric casts, mutability and explicit `void`. Pointer, slice and fixed-array types now compare structurally, including nested forms and zero-length arrays; `str` is canonical with `[]const u8`; mutable pointer/slice weakening is one-way; contextual `nil` covers pointers and slices; and address-of/dereference enforce binding and pointee mutability while rejecting `*void` dereference. Array lengths accept all integer-literal bases and checked `usize` literal arithmetic with overflow and division-by-zero rejection. The bootstrap's x64 emitter now uses unsigned division, remainder and relational instructions for unsigned operand types, including the full `usize` range. Qualified calls into loaded source modules now use the target function's collected signature for arity, argument and return checking. Non-generic aliases now canonicalize recursively through imported names, pointers, slices and arrays while nominal aggregates retain identity; every alias cycle is rejected even when it passes through pointer indirection. Integer constants now use an order-independent dependency graph across loaded modules. Suffixed initialisers may infer the constant type; forward and qualified references, cycles, invalid runtime-state references, division by zero, type mismatches and checked range or bootstrap-word arithmetic overflow are covered. Evaluated constants type-check in function bodies and drive array lengths in function, local and alias type positions; a preliminary/final alias pass makes local and qualified constants available to aliases independent of declaration order. Unary minus and `+`, `-`, `*`, `/`, `%` are the current folding subset. The checker now installs all fixed `os` signatures plus `mem.mark`, `mem.reset` and `mem.stats` as compiler-owned declarations, then checks their arity and structural pointer/slice arguments. Signatures retain complete ordered return lists; source and intrinsic result sets are consumed through tuple bindings or assignments with `_ discards`, and multi-value function bodies check every returned expression. Statement-level `try` consumes a trailing `err` in a whole binding initializer, assignment RHS or error-only call statement, while enforcing fallible caller/callee signatures. Returned values cannot be silently ignored. Compiler-owned `mem.alloc[T]` now specializes its arena/count arguments and `([T], err)` results for primitive, local or qualified named, aliased-composite and pointer `T`. Bracket arguments retain directly written pointer, slice and fixed-array types as type nodes without misclassifying array-literal arguments, covering every `neper-0` allocation element form. Generic source functions now collect `[T: type]` and `[N: usize]` parameters, bind explicit or structurally inferred trailing arguments, substitute composite signatures and symbolic array lengths, cache concrete instances, and check each instantiated body, including forwarded, fallible and qualified calls. Array, slice and `str` indices and slice bounds take `usize` context; `.len`, open ranges, pointee-aware slice mutability, indexed address-of, element assignment and

explicit-dereference assignment use the same place rules. Typed `zero/undef` initializers inherit their declared context. Non-generic struct, union, tagged-union and fixed/inferred-array literals now validate complete field or element sets, including nested and qualified forms. Named field access auto-dereferences pointers; field address-of and assignment share the same pointee-aware place mutability rules. Generic aggregate types preserve explicit type and `usize` arguments in nominal identity, specialize symbolic array fields at concrete use sites, and flow through generic function parameters and instantiated bodies. Recursive specialization reserves stable field ranges for nested generic aggregates, and concrete aliases preserve their nominal instance identity. Enum declarations and contextual, local type-qualified or imported type-qualified members now check nominally; source and fixed intrinsic error values are typed as `err`. Integer shifts require an independently unsigned count, every compound assignment checks its place and operator family, and integer-range, array, slice and string `for` loops install scoped index/value bindings with loop-valid `break/continue`. Pointer equality and enum equality/ordering are checked, and an unbroken `while true` is non-fallthrough for guaranteed-return analysis. The entire self-hosted compiler, including `src/check.e` and `src/main.e`, now passes this checker on Windows and Linux. `defer` now checks captured calls, discarded fallible calls and block bodies under the no-escaping-control rule. Enum, tagged-union, integer, `bool` and `err` switches validate compile-time cases, duplicates, payload captures and exhaustiveness; caller-owned switch facts feed guaranteed-return analysis. Tagged-union `.tag`, contextual tag members and local/imported `Union.Tag.Member` values use a distinct nominal tag type. Compile-time integer evaluation now covers checked arithmetic, width-specific bitwise complement and binary operators, independently unsigned checked shifts, and width-preserving wrapping arithmetic. One result-sensitive corpus exercises ordinary constants, direct array lengths, nested generic aggregate bounds and instantiated generic function bounds on both hosts. Unsupported expression and statement forms fail explicitly rather than being accepted unchecked. Generic function declarations now receive a dependency-aware first check: fixed type, shape, arity, mutability, control-flow and return rules fail before any call, while operations on an unknown type shape are deferred and rechecked in each concrete specialization. The cross-host corpus distinguishes valid dependent arithmetic, fields, indexing, iteration, switching, mutation, calls and literals from independent declaration errors and invalid concrete instances. Bootstrap/self-hosted accept/reject parity is now enforced over all 51 `tests/neper0` sources on Windows and Linux, including ranges, contextual zero values, enum discriminants, recursive aggregate layouts, nested aggregate literals, intrinsic aggregates and iterator protocols. Rejected fixtures emit byte-identical ordered versioned diagnostics from exact source-token spans on both hosts. The self-hosted NIR and code-generation rewrite is next.

- Everything in M0
- Slices, arrays, union and union `enum`, `enum`, `defer`, `switch` (exhaustive), `for`
- `err/try/ok`, arenas (`e.mem`), `let/var with = zero/= undef`
- `[T: type]` and `[N: usize]` comptime parameters with monomorphisation. The interpreter does **integer constant folding only**: `const N: usize = 4096` and `[N*2]u8` evaluate (an array length is `usize`, spec §3); a function call in a `const` does not
- The debug-mode checks of spec §11 and the trap protocol
- Line tables, symbols, unwind info, and the fixed DWARF/CodeView locals-and-types subset (spec §13) — hundreds of lines, and it means the self-hosted compiler can be debugged with locals while the bootstrap still builds it
- A **fixed intrinsic set** standing in for `extern` and `e.os: os.open, os.read, os.write, os.close, os.stdout, os.stderr, os.readdir, os.spawn, os.wait, os.exit, os.args, os.reserve, os.commit, os.clock` — what a single-threaded compiler needs to read sources, write `.em` files and objects, spawn `--linker=system` and time itself

Not in `neper-0`, and therefore not written in C: argument packs and `printf/format` (the self-hosted compiler formats its diagnostics with `str.push_*`), the general compile-time interpreter, threads and atomics, function pointers and the `K: fn comptime kind, @test` and `neper test, neper fmt, neper tokens, neper parse, neper index` and `neper info`, general `extern` and `e.os, Vec[T, N]/simd, when/target, @gpu`, generic protocol dispatch and `e.meta` (the compiler-generated iterator next lookup is the sole protocol exception; the bootstrap's own containers are otherwise written per element type, by hand). Each is implemented once, in the self-hosted compiler, after it compiles itself.

Size, recorded against D4. `neper-0` in C99 is roughly 15k lines: lexer 1k, parser 2.5k, resolve and typecheck 3.5k, NIR and monomorphisation 2.5k, linear-scan allocator 1.5k, x64 emitter 2.5k, ELF/COFF object writer with the debug subset 1.5k. The full M1 language in C would be some 35k — the interpreter, packs, threads, the test runner, `fmt`, `index`, the `extern ABI`, `simd` and `e.os` on top — and every line of it thrown away. The subset halves what is written twice.

Done when: the language can express a compiler — containers over arenas, interning, hash maps — with no heap, and the bootstrap compiles the self-hosted compiler's source; the bootstrap and self-hosted front ends accept and reject the same `neper-0` corpus and emit the same versioned diagnostics and recovery spans. M2 gates on this.

M1 — The full CPU language

Everything in the spec that is not GPU-specific. The bootstrap stops at `neper-0` (above); every item below that `neper-0` lacks is implemented in the self-hosted compiler, so M1 completes after M2's rewrite compiles itself, not before.

- Slices, arrays, unions (untagged and `union enum`), enums, `defer`, `switch` (exhaustive over enums and tagged unions), `for` — no tuples: `(A, B)` is a return convention only (spec §5, D17)
- `err + try`, arenas, compile-time parameters in `[...]` and monomorphisation
- Compile-time interpreter for `const` and `[...]` arguments
- The full debug-mode check table of spec §11 — bounds, null, tag, overflow, narrow, shift, enum, align — the trap protocol, `unreachable()`, and the arena fills
- `Vec[T, N]`, `Mask[T, N]` and the `simd` module (spec §4, D40): the closed width table, register-class passing, and lowering at every `--cpu` level including the split below the vector's width
- Debug info on the default path (spec §13, D9, D64): the fixed DWARF (ELF/Mach-O) and CodeView (PE) locals-and-types subset with `DW_OP_fbreg` locations, so `lldb`, `gdb`, `WinDbg`, `lldb-dap` and `code1ldb` show locals from here — VS Code and Zed debugging with zero adapters of our own. Debug builds do not inline; `--g` adds the subset and `inlined_subroutine` records to a release build
- The finite-command JSONL contract for `build`, `check`, `run`, `test`, `fmt`, `tokens`, `parse`, `index`, `dis` and `info`: one version header, advertised `language_profiles`, closed record discriminators, diagnostics in-stream, one final result and no human text on stdout. Validate every record with the v1 schema and every non-schema ordering rule with the S0 tooling corpus (D65)
- Lossless `tokens/parse` output over grammar revision 1's closed 94-token and 54-syntax-node registries in `grammar.ebnf`: trivia, BOM, physical newline spelling, invalid-byte capture, `ErrorNode` recovery and reconstruction of every original byte. Spans retain original byte offsets plus normalized one-based scalar and UTF-16 columns
- Complete deterministic `index` output for every specified symbol and reference, including unresolved references and compiler-origin protocol, iterator and formatting calls. Stable diagnostic codes come only from `diagnostics.md`; fixes carry non-overlapping original-byte edits and expected-source hashes
- `neper fmt` implements the complete canonical-layout contract: LF/UTF-8 output, four-space indentation, deterministic 100-scalar wrapping, preserved comments and literal spelling, sorted eligible `use` declarations, exact failure behavior, idempotence and golden output
- `@test` discovery uses one child process and fresh arena per test, parallel scheduling with deterministic report order, capture, the 60-second timeout and structured outcomes. `run --json` captures arbitrary child bytes without corrupting JSONL; every duration is the sole normalized volatile field in golden comparisons
- Every build writes the canonical v1 build manifest with language/grammar versions, normalized source identities, SHA-256 inputs/dependencies/libraries/artifacts and effective options. Generated-source maps use exact tooling spans and hashes; stale or malformed maps fail with `E-TOOL-0001` and never change compilation semantics
- extern declarations with `@import/@cc` (spec §5, D32): the C ABI for every crossing type on System V and win64, C variadics, `*void` and `mem.cast`, `@cc` callbacks; `e.os` per target — files, directories, processes, memory reservation, clock, threads, wait/wake, sockets, polling and dynamic loading — over raw syscalls on Linux and `kernel32` on Windows. **M1 depends on this landing first:** every other `lib/e` module that touches the OS is written over `e.os`
- Builtin `Atomic[T]` and orderings; `e.thread` over OS threads
- Protocols and `e.meta` (spec §9, D52, D53): `T.f(...)` resolution at instantiation, `for` over a `next`, the `comptime-unrolled` `for`, and `fields/members/type_name/get/set`. `e.data.map`, `e.data.sort` **and every** `fmt.*` **module depend on these**
- M1 library set, using canonical qualified names from `docs/modules.json`: `e.mem`, `e.meta`, `e.math`, `e.simd`, `e.atomic`, `e.bytes`, `e.str`, `e.path`, `e.data.list`, `e.data.map`, `e.data.sort`, `e.data.iter`, `e.os`, `e.io`, `e.thread`, `e.time`, `e.test`. Implement exactly the M1 declarations frozen in `module-apis.md`; a module is not delivered while a declared surface is missing

Done when: every item above is implemented in the self-hosted compiler, and `lib/e` builds and its tests pass under `neper test`; all applicable conformance fixtures validate byte-for-byte after the specified duration normalization; tooling streams reconstruct source exactly and validate against the schema; formatter idempotence holds; every M1 API fence matches extracted source declarations; and the S0 generated-code benchmark has been rerun with no unexplained regression.

M2 — Self-hosting and `.em` modules

- Rewrite the compiler in `neper-0`, compiled by the bootstrap; single-threaded at first, gaining the pool and the rest of M1 once it compiles itself
- Bootstrap compiler frozen, then deleted
- Work-stealing thread pool, per-thread arenas, sharded intern table
- Parallel parse phase, then function-granular parallel codegen
- Determinism harness: byte-identical output across `-j 1` and `-j N`, **and byte-identical output from an incremental rebuild and a clean build** of the same sources, over an edit script that touches bodies of inlined, generic and comptime-executed functions (spec §12, D36); the device-reached case is added to the same harness at M3, when `@gpu` exists
- `.em` format: per-declaration signature and body hashes, fine-grained `Deps` edges, NIR with body hashes, machine code, standard-format debug sections
- Incremental and parallel module compilation on the edge rule of spec §12
- Cross-module inlining via NIR, capped at 40 NIR instructions per callee, each inlined body recorded as a body-hash edge
- Monomorphised instances and device-compiled helpers emitted into the instantiating module's `.em` with module-local linkage; the own linker folds copies by content hash
- M2 library set from the machine plan: `e.data.deque`, `e.data.ring`, `e.data.heap`, `algo.rand`, `algo.uuid`, `algo.hash`, `algo.bitset`, `e.fs`, `e.proc`, `e.sync`, `e.channel`, `fmt.json`, `fmt.csv` and `fmt.ini`, with the exact frozen APIs and only their declared direct dependencies
- Module-plan validation in CI: source imports are a subset of each module's `direct_dependencies`; no layer violation, cycle, undeclared public symbol or package reservation enters the toolchain graph; implemented surfaces advance to `surface:"source"` only in the same plan revision that verifies their extracted public declarations
- Own linker, easy case first: freestanding ELF executables, and PE executables with a fixed `kernel32` import table (spec §13, D13), so `hello.e`, the compiler and `lib/e` link on Windows too with no object files and no process spawn. No PDB yet: Windows symbolication by external tools stays on `--linker=system` until M4

Done when: `neper` compiles itself; GP-01 passes its applicable correctness, failure-injection and deterministic-build cases; a one-function edit rebuilds in milliseconds on Linux and Windows; clean, incremental, relocated, `-j 1`, every supported worker count and perturbed-schedule builds produce byte-identical deterministic artifacts; every M2 API matches source; and the language- and tool-complete release gates below pass.

M2.5 — LLM language and compiler hardening

Scheduled; not started. Entry: M2 complete. Exit blocks M3. `post-m2-llm-hardening.md` records the motivating article, deep design evaluation, H01–H29 requirements, alternatives, migration and acceptance evidence.

`llm-hardening-recommendations.md` distills that review into the prioritized R01–R11 recommendation set and registers R08–R11 as H26–H29. It is a cross-cutting gate, not a new module tier or a module-manifest milestone value. All H items begin unresolved; this document marks none delivered.

- Archive the reproducible M2 compiler/source and measure the baseline before changing semantics. Freeze held-out CPU workloads and performance budgets.
- Resolve and implement checked ownership/cleanup, region and container-view validity, checked optimized execution, unsafe boundaries and scoped concurrency. Define the analysis limits; move syntax alone is not lifetime or race safety.
- Resolve the large-aggregate by-value aliasing rule, explicit native error detail, partial-failure resource states and explainable/explicit protocol strategies.
- Deliver bounded compiler-derived semantic context, causal diagnostics, snapshot-checked transactional repairs, executable API guidance and fast, cancellable checks backed by sound cache invalidation and IR verification.
- Apply `D84/stdlib-hardening.md` SL01–SL11 under H11: keep no-shadow syntax, fix parameter/import collisions, execute declaration validation, migrate delivered CPU I/O/JSON/iteration/filesystem/process contracts, and deliver common `e.cancel` plus core `text.utf8`. Their cross-cutting gate supplements the numbered module milestones without changing the preserved M2 baseline. Static catalogue checks do not substitute for real resolver and executable behavior tests.
- Implement declaration/query-level reuse, immutable workspace snapshots and multi-file overlays, bounded request/cache lifetimes, semantic rename/move/delete plans, compact bounded streams and regeneration-aware

provenance (H14–H19).

- Harden implemented artifact readers, collision handling, concurrent publication and aggregate work limits (H24). Verify invalidation and fault recovery against clean builds; resource exhaustion cannot become a successful partial check.
- Freeze measured CPU build/optimization policy and publish supported optimization explanations (H20). H25 registers cold/warm/edit/refactor/runtime/resource budgets and reports full harness cost per verified success. Advanced optimizers require separate delivery decisions; they are not implicitly required before M3.
- Version affected language/grammar/protocol/ABI/cache contracts, migrate the self-hosted compiler and delivered M2 APIs, and update all affected specification, schema, decision, library and conformance documents together. Keep the M2 predecessor reproducible without adding these features to the C bootstrap.
- Pass the H12 held-out evaluation on at least two independently trained model families using actual compiler tools and independent behavioral oracles. Report correctness, escaped defects, false rejection, unsafe additions, full token/repair costs and compiler/runtime distributions; synthetic editing success is insufficient.
- Freeze the GPU integration of the new ownership, sharing, safety and tooling contracts, with named M3 fixtures. Device implementation/testing remains in M3.
- Freeze completion-token/cross-queue/range-lifetime contracts and end-to-end GPU staging/pipeline-cache accounting; correct subgroup-equivalence and floating-point capability/execution-mode claims (H21–H23). Record M3 runtime fixtures explicitly pending. Execution graphs and a fast CPU kernel backend remain separately decided.

Done when: every H01–H29 M2.5 obligation has its closure record; all selected CPU changes and tooling are implemented; migrated self-hosting, applicable conformance, ABI and clean/incremental/parallel determinism checks pass on Windows and Linux; H12/H25 CPU evidence and budget gates pass; and H13/H21–H23 GPU contracts and future fixtures are frozen, without claiming device runtime verification. Merely documenting a risk, deferring it past M3 or silently weakening a threshold does not close it. The broader general-purpose release gate retains its later prerequisites.

M3 — GPU

Entry requirement: M2.5 is complete. No GPU implementation starts while an H01–H29 M2.5 closure requirement remains open. M3 supplies actual device/runtime evidence for the contracts and fixtures frozen under H13/H21–H23.

- `@gpu(X, Y, Z)` profile enforcement with call-chain diagnostics; device types, device slices and address spaces (`[]shared T`); `shared var` (spec §10, D37)
- SPIR-V emitter on the Vulkan 1.2 floor — `PhysicalStorageBuffer64`, scalar block layout, `NoContraction` on every result, correctly rounded division and square root sequences; Vulkan compute runtime written over `extern` (spec §5, D32). Its `libvulkan` import needs the M4 dynamic-import linking stage, so a program that opens a `.Vulkan` (or, once the PTX emitter exists, a `.Cuda`) device links under `--linker=system` until that stage lands (spec §13)
- CPU backend (spec §10, D38): a launch runs workgroup by workgroup on the calling thread; a workgroup runs on one host thread by barrier loop-fission, with per-invocation slots for locals that cross a barrier, the barrier divergence trap and the `0xCD` fill of `shared` — same semantics, steppable
- `e.gpu`: Device, Queue, `Buf[T]`, synchronous staging `upload/write`, asynchronous `launch[K]` with the compile-time pack check, in-order release, syncing download, `gpu.sync`, `grid1/2/3` as invocation counts, `gpu.barrier`, `gpu.memory_barrier`, `gpu.atomic_*` with scopes, the subgroup set, capability inference into the `.em` and the launch check — the implementation and documentation must match the exact `e.gpu` API fence in `module-apis.md`
- Device selection: bounded `devices`, `DeviceInfo/DeviceKind/DeviceKey`, exact `open_id`, and opening-time `info` (spec §10, D83). Expose names, capacity and capabilities without treating ordinals as persistent IDs. Reject missing or ambiguous keys and cross-device handles; never silently choose a substitute. Test zero/one/many devices, duplicate names/keys, changed ordinals, hotplug, unsupported floors, arena/limit failure and simultaneous devices. CPU/Vulkan run in M3; CUDA/partition-identity cases run when M4 adds that backend. Require a hardware-gated two-device integration job; CPU mocks are supplemental evidence, not reported as multi-GPU hardware passes. GPU tests cannot silently CPU-fallback.
- Device-callable inference: plain functions reached from a kernel compile for the device into the kernel-owning module's `.em` (spec §12, D24), with call-chain diagnostics on violations
- Floating point (spec §11, D39): no contraction anywhere, `math.fma` the only FMA, denormals preserved, `@gpu(..., ftz)` as the opt-in
- The M2 determinism harness gains the device-reached edit case (spec §12, D36)

Done when: GP-09 and GP-10 pass on every backend available at M3, and every kernel in the test suite that uses no approximate builtin — the `e.math` transcendentals `sin cos tan asin acos atan atan2 exp exp2 log log2 log10 pow`, `math.rsqrt`, and the float `gpu.subgroup_add/min/max` reductions (spec §11) — produces **bit-identical** output on the CPU backend and on a Vulkan device (linked with `--linker=system`, above), the `saxpy` kernel of `examples/saxpy.e` among them (the example is a program root run as `neper run examples/saxpy.e`, spec §2; it runs on `.Cpu`, then on `.Vulkan` when one is present, and prints both checksums); and every kernel that uses one agrees within its stated ULP bound. The performance-language gate is rerun whenever M4 or M5 adds an applicable CPU or GPU backend; published comparisons use identical algorithms and report distributions, compile time, runtime and memory rather than isolated best cases.

M4 — Breadth and depth

- `aarch64` emitter (macOS, Linux, Windows on ARM)
- `x86-32` emitter
- `PTX` emitter
- Own linker, hard case: dynamic imports by name from any `.dll/.so/.dylib`, plus ad-hoc code signing on Apple Silicon, plus the PDB writer — MSF container, DBI, module, symbol and line streams, no type stream (spec §13, D13).
`--linker=system` remains for static archives
- `neper-format` debug side table in `.em` — types, locals, scopes, variable locations over the serialised type table — beside the standard subset, never instead of it (spec §13, D9)
- Debug engine: process control, breakpoints, single-step, stack walking, location rendering
- `neper dap` over that engine, plus VS Code and Zed extensions — the optimisation over the `M1 lldb-dap/code11db` path. DAP keeps its framed JSON transport and rejects `--json` and `--absolute-paths`; it is the intentional exception to the finite-command JSONL envelope
- Optimiser depth: better inlining heuristics, load/store forwarding, scheduling; cross-`.em` inlining stays under the M2 cap

M4 expands the platform matrix for all applicable conformance, ABI, determinism, debugging and GP workloads. It does not by itself claim general-purpose completion: networking, advanced pure domains, interchange formats and multi-package verification remain in the later library waves and M6.

Done when: every promised M4 target passes the applicable language/tooling corpus and bidirectional C ABI fixtures; clean and incremental outputs remain deterministic; the native debug path agrees with standard debugger locations on shared fixtures; and GP-09/GP-10 have been rerun for every new applicable backend.

Later toolchain-library waves — unnumbered

The 96 entries with `schedule:"later"` and `milestone:null` in `modules.json` are real proposed toolchain modules, but this roadmap does not disguise them as part of M4, M5 or M6. They may begin when their declared dependencies exist. Before each wave starts, its `surface:"planned"` API is frozen; delivery requires tests, extracted source/API equality and a same-change transition to `surface:"source"`. The core `e.cancel/text.utf8` exceptions have explicit M2.5 cross-cutting delivery obligations in `stdlib-hardening.md`; null Mn milestones do not waive that gate. Their position here does not order them before M5 or M6; independent waves may run in parallel once their prerequisites and specifications are ready.

- **Extended containers and graph algorithms:** `e.data.tree`, `e.data.disjoint_set`, `e.data.graph`, `e.data.slot_map` and `algo.graph`. Existing collections gain deterministic non-mutating iterators; `e.data.heap` also gains comparator/context and linear-time bulk construction. Graph delivery covers BFS, DFS, deterministic topological sorting, weak/strong components and Dijkstra over immutable CSR views.
- **Extended pure algorithms, text and cryptography:** `algo.stat`, `algo.complex`, `algo.decimal`, `algo.bignum`, `algo.deflate`, `algo.linalg.matrix`, `algo.linalg.tensor`, `text.unicode`, `text.encoding`, `text.normalize`, `text.collate`, `text.regex`, `text.locale`, `crypto.hash`, `crypto.mac`, `crypto.kdf`, `crypto.aead`, `crypto.sign`, `crypto.kx`, `crypto.random` and `crypto.x509`. Preserve caller-owned allocation, caller-supplied entropy and the declared dependency edges. Cryptographic delivery requires published standard vectors, malformed-input cases and verification of every API that explicitly promises constant-time behavior.
- **Extended host and application services:** `text.io`, `e.task`, `e.time.calendar`, `e.tz`, `e.fs mmap`, `e.fs.watch`, `e.concurrent.queue`, `e.concurrent.map`, `e.debug`, `e.metrics`, `e.log`, `e.cli`, `e.async`, `e.async.io`, `e.net`, `e.net.tls`, `e.net.http`, `e.net.ws`, `e.db`, `e.test.support`, `e.test.coverage` and `e.test.fuzz`. These build over the M1/M2 platform boundary and must demonstrate cancellation, backpressure, bounded buffers, partial I/O, deterministic shutdown and

no hidden allocation or entropy. They unlock the GP-04 service workload.

- **Interchange formats:** `fmt.uri`, `fmt.mime`, `fmt.asn1`, `fmt.pem`, `fmt.multipart`, `fmt.mail`, `fmt.quoted_printable`, `fmt.gzip`, `fmt.zstd`, `fmt.bzip2`, `fmt.lzw`, `fmt.zlib`, `fmt.zip`, `fmt.tar`, `fmt.yaml`, `fmt.xml`, `fmt.html`, `fmt.png`, `fmt.jpeg`, `fmt.webp`, `fmt.bson`, `fmt.msgpack` and `fmt.protobuf`, plus `text.template` and its context-safe `fmt.html.template` specialization. These consume caller-provided slices/readers and writers, never open resources themselves, and require malformed, streaming, bounds and round-trip corpora in addition to API equality. `fmt.html` additionally runs the pinned `html5lib` tokenizer and tree-construction fixtures for the WHATWG behavior frozen by that toolchain release.
- **Stable pure images and codecs:** deliver extended `gfx.geometry`, `gfx.paint`, `gfx.image`, `fmt.png`, `fmt.jpeg` and `fmt.webp` as one compatibility cohort under `image-codec-conformance` (D84). Test independent decoders, pixel/stride/alpha contracts and hostile inputs. This wave requires no GPU/window or GP-15 UI gate.
- **Experimental collection conveniences:** `e.data.stack`, `e.data.queue` and `e.data.linked` remain available for workload evaluation but have no compatibility promise. `e.gpu.tensor` is likewise experimental, follows both M3 `e.gpu` and extended `algo.linalg.tensor`, and keeps every operation as an explicit queue submission. Promotion requires an applicable general-purpose workload and a recorded compatibility decision.
- **Experimental declarative GPU UI:** extended pure image/geometry/paint values support experimental `text.shape`, `text.layout`, `ui.style`, `ui.layout`, `gfx.scene`, `e.asset`, `ui.asset`, `ui.window`, `ui.input`, `ui.widget`, `ui.animation`, `ui.accessibility`, `ui.testing` and `ui.app`. Delivery follows the staged vertical slice and exact lifetime/reconciliation contracts in `ui-framework.md`; implementation remains blocked on reviewed embedded-asset linking, native-window, GPU-presentation and accessibility primitives. GP-15 must pass before any module in this family is promoted.

The 9 owner-qualified `x.*` reservations are actual vendor packages, not a fifth wave. Each gets a separate package specification only after its upstream API version, supported targets, ownership rules and licensing boundary are selected. `x.neper.*` is forbidden: Neper-owned locale, URI/MIME, TLS, compression, archive and time-zone facilities use toolchain namespaces and pin their standards/data snapshots to the toolchain version.

M5 — Native Metal

- Metal backend: MSL or AIR emitter plus a Metal runtime, replacing MoltenVK as the first-tier Apple GPU path
- Metal-only capabilities exposed where they map onto the GPU profile

Sequenced after M4 because a GPU backend costs an emitter *and* a runtime, and Apple is one vendor. Until then Apple GPUs are reached through MoltenVK, supported but second-tier (spec §10).

Done when: the Metal backend passes the same capability, ABI, source-map, determinism and GP-10 correctness/ULP fixtures as the other applicable GPU backends, and its published performance results satisfy the performance-language gate.

M6 — pacman package manager

Pacman starts after M2 stabilizes self-hosting and `.em`; its remote and task stages land after M4 stabilizes the platform and dynamic-linking path. It does not wait for M5's independent Metal backend. It is bundled as the portable `neper pacman` command group; the package manager remains outside compilation, and the compiler never accesses the network. The normative design is `pacman.md`.

- **P0, formats and resolution:** strict `project.yaml` subset including normalized root asset declarations, canonical `project.lock`, SemVer 2.0 constraints, deterministic PubGrub resolution and explanations, module-export collision checks, language-version checks, and golden fixtures
- **P1, local and offline:** deterministic asset hashing and read-only executable embedding, path dependencies, global immutable content-addressed cache, atomic concurrent population, `sync --frozen --offline`, a read-only virtual package-module map, reproducible dependency source identities, and compiler build- manifest/cache integration. Every vendored root has the canonical `neper-package.json` shape from the v1 schema; package source identities use the same root/path rules as compiler tooling
- **P2, remote sources:** signed central/private registry protocol, immutable release archives, Git dependencies pinned to commit and tree hash through an external Git adapter, credential-store integration, publish/yank, cache verification and explicit garbage collection
- **P3, generated inputs:** root-only explicit tasks instead of dependency lifecycle hooks, argument-array execution, scratch directories, capability approval, locked tools, content-addressed outputs and v1 generated-source maps with

exact hashes and spans. Install and sync never execute package code

- Portable CLI: `init`, `add`, `remove`, `resolve`, `sync`, `upgrade`, `build`, `run`, `task`, `vendor`, `publish`, `cache`, and `info`, all with the toolchain's versioned JSON output conventions, shared diagnostic registry, deterministic record ordering and schema validation
- The public namespace policy reserves `e.*`, `algo.*`, `text.*`, `crypto.*`, `fmt.*`, `gfx.*` and `ui.*` to the toolchain; registry packages normally export `x.<owner>.*`

Done when: GP-12 passes; the same manifest and signed registry snapshot resolve to a byte-identical lockfile on Windows, Linux, macOS, and in a container; a clean cache can be populated from the lock and then build byte-identically under `sync --frozen --offline`; mutable Git refs cannot alter a locked sync; concurrent syncs expose no partial entry; and neither transitive packages nor denied tasks can execute code.

Cross-milestone verification and release claims

`general-purpose-verification.md` is the authority for the workloads, metrics and decision rule. The roadmap supplies sequencing only. A workload starts as soon as its prerequisites exist and is rerun when a relevant backend, ABI, module, package or protocol changes.

Workloads	Primary enabling delivery
GP-01 self-hosted compiler	neper-0 and M2
GP-02 CLI, GP-03 parallel executor, GP-06 streaming parser, GP-07 bounded cache, GP-13 binary format	M1 tooling plus the M2 library set
GP-04 HTTP service	later <code>e.async</code> , <code>e.net</code> and <code>e.net.http</code>
GP-05 database client, GP-08 plugin C ABI	M1 FFI, M4 dynamic linking, <code>e.db</code> , and one of <code>x.sqlite.sqlite</code> , <code>x.oracle.mysql</code> or <code>x.postgresql.libpq</code>
GP-09 SIMD codec	M1 SIMD; rerun for each M4 CPU backend
GP-10 CPU/GPU numerical work	M3; rerun for M4 PTX, M5 Metal and later <code>e.gpu.tensor</code>
GP-11 terminal application	platform support plus a separately specified terminal package owned by its actual provider
GP-12 multi-package application	M6
GP-14 image/audio pipeline	M2 concurrency, M4 target/FFI coverage and a separately specified media package
GP-15 declarative GPU desktop app	experimental <code>e.asset/gfx.*</code> / <code>ui.*</code> family plus asset linking, native window, presentation and accessibility blockers

The four named release gates are cumulative. In addition, M2.5 is a mandatory transition gate between completed M2 and the start of M3. Its bounded CPU evaluation does not claim that the later general-purpose or GPU gates have passed:

1. **Language-complete:** `spec.md` and `grammar.ebnf` have no unresolved contradiction; every syntax/semantic rule has planned conformance coverage; non-goals have constructions or excluded workloads; no GP design requires an unapproved language feature. S0 must satisfy this gate before M0.
2. **Tool-complete:** formatter, lossless tokens/parse, complete index, stable diagnostics/fixes, tests, build manifests and source maps obey their versioned contracts; all locations have exact original-byte spans; clean, incremental, parallel and relocated builds are deterministic. M2 cannot complete without it.
3. **General-purpose:** GP-01 through GP-08 and GP-12 pass on Windows and Linux; long-running profiles have bounded resources; FFI and failure-injection suites pass every promised host ABI; and the LLM thresholds pass on at least two independently trained model families. This gate necessarily waits for M6 and the required later networking/package work; no earlier milestone implies it.
4. **Performance-language:** GP-09 and GP-10 are correct on every applicable backend, and comparable C, C++, Rust and Go results publish distributions for compile time, runtime and memory using the same algorithms. M3 establishes the first result; M4, M5 and later GPU work reopen it.

The initial LLM thresholds are release criteria, not aspirations: at least 95% of local edits parse first pass, at least 90% type-check first pass, median unrelated formatted diff is zero, at least 95% of compiler-error repairs finish in one additional turn, no task requires parsing human diagnostics, and no accepted task uses undocumented behavior or intrinsics. Thresholds may tighten after a versioned baseline, but may not be silently relaxed. A syntax/API change is

justified by total model effort, repair behavior and review risk across the supported model/tokenizer matrix—not by character count or a single tokenizer.

Deliberately not scheduled

Auto-vectorisation, an LSP beyond the public `neper parse/neper index` data, and any form of runtime reflection. Each is a real cost against the two headline goals. Package management is scheduled as M6 under the deliberately constrained pacman design above.

Decisions

docs/decisions.md | 102251 bytes | sha256 8a97c866b02d739e

neper — locked design decisions

Decisions that are settled. Each one constrains everything downstream, so changing one means revisiting the spec. Recorded with the reasoning so a future reader (human or model) does not relitigate them. Rows D1–D26 are the original design; D27–D46 were added by the 2026-09-04 review and its three verification passes (2026-09-04); D47–D50 by the design calls accepted on 2026-09-04 that closed §17's open items; D51–D53 by the standard-library plan and sample program of 2026-09-04 (`docs/modules.md`, `examples/sample.e`); D54–D64 by the implementation-readiness audit of 2026-09-04, with later rows superseding the narrower clauses they name; and D65–D73 by the machine-interface and generated-code audit of 2026-09-04. Later rows explicitly supersede narrower clauses in earlier rows; historical rationale is retained but is not normative after supersession. A row that any pass or call changed carries an `(amended 2026-09-04: ...)` clause in its Why column — D9 was rewritten outright. Rows that predate D51 spelled the standard library `core.<module>` and its directory `lib/core`; every one of them has been rewritten in place to the `e.*` names and the `lib/e` path D51 introduced, which renames no decision and moves no call site. Ids are sequential and are never reused.

#	Decision	Chosen	Why
D1	Codegen backend	Own direct machine-code emitter (x64, aarch64, x86-32) + SPIR-V/PTX	Compile speed is a headline goal; LLVM costs 10-100x. No external dependency, full control of the binary format.
D2	GPU model	Explicit <code>@gpu</code> kernels, restricted GPU subset	A GPU has no unified stack, no recursion, no function pointers. A restricted profile is honest; auto-mapping produces errors far from their cause.
D3	Memory	Stack + explicit arenas. No global allocator, no GC.	Deterministic, trivially fast, maps onto GPU scratch memory, and guarantees no function allocates in secret — which is what makes cross-module inlining cheap. (amended 2026-09-04: "cheap" was about the cost of a call, not about invalidation — an inlined body is recorded as a body-hash edge in Deps (§12, D36), so the inliner is recompiled when the callee changes.) (amended 2026-09-04: one exception to "no function allocates in secret" — <code>gpu.upload/gpu.write</code> take a driver-owned staging block outside every arena for the duration of the transfer (§10); it is named on the page and <code>gpu.OutOfMemory</code> is its failure.)
D4	Bootstrap	Throwaway C99 compiler implementing only the <code>neper-0</code> subset (roadmap); the self-hosted compiler is written in <code>neper-0</code> ; M2 gates on the bootstrap compiling it; everything beyond <code>neper-0</code> is implemented once in the self-hosted compiler	C99 builds anywhere with no toolchain install. The bootstrap is discarded, so its ergonomics matter less than its portability. (amended 2026-09-04: the bootstrap's scope was never sized — as "everything M1 lists" it was a second full compiler; <code>neper-0</code> is what <code>list.e</code> -shaped code needs, roughly 15k lines of C99 against some 35k for the full M1 language in C, and it halves what is written twice.)

#	Decision	Chosen	Why
D5	Compile-time abstraction	Compile-time parameters in [...] (§9), monomorphised; one interpreter for four sites — [...] arguments, <code>const</code> initialisers, <code>when</code>, and a call to a comptime-only core function (<code>meta.fields</code>, <code>meta.members</code>, <code>meta.type_name</code>) standing in an ordinary body with every argument comptime-known, folded at the instantiation — the four being what "comptime context" names; with byte-addressed compiler-owned memory in the target's layout and endianness; runtime state (module <code>var</code>, atomics other than <code>atomic.init</code>, <code>threads</code>, <code>extern/OS</code>, <code>gpu.*</code>) is a compile error naming the call chain; a <code>const</code> result is pointer-free save a []const u8 into compiler-emitted read-only data — a string literal or a compiler-synthesised name; 10M steps, depth 1024 and 64 MiB of interpreter memory per evaluation	One mechanism covers generics, compile-time constants and specialisation. Required to write the self-hosted compiler's containers over arenas. (amended 2026-09-04: "touches no runtime state" defined neither runtime state nor the interpreter's memory — with untagged unions and arenas a value-tree interpreter is wrong, and the bootstrap and the self-hosted compiler must produce the same bytes, so the model is byte-addressed and target-laid-out; the budget is what stops a <code>const</code> from hanging the build.) (amended 2026-09-04: the step budget bounded time and not space, so a <code>const</code> that allocated without looping was unbounded; 64 MiB is the third limit.) (amended 2026-09-04: the runtime-state clause named every atomic operation while §8 makes <code>atomic.init</code> comptime-evaluatable and legal in a <code>const</code>; <code>init</code> builds a value and touches no location, so it is carved out here and in §9's list.) (amended 2026-09-04: three evaluation sites left <code>meta.type_name[T]()</code> and <code>meta.fields[T]()</code> illegal in an ordinary body, which is where §9's own worked example calls them, and left "comptime context" undefined; a comptime-known call to a comptime-only function is the fourth site, and the four define the term.) (amended 2026-09-04: "a literal []const u8" made <code>const NAME: str = meta.type_name[T]()</code> illegal, where §9 already lets a <code>const</code> hold any []const u8 the compiler emits into read-only data.)
D6	Errors	Multi-return (<code>T, err</code>) + the <code>try</code> form — a keyword prefixing one call, not an operator (§6); error names are module-scoped and qualified cross-module (<code>os.NotFound</code>); the value is the 32-bit FNV-1a hash of the fully qualified name, and a name whose hash is 0 is a compile error at the declaration; each <code>.em</code> carries a value-to-name table, merged and collision-checked at link; <code>err</code> formats as its qualified name, and a value absent from the table as <code>err(0x<hex>)</code>	Zero cost, no allocation, no hidden control flow, and every propagation site is one greppable token. (amended 2026-09-04: "globally unique" was undefined — names are unique per module like every declaration, and a hash of the qualified name makes the value a comptime constant with no numbering pass, so adding an error in one module rennumbers nothing and -j N output stays byte-identical; the per-module table plus a link-time collision check gives <code>err</code> a printable name with no reflection.) (amended 2026-09-04: the example was <code>io.NotFound</code>, an error <code>e.io</code> never declares (D43); it is <code>os.NotFound</code>.) (amended 2026-09-04: <code>try</code> was called an operator here and in the README where §6 says it is not; §7's hash-is-0 error and the <code>err(0x...)</code> form for a value absent from the table — reachable only through <code>undef</code> or a union pun — had no row.)

#	Decision	Chosen	Why
D7	Safety checks	On in debug, elided in release; every arithmetic check has a defined release result (wrap, truncate, saturate, masked count, unchecked enum) — never undefined behaviour; an elided memory check (bounds, null, tag, align, barrier) leaves the operation to the hardware, which is undefined behaviour as in C; division by zero and <code>MIN / -1</code> trap in every mode; <code>unreachable()</code> is never elided	Serves "easy to debug" without taxing release builds. Semantics are identical up to the point a check would fire. (amended 2026-09-04: §11 granted C-style "undefined" for release division, which contradicted "elided" — x64 raises #DE regardless, so "undefined" bought nothing but licence to miscompile; trapping in every mode costs one compare on the targets that do not trap for free, and keeps release behaviour target-independent.) (amended 2026-09-04: "never undefined behaviour" overclaimed — an out-of-range write with the bounds check off is undefined by any reading, and the <code>align</code> row's release result is the hardware's; the claim is narrowed to the arithmetic rows, where it is true and testable.)
D8	Source extension	<code>.e</code>	Single-letter, collides with nothing live, keeps harness output short. Compiled modules are <code><name>.<target>.em</code> .
D9	Debug info	Three things, kept distinct. For external tools, standard formats : line tables + symbols + unwind always, and locals and types in a fixed twelve-tag DWARF/CodeView subset with <code>DW_OP_fbreg</code> locations, on the default path from <code>neper-0</code> (the bootstrap emits it too, so the self-hosted compiler is debugged with locals from its first build). For the runtime's own trap backtrace, <code>.nepersym</code> (D46), on every platform under either linker. Debug builds do not inline; <code>--g</code> adds the subset plus <code>inlined_subroutine</code> records to a release build. The neper-format locals/types side table , debug engine and <code>neper dap</code> are the M4 optimisation over that path, never a replacement for it	Rewritten 2026-09-04; the earlier rationale was wrong on two counts. It costed hand-written DWARF against an own format and never costed the debug engine the own format is worthless without — process control, breakpoint patching, stepping, stack walking, location rendering — which is the expensive part and cannot land before M4; the old sequencing left every developer with line tables and no locals through self-hosting and the whole GPU milestone. And it counted editor support as "one adapter" when VS Code and Zed already ship <code>lldb-dap</code> and <code>codelldb</code> over <code>lldb</code> , so the standard format costs zero adapters and an own format costs one plus an engine. A twelve-tag subset with one fixed abbreviation table and frame-relative locations is hundreds of lines, not the "thousands of lines of DIE trees" the old row feared — that cost lives in location lists and expression programs, which are not emitted — and it is read by <code>lldb</code> , <code>gdb</code> , <code>WinDbg</code> and Visual Studio from <code>neper-0</code> . The side table survives as the later optimisation because the <code>.em</code> already serialises the type table. Not inlining in debug builds is what makes every frame, breakpoint and step correspond to the page. (amended 2026-09-04: <code>.nepersym</code> was used by §11 and §13 with no row of its own and sat beside this one as if it were a fourth standard format; the three are now distinguished — standard formats for external tools, <code>.nepersym</code> for the runtime's backtrace (D46), and the side table as the M4 optimisation.)

#	Decision	Chosen	Why
D10	Baseline ISA	x64-v3 (AVX2/FMA/BMI) by default, x64-v4 opt-in, native never the default; aarch64-v8.2 the aarch64 default, v8.0 and v8.6 (native BF16) opt-in; x86-v1 the x86-32 default and its only level	AVX-512 is absent from Alder Lake and later Intel consumer parts, and throttles some earlier Intel server parts. v3 is near-universal on the last decade of hardware and carries the instructions that matter. Standard microarchitecture levels are adopted verbatim rather than inventing feature flags. (amended 2026-09-04: aarch64 had no default level; v8.2 is what Apple Silicon and every recent ARM server have, and v8.0 is 2011-era hardware; x86-32 has one level, so it is the default.) (amended 2026-09-04: §13's table listed aarch64-v8.6 and this row did not.)
D11	Naming	Compiler-enforced casing, the §3 table being canonical: <code>snake_case</code> functions and values (variables, parameters, fields) and modules; <code>PascalCase</code> types, enum members, errors and comptime parameters of kind <code>type</code> or <code>fn</code> (T, Ctx, K); <code>SCREAMING_SNAKE</code> constants and comptime parameters of a value kind (N, FMT, IDX, FIELD); <code>snake_case</code> the comptime binding an unrolled <code>for</code> introduces (§9), which is a local binding and not a parameter	Casing tells you a name's kind on sight, and a <code>grep</code> pattern never has to guess. Enforced by <code>neper fmt --check</code> , not left to a style guide, because naming drift is exactly what generated code produces. (amended 2026-09-04: the table had no row for comptime parameters, so <code>neper fmt --check</code> had no rule for them; a type-kind parameter names a type and takes a type's casing, a value-kind one is a compile-time constant and takes a constant's, and every use in the spec already conforms.) (amended 2026-09-04: the rule was stated three ways — here, in §3 and in §14 invariant 10 — with modules and enum members in only one of them; §3's table is canonical, and this row and invariant 10 name every row of it.) (amended 2026-09-04: the unrolled comptime <code>for</code> of D53 binds a comptime value that no row covered, and the <code>SCREAMING_SNAKE</code> rule would have claimed it; the rule covers comptime <i>parameters</i> only, and <code>f</code> in <code>for f in meta.fields[T]()</code> is <code>snake_case</code> .)
D12	Compiler parallelism	Work-stealing pool, parse phase fully parallel, codegen parallel per function, per-thread arenas, deterministic output	Scaling is architectural, not an afterthought. No preprocessor or macros means every file parses with zero coordination; no global allocator means no allocator contention; work-stealing absorbs hybrid P-core/E-core speed differences that static partitioning cannot.

#	Decision	Chosen	Why
D13	Linking	Own linker and direct executable emission as the default; the M2 easy case is a freestanding ELF and a PE with a fixed <code>kernel32</code> import table; general dynamic imports and the PDB writer (module, symbol and line streams, no type stream) at M4; <code>--linker=system</code> kept permanently for static archives and for a Windows build that wants locals in an external debugger (no type stream is scheduled, so only symbolication and line tables move to the own PDB at M4); until M4 it is also required for any Windows symbolication by external tools	Linking is the largest serial section of a build, so it caps the speedup no matter how many cores are available. Code and relocations are already in memory: skipping object files and the process spawn takes a full relink from 100ms–1s to realistically sub-10ms. Staged by difficulty — static pure-neper first, dynamic imports next, static archives never. (amended 2026-09-04: a pure-neper static executable does I/O only on Linux, so without a <code>kernel32</code> table the M2 own linker served nothing on Windows, one of the two M0 platforms, and "rebuilt in milliseconds" held on Linux only; <code>e.io</code> , <code>e.thread</code> and <code>e.time</code> on Windows are over dynamic imports by construction, and one fixed DLL is a few hundred bytes. The PDB was promised in §13 and absent from the obligations.) (amended 2026-09-04: "until the PDB writer lands" read as if locals would follow; they do not without a type stream, so the system-linker dependency for Windows locals is permanent and stated.)
D14	Threads	OS threads only, no runtime or scheduler; builtin <code>Atomic[T]</code> with explicit orderings, <code>T</code> any integer type of §4 or a pointer on the CPU (32- and 64-bit integers in device code); one arena per thread; explicit context pointer instead of TLS; no compile-time race protection	The fast choice and the simple choice coincide. A scheduler adds latency; platform TLS costs a lookup and hides state; race checking would need an ownership system neper does not have (D3). Races on non-atomic locations are undefined behaviour, as in C. (amended 2026-09-04: <code>Atomic[T]</code> 's legal <code>T</code> was shown by example only; the CPU set is every integer width and pointers because every target has native atomics at each, and the device set is narrower for the same reason.)

#	Decision	Chosen	Why
D15	Testing	Built in: <code>@test</code> attribute, one arena-taking signature, <code>neper test</code>, which takes the same optional <code>FILE.e</code> operand as <code>neper fmt</code> and <code>neper index</code> (§13) and then runs that file's module's <code>@tests</code> and nothing else. Compiler owns discovery, isolation and reporting only; <code>e.test</code> declares <code>error Failed</code> and the assertion helpers — <code>test.assert(cond: bool, msg: str) -> err</code> returns <code>Failed</code> and records <code>msg</code> in the test's JSON, never traps; <code>unreachable()</code> stays for impossible paths. Execution: a synthesised program root per module — a generated <code>main</code> that takes the module name, the start-at test and the control-handle number through <code>args</code>, runs the module's <code>@tests</code> serially in source order from that test and writes one JSON record per test to the control handle, reaching the module under test through a compiler-internal qualifier no source file can spell, exempt from §2's no-alias rule; one child process per module, modules in parallel under <code>-j N</code>, tests within a module serial in source order, a fresh arena per test, <code>stdout/stderr</code> captured per test, a 60-second default per-test timeout (<code>--timeout</code>), a crash or timeout recorded as <code>outcome crashed/timeout</code> with the trap record in that test's JSON object and the module process restarted from the next test; JSON fields <code>name</code>, <code>module</code>, <code>file</code>, <code>line</code>, <code>outcome</code>, <code>error</code>, <code>message</code>, <code>duration_ms</code>, <code>timeout_s</code>, <code>stdout</code>, <code>stderr</code>, <code>trap</code>	A library cannot discover tests — no macros, no reflection, no static initialisers — leaving only a hand-maintained registry whose failure mode is a test that silently never runs. Assertions, fakes and generators stay libraries because they need no discovery. Host-native only; cross-target testing needs an external runner. (amended 2026-09-04: a library assertion needs a primitive that survives release, since every other check is elided — <code>unreachable()</code> is that one builtin, and <code>test.Failed</code> gives the §13 example a declared error to return.) (amended 2026-09-04: every test in one process on the §15 pool meant the first trap killed the run and the JSON the §14 tooling consumes was never written, while module-scope <code>var</code> raced between parallel tests by construction; a process per module isolates a trap at the cost of one spawn per module, attributes the crash to the test that was running, and turns the shared global into an ordered, reproducible hazard instead of a race.) (amended 2026-09-04: the runner was specified over an <code>os.spawn</code> that inherited the standard streams and an <code>e.os</code> with no pipe or kill, so capture, the control stream and the timeout could not be written over the surface the spec named; <code>os.pipe</code>, a <code>Stdio</code> argument to <code>os.spawn</code> and <code>os.kill</code> (D32) are what it uses, and nothing outside <code>e.os</code>.) (amended 2026-09-04: the field list omitted the four identity fields §13's Report table requires.) (amended 2026-09-04: the module process had no entry point, so <code>neper test</code> synthesises a program root per module — the generated <code>main</code> above; and <code>test.assert</code> could not be written over <code>unreachable()</code>, which takes a <code>str</code> literal only and traps, so it is a signature of its own that returns <code>Failed</code> and records its message.) (amended 2026-09-04: the synthesised root had to use the module under test, and for a module named <code>main</code> that qualifier collided with the generated entry point, while §2 as written made a library's <code>fn main</code> a second program root; <code>main</code> is special only in the root module (D41), and generated code reaches the module under test through an internal name that source-level invariants do not govern.) (amended 2026-09-04: <code>neper test</code> took no <code>FILE.e</code> where <code>fmt</code> and <code>index</code> did, so no example's <code>@test</code> could ever run — a file outside every source root is a module only when a command names it (§2) — and it now takes that same operand with that same meaning.)
D16	Attribute placement	Attributes sit on their own line at column 0, above the declaration	<code>@gpu fn saxpy(...)</code> would break invariant 2 — <code>grep "^fn saxpy"</code> would miss exactly the annotated functions. Keeping the keyword first on its own line preserves every §14 guarantee.

#	Decision	Chosen	Why
D17	Tuples	None. (A, B) is a return calling convention, not a type — no storing, no passing, no .0. Two return classes — integer (integers, pointers, enums, err, bool) and vector (floats of every width, Vec[T, N]) — each with a per-target budget in values (x64 2+2, aarch64 2+2, x86-32 2+1), assigned to the class's registers in declaration order; when either budget is exhausted the whole set returns through a caller-provided slot; an aggregate — struct, union, union enum, [N]T, a slice, str — belongs to neither class and is never returned in registers, a set containing one going entirely through the slot	Positional access is unanchored for search and unreadable in generated code; a named struct costs one line and gives every field a name. No layout is visible to the program, so there are no equality or ordering rules to define, and there is exactly one way to spell a product type. gpu.launch takes trailing compile-checked arguments instead of an argument tuple. (amended 2026-09-04: "returns stay in registers" was false for (List[T], err); the convention is registers up to N per target and a caller slot beyond, and the semantic claims stand.) (amended 2026-09-04: three independent budgets gave scalar floats and Vec values the same physical registers, so a mixed set such as (f32, Vec[f32, 4]) had no assignment; two classes with one register sequence each, filled in declaration order, give every set exactly one.) (amended 2026-09-04: an aggregate belonged to neither class, so (List[T], err) had no rule after all; aggregates take the slot, and only the two scalar classes use registers.)
D18	Const qualifier	*const T and []const T; string literals are []const u8; []T → []const T and *T → *const T are the mutability-losing conversions in spec §6's list of implicit operations, which is the one canonical list; the constness of &e, e[lo..hi] and an assignment through e is that of the innermost pointer or slice step on the path to the place, and of the binding (var mutable; let, parameter, const immutable) only when there is no such step; read-only library slice parameters are []const T; []const T is shallow	Without it let s = "abc"; s[0] = 'x' compiles and segfaults — a runtime fault where a compile error belongs, in a language selling debuggability. Qualifies the pointee only, cannot be cast away, exists in two positions and nowhere else. (amended 2026-09-04: with no rule for &x and x[lo..hi] on a let place the guarantee was undelivered — let p = &v; p.x = 1.0 compiled; the const forms close it, and []const T library parameters let a literal or a let array fit without a cast.) (amended 2026-09-04: "the one implicit conversion" disagreed with §4's two and §6's two, and §10's Buf[T] mapping was a fourth nobody listed; §6 now holds the single list of four and every other section points at it.) (amended 2026-09-04: "on a let, a parameter or a const path" matched a pointer-typed parameter and its pointee's step with opposite results, so list.e's l.items[0..l.len] had two types; the innermost step decides, and a parameter of pointer or slice type contributes its own type as that step.) (amended 2026-09-04: "two positions and nowhere else" is stated as the pointee of a pointer or slice type, which includes §10's address-space-qualified device forms *const shared T and []const shared T.)

#	Decision	Chosen	Why
D19	Argument packs	Trailing ... parameter, comptime-only, monomorphised per shape; in v1 it appears only in the intrinsics <code>io.printf</code> , <code>str.format</code> and <code>gpu.launch</code> — not declarable as a pack in user or library code (the ... on an extern, D32, is a C variadic under a separate rule), no pack API; <code>gpu.launch[K: fn]</code> takes its kernel as a comptime parameter of kind <code>fn</code> , never as a runtime argument	Formatting and kernel launch both need variadics; one token serves the three, with no <code>va_list</code> , no runtime variadics, no boxing. (amended 2026-09-04: the "one general mechanism" claim was wrong — with no length, indexing or type dispatch a pack is usable only by the compiler, so packs hide <code>gpu.launch</code> 's exception behind a token rather than removing it; recorded as three intrinsics, which is one paragraph and costs no interpreter feature.) (amended 2026-09-04: <code>launch</code> took <code>k: kernel</code> at runtime, a type defined nowhere, so the compile-time check of the pack it promised had no mechanism — the kernel is now a comptime <code>K: fn</code> , which is what <code>printf[FMT]</code> and <code>thread.spawn[Ctx]</code> already do.)
D20	Equality	<code>==</code> on primitives, pointers and enums only; slices, structs, unions and arrays use explicit functions	<code>==</code> on a slice would compare pointer and length rather than contents — a silent wrong answer, which is the worst kind.
D21	Time	<code>e.time</code> is UTC-only and allocation-free; <code>Timestamp/Instant/Duration/Date/Time</code> are distinct types; zones are an explicit minute offset; leap seconds ignored; every comparator and arithmetic function over a clock type is spelled <code><t>_<op></code> — <code>timestamp_cmp</code> , <code>instant_cmp</code> , <code>duration_cmp</code> , <code>timestamp_add</code> , <code>instant_add</code> , <code>duration_add</code> , <code>timestamp_diff</code> , <code>instant_diff</code> , <code>duration_sub</code> , <code>duration_neg</code> , <code>duration_scale</code> — so each type satisfies §9's protocols	IANA tzdata is megabytes plus file I/O plus a parser, none of which fits a freestanding core that allocates nothing. Distinct types make "measured elapsed time with the wall clock" a compile error rather than a production bug, and cost nothing given no implicit conversions. A real tz database is a separate library. (amended 2026-09-04: spelled <code>cmp</code> , <code>cmp_instant</code> , <code>cmp_duration</code> , no <code>e.time</code> type had a protocol <code>cmp</code> and none could be sorted; under D52's <code><t>_<op></code> the suffixes are the protocol convention rather than a workaround for having no overloading.)

#	Decision	Chosen	Why
D22	Device memory	<code>gpu.Buf[T]</code> is a distinct host type whose <code>T</code> is a device type (plain data: no pointers, slices or <code>bool</code>; <code>err</code> is a <code>u32</code> and is one), mapped onto a kernel's <code>[]T</code>, <code>[]const T</code> or <code>[]Atomic[T]</code> parameter by <code>gpu.launch</code>; the SPIR-V floor is Vulkan 1.2 with <code>bufferDeviceAddress</code> and <code>scalarBlockLayout</code>, so a device slice is §4's { 64-bit address, len } header under <code>PhysicalStorageBuffer64</code> with the CPU's sub-slicing and the <code>len</code> at the device's <code>usize</code> width; <code>usize</code> is 32 bits on <code>spv</code> and, with <code>isize</code>, is therefore not a device type — legal as a kernel local, never in a <code>Buf[T]</code> element or a kernel parameter; address spaces are part of the type — <code>[]T</code> device, <code>[]shared T</code> workgroup, no conversion — and private memory has no pointer or slice type in device code	A host pointer is meaningless on the device and the reverse. Making them different types means the compiler enforces the boundary instead of a naming convention. No unified address space in v1. (amended 2026-09-04: a <code>[]T</code> had no device representation at all — under logical addressing a pointer carries a storage class, cannot be re-based and cannot be stored, so <code>x[k..]</code> and one helper over a buffer and a shared array were inexpressible; the 1.2 floor makes the slice one type on both profiles at the cost of excluding Vulkan 1.1 devices, POD-only elements under scalar layout make an upload a memcpy, and a 32-bit <code>usize</code> keeps indexing off the optional <code>Int64</code>.) (amended 2026-09-04: a <code>usize</code> field in a buffer would have had two layouts, one per side, and "an upload is a memcpy" was false for it; excluding the two target-width integers from device types keeps the layout rule true, and <code>launch</code> writes each slice's <code>len</code> at the device's width.) (amended 2026-09-04: on <code>.Cpu</code> there is no driver, so <code>gpu.alloc</code> and the staging block had no memory source — a hidden allocation D3 forbids; the CPU backend takes every <code>Buf[T]</code> from the arena passed to <code>gpu.open</code>, exhaustion being that arena's <code>mem.Exhausted</code>, while the driver backends use device memory and that arena for the bookkeeping block alone.) (amended 2026-09-04: whether an <code>err</code> could exist in device code was unstated; it is an ordinary <code>u32</code>-backed value there — helpers return <code>(T, err)</code> and try between them, only a kernel entry point returns nothing — and as a <code>u32</code> in every layout it is a device type, so a kernel can report failure by writing one into a <code>Buf</code>.) (amended 2026-09-04: the previous clause also gave <code>.Cpu</code> a staging block, which §10 elsewhere called a plain memcpy; on <code>.Cpu</code> there is no staging — upload and write copy straight into the <code>Buf</code>'s storage, allocated once at <code>alloc/upload</code> — and staging exists on the driver backends only.)

#	Decision	Chosen	Why
D23	GPU async model	<code>gpu.Queue</code> from <code>gpu.queue(dev)</code>, and every call that submits work — <code>alloc</code>, <code>upload</code>, <code>write</code>, <code>launch</code>, <code>download</code>, <code>sync</code>, <code>release</code> — takes a <code>*Queue</code>, while <code>open</code>, <code>close</code>, <code>has</code>, <code>queue</code>, <code>len</code> and <code>grid*</code> take none; a queue is an in-order stream used by one thread at a time, a Device is thread-safe because <code>open</code> takes one bookkeeping block from its arena and guards that block with its own lock — the one locked arena memory in <code>lib/e</code>; <code>launch</code> queues and returns; <code>upload/write</code> copy the source into driver-owned staging synchronously, then queue the transfer — the source is free on return, the cost is one memcpy; <code>download</code> synchronises that queue, then copies; <code>gpu.sync(q)</code> waits for that queue only; <code>release</code> is queued behind prior submissions, so <code>defer gpu.release</code> is safe by construction; queues are unordered with respect to each other and a buffer shared across them without a <code>sync</code> is a race	Overlap between host and device is the reason to have a GPU. <code>download</code> syncs implicitly because a stale read is a silent wrong answer, the failure mode this language works hardest to eliminate. (amended 2026-09-04: no <code>Queue</code> type existed, so <code>sync</code> and <code>download</code> could only drain the whole device and stall the independent streams the same paragraph promised; an asynchronous <code>upload</code> from a host slice with no lifetime rule corrupted the double-buffer pattern silently, and the synchronous staging copy costs one memcpy where a lifetime rule costs a check the compiler cannot make; <code>release</code> appeared in every example with no ordering.) (amended 2026-09-04: "usable from any thread" while allocating from an arena D14 declares not thread-safe was a race by construction; a single block taken at <code>open</code> and a lock inside the Device resolve it without making arenas thread-safe.) (amended 2026-09-04: "every host call takes a <code>*Queue</code>" was false for the six device-level calls, and §10's thread rule listed <code>alloc</code> among calls made on a <code>*Device</code>; <code>alloc</code> is a queue call and the concurrent set is <code>queue</code>, <code>has</code>, <code>close</code>.)
D24	Device-callable inference	<code>@gpu</code> marks kernel entry points; a plain <code>fn</code> reached from a kernel is compiled for the device too, and must satisfy the GPU restrictions; the device copy is emitted into the kernel-owning module's <code>.em</code> with module-local linkage and recorded as a body-hash edge; a helper that uses any <code>gpu.* builtin</code> or names a shared type is device-only, and a helper over both address spaces is a generic over the slice type, monomorphised per space	Otherwise <code>@gpu</code> spreads down through every leaf helper. Errors are reported at the kernel with the call chain reaching the violation. (amended 2026-09-04: inference compiled another module's body into the kernel's module with no invalidation rule and no stated owner; D36 supplies the rule, and module-local emission means no ownership protocol between modules.) (amended 2026-09-04: address spaces are part of the slice type, so a helper reached with <code>[]shared T</code> is one more instance in the kernel-owning <code>.em</code>, placed by the same rule.) (amended 2026-09-04: the device-only rule named three builtins where D37 and §10 say any <code>gpu.* builtin</code> — <code>gpu.gid</code>, the barriers included; aligned.)
D25	Narrow floats	<code>f16</code> (binary16) and <code>bf16</code> are primitive types; arithmetic is "compute in <code>f32</code>, round after every op"; native instructions used where the target has them	Storage bandwidth is the reason they exist and any GPU or AI-adjacent workload needs them. The compute-then-round rule is bit-identical to native half hardware for <code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>sqrt</code>, so results are deterministic across targets whether or not the CPU has FP16/BF16 instructions.
D26	Apple GPUs	MoltenVK is the supported-but-second-tier path now; a native Metal backend is scheduled as M5; GPU capabilities (<code>f64</code>, <code>f16</code>, subgroups, 64-bit atomics) are checked per device at <code>gpu.launch</code>	Apple ships no Vulkan. A GPU backend is the most expensive kind of target — emitter plus runtime — and Apple is one vendor, so it follows the core targets rather than delaying them. Capability checks must be at launch because the compiler cannot know the device.

#	Decision	Chosen	Why
D27	Literal and index typing	An unsuffixed literal is an untyped comptime value typed by its immediate context; with no context it is a compile error, never <code>i32</code> ; type suffixes <code>42u64</code> , <code>1.5f32</code> ; indices and <code>.len</code> are <code>usize</code> ; <code>a..b</code> has its bounds' type; GPU ids are <code>u32</code> ; comptime inference is unambiguous only when every non-literal argument whose parameter type mentions the parameter agrees on one type, and the literals are then typed by the parameter	With no implicit conversions, the type of every literal must be legible on its line, because under §11 it decides where arithmetic wraps. A default of <code>i32</code> would have <code>atomic.add(&c.hits, 1, ...)</code> and <code>max(3, 9)</code> disagree on what a bare literal is. Go's untyped constants and Zig's <code>comptime_int</code> show the rule costs nothing; the error-without-context is the price of never guessing on a model's behalf. (amended 2026-09-04: the row dropped "whose parameter type mentions the parameter", which let an unrelated argument appear to vote; aligned with §9.)
D28	Local initialisers and zero values	Every local requires an initialiser; <code>= zero</code> is a visible memset, legal on an annotated <code>let</code> or <code>var</code> , and <code>= undef</code> is no store (<code>0xCD</code> fill in debug), legal on an annotated <code>var</code> only and banned inside <code>@gpu</code> ; <code>try</code> returns the zero value in every non-err slot; the zero value is defined for every type, and an enum with no member at <code>0</code> has none	An uninitialised local is either a hidden memset or an unchecked read, against Goals 2 and 6 at once. Two spellings put the cost on the page, and a definite value beside a failed <code>try</code> means two conforming compilers produce the same binary instead of two different ones. (amended 2026-09-04: §5 said both spellings were <code>var</code> -only and then banned only <code>undef</code> on <code>let</code> ; <code>let x: T = zero</code> is a definite constant and is legal.)
D29	Tagged unions	<code>union enum u8 { A: X, B }</code> beside bare <code>union</code> ; payload access is tag-checked in debug; <code>case .A as x</code> binds the payload; <code>switch</code> on an enum or tagged union is exhaustive unless <code>default</code> is present; the optional is <code>union enum u8 { Some: T, None }</code> with no keyword	A compiler is mostly variant types, and a wrong-member read on a bare union is a silent wrong answer with no check in §11. The tag-then-payload layout is byte-identical to the hand-rolled <code>struct { kind, u }</code> , so placing the tag hides no cost — the earlier "will not hide the cost" rationale was wrong. Exhaustive <code>switch</code> turns an added variant into a compile error at every stale switch. (amended 2026-09-04: the byte-identity holds on the CPU; in device code the payloads are laid out without overlap, since SPIR-V cannot alias typed members over one storage, and the program cannot observe which — so neither union form is a <code>Buf[T]</code> element type. Bare union is kept as the C-ABI escape hatch, not as the punning form: <code>mem.bitcast</code> puns without one (D49).)
D30	By-value argument aliasing	The storage of a by-value argument is written by no one for the duration of the call; doing so is undefined behaviour	Mirrors D14. It makes hidden-reference passing a pure ABI choice: the compiler may pass the caller's storage by address with no copy and no alias analysis, which the -O1 optimiser does not have. Without it either every 64-byte matrix is a <code>memcpy</code> per call or a write through another pointer changes a by-value parameter.
D31	Conditional compilation and reserved names	<code>target</code> is a builtin comptime namespace (<code>target.arch: target.Arch</code> , <code>target.os: target.Os</code> , PascalCase members) needing no use; when is statement-level only; target-specific module-scope declarations live in per-target files <code><name>.<arch os>.e</code> ; a use qualifier cannot be shadowed by any declaration; <code>_</code> is reserved; <code>@cpu</code> is removed until runtime dispatch lands, then spelled <code>@cpu("x64-v4")</code>	when around a declaration would break §14's column-0 and order-independence invariants, while a per-target file is found by glob like every other module. Reserving the qualifier keeps <code>mem.alloc</code> one thing in every scope, which is the reason qualified references exist; a function-pointer field named <code>mem</code> would otherwise make a call site ambiguous.

#	Decision	Chosen	Why
D32	Foreign functions	Column-0 <code>extern fn</code> with <code>@import(LIB, SYM)</code> and <code>@cc(c sysv win64 stdcall)</code> attribute lines — <code>sysv</code> and <code>win64</code> compile errors on any target but <code>x64</code> — they exist only to force a non-native convention there, <code>c</code> being the native convention everywhere — and the convention part of an <code>extern fn</code> pointer type, so two conventions are two distinct types, a pointer type naming its convention by a <code>@cc(CONV)</code> line above type <code>Cb = extern fn(...) -> R</code> and an <code>extern fn</code> type without one being <code>c</code>; a closed C-ABI mapping — primitives, pointers, <code>*void</code>, enums, by-value structs and unions cross; slices, <code>err</code>, multiple returns, tagged unions, vectors and arrays by value do not; ... on an <code>extern</code> is a C variadic with no default promotions; no <code>err/try</code> on an <code>extern</code>; <code>@cc</code> on a plain <code>fn</code> makes it C-callable, a generic <code>@cc fn</code> instantiating one C-callable function per instance, under the same signature rule as an <code>extern</code>; a pointer to a <code>Vec[T, N]</code> or an <code>Atomic[T]</code> crosses as <code>T*</code> although the value does not; banned inside <code>@gpu</code> and at compile time; <code>e.os</code> is the sole OS surface of <code>lib/e</code>, written per target over raw syscalls (Linux, via the <code>os.syscall</code> and <code>os.thread_start</code> intrinsics), <code>kernel32</code> (Windows) and <code>libSystem</code> (macOS), and includes <code>os.pipe</code>, an <code>os.spawn</code> that takes the child's three standard streams, and <code>os.kill</code>	The language could not name an external symbol, a library or a calling convention, so <code>e.io</code>, the thread pool, the root arena, <code>--linker=system's process spawn</code> and the Vulkan runtime could not be written in it; a fixed intrinsic set gets <code>hello.e</code> running and cannot scale to a driver's hundreds of entry points. The foreign name is a string so the neper-side name keeps D11; a separate keyword keeps §14's column-0 set exact. The mapping is closed so "does this type cross" is answered at the declaration, and <code>err</code> stays out because C has no such value — the <code>e.os</code> wrapper is where a platform code becomes one. Raw syscalls on Linux keep the freestanding executable D13 stages first; <code>libSystem</code> on macOS is the lesson Go learned in 1.12. (amended 2026-09-04: the pointer row and the <code>Vec/Atomic</code> rows of the ABI table disagreed, the Linux stack switch had no name a source file could call, <code>thread_create[Ctx]</code> on Windows needed a trampoline the text could not express, and <code>neper test</code> (D15) needed pipes and a kill the surface lacked.) (amended 2026-09-04: the three standard streams could not carry the test runner's control pipe, so <code>Stdio</code> also carries <code>inherit</code>, extra handles the child inherits, their numbers passed to it through <code>argv</code>.) (amended 2026-09-04: §9 said no generic could be taken as a value, which the <code>per-Ctx</code> trampoline needs; an explicitly bracketed instance <code>f[Ctx]</code> is a value — a pointer to that one instance — and the inferred form still is not. An inherited handle keeps its raw value in the child on both platforms, so that is the number <code>argv</code> carries; <code>os.syscall</code> and <code>os.thread_start</code> are declared in <code>os.linux.e</code> alone.) (amended 2026-09-04: what <code>@cc(sysv)/@cc(win64)</code> meant on a non-<code>x64</code> target, and whether the convention was part of the pointer type, were unstated; a convention the target cannot honour is an error at the declaration, and a pointer type that dropped it could be called under the wrong one.) (amended 2026-09-04: <code>@cc</code> had no type-level syntax, so no pointer type could name its convention; it is an attribute line on the type declaration. The <code>kernel32</code> example carried <code>@cc(win64)</code>, an error on <code>aarch64</code> — it is <code>@cc(c)</code>. And <code>os.syscall/os.thread_start</code> are intrinsics: they appear in no <code>.e</code> file, <code>os.linux.e</code> included, and are compiler-known on Linux alone.)

#	Decision	Chosen	Why
D33	Trap protocol and checked operations	A failed check writes <code>file:line:col: trap[kind]: <values></code> plus a symbolised backtrace to <code>stderr</code> and exits 134; under <code>neper test</code> the same record lands in the test's JSON; <code>unreachable()</code> is the one always-on builtin (a keyword, no use); integer narrowing casts trap in debug and truncate in release, <code>T.trunc(x)</code> truncates by intent; a float-to-integer cast rounds toward zero and, out of range or NaN, traps in debug; in release an out-of-range value saturates and NaN yields 0, on every target; a cast is not a typing context for an untyped literal; shifts by <code>>= width trap / mask</code> , the count being independent of the left operand — any unsigned integer type, a literal typed <code>u32</code> ; out-of-range enum casts trap / unchecked; <code>/ %</code> by zero and <code>MIN / -1</code> trap in every mode	"Illegal instruction" at a <code>ud2</code> is a debugger session where one line would do, and the line tables, symbols and unwind info are emitted in every build already, so the record and the backtrace are nearly free. The record has a compiler diagnostic's shape so §14 tooling parses one format. Every cast and shift must have a stated result in both modes or "behaves identically until a check fires" is unfalsifiable; the release results are the C results because the hardware gives them for free. 134 is <code>128 + SIGABRT</code> , what every crash pipeline already classifies as an abort. (amended 2026-09-04: float-to-integer had no rule at all, and the two obvious hardware results differ — <code>x64</code> gives the integer-indefinite value, <code>aarch64</code> saturates — so a target-independent release result had to be chosen; saturation is <code>aarch64</code> 's for free and a few instructions on <code>x64</code> , and it is what <code>simd.convert</code> already promised lane-wise. The shift count's type was unstated and is load-bearing under "no implicit conversions".) (amended 2026-09-04: a literal count typed <code>u8</code> contradicted §3's rule that a literal takes the other operand's type when the left operand is signed; the count is independent of the left operand, must be unsigned, and a literal is <code>u32</code> .) (amended 2026-09-04: the row said NaN saturates where §4 and §11 give 0; aligned to the spec.)
D34	Program entry	<code>fn main(a: *mem.Arena, args: []str) -> err</code> in the root module, no variants; ok exits 0; any other value writes error: <code><qualified name></code> to <code>stderr</code> and exits 1; a trap exits 134; the root arena is the one passed in, sized by a link option; <code>args</code> live in it	Mirrors <code>@test</code> : one signature, and a different one is a compile error rather than a convention. The root arena has to reach the program through some door, and a parameter is greppable where a global or a <code>mem.root()</code> call is not; the examples were carving stack buffers because no door existed. The failure line reuses the error table D6 already pays for, so no reflection primitive is needed and the roadmap's exclusion of runtime reflection stands.
D35	Arena fills and builder precondition	Debug only: <code>mem.reset</code> fills <code>[m, off)</code> with <code>0xDD</code> , <code>mem.alloc</code> fills fresh memory with <code>0xCD</code> ; <code>str.push</code> checks in every mode that the builder still owns the arena top and returns <code>str.NotOnTop</code> ; a growable container's contract states that slices do not survive a push	The one lifetime hazard arenas create is a slice outliving <code>reset</code> , and a range <code>memset</code> needs no per-object bookkeeping, so D3 is untouched; two distinct bytes tell "freed" from "never written" in a debugger. The builder's in-place growth is a precondition the compiler cannot prove, and one compare turns a silent overwrite into an <code>err</code> . The push-abandons-buffer hazard cannot be poisoned — the old memory stays valid — so it is a documented contract, as in Zig's arena-backed lists.

#	Decision	Chosen	Why
D36	Incremental rebuild invalidation	Per-declaration signature and body hashes; Deps holds (module, symbol, hash) edges — signature edges for references, body edges for anything inlined, instantiated, comptime-executed or device-compiled, and negative edges (module, name, absent) for every protocol name (§9) looked up and not found, every compiler-supplied fallback included; a module recompiles when any edge changes, a negative edge changing when the name is later declared; cross-module inlining capped at 40 NIR instructions per callee; monomorphised instances and device helpers are emitted into the instantiating or kernel-owning .em with module-local linkage and folded by content hash in the own linker; an incremental build's output must equal a clean build's, checked by the M2 harness	A per-module interface hash with "body edits never disturb dependents" shipped stale code the moment a body crossed a .em boundary — and four mechanisms make bodies cross: inlining, monomorphisation, the comptime interpreter and D24. Recording exactly which bodies were compiled in is the only rule that is both correct and cheap: a signature edge is what a call needs, a body edge is what an expansion needs, and the cap keeps the second kind to leaf helpers so ordinary edits stay local. Delphi's DCUs learned the same lesson with inline bodies. Module-local instances need no cross-module ownership, which keeps compilation parallel; folding by content hash is free given §15's determinism. (amended 2026-09-04: edges recorded only symbols that exist, so declaring a fn sensor_hash after an instantiation had fallen back to the compiler-supplied hash invalidated nothing and the fallback shipped for ever; a negative edge records the absence, and declaring the name changes it.)
D37	Kernel geometry, shared memory and device synchronisation	@gpu(X, Y, Z) carries the workgroup size and bare @gpu is a compile error; gpu.launch[K] is a kernel's only caller on both profiles — a direct call to an @gpu function is a compile error; a helper using any gpu.* builtin or a shared type is device-only; gpu.subgroup_ballot adds .Int64 to the inferred set; gpu.grid1/2/3 count invocations and launch rounds up to workgroups, returning gpu.TooLarge/gpu.Unsupported when a device limit is exceeded; shared is a keyword and shared var name: T an uninitialised statement legal only directly in a kernel body; helpers receive it as []shared T; gpu.barrier() is a Workgroup control barrier with AcquireRelease over workgroup memory and must be in uniform control flow, gpu.memory_barrier(.Workgroup .Device) orders device memory; device atomics are gpu.atomic_* with a scope, on Buf[Atomic[T]], []Atomic[T] and shared Atomic[T]; a minimal subgroup set (size, sid, reductions, all/any/ballot, broadcast/shuffle, elect); the capability set is inferred per kernel into its .em Interface entry and checked at launch, caps(...) in the attribute is a compile-time upper bound; a type-to-capability table; Vec[T, N > 4] splits into four-lane vectors on SPIR-V	SPIR-V needs LocalSize at compile time and PTX the block size at launch, and lid, shared and barrier mean nothing without it; an invocation count is what a caller holds (x.len), so the division belongs in launch and the guard stays in the kernel where it is visible. A statement keyword keeps §14.2's column-0 set exact and the module-scope var ban intact; no initialiser because no single invocation could run one. A barrier with no stated scope and memory semantics cannot be emitted, and scoped atomics are the device's model — hiding the scope would make .Device a silent default. gpu.atomic_* rather than a second atomic.* signature keeps one signature per name (§14). Inference keeps the capability set honest; the upper bound exists because a helper edit could otherwise add a desktop-only capability to a mobile kernel with no diagnostic until launch. (amended 2026-09-04: "on the CPU it is also an ordinary function" gave gpu.gid, a shared var and gpu.barrier() no meaning in a direct call — there is no grid — so the call is an error rather than a hidden behaviour; the ballot's u64 silently needed Int64 and now says so.)

#	Decision	Chosen	Why
D38	CPU execution model for kernels	The CPU backend (<code>gpu.open(a, .Cpu, 0)</code>) runs a launch on the calling thread, one workgroup after another; a workgroup runs on one host thread by barrier loop-fission — the body cut at every barrier and subgroup builtin into regions, each looped over the invocations in <code>lid</code> order, one slot per invocation for a local that crosses a barrier, one shared storage per workgroup filled <code>0xCD</code> in debug; a breakpoint stops at one invocation's iteration with the workgroup's state visible; a debug check traps with kind <code>barrier</code> when invocations reach different barriers or return before one; subgroups are 32 consecutive invocations, the last of a workgroup partial when the size is not a multiple of 32, with inactive lanes reading <code>false</code> and contributing nothing; address-space qualifiers are erased	The CPU build is the only debugger and the only test vehicle for kernels, so it must have the device's semantics rather than a serial approximation in which a missing barrier or a shared-memory race disappears — CUDA dropped its emulation mode because it diverged from the device. One OS thread per invocation with a host barrier is the obvious alternative and is hundreds of context switches per barrier, unsteppable in practice; loop-fission (pocl, Intel's CPU OpenCL) reproduces the barrier's exact ordering on one thread with no runtime, and makes "step one work item" a plain loop iteration. The divergence check is the one thing the CPU can do that the device cannot, and it catches the bug class that hangs a GPU. (amended 2026-09-04: a workgroup size such as <code>@gpu(100)</code> left the last subgroup's size and ballot bits unspecified; the partial-subgroup rule mirrors the device.)
D39	Floating point	No implicit contraction in any build mode on any target; <code>math.fma</code> is the only FMA; every SPIR-V result carries <code>NoContraction</code> , with correctly rounded division and square-root sequences; PTX uses the <code>.rn</code> forms and never <code>.approx</code> ; <code>min/max</code> are IEEE minimum/maximum; denormals preserved by default, with <code>@gpu(..., ftz)</code> opting a kernel into flush-to-zero on device and CPU build alike; the approximate builtins — the <code>e.math</code> transcendentals, <code>math.rsqrt</code> , float subgroup reductions — are listed in §11 and are the only permitted divergence	Every GPU compiler fuses <code>a*b + c</code> by default and the default CPU level carries FMA, so without a rule the CPU build and the device disagreed in the last bit with no check firing — the silent wrong answer D23 names, inside the debugger itself. Exactness by default costs one rounding per operation, which is what the source said; a generator that wants the fused result writes <code>math.fma</code> and the reviewer sees it. Vulkan bounds division to 2.5 ULP, so bit-identity needs a software sequence there; it is a handful of instructions and the only way M3's criterion is testable. Flush-to-zero is a per-kernel opt-in because it is the one IEEE deviation with a real GPU cost, and it is applied to the CPU build too so the two still agree.

#	Decision	Chosen	Why
D40	SIMD	The vector type is the builtin generic <code>Vec[T, N]</code> — <code>T</code> an integer or float primitive, width <code>N * mem.size_of[T]()</code> of 16, 32 or 64 bytes, a closed table of 36 pairs — with <code>Mask[T, N]</code> as its mask type and no <code>f32x8</code>-style aliases; a <code>simd</code> module of comptime-generic intrinsics: <code>splat</code>, <code>load/store</code> (unaligned, aligned with an <code>align</code> check, masked), <code>shuffle</code> with a comptime index array, <code>cmp_*</code> to masks, <code>select</code>, <code>any/all/bits</code> reading a mask, <code>reduce_add/min/max</code> in a fixed pairwise tree, <code>convert</code>, <code>gather</code>, <code>fma</code>, <code>pdep/pext</code>; integer lanes take only the wrapping operators; vectors are a register class passed in vector registers at any width, exempt from the hidden-reference rule, and returned in them up to the per-target budget (two on x64 and aarch64, one on x86-32), through the caller slot beyond it; <code>N > 4</code> splits into four-lane vectors on SPIR-V, and <code>Mask[T, N]</code> is a register-only type on both profiles — a local, a parameter, a return value (passed and returned as the <code>Vec[T, N]</code> it masks, in the vector class under §5's budget) or an intrinsic operand, never a field, an element, a module-scope var, a <code>Buf[T]</code> element or a pointee, <code>mem.size_of</code> on it a compile error, <code>= zero</code> on a local all lanes <code>false</code>, <code>simd.bits/simd.mask</code> converting to and from an integer bitmask for storage — and never a device type; in device code the slice parameter of the <code>load/store/gather</code> intrinsics is generic over the address space; <code>simd</code> ships in M1	A vector type with <code>splat</code> and elementwise arithmetic and no load, store, mask, shuffle, compare, reduction or FMA could not deliver Goal 3, and an open-ended type list meant a generator could not know whether <code>i16x8</code> was legal. <code>Vec[T, N]</code> is the D5 form every other parameterised builtin uses, so a suffixed family with suffixed intrinsics (<code>splat_f32x8</code>) was the one naming exception in the language. Wrapping-only integer lanes tell the truth about hardware with no overflow flag, instead of letting <code>+ mean +%</code> in one type; the fixed reduction tree and lane-wise <code>fma</code> keep vector code under D39's bit-identity rule; the register class is what every ABI does with vectors and what the hidden-reference rule would have got wrong. (amended 2026-09-04: §5's two-word vector budget sent a <code>Vec[u8, 64]</code> to the slot that §4 said it never entered; <code>Mask</code> had no SPIR-V form and the loads took only device-space slices, so no vector could be loaded from shared memory.) (amended 2026-09-04: "never entering the return slot" still contradicted §5's budget; §5 is right — vectors return in vector registers up to the per-target budget of two and through the caller slot beyond it, like any aggregate.) (amended 2026-09-04: "device type" is §10's term for what a <code>Buf[T]</code> element or kernel parameter may be, and <code>Mask</code> is not one; the budget is §5's per target, not a universal two; <code>any/all/bits</code> were defined in prose and missing from the intrinsic table.) (amended 2026-09-04: register-only was stated for device code alone, which left a <code>Mask</code> field or array on the CPU with a size that varied by level — a <code>k</code> register at x64-v4, a full vector below it — visible through <code>mem.size_of</code>; a type whose representation the program cannot see cannot be stored, so it is register-only everywhere, and the bitmask conversion is its storage form.) (amended 2026-09-04: a <code>Mask</code> parameter or return value had no convention — it was in neither §5 return class, has no size for the hidden-reference rule and cannot be a pointee for the slot; it travels as the same-width all-ones/all-zeros vector every level can produce, which is the <code>Vec[T, N]</code> convention.)

#	Decision	Chosen	Why
D41	Source roots and module names	<code>lib/</code> and <code>src/</code> are the two source roots; a module's name is its path below the root with <code>/</code> as <code>.</code> (<code>lib/e/mem.e</code> $\rightarrow$ <code>e.mem</code>, <code>src/main.e</code> $\rightarrow$ <code>main</code>); a per-target suffix is not part of the name; a file declaring <code>fn main</code> is a program root when it is the file named to the command, and <code>main</code> is an ordinary function in every other module; <code>neper run <file.e></code> takes one, and <code>neper build <file.e></code> takes one or any other <code>.e</code> file, compiling a file without <code>main</code> to <code>.em</code> files and no executable; the named file is a module — named from its root when it lies under one, by its bare filename otherwise, so <code>examples/hello.e</code> is <code>hello</code> and runs from any directory; the project root is the nearest ancestor of the named file (of the working directory, for <code>fmt/test/index</code>) holding <code>lib/</code> or <code>src/</code>, else that directory itself; the toolchain's own <code>lib/</code> beside the binary is a third source root on every command, consulted for a module name the project's roots do not define, which is where <code>e.*</code> lives; <code>neper test</code> runs every module under the project's roots; a backtrace frame is <code>module.function</code>	§2's one example (<code>lib/e/mem.e</code> $\rightarrow$ <code>e.mem</code>) already dropped a segment by an unstated rule, the trap example mapped <code>src/lex.e</code> to <code>e.lex</code>, the root module had no name, and <code>neper build</code> took no file. Two fixed roots keep "path is module name" true and a glob sufficient; a configurable root would put the name in a config file, which §2 exists to avoid. (amended 2026-09-04: under the two-root rule the shipped examples were not modules, so M0's <code>neper run examples/hello.e</code> named a file the spec could not compile; a program root is named from its own directory, which makes <code>examples/hello.e</code> module <code>hello</code> without <code>examples/</code> being a package.) (amended 2026-09-04: the project root was defined only for a program root and only for naming, so use <code>e.mem</code> in <code>hello.e</code> had no directory to resolve from, <code>neper test</code> no scope, <code>lib/e</code> no way to be built, and <code>vec3.e/list.e</code> — no <code>main</code> — no command that compiled them; the toolchain's <code>lib/</code> as a third root with the project's roots taking precedence answers all four without a config file, and a non-root file given to <code>neper build</code> is a module like any other.) (amended 2026-09-04: "a file that declares <code>fn main</code> is a program root" made <code>main</code> special in every module, so a library could not declare one and a module named <code>main</code> could not be tested; <code>main</code> is special only in the root module, and the test root reaches the module under test through a compiler-internal name exempt from the no-alias rule, generated code not being subject to source-level invariants (D15).)

#	Decision	Chosen	Why
D42	Operator, iteration and naming rules the text had left implicit	Integer / truncates toward zero and % takes the dividend's sign; >> is arithmetic on signed and logical on unsigned; unary - is an operator and -MIN an overflow check; enum members may carry explicit values, the unassigned ones counting on from the previous; for v in s binds a let copy and writes go through s[i]; a range with b <= a is empty; an unused binding or parameter is not an error; a bracket is a comptime argument list after a name resolved to a function or type and an index after a value, decided by the symbol table; a name before . resolves as a use qualifier, then a type, then a comptime type parameter or a comptime type value, then a value, and the builtin type names (Vec, Mask, Atomic, the primitives) are reserved; a newline is whitespace inside (, [, a literal's { } and a type body's { } only, never inside a block's; defer runs at the exit of its enclosing block — by break, continue or ret included — in reverse order, a loop body's once per iteration, and a trap or os.exit runs none; every returned value is bound or discarded with _ (let _ = f()), so a call returning anything cannot stand as a statement; a local or parameter is declared once per function and shadows nothing — no earlier local, no module-scope name of its own module, no qualifier; a struct literal names every field exactly once, an array or Vec literal supplies exactly N elements, a union literal names one member; % is integer-only; neper fmt writes a payload-less union enum member as .Name wherever context types it	Each is a rule a generator needs on an ordinary line and a reader needs to check one — the sign of %, whether v aliases the element, whether xs[i] after a const named xs is an index — and each had either no statement or two. The C results for /, % and >> are what the hardware gives and what every reader expects; a copy in for keeps "what is on the page is what runs" (a hidden reference would make v.x = 1.0 write the array); an unused binding as an error would reject main's args in every example. The newline rule as first written made every statement in a block whitespace-separated, which no lexer could implement. (amended 2026-09-04: the rule as rewritten made every multi-line type X = struct { a syntax error, the spec's own declarations and both record examples included — a type body's brace is a third case beside a literal's and a block's. defer's scope, the single-value discard, redeclaration, literal completeness and float % each had no statement; block-scoped defer is what "scope exit" already said and what defer inside a loop needs, an explicit let _ keeps an ignored err on the page, no-shadowing keeps one name one meaning per function as §14 keeps it per module, a complete literal is the counterpart of no hidden memset, and % on floats is fmod, which nothing defined.) (amended 2026-09-04: the resolution order named three sets where §2 has four — a comptime type parameter or a comptime type value, whose protocol function is looked up at the instantiation (§9, D52), sits between a type and a value.)

#	Decision	Chosen	Why
D43	Library surfaces fixed by the spec	e.mem: error Exhausted, copy[T] returning nothing, eq[T], cast[P], bitcast[T] (D49), type Stats = struct { used: usize, capacity: usize } with an allocation-free stats(a: *const Arena) -> Stats deriving both from off and cap, the Arena carrying no counters so that alloc stays a bump and a bounds compare, and the comptime intrinsics size_of[T]() and align_of[T](), alloc aligned to align_of[T]; e.str: type Builder, type Sink (a *void context and a plain fn(ctx: *void, bytes: []const u8) -> err), error NotOnTop, builder (whose cap is an initial reservation, not a limit), builder_to (the same, carrying a flush sink: a push that would exhaust the arena drains through the sink and proceeds, so mem.Exhausted never leaves a push on a sink-backed builder), done, concat, join, eq, error BadNumber with the four parsers parse_f32, parse_f64, parse_i64 and parse_u64 that return it on a malformed or out-of-range input, and a push per integer type — usize and isize included — so an expansion contains no source-level conversion (the formatter's internal widening of narrow integers to the push width under {x}/{b} is not one), {x}/{b} printing a signed value's two's-complement bits at its width; e.io: print(s: []const u8) -> err and printf[FMT] — pushing into a sink-backed builder over a 4 KiB stack arena of its own, whose sink is print's loop over os.write, so the flushing is the builder's and printf carries no retry — no error of its own; the formattable set is every type §9 rule 4 supplies a format for — the integers, the floats, bool, str and every other slice, err, pointers, arrays, Vec/Mask, and every enum and union enum whose own module declares none — plus any type whose module declares fn <t>_format(v: T, b: *str.Builder) -> err (§9), a verb on which expands to one call to that function; e.atomic: type Ordering and the intrinsics init (comptime-evaluable), load, store, xchg, cas -> (bool, T), add, sub, and, or, xor, min, max (the last seven on integer T only), fence; e.time: the signature table of §16 and error Invalid; e.math: every function comptime-generic over the float type, fma an intrinsic; e.gpu: len[T](b: Buf[T])	The examples called a dozen functions the spec never declared, so their argument and return types could not be checked from the text, and size_of was used as a builtin that existed nowhere. Naming a size needs an intrinsic because nothing else in the language can compute layout; the narrow pushes leave no source-level conversion in a printf expansion — the formatter's internal widening of narrow integers to the push width is not one — so §6's list of four implicit operations stays true. (amended 2026-09-04: §7 and §13 still used io.read and io.NotFound, which the fixed e.io surface never had — the examples are over e.os; e.atomic was five calls by example with no cas result and no ordering type, and §10 named device operations "§8's" that §8 lacked; push_usize/push_isize were missing from a family that exists so no cast is needed; str.Builder was used as a type and never declared.) (amended 2026-09-04: "no widening remains in a printf expansion" contradicted §4's {x}/{b} rule, under which the formatter widens a narrow integer to push_hex_u32's width internally; the claim is that no source-level conversion remains.) (amended 2026-09-04: io.printf was said to push into a 4 KiB stack builder with no arena, which the push family's NotOnTop contract cannot satisfy — every push is a claim on an arena top; it pushes into a builder over a 4 KiB stack arena of its own, carrying a sink whose write is print's loop over os.write, so a push that would exhaust the arena drains the buffer through the sink and proceeds, a str argument longer than the whole buffer drains through the sink as it is pushed, and printf sends what str.done leaves through the sink once at the end — every verb still a push, no caller's arena involved, and no retry in printf itself.) (amended 2026-09-04: the fixed surfaces omitted atomic.init and mem.bitcast, both declared in §8.) (amended 2026-09-04: D52 made a user type formattable through its format, but printf's flush-and-retry wrapped only printf's own pushes, so a user format body's try str.push propagated mem.Exhausted out of printf; the flush moves into the builder as str.builder_to's sink, printf becomes an ordinary sequence of pushes, and the formattable set gains every type with a format.) (amended 2026-09-04: three gaps in the fixed surfaces — mem.stats, which docs/modules.md lists in e.mem and §8 now declares; number parsing, which e.cli, fmt.json and fmt.csv all need and examples/sample.e already called, now str.parse_* over str.BadNumber; and a formattable set closed at six shapes while §9 rule 4 supplies a format for far more, now stated as rule 4's set plus the declared ones.)

#	Decision	Chosen	Why
D44	Operator precedence	C's precedence and associativity with two changes: every bitwise operator (&, ^, , and the shifts) binds tighter than every comparison, and comparisons do not chain; eleven levels, tightest first: postfix (., [], [a . b], call, comptime bracket), prefix (- ! ~ & *), * / % **%, + - +% -%, << >>, &, ^, , comparison (non-associative), &&, ; assignment is a statement, not an expression; try prefixes one call; neper fmt neither adds nor removes parentheses	No table existed, so a & b == c had no parse and §3's "the other operand of a binary operator" named a tree that was never defined. C's order is what every reader and every emitter expects, and its one famous trap — comparison binding tighter than &, a historical accident — is the one thing worth fixing, as Go did; a non-chaining comparison turns a < b < c, which is never what a generator meant, into a compile error instead of a bool compared with a number.
D45	GPU targets in a build	--gpu spv,ptx none, default spv; --gpu none is a host-only build and a kernel present under it is a compile error naming the kernel; a host build compiles every kernel-owning module — only its device-reachable functions and those of the modules they reach — once per listed GPU target into <module>.<gpu-target>.em and embeds those Code sections in the executable; --target spv ptx alone builds device modules for neper dis and no executable; the device half of a module includes only device-reachable functions (D24, D37), so target.os is .None there and host-only names, use e.os included, are simply absent	One binary embeds the GPU module (§10) but the only build option was a single --target, so nothing said how a host build produced or chose the .spv.em/.ptx.em it embedded, what --target spv meant on its own, which modules received device .em files, or how use e.os resolved for the device half of a kernel-owning module. spv by default is what every desktop GPU on the two M0 platforms speaks; ptx is opt-in because it needs a CUDA driver. Restricting device compilation to the device-reachable set is what D24 already does, and it is what makes a host-only import invisible on the device without a second module system.
D46	.nepersym	The compiler emits its line and symbol tables as a non-loaded section .nepersym in every object and executable from M0, read by the runtime for the trap backtrack on every platform regardless of linker	§11 promises a symbolised backtrack in every build mode under either linker, and the standard formats cannot deliver that on Windows until the M4 PDB writer exists — nor should the runtime carry a DWARF or PDB reader. One compact section, written from data the compiler already holds and found through the executable's own section table, gives the backtrack one source everywhere. It is distinct from the standard-format debug information external tools read (D9) and from the neper-format locals/types side table that is the M4 optimisation.
D47	Module graph	use cycles are forbidden: the module graph is a DAG, and a cycle — direct or through any number of modules — is a compile error naming every module in it (§2, §12, §15)	Interfaces are built from the interfaces of the modules a file uses (§15) and Deps edges follow use (§12); a cycle has no first interface to build and no finite recompilation set, and a language with one file per module and no forward declarations gains nothing from allowing one. Naming the whole cycle is what makes the error actionable in generated code.
D48	Type aliases	type X = T over anything but a struct, union, enum or union enum body is an alias — the same type, interchangeable, no conversion in either direction; a distinct type is a single-field struct, which is what e.time and e.os do; str is exactly such an alias of []const u8 (§4)	The form existed with no rule for whether X and T were one type or two. An alias is the cheap reading: it costs no cast and no rule, and the distinct-type need is already served by the one-field struct, which every library type in this document uses — a second distinct-type mechanism would be the redundancy a small language cannot afford.

#	Decision	Chosen	Why
D49	Bare union	Kept as the C-layout escape hatch: it exists for C ABI compatibility across <code>extern (INPUT, VkClearColor, sockaddr)</code> and deliberate punning, and <code>union enum</code> is the default for a variant type; <code>mem.bitcast[T](x)</code> reinterprets a value as a <code>T</code> of the same size, checked at compile time, legal on CPU and device, so a pun never needs a union; bare union is banned in device code; a <code>union enum</code> in device code has non-overlapping payloads; neither union form is a <code>Buf[T]</code> element type (§4, §8, §10)	The earlier rationale — punning, and not hiding a cost — was wrong on both counts: a compiler-placed tag has the identical layout, and punning is a bitcast, not a type. What only a bare union can do is match a C union's layout at an extern boundary, which the Win32, Vulkan and socket APIs all require. SPIR-V's logical addressing cannot reinterpret a local, so the device gets the bitcast and not the union, and non-overlapping payloads are legal there because the program cannot see the layout — which is also why neither form may be a <code>Buf[T]</code> element, where the layout crosses the boundary as a memcopy.

#	Decision	Chosen	Why
D50	Implementation constants	(1) root arena 64 MiB by default, <code>--arena SIZE</code> raises or lowers it (§8, §13); (2) no default parameters exist, so <code>thread.spawn[Ctx](f, ctx, stack: usize)</code> takes the stack size explicitly, <code>const thread.DEFAULT_STACK: usize = 1 MiB</code> is provided, and <code>os.thread_create</code> takes the same stack (§5, §8); (3) xxHash64 for signature hashes, body hashes and content-hash folding (§12, §13), FNV-1a 32-bit for error values alone (§7); (4) ULP bounds — the Vulkan specification's precision table on the device, every <code>e.math</code> transcendental within 2 ULP on the CPU, <code>math.rsqrt</code> within 2 ULP on both, per-function bounds recorded in <code>lib/e/math.e</code> when implemented (§11); (5) PTX levels <code>--cpu sm_50 sm_70 sm_80 sm_90</code>, default <code>sm_50</code>, as rows of the §13 table beside x64 and aarch64, <code>--cpu</code> given at most once per target family so a host build with <code>--gpu ptx</code> names both its host level and its <code>sm_*</code> level, a level whose family the build does not target being an error (§10, §13); (6) diagnostic codes numbered by spec section, <code>E<SS><NN></code>, <code>E0403</code> being §4's third; the catalogue is written with the checker (§13); (7) <code>neper fmt</code>: four-space indentation, 100-column lines, a breaking bracketed list one element per line, comments never column-aligned, frozen at 1.0 (§3, §14); (8) <code>e.test</code> ships <code>assert(cond, msg), eq[T](a, b, msg)</code> — <code>T</code> a type with an <code>eq</code> (§9), which includes every type <code>==</code> takes — <code>near(a, b, abs, rel, msg)</code> and <code>fail(msg)</code>, each <code>-> err</code>; golden-file comparison deferred (§13, §17); (9) <code>neper test</code> builds debug by default and accepts <code>--release</code>, running the same tests against release semantics (§13); (10) per-test arena 16 MiB by default, <code>--test-arena SIZE</code> with <code>--arena</code>'s <code>SIZE</code> syntax (§13); (11) <code>.em</code> files are written to <code>.neper/<mode>/<module>.<target>.em</code> under the project root, debug and release coexisting by directory (§12); (12) <code>e.data.sort</code>, <code>e.data.map</code> and the rest of <code>e.thread</code> — beyond <code>spawn</code>, <code>join</code>, <code>detach</code> and <code>DEFAULT_STACK</code> (§8) — are specified by their <code>lib/e</code> source and indexed by <code>neper index</code>, the spec authoritative only where it writes a signature; <code>e.math</code> arities <code>atan2(y, x)</code>, <code>pow(x, y)</code>, <code>fma(a, b, c)</code>, <code>min(a, b)</code>, <code>max(a, b)</code>, <code>copysign(x, y)</code>; <code>round</code> is ties-to-even — <code>RoundEven</code> on SPIR-V, the round-to-even instruction on CPUs — so device and host agree (§11, §13)	These were §17's twelve open items: each a number, a catalogue or a surface with no goal to decide it, listed so that an implementer knew it was choosing rather than reading. Each is now fixed by the smallest defensible value — the Windows default for a stack, a small arena a program raises, one 64-bit non-cryptographic hash for everything that is not an error value, published precision tables rather than measured ones, the standard PTX levels verbatim as the x64 levels are, section-numbered diagnostics so a code says where its rule lives — so that two implementations, and the bootstrap and the self-hosted compiler, agree without a second document. <code>round</code>'s tie rule is recorded because it is the one exact <code>e.math</code> function whose device and host instructions could otherwise silently disagree. (amended 2026-09-04: a single <code>--cpu</code> could not name both a host level and an embedded ptx module's <code>sm_*</code> level; one option per target family needs no second option, since every level names its family. Item 12 also omitted <code>thread.detach</code>, whose signature §8 fixes.) (amended 2026-09-04: item 8 fixed <code>test.eq</code>'s <code>T</code> as a type <code>==</code> takes while D52 lists <code>e.test</code> as a consumer of the <code>eq</code> protocol; <code>T</code> is a type with an <code>eq</code>, which is the wider set.)

#	Decision	Chosen	Why
D51	Standard-library namespaces	<code>e.*</code> is the language-owned standard library (replacing <code>core.*</code>), <code>algo.*</code> is pure computation, <code>text.*</code> is Unicode/text processing, <code>crypto.*</code> is pure cryptography with caller-supplied entropy, <code>fmt.*</code> is interchange formats, <code>gfx.*</code> is graphics/rendering and <code>ui.*</code> is the declarative application framework; <code>x.<owner>.<package>.*</code> is separately versioned source; <code>lib/e/mem.e</code> is <code>e.mem</code> . D66 supersedes the former ecosystem-wide final-segment reservation; D79 adds <code>gfx.*</code> and <code>ui.*</code> .	Domain prefixes make ownership and purity visible without placing unrelated text and security surfaces into algorithm grab bags. Owner-qualified optional names avoid provenance collisions, while explicit import aliases handle local final-segment collisions.
D52	Protocols	<code>T.f(...)</code> on a comptime type parameter or a comptime type value resolves, at instantiation, to <code>fn <t>_f</code> in the module declaring <code>T</code> — <code>Sensor.hash</code> is <code>fn sensor_hash</code> ; conventional <code>hash</code> , <code>eq</code> , <code>cmp</code> , <code>format</code> , <code>next</code> ; for <code>v in it</code> uses <code>next</code>	A generic container needs an operation on its element type, and neper has no interfaces, traits or closures by design. Name lookup at compile time gives it with no <code>vtable</code> , no tag and no dispatch — a direct monomorphised call — and <code>grep "^fn sensor_hash"</code> still finds the one that runs. Legal only where the module is genuinely unknowable, so §14 invariant 4 survives. The type's name is in the function's name because invariant 1 admits no overloading: a module declaring two types would otherwise need two <code>fn hash</code> lines. (amended 2026-09-04: the row admitted only a comptime type parameter as the receiver, where §9 rule 1, §2's resolution order, §14 invariant 4 and <code>docs/modules.md</code> all admit a comptime type value — <code>f.ty.format(x, b)</code> , which §9's own worked example relies on.)
D53	Compile-time introspection	<code>e.meta: fields</code> , <code>members</code> and <code>type_name</code> are comptime-only, returning comptime values and callable only in a comptime context (D5); <code>get</code> and <code>set</code> are comptime- parameterised , only their <code>FIELD</code> argument comptime, each compiling to an ordinary field load or store and legal in any body; <code>Field</code> and <code>Member</code> are comptime-only types, never storage; a <code>for</code> over a comptime slice is unrolled	Every <code>fmt.*</code> module, <code>e.cli</code> , <code>e.log</code> , <code>e.bytes</code> and owner-qualified <code>x.<owner>.db.*</code> driver otherwise needs hand-written marshalling per type — the boilerplate generated code gets wrong most often. Compile-time only keeps runtime reflection a non-goal (§1): no type table in the binary, dead-code elimination intact, and the unrolled loop is gone by codegen. <code>fields</code> on a bare union is an error, since nothing says which member is live. (amended 2026-09-04: "all comptime-only" made §9's own <code>format_struct</code> illegal, because <code>meta.get</code> reads a field of a runtime <code>*const T</code> and the read has to happen at runtime; only <code>FIELD</code> is comptime there, and <code>Field/Member</code> are declared comptime-only types instead, since a type-valued field has no layout, no zero value and no place in a struct, a buffer or extern.)
D54	Continuation barrier	An unclosed <code>(</code> , <code>[</code> or literal/type-body <code>{</code> may not cross a column-0 declaration keyword or attribute line; the error is reported at the opening bracket	Implicit continuation's standard failure is a dropped bracket swallowing the rest of the file and erroring far from the cause — exactly what generated code produces. §14 invariant 2 already guarantees nothing else starts at column 0, so the parser gets a barrier for free: one dropped bracket, one precise diagnostic. The property that makes <code>^fn</code> greps exact is what bounds the cascade.

#	Decision	Chosen	Why
D55	try positions	Statement-level only: a whole statement, the entire initialiser of a <code>let/var</code> , or the entire right-hand side of an assignment. Not nested in an argument list, an operand, a literal, an index, a condition or after <code>ret</code> ; not inside <code>defer</code> ; not lexically in a comptime root expression. A helper dynamically executed by the interpreter may use <code>try</code> ; an escaping error fails the root.	<code>try</code> is control flow, and neper keeps control flow at statement level. The lexical/dynamic distinction permits ordinary fallible helpers during evaluation without inventing a return target for a <code>const</code> . A deferred fallible call is <code>defer let _ = f()</code> .
D56	Evaluation and type completeness	Subexpressions evaluate left to right; conditions are <code>bool</code> ; non-void paths return; operator operand/result types, switch duplicates and break targets are fixed by spec §6; sized/runtime-storable types, recursive-type rejection, empty aggregates and exact layout are fixed by §4.	Two conforming implementations must not differ in side effects, accepted operands or layout. These rules close what the earlier operator and ABI decisions left implicit.
D57	Representations and arena/resource state	<code>bitcast</code> cannot contain pointers or strip provenance; invalid representations are a debug <code>invalid</code> trap and release undefined behavior; an integer-to-enum cast naming no member traps in every mode, superseding D7's unchecked-enum clause; <code>mem.eq</code> is field/element value equality; arenas, builders and OS/GPU handles have the single-live-copy and consumption contracts in §§5, 8 and 10.	Raw byte equality reads padding, unrestricted bitcasts defeat <code>const</code> , and freely copied cursors/handles create overlapping allocation or double release. An invalid enum would invalidate exhaustive control flow, so it has no useful unchecked result.
D58	Generic, protocol and reflection completion	Generic inference is structural and checking dependent on a comptime parameter occurs at instantiation; aliases preserve the canonical nominal declaration; protocol snake-case, wrong-signature errors, fallbacks and lookup edges are normative; <code>e.meta</code> includes type-kind, element, length and backing-type queries.	These rules make protocol lookup deterministic and support structural encoding/decoding without claiming tagged unions and opaque resources are generically decodable.
D59	GPU host semantics	Driver launches are asynchronous and <code>.Cpu</code> launch is synchronous; device storage/execution/argument types are separate sets; host handles carry owner/generation validation with <code>WrongDevice/InvalidHandle</code> ; buffer release and device close are fallible; workgroup/grid bounds, barriers, subgroup lanes and capability inference follow §10.	The previous single "device type" and unconditional asynchronous statement contradicted legal locals and the CPU backend, while copyable opaque handles needed an enforceable lifecycle.
D60	Floating determinism	Narrow floats use widen-to-f32, operate, narrow semantics; arithmetic NaNs canonicalize; special values, FTZ widths and approximate-function domains are fixed by §§4 and 11.	Direct narrow rounding and f32 double rounding can differ, and NaN payload choices made the earlier bit-identical promise untestable.
D61	Incremental and artifact integrity	Foreign <code>const</code> reads take value/body-hash edges; comptime cache keys include expression, arguments, target and semantic versions; <code>xxHash64</code> is a candidate index followed by full identity comparison; <code>.em</code> and <code>.neper.sym</code> have versioned bounded formats.	Hash collisions must affect performance only, and target- or value-dependent compiled constants must never survive stale incremental builds.

#	Decision	Chosen	Why
D62	Toolchain and test isolation	Host targets are architecture-OS triples; run is host-only; under <code>--gpu</code> none only kernels in the reachable module graph are errors; linker inputs, index schema, diagnostic/exit ordering and startup failure are fixed by §13. Each test runs in its own child process with fresh globals, dedicated output pipes and nonce-authenticated framed control records. This supersedes D15's process-per-module execution details and narrows D45's "kernel present" wording.	Process-per-test removes post-crash global ambiguity and cross-pipe attribution races. Authenticated control records prevent user stderr or a guessed inherited handle from spoofing a trap.
D63	Module and platform boundaries	<code>e.os</code> is the sole ordinary host-platform binding surface, with GPU-driver and owner-qualified <code>x.*</code> exceptions; standard modules needing new OS primitives remain blocked until those primitives are added. <code>x.*</code> is vendored source in v1; pure <code>algo.*</code> and <code>crypto.*</code> receive entropy explicitly; fallible iterators are explicit rather than language <code>for</code> ; WebSocket and format transports are separate modules. D78 supersedes the former provisional <code>x.neper.*</code> package placement.	The catalogue previously scheduled modules that its own extern rule made impossible, classified externally backed security and OS-seeded randomness under the wrong namespaces, grouped unrelated security concerns, and used external package names with no owner provenance.
D64	Debug type and symbol completeness	The fixed DWARF/CodeView subset includes const-qualified pointees and subroutine types for function pointers; Mach-O symbol tables are explicit; retained <code>.nepersym</code> contributions are merged, relocated and indexed rather than carried as independent opaque sections. This supersedes D9's "twelve-tag" count while preserving its intentionally small subset.	A pointer tag alone cannot describe a callback signature, and system linkers do not preserve or combine custom sections portably without retention and merge rules.
D65	Public tooling protocol	<code>docs/tooling.md</code> version 1 fixes a JSONL envelope and discriminated records, original-byte and Unicode/UTF-16 spans, structured fixes, lossless tokens/parse, complete definition/reference index, deterministic test records, reproducible build manifests, info capability discovery, stdin operation and generated-source maps; <code>docs/schemas/neper-v1.schema.json</code> is its machine schema. Diagnostic codes come from <code>docs/diagnostics.md</code> as stable semantic <code>E-<CATEGORY>-<NNNN></code> values. This supersedes D50 item 6 and D62's narrower index/diagnostic clauses.	A harness must never scrape human text, infer compiler capabilities, duplicate parsing or name resolution, depend on checkout paths, or guess how to apply a fix. Section-numbered codes and point-only locations were documentation conveniences rather than stable APIs.
D66	Composable names and imports	<code>use path</code> binds the last segment by default and <code>use path as qualifier</code> supplies a module-local explicit alias; wildcard imports remain forbidden and every cross-module source reference remains qualified. A local cannot shadow an active outer, module, qualifier or builtin name, but a name may be reused in disjoint sibling scopes. This supersedes D15/D41's "no-alias" wording, D42's once-per-function rule and D51's ecosystem-wide reservation.	Last-segment uniqueness made unrelated packages coordinate globally, while function-wide uniqueness made independently generated blocks coordinate outside their lexical interface. Local aliases and lexical non-shadowing preserve one meaning at every source position without either global burden.

#	Decision	Chosen	Why
D67	Formal syntax and bounded recovery	<code>docs/grammar.ebnf</code> revision 1 is the normative concrete grammar. The lexer always emits <code>NEWLINE</code> ; the parser alone classifies it as soft in delimiter contexts. UTF-8, BOM, line-ending normalization, controls, tabs, longest match, literal lexing, raw strings and closed token/node registries are fixed. A continuation cannot cross a declaration, containing-block, <code>case</code> , <code>default</code> or EOF barrier. This extends D54.	Standalone tokenizers, syntax highlighters and lossless editors need parser-independent tokens, and a dropped delimiter inside one large function must not consume the rest of that function. Raw strings reduce generated embedded-data escaping without adding interpolation or hidden work.
D68	Error-propagation spelling	The propagation form remains <code>try call()</code> , not <code>?call()</code> , under D55's three positions.	Both spellings are one lexical token and model-token cost is tokenizer-specific. The word is greppable, states control flow, and avoids spending punctuation on an unmeasured character-count optimization.
D69	Machine-readable module plan	<code>docs/modules.json</code> version 1 is authoritative for module names, normalized surface/schedule status, milestones, layers and blockers; <code>docs/modules.md</code> carries rationale. Vendored packages carry the reproducibility-only <code>neper-package.json</code> manifest described there.	A prose table and ASCII layering diagram cannot be checked reliably, and "Spec, M1, Later" mixed independent dimensions. Structured plan data lets CI and generation harnesses reason about availability without making it a compiler package resolver.
D70	Pre-implementation conformance and LLM measurement	Before compiler implementation, the specification ships the <code>tests/conformance/corpus</code> defined by tooling protocol §9 and an initial multi-tokenizer generated-code report. Grammar examples, primary diagnostics, formatting, token trees and tool records are byte-exact fixtures.	"LLM optimal" is not a property that character count can establish: tokenizers and repair behavior differ. A versioned corpus prevents two implementations from making different recovery or formatting choices, while the report makes syntax tradeoffs measurable.
D71	Module decomposition and API ownership	<code>docs/modules.md</code> , <code>docs/module-apis.md</code> and <code>docs/modules.json</code> jointly define the toolchain library plan: the prose fixes namespace and dependency policy, the API catalogue enumerates the exact proposed declarations, and the JSON manifest is authoritative for names, layers, delivery state, direct edges and blockers. Broad historical buckets such as <code>e.data</code> , <code>algo.text</code> , <code>algo.crypt</code> , <code>e.http</code> , <code>e.ws</code> and <code>x.sec</code> are superseded by the decomposed owner-qualified names in that plan.	One concern per module improves retrieval, generation context and dependency auditing. Separating Unicode, cryptography, linear algebra, network protocols and optional vendor packages prevents unrelated APIs from sharing a qualifier, while an exact catalogue prevents implementation-time API invention.
D72	Canonical implicit-operation count	Spec §6's table is the only authoritative list and contains five operations. This supersedes the historical counts embedded in D18 and D43 without changing either decision's substantive rule.	A generated checker needs one enumerable source of truth. Repeating a count across historical rationale created a contradiction when the compiler-generated iterator-place operation was added.
D73	Compact lossless syntax interface	Tooling protocol v1 exposes 94 explicitly named token kinds and 54 explicitly named structural syntax-node kinds. Grammar helper productions are inlined rather than serialized. Original newline spellings, a leading BOM, trivia and invalid bytes remain reconstructible, while <code>info.language_profiles</code> relates language, grammar and stream versions in records rather than parallel arrays.	A grammar-shaped tree is correct but unnecessarily deep and expensive for model context; an AST without tokens is cheap but unsafe for source edits. A compact lossless tree gives harnesses stable structure and exact edit provenance without leaking parser implementation details or making them guess version relationships.

#	Decision	Chosen	Why
D74	General-purpose RTL completion proposal	The module plan adds <code>algo.bitset</code> , <code>algo.complex</code> , <code>text.encoding</code> , <code>text.io</code> , <code>e.task</code> , <code>e.time.calendar</code> and toolchain-owned <code>e.tz</code> ; expands <code>e.str</code> , <code>e.io</code> , <code>e.fs</code> and <code>e.sync</code> ; replaces <code>raw os.last_error</code> with explicit <code>os.ErrorDetail</code> ; and gives locale, URI and MIME exact initial toolchain-module surfaces. <code>e.tz</code> pins its built-in IANA data to the toolchain version while accepting explicit compatible data. These additions preserve caller-owned memory, bounded I/O, explicit locale/zone data, cooperative cancellation and portable <code>err</code> propagation. Runtime variants, exceptions, object/component streaming, COM and dynamic invocation remain outside the language-owned runtime.	Comparison with Delphi's RTL exposed missing everyday substrate rather than missing advanced algorithms: substring manipulation, encoding-aware line I/O, civil calendars and zones, secure temporary files, rich metadata, task pools, timed synchronization, locale formatting and URI/MIME processing. Making state and data versions explicit gains the useful coverage without importing Delphi's global locale, managed-object or exception model.
D75	HTML is a format module	<code>fmt.html</code> parses UTF-8 slices or readers into a bounded, arena-owned, index-linked, read-only tree and serializes it. Its tokenizer and tree builder follow the WHATWG algorithm frozen by the toolchain release, including recovery, implied elements and HTML/SVG/MathML transitions. It does not reuse <code>fmt.xml</code> , open files, fetch resources, execute scripts, apply CSS or provide a mutable browser DOM.	Real HTML is not well-formed XML, and useful parsing requires deterministic recovery and tree construction. Owning it as <code>fmt.html</code> matches the existing format namespace without creating a pseudo-external <code>x.neper.*</code> standard library. Node indices keep relationships stable under arena allocation and explicit limits preserve the library's bounded-input rule.
D76	Complete regular collection families	The plan adds semantic <code>e.data.stack</code> and <code>e.data.queue</code> facades, stable non-reusing node identifiers in <code>e.data.linked</code> , caller-storage <code>e.data.disjoint_set</code> , immutable CSR <code>e.data.graph</code> , and pure <code>algo.graph</code> traversal/component/shortest-path algorithms. Every existing collection exposes deterministic non-mutating iteration, ordered trees add upper-bound/range iteration, and heaps add comparator contexts and linear-time bulk construction.	Lists, hash/ordered maps and sets, deques, rings and heaps supplied the storage foundation but left ordinary names, traversal and graphs to every application. Thin stack/queue facades improve discovery; stable indices avoid pointer invalidation under arena growth; separating graph storage from algorithms keeps representation reusable and dependency direction acyclic.
D77	Ordered delivery tiers	Every toolchain module appears exactly once in <code>modules.json</code> 's ordered core, extended or experimental tier. Core gates the first stable CPU release; extended modules become stable when independently delivered but do not gate it; experimental APIs have no compatibility promise. Dependency layers remain separate from delivery tiers. The lean core excludes ordered trees, application frameworks, advanced formats, specialized numerical/text/crypto domains and thin collection facades; <code>algo.bitset</code> moves into M2 core while <code>e.data.tree</code> , <code>e.debug</code> , <code>e.metrics</code> , <code>e.log</code> and <code>e.cli</code> cease to gate M2.	A large proposal catalogue is useful, but presenting all modules as one product commitment makes the language appear bloated and lets unrelated breadth delay stability. An explicit exhaustive order preserves future designs while making release scope and compatibility honest and mechanically checkable.

#	Decision	Chosen	Why
D78	Priority RTL expansion and namespace ownership	The extended tier adds <code>e.data.slot_map</code> , fuller <code>e.data.iter</code> , <code>algo.decimal</code> , <code>algo.deflate</code> , <code>text.locale</code> , memory mapping and file watching, concurrent queue/map, typed asynchronous I/O, TLS, URI/MIME, gzip/zstd and zip/tar. Neper-owned APIs live under the toolchain domain namespaces; <code>x.neper.*</code> is forbidden and all such reservations are removed. <code>x.*</code> is reserved for optional packages named after their actual external owner. This supersedes D63's provisional <code>x.neper.*</code> placement and D74's package wording; D79 subsequently adds the <code>gfx.*</code> and <code>ui.*</code> domains.	The priority set closes major everyday gaps against .NET/Delphi without enlarging the first-stable core. A project cannot meaningfully be an external vendor to itself; ownership namespaces make discovery, support and reproducibility boundaries honest.
D79	Declarative GPU UI proposal	Reserve toolchain-owned <code>gfx.*</code> and <code>ui.*</code> namespaces and add fifteen experimental modules for pure geometry/paint/images, shaping/layout, display lists, windows/input, immutable widgets, explicit state reconciliation, animation, accessibility, testing and application scheduling. Widgets are frame-arena values; persistent elements/state and GPU resources use bounded generation-checked stores. No new syntax, objects, closures, GC, reflection or hidden allocation is introduced. Delivery is blocked on reviewed <code>e.os</code> native-window/accessibility and <code>e.gpu</code> presentation designs and must pass GP-15 before promotion.	A Flutter-like value tree and GPU renderer fit Neper's explicit memory model better than a Swing-style object hierarchy. Experimental status permits workload-led API correction without expanding the first-stable commitment, while explicit blockers preserve the sole-platform-boundary rule.
D80	Embedded application assets	Add experimental <code>e.asset</code> and <code>ui.asset</code> . Root <code>project.yaml</code> declarations become a sorted linker-generated immutable registry; logical name, source, media type, attributes, size and SHA-256 enter build identity and the canonical build manifest. <code>e.asset</code> performs allocation-free zero-copy lookup. <code>ui.asset</code> deterministically selects locale/theme/scale variants, exposes fonts without copying and owns a caller-bounded GPU texture cache driven by a caller-supplied decoder. Raw bytes are embedded unchanged; package-provided, compressed and external runtime assets are deferred.	Flutter-style applications need reproducible fonts, images and other resources without runtime path assumptions. Keeping codecs injectable avoids a hidden format dependency, immutable executable slices fit the existing data model, and explicit bounded caches preserve Neper's allocation and ownership rules.
D81	Interoperability and verification expansion	Add extended <code>crypto.x509</code> , ASN.1/PEM, multipart/mail, quoted-printable, PNG/JPEG/WebP, bzip2/LZW/zlib, deterministic text and context-safe HTML templates, compiler-backed coverage/fuzzing, and generic <code>e.db</code> SQL contracts. Extend <code>e.bytes</code> with Base32 and Base85. Encodings/codecs remain bounded and streaming; image pixels are caller-owned; certificate roots and time are explicit. Concrete drivers are the owner-qualified packages <code>x.sqlite.sqlite</code> , <code>x.oracle.mysql</code> and <code>x.postgresql.libpq</code> .	Comparison with Go's standard library exposed operational gaps in TLS identity, common Internet messages, image interchange, SQL portability and verification tooling. Toolchain-owned protocol contracts improve interoperability without importing hidden allocation, ambient trust stores, driver registries or database implementations into the stable core.

#	Decision	Chosen	Why
D82	Post-M2 LLM hardening gate	Complete M2 against its existing contract, then execute M2.5 before any M3 implementation. post-m2-llm-hardening.md owns H01–H29 evaluation, required outcomes, migration and evidence; llm-hardening-recommendations.md distills that review into the prioritized R01–R11 set and registers R08–R11 as H26–H29. All items start scheduled/unresolved; adopted language/API/protocol changes require versioned normative amendments and implemented verification. CPU/tooling obligations close in M2.5; explicitly future GPU runtime evidence remains pending against frozen contracts.	The hidden-semantic-context concern raised by Jose Crespo's 2026-08-22 article motivates checked ownership/lifetimes, explicit unsafe and concurrency contracts, value/alias and error-state review, compiler-derived context and real LLM evaluation. The follow-up review adds incremental queries, snapshots, bounded semantic editing/transport, artifact hardening and measured CPU/GPU performance contracts. This decision fixes sequencing and the mandatory gate; it does not yet supersede D3/D14 or adopt proposed keywords. Documentation alone cannot close implemented-delivery obligations.

| D83 | Explicit multi-GPU discovery and selection | Extend planned `e.gpu` with bounded caller-owned `devices`, `DeviceInfo/DeviceKind/DeviceKey`, exact `open_id` and opening-time `info`. Backend-scoped UUIDs are revalidated selectors, not immutable serial numbers; missing/duplicate keys fail without substitution. Indices are temporary; memory reports capacity, not availability; every queue/buffer belongs to one logical open device. | Applications and harnesses must be able to choose among multiple GPUs without assuming ordinal stability, unique names or Vulkan/CUDA identity equivalence. Freeze contracts and versioned tooling under M2.5 H18/H21/H22; implement CPU/Vulkan in M3 and CUDA in M4. No implicit offload, automatic fallback, peer sharing, device migration or optimized CPU kernel backend is adopted. |

D84 — standard-library composition and stable dependency closure

Adopt `stdlib-hardening.md` SL01–SL11 and the synchronized next-contract catalogue. Keep ordinary parameters and the current shadowing rule; fix signature/import collisions and validate declarations with the real resolver during M2.5. Add core `e.cancel` and promote basic `text.utf8` to core through that cross-cutting gate. Add extended `crypto.mac`, `crypto.kdf` and `e.test.support`; extend existing modules for composable buffering, lossless JSON/Pointer/Patch, bounded globbing, fallible iteration, safe directory-relative operations, supervised processes and HTTP/SSE streaming. Installed capabilities remain separate from design metadata.

Promote the pure `gfx.geometry/gfx.paint/gfx.image` dependency closure to extended and stabilize it with the already-extended PNG/JPEG/WebP codecs. GPU rendering and UI remain experimental; stable public types cannot depend on experimental types. This amends D77/D79's planned tiers, not implementation status. Version/migrate delivered CPU contracts in M2.5 without moving M2's preserved baseline. Future libraries retain their own implementation gates and independent runtime evidence.

Consequences accepted

- **We own the optimiser.** v1 targets roughly -01 quality: inlining, constant folding, DCE, good register allocation. Explicit SIMD carries the vector performance rather than an auto-vectoriser. Beating LLVM on scalar codegen is a multi-year project; beating it on compile time is immediate and permanent.
- **x86-32 is in scope** and is a fourth emitter. It is sequenced last.
- **Self-hosting sets the language floor.** neper must be able to express a compiler — interned strings, growable arrays, hash maps — with no heap. This is a real constraint on minimalism and a useful forcing function.

Diagnostic fixtures

tests/selfhost/fixtures/diagnostics | 2 files

tests/selfhost/fixtures/diagnostics/src/broken.e

```
fn invalid() -> Missing {  
    ret zero  
}
```

tests/selfhost/fixtures/diagnostics/src/main.e

```
use e.mem  
use broken  
  
fn main(a: *mem.Arena, args: []str) -> err {  
    ret ok  
}
```

General-purpose verification

docs/general-purpose-verification.md | 20066 bytes | sha256 bf0a3306f4d0f68d

neper — general-purpose replacement verification

Status: verification plan, not language specification. This document adds no syntax, runtime facility or standard-library API. `spec.md` remains authoritative for the language, `grammar.ebnf` for concrete syntax, `tooling.md` for machine interfaces, `modules.md` for the library plan, and `roadmap.md` for sequencing.

The mandatory M2.5 gate in `post-m2-llm-hardening.md` adds a post-M2 CPU evaluation before M3 may start. It revisits ownership, lifetimes, concurrency and compiler feedback through versioned changes; current language guarantees remain those in `spec.md` until those changes land. It does not imply completion of the broader general-purpose workloads that require M3, M4, M6 or later libraries.

1. Claim under test

Neper's intended claim is:

Neper is a compact, ahead-of-time general-purpose language for model-generated software, with explicit memory, deterministic semantics, inexpensive abstraction, native interoperability, and first-class CPU/GPU execution.

This is not a claim of source compatibility, runtime compatibility, ecosystem parity, or feature parity with Python, Java, C#, C++, Rust or Go. The verification target is that one compact language can cover a large, useful intersection of their workloads without needing an unplanned language feature.

The expected replacement envelope is:

- C, C++, Rust and Go workloads that accept ahead-of-time compilation, explicit arenas, explicit concurrency and native interoperability.
- Python, Java and C# workloads that are deployed as compiled command-line tools, services, data processors, build tools, numerical programs or automation, and do not depend on a dynamic language runtime.

The following are deliberate non-claims, not verification failures:

- Rust-equivalent compile-time memory or data-race safety.
- Java/C#-equivalent managed object graphs or garbage collection.
- Python-compatible runtime evaluation, monkey patching or dynamic reflection.
- Language-level exceptions, closures, coroutines, classes or inheritance.
- Drop-in access to another language's package ecosystem.

If a workload requires one of those properties, neper is not its replacement. The project must say that plainly rather than silently broadening the language.

2. Questions that must be answered with programs

The core language is considered general purpose only when representative programs answer all of these questions without compiler-private escape hatches or an unplanned language construct.

Memory and lifetime

- Can a request, compiler pass, frame or batch own an arena and release all of its temporary state at one visible boundary?
- Can a long-running server, cache, editor and database client keep memory bounded while values have different lifetimes?
- Can persistent structures use explicit pools, free lists or arena generations without making their ordinary call sites error-prone?

- Which stale slices after `mem.reset`, views across container growth and copied arena cursors are rejected or detected, and which remain unprotected? The current spec explicitly leaves a pre-growth slice silently stale; poison bytes are not reliable lifetime detection. M2.5 H01–H03 require an enforced checked subset and separately reported unsafe/unknown cases rather than assuming debug fills suffice.

Passing this section does not imply memory safety. Release-mode invalid memory access and unsynchronised data races retain the semantics specified in `spec.md`; reports must never describe neper as having Rust's static safety guarantees.

Resource cleanup

- Does `defer` correctly clean up files, processes, threads, dynamic libraries, queues and buffers on ordinary return, every `try` propagation path and partial initialisation?
- Do debug checks reliably diagnose double close, use after close, wrong-device use and use of a consumed logical handle?
- Can APIs make ownership transfer legible without compiler-enforced linear types?

Abstraction

- Can comptime parameters and protocols implement containers, sorting, formatting, parsing and serialization without duplicated algorithms?
- Can tagged unions and explicit function tables express state machines and dynamic dispatch where another language would use interfaces or classes?
- Can an explicit context pointer plus a function pointer express callbacks for an event loop, GUI binding and C API without closures?
- Do generated specialisations remain bounded in compile time and binary size?

Errors and diagnostics

- Is `err` sufficient for routine propagation while explicit result/diagnostic structs carry payloads, source positions and causal context where required?
- Can parsers, network clients and database bindings report actionable failures without hidden allocation or exception state?
- Are cleanup and diagnostic context preserved when an error crosses several module boundaries?

Concurrency and services

- Can the library express a bounded worker pool, channel, timeout, cancellation, backpressure and graceful shutdown over OS threads and explicit synchronization?
- Can an event loop serve many connections without one OS thread per connection?
- Can shutdown join or terminate every worker and release every resource deterministically?
- Are all shared-memory and ownership rules clear enough that model-generated code does not introduce races at an unacceptable rate?

Interoperability and platforms

- Do all promised C ABI shapes round-trip on every host target?
- Do callbacks, opaque handles, calling conventions, packed records, dynamic libraries and ownership transfer behave identically under both linkers?
- Can existing C libraries supply capabilities that are intentionally outside the standard library without requiring special compiler knowledge?
- Are paths, process arguments, environment strings and filenames handled correctly on Windows, Linux and macOS, including non-ASCII input?

Project scale

- Can a large program keep stable module boundaries when every module-scope declaration is exported?

- Can vendored dependencies be resolved reproducibly with versions, checksums, transitive dependencies and licences recorded outside the compiler?
- Do incremental and clean builds remain byte-identical as the module graph grows?
- Can tools rename, index and edit code without reparsing human diagnostic text?

3. Reference workload corpus

These programs are acceptance artifacts. Their purpose is to expose missing semantics; they do not justify adding a feature automatically. When a program is awkward, first attempt a library or design solution using the existing language. If that remains unacceptable, record the workload as outside the replacement envelope before proposing a language change.

ID	Program	What it proves
GP-01	Self-hosted neper compiler	Large program structure, parsers, interned strings, generic containers, incremental artifacts and deterministic parallel compilation
GP-02	Cross-platform CLI	Arguments, configuration, Unicode input, JSON, filesystem traversal, diagnostics, exit codes and subprocesses
GP-03	Parallel build executor	Worker pool, dependency scheduling, cancellation, timeouts, captured output and graceful shutdown
GP-04	Long-running HTTP service	Bounded steady-state memory, sockets, event loop, backpressure, request arenas, logging and concurrent shutdown
GP-05	Database client implementing e.db over a C library	FFI, callbacks, opaque handles, streaming row reads, transactions, rich errors, ownership transfer and dynamic linking
GP-06	Streaming data parser	Partial input, malformed input, fallible iteration, source spans, bounded buffers and recovery
GP-07	Persistent bounded cache	Mixed lifetimes, insertion, lookup, eviction, fragmentation control and concurrency
GP-08	Plugin-style C ABI	Stable exported ABI, callbacks with context pointers, version checks, foreign ownership and failure isolation
GP-09	SIMD codec or compressor	Explicit vectors, masks, tails, alignment, target levels, scalar fallback and output equivalence
GP-10	CPU/GPU numerical workload	Kernel reachability, transfers, capabilities, launch errors and deterministic CPU/device results within specified bounds
GP-11	Interactive terminal application	Incremental input, terminal state cleanup, redraw, signals, resize and event handling
GP-12	Multi-package application	Reproducible vendoring, checksums, transitive resolution, namespace composition, licences and clean-room rebuilds
GP-13	Binary format reader/writer	Endianness, packed/external layout, bounds checks, zero-copy slices, malformed files and round trips
GP-14	Image or audio transform pipeline	Large buffers, staged arenas, native library interop, parallel work and sustained memory bounds
GP-15	Declarative GPU desktop application	Embedded asset and DPI/theme/locale variants, frame-arena widgets, keyed state reconciliation, constraint layout, shaped text, ordered input, accessibility, deterministic animation, GPU presentation and headless snapshots

Each workload must have a written design before implementation identifying its arena topology, resource ownership, concurrency model, error representation, foreign boundaries and expected module graph. That design is part of the verification record.

4. LLM-generation evaluation

Character count is not token count, and no syntax is universally optimal across model tokenizers. Token efficiency is therefore measured on the target model families and never inferred from keyword length.

The prioritized recommendation set in `llm-hardening-recommendations.md` elevates this principle from a measurement obligation to a **design input** (R03, R08/H26): a grammar-versioned tokenizer profile per supported family must exist, and a proposed syntax spelling is changed only when a measured token or repair-turn improvement on that profile justifies it, never on a character-count argument. The per-tokenizer report below is a required input to that decision and to the H12/H25 gate.

Task set

For every workload, evaluate at least these task shapes:

1. Generate one complete module from a prose contract.
2. Add a function to an existing module without unrelated edits.
3. Change one constant or condition exactly as requested.
4. Repair a compiler diagnostic using only structured tool output.
5. Add a cross-module call and its import.
6. Add an error path with correct cleanup.
7. Refactor a type or function across multiple modules.
8. Complete an intentionally truncated or delimiter-damaged file.
9. Review code containing a lifetime, race, bounds or ABI defect.
10. Generate tests from a public signature and behavioral contract.

Metrics

Record all of the following by model and model version:

- First-pass parse rate and first-pass type-check rate.
- First-pass test pass rate.
- Mean and 95th-percentile repair turns.
- Input, output and total model tokens.
- Tokens per non-comment source line and per syntax-tree node.
- Unrequested diff lines after canonical formatting.
- Hallucinated keywords, APIs, implicit conversions and language features.
- Identifier/import collisions and incorrect protocol names.
- Missing cleanup, ignored `err`, invalid lifetime and race defects.
- Percentage of repairs completed from JSON diagnostics without opening unrelated files.

The benchmark corpus, prompts, compiler version, formatter version, schemas, model identifier and tokenizer identifier are versioned together. A syntax spelling is not called more token-efficient unless the same semantic tasks improve total-token or repair-turn results across the supported model set.

Initial acceptance thresholds

Thresholds may be tightened after a baseline exists, but may not be silently relaxed:

- At least 95% of local edits parse on the first attempt.
- At least 90% of local edits type-check on the first attempt.
- Median unrelated diff lines after formatting: zero.
- At least 95% of compiler-error repairs complete in one additional turn.
- No benchmark task requires parsing human-formatted diagnostics.
- No accepted task relies on an undocumented compiler behavior or intrinsic.

Generation quality must also be compared with equivalent C, C++, Rust, Go, Python, Java and C# tasks. Neper succeeds by reducing total model effort and review risk, not merely by producing shorter files.

Post-M2 evaluation before M3

M2.5 H12 in `post-m2-llm-hardening.md` is the additional acceptance protocol for this transition. It requires a preserved M2 baseline, pre-registered held-out tasks, at least two independently trained model families, real compiler/query/repair tools,

independent behavioral oracles and reporting by task stratum. Each family must meet the local-edit and repair thresholds above on the supported CPU corpus; synthetic index or regex success does not count as semantic verification.

Include resource/region escape, aliasing, error/cleanup, scoped concurrency, protocol and target context, malformed source, stale transactions and long-running resource tests. Measure defect escape and false rejection alongside tokens, repair turns and completion. Report failures/timeouts and all retry costs, unsafe/check- suppression additions, confidence intervals and paired baseline effects. The full H12 sample sizes, decision rules and compiler/runtime budgets are mandatory for M2.5 closure; they do not replace the later cross-platform workload gate.

The accepted library review in `stdlib-hardening.md` adds SL01–SL11 fixtures to H11/H12: catalogue signatures must obey the real resolver's shadowing/import rules, buffered I/O must compose through public adapters, generic JSON must preserve large integer values, and process/filesystem/control operations must expose partial effects and bounded cleanup. Exercise the migrated CPU paths before M3; do not require unfinished TLS/image implementations in the M2.5 runtime report. Activate streaming HTTP/SSE, HMAC/HKDF, fake-I/O and image/codecs corpora when their extended modules are delivered. Image value types/codecs stabilize together without claiming that experimental GPU/UI modules passed GP-15.

5. Semantic and implementation verification

Conformance

- Every grammar production has at least one accepted case and every restriction has at least one rejected case with a stable diagnostic code.
- Operator, literal, cast, layout, control-flow, `try`, protocol and comptime rules have boundary-value tests.
- The bootstrap and self-hosted compilers accept and reject the same `neper-0` programs and emit the same structured diagnostics.
- Debug and release differences are enumerated; no unlisted semantic difference is accepted.

Determinism

- Rebuild each workload clean and incrementally.
- Build with `-j 1`, every supported worker count and perturbed scheduling.
- Relocate the project and toolchain directories.
- Compare all deterministic artifacts byte for byte after excluding only fields the specification explicitly marks nondeterministic.
- Verify that JSONL records retain their specified order independent of completion order.

Adversarial safety

Inject bounds errors, integer boundaries, stale slices, use-after-reset, invalid enum representations, duplicate handle consumption, allocation exhaustion, failed partial initialisation, races, malformed foreign input, device loss and process termination. The observed result must be exactly the debug trap, returned `err`, defined release result or documented undefined behavior promised by the specification.

Portability

Every CPU-only workload runs on all supported host triples. Portable output and behavior agree, excluding only explicitly platform-defined data such as path syntax, clock values and enumeration order. ABI fixtures are compiled on both sides of every C boundary and validate sizes, alignments, offsets, calling conventions and return classes.

Operational behavior

For every long-running workload, record peak memory, steady-state memory after warmup, open handles, thread count, shutdown duration and behavior under repeated failure. General-purpose readiness requires bounded resource use, not merely successful short tests.

6. Replacement matrix

This matrix states what must be demonstrated before comparison language names appear in project claims.

Comparison	Neper must demonstrate	Claim that remains prohibited
C	Equivalent ABI reach, freestanding/native programs, predictable layout and competitive low-level performance	That release memory errors are defined or safe
C++	Large native applications, generic containers, explicit polymorphism and reliable cleanup without constructors or RAI	Feature or ecosystem parity
Rust	Native performance, deterministic cleanup patterns and robust debug detection of lifetime/resource mistakes	Borrow-checker, memory-safety or race-safety equivalence
Go	Fast builds, cross-platform services, worker pools, networking and simple deployment	Goroutine, GC or language-level channel equivalence
Python	CLI tools, automation, parsers, data processing and rapid model-driven iteration	Dynamic evaluation, runtime reflection or source/package compatibility
Java/C#	Long-running services, FFI-backed applications, structured tooling and maintainable large projects	Managed-runtime, GC, reflection or exception equivalence

Passing a row means neper is a viable implementation-language alternative for the demonstrated workload class. It never means every program in the comparison language should be ported.

7. Release gates

Language-complete gate

- `spec.md` and `grammar.ebnf` contain no unresolved contradiction.
- The conformance plan covers every syntax and semantic rule.
- Each deliberate non-goal has an explicit existing-language construction or a documented excluded workload.
- No reference workload design identifies a required new language feature.

Tool-complete gate

- Formatter, tokenizer/parser output, index, diagnostics, tests and build manifests use their versioned machine contracts from `tooling.md`.
- All source locations have exact byte spans suitable for automated edits.
- Clean, incremental, parallel and relocated builds pass determinism checks.

General-purpose gate

- GP-01 through GP-08 and GP-12 pass on Windows and Linux.
- No long-running workload shows unbounded resource growth under its documented operating profile.
- The FFI and failure-injection suites pass on every promised host ABI.
- The LLM thresholds in §4 pass on at least two independently trained model families.

Performance-language gate

- GP-09 and GP-10 meet their correctness requirements on every applicable backend.
- Compile-time, runtime and memory results are published against comparable C, C++, Rust and Go implementations using the same algorithms.
- Performance claims report distributions and workloads, not isolated best cases.

8. Decision rule when verification fails

A failed workload does not automatically authorize a language feature. Resolve it in this order:

1. Correct an ambiguity or contradiction in the existing specification.
2. Use an existing construct more directly.
3. Move reusable policy into an ordinary library.
4. Use the specified C ABI to reuse a mature external implementation.
5. Narrow the replacement claim and document the excluded workload.
6. Only then open a separate language proposal, measured against the complexity and LLM-generation costs it introduces.

This order preserves the compact-language goal. The verification program exists to test whether the chosen small core is sufficient, not to turn comparison-language feature lists into neper's roadmap.

Grammar

docs/grammar.ebnf | 14393 bytes | sha256 aca153df102d4e29

neper concrete grammar – revision 1

Status: normative companion to docs/spec.md. This file specifies concrete syntax;
spec.md specifies typing, legality, evaluation and runtime semantics.

#

Notation: `x` is a terminal; $x?$ is optional; x^* repeats zero or more times; x^+
repeats one or more times; $(a \mid b)$ chooses one. UPPER_CASE names lexer tokens.
`soft` NEWLINE tokens are retained by the lexer and ignored by the parser only in
the delimiter contexts listed under line structure below.

file = separator* top_decl (separator+ top_decl)* separator* EOF ;
separator = NEWLINE ;

top_decl = use_decl
| attribute* type_decl
| attribute* const_decl
| attribute* var_decl
| attribute* error_decl
| attribute* fn_decl
| attribute* extern_decl ;

use_decl = `use` module_path (`as` value_name)? ;
module_path = module_name (`.` module_name)* ;

attribute = `@` value_name (`(` argument_list? `)`)? NEWLINE ;
type_decl = `type` type_name comptime_params? `=` type_rhs ;
const_decl = `const` const_name (`:` type_expr)? `=` expression ;
var_decl = `var` value_name (`:` type_expr)? (`=` initializer)? ;
error_decl = `error` type_name ;
fn_decl = `fn` value_name comptime_params? signature block ;
extern_decl = `extern` `fn` value_name comptime_params? signature ;

signature = (`(` parameter_list? `)` (`->` return_spec)? ;
parameter_list = parameter (``,` parameter)\* `,`? ;
parameter = value_name `:` type_expr
| value_name `:` `...`
| `...` ;
return_spec = type_expr | (`(` type_list `)`) ;
type_list = type_expr (``,` type\_expr)+ `,`? ;
comptime_params = `[` comptime_param (``,` comptime\_param)\* `,`? `]` ;
comptime_param = comptime_name `:` comptime_kind ;
comptime_kind = `type` | `fn` | type_expr ;

type_rhs = struct_type | union_type | enum_type | union_enum_type | type_expr ;
struct_type = `struct` type_body ;
union_type = `union` type_body ;
enum_type = `enum` type_expr? enum_body ;
union_enum_type = `union` `enum` type_expr? union_enum_body ;
type_body = `{` soft* (field_decl (field_sep field_decl)* field_sep)? soft* `}` ;
enum_body = `{` soft* enum_member (field_sep enum_member)* field_sep? soft* `}` ;
union_enum_body = `{` soft* union_member (field_sep union_member)* field_sep? soft* `}` ;
field_sep = ``,` soft* ;
field_decl = value_name `:` type_expr ;
enum_member = type_name (`=` expression)? ;
union_member = type_name (`:` type_expr)? ;

type_expr = pointer_type
| slice_type
| array_type
| function_type
| comptime_type
| named_type ;
comptime_type = `type` ;
pointer_type = `\*` `const`? `shared`? type_expr ;
slice_type = `[` ` `]` `const`? `shared`? type_expr ;
array_type = `[` (expression | `\_`) `]` type_expr ;

```

function_type = `extern`? `fn` `(` function_type_parameters? `)`
              (`->` return_spec)? ;
function_type_parameters = function_type_parameter
                          (``,` function_type_parameter)*`,`? ;
function_type_parameter = (value_name `:`)? type_expr | `...` ;
named_type = qualified_name comptime_args? ;

block = `{` separator* statement_list? separator* `}` ;
statement_list = statement (separator+ statement)* ;
statement = binding_stmt
            | assignment_stmt
            | call_stmt
            | try_stmt
            | if_stmt
            | while_stmt
            | for_stmt
            | switch_stmt
            | when_stmt
            | defer_stmt
            | nocheck_stmt
            | shared_var_stmt
            | `break`
            | `continue`
            | return_stmt ;

binding_stmt = (`let` | `var`) binding (`:` type_expr)? `=` initializer ;
binding = value_name | `__` | `(` binding_item (``,` binding_item)+`,`? `)` ;
binding_item = value_name | `__` ;
initializer = `zero` | `undef` | try_call | expression ;
assignment_stmt = assignment_target assignment_op (`zero` | try_call | expression) ;
assignment_target = place | `(` place (``,` place)+`,`? `)` ;
assignment_op = `=` | `+=` | `-=` | `*=` | `/=` | `%=`
               | `+=` | `-=` | `*%` | `<<=` | `>>=` | `&=` | `^=` | `|=` ;
call_stmt = call_expr ;
try_stmt = `try` call_expr ;
try_call = `try` call_expr ;
return_stmt = `ret` return_values? ;
return_values = expression | `(` argument_list `)` ;
defer_stmt = `defer` (simple_statement | block) ;
simple_statement = binding_stmt | assignment_stmt | call_stmt ;
nocheck_stmt = `@` `nocheck` block ;
shared_var_stmt = `shared` `var` value_name `:` type_expr (`=` initializer)? ;

if_stmt = `if` expression block (`else` (`if` expression block | block))? ;
while_stmt = `while` expression block ;
for_stmt = `for` binding_item (``,` binding_item)? `in` for_subject block ;
for_subject = expression (`..` expression)? ;
when_stmt = `when` expression block (`else` block)? ;
switch_stmt = `switch` expression `{` separator* switch_arm+ `}` ;
switch_arm = (`case` case_pattern | `default`) `:` separator+
             statement_list? separator+ ;
case_pattern = expression (``,` expression)* (`as` value_name)? ;

expression_list = expression (``,` expression)+`,`? ;
expression = logical_or ;
logical_or = logical_and (`||` logical_and)* ;
logical_and = comparison (`&&` comparison)* ;
comparison = bit_or (compare_op bit_or)? ;
compare_op = `==` | `!=` | `<` | `<=` | `>` | `>=` ;
bit_or = bit_xor (`|` bit_xor)* ;
bit_xor = bit_and (`^` bit_and)* ;
bit_and = shift (`&` shift)* ;
shift = additive ((`<<` | `>>`) additive)* ;
additive = multiplicative ((`+` | `-` | `+`%` | `-`%) multiplicative)* ;
multiplicative = prefix ((`*` | `/` | `%` | `*`%) prefix)* ;
prefix = (`-` | `!` | `~` | `&` | `**`) postfix ;
postfix = primary postfix_part* ;
postfix_part = `.` identifier
              | `[` bracket_contents? `]`
              | `(` argument_list? `)` ;
bracket_contents = expression (`..` expression)?
                 | `...` expression?

```

```

        | argument_list ;
argument_list = expression (`,` expression)*`,`? ;
call_expr    = postfix ;
place        = `**` postfix ;

primary      = literal
              | qualified_name
              | `.` type_name
              | `unreachable`
              | `(` expression `)`
              | aggregate_literal ;
aggregate_literal = aggregate_type `{` soft* literal_item
                    (field_sep literal_item)* field_sep? soft* `}` ;
aggregate_type   = named_type | `[` (expression | `.`) `]` type_expr ;
literal_item     = value_name `:` expression | type_name (`:` expression)? | expression ;

literal       = INTEGER | FLOAT | STRING | RAW_STRING | CHARACTER
              | `true` | `false` | `nil` | `ok` ;
qualified_name = identifier (`.` identifier)* ;
comptime_args = `[` argument_list? `]` ;

identifier    = IDENTIFIER ;
value_name    = IDENTIFIER ;
module_name   = IDENTIFIER ;
type_name     = IDENTIFIER ;
const_name    = IDENTIFIER ;
comptime_name = IDENTIFIER ;

# Lexical productions. Keyword recognition happens after IDENTIFIER scanning.
KEYWORD       = `use` | `type` | `const` | `var` | `let` | `fn` | `ret`
              | `if` | `else` | `while` | `for` | `in` | `switch` | `case`
              | `default` | `break` | `continue` | `defer` | `try`
              | `struct` | `union` | `enum` | `error` | `when`
              | `true` | `false` | `nil` | `ok` | `as` | `zero` | `undef`
              | `extern` | `unreachable` | `shared` ;
IDENTIFIER    = ASCII_LETTER_OR_UNDERSCORE ASCII_LETTER_OR_UNDERSCORE* ;
INTEGER       = decimal_integer integer_suffix?
              | hex_integer integer_suffix?
              | octal_integer integer_suffix?
              | binary_integer integer_suffix? ;
decimal_integer = DIGIT (`_`? DIGIT)* ;
hex_integer     = `0x` HEX_DIGIT (`_`? HEX_DIGIT)* ;
octal_integer   = `0o` OCT_DIGIT (`_`? OCT_DIGIT)* ;
binary_integer  = `0b` BIN_DIGIT (`_`? BIN_DIGIT)* ;
integer_suffix  = `i8` | `i16` | `i32` | `i64` | `isize`
              | `u8` | `u16` | `u32` | `u64` | `usize` ;
FLOAT          = decimal_float float_suffix? ;
decimal_float  = decimal_integer `.` decimal_integer exponent?
              | decimal_integer exponent ;
exponent       = (`e` | `E`) (`+` | `-`)? decimal_integer ;
float_suffix   = `f16` | `bf16` | `f32` | `f64` ;
STRING         = `` string_element* `` ;
CHARACTER     = `` character_element `` ;
string_element = STRING_SCALAR | escape_sequence ;
character_element = CHARACTER_SCALAR | escape_sequence ;
escape_sequence = `\\n` | `\\t` | `\\r` | `\\\\` | `\\`" | `\\`" | `\\`"
              | `\\x` HEX_DIGIT HEX_DIGIT ;
RAW_STRING     = raw_string_0 | raw_string_1 | raw_string_2 | raw_string_3
              | raw_string_4 | raw_string_5 | raw_string_6 | raw_string_7
              | raw_string_8 ;

raw_string_0   = `r` RAW_BODY_0 `` ;
raw_string_1   = `r#` RAW_BODY_1 `#` ;
raw_string_2   = `r##` RAW_BODY_2 `##` ;
raw_string_3   = `r###` RAW_BODY_3 `###` ;
raw_string_4   = `r####` RAW_BODY_4 `####` ;
raw_string_5   = `r#####` RAW_BODY_5 `#####` ;
raw_string_6   = `r#####` RAW_BODY_6 `#####` ;
raw_string_7   = `r#####` RAW_BODY_7 `#####` ;
raw_string_8   = `r#####` RAW_BODY_8 `#####` ;

TRIVIA        = SPACE | DOC_COMMENT | LINE_COMMENT ;

```

```

SPACE          = ` `+ ;
DOC_COMMENT    = `///<` COMMENT_SCALAR* ;
LINE_COMMENT    = `///<` COMMENT_SCALAR* ;

# STRING excludes quote, backslash, LF and forbidden controls. A backslash begins
# exactly one of: n t r \\ \" \' 0 x HEX_DIGIT HEX_DIGIT. CHARACTER uses the same
# escapes.
# STRING_SCALAR is any Unicode scalar except quote, backslash, LF or a forbidden
# control. CHARACTER_SCALAR additionally must encode as one UTF-8 byte. A character's
# decoded value must be exactly one byte. RAW_BODY_N is the longest sequence of
# permitted UTF-8 source scalars not containing the corresponding closing delimiter;
# it may contain normalized LF and TAB but not NUL or another forbidden control.
# ASCII_LETTER is A-Z or a-z; DIGIT is 0-9; HEX_DIGIT adds
# A-F and a-f; OCT_DIGIT is 0-7; BIN_DIGIT is 0 or 1. The `_OR_UNDERSCORE` and
# `_DIGIT_OR_UNDERSCORE` classes have their literal meanings. These named character
# classes, COMMENT_SCALAR and RAW_BODY_N are lexer primitives, not parser
# productions. COMMENT_SCALAR excludes LF, NUL and forbidden controls but admits
# TAB. DOC_COMMENT is tested before LINE_COMMENT; both emit tooling trivia kind
# `comment`, with exact text retained for documentation attachment.

# Line structure. NEWLINE is always emitted by the lexer. It is `soft` only while
# parsing parentheses, brackets, an aggregate literal body, or a type body. It is a
# separator everywhere else, including ordinary blocks. A soft delimiter cannot
# cross the declaration, containing-block, case, default or EOF recovery barriers
# specified by spec.md §3.
soft          = NEWLINE ;

# Longest-match punctuation, in descending ambiguity order:
# ... .. -> == != <= >= << >> +% -% +% += -= *= /= %= +%= -%= *%=<=>= &= ^= |=
# && ||, then the remaining one-character punctuation and operators.

# Public keyword kinds are KW_USE, KW_TYPE, KW_CONST, KW_VAR, KW_LET, KW_FN,
# KW_RET, KW_IF, KW_ELSE, KW_WHILE, KW_FOR, KW_IN, KW_SWITCH, KW_CASE,
# KW_DEFAULT, KW_BREAK, KW_CONTINUE, KW_DEFER, KW_TRY, KW_STRUCT, KW_UNION,
# KW_ENUM, KW_ERROR, KW_WHEN, KW_TRUE, KW_FALSE, KW_NIL, KW_OK, KW_AS, KW_ZERO,
# KW_UNDEF, KW_EXTERN, KW_UNREACHABLE and KW_SHARED.
#
# Public punctuation kinds map exactly as follows:
# PUNCT_ELLIPSIS `...`; PUNCT_RANGE `..`; PUNCT_ARROW `->`;
# PUNCT_EQ_EQ `==`; PUNCT_BANG_EQ `!=`; PUNCT_LT_EQ `<=`; PUNCT_GT_EQ `>=`;
# PUNCT_SHIFT_LEFT `<<`; PUNCT_SHIFT_RIGHT `>>`; PUNCT_ADD_WRAP `+`;
# PUNCT_SUB_WRAP `-`; PUNCT_MUL_WRAP `*`; PUNCT_ADD_ASSIGN `+=`;
# PUNCT_SUB_ASSIGN `-=`; PUNCT_MUL_ASSIGN `*=`; PUNCT_DIV_ASSIGN `/=`;
# PUNCT_REM_ASSIGN `%=`; PUNCT_ADD_WRAP_ASSIGN `+%=`;
# PUNCT_SUB_WRAP_ASSIGN `-%=`; PUNCT_MUL_WRAP_ASSIGN `*%=`;
# PUNCT_SHIFT_LEFT_ASSIGN `<<=`; PUNCT_SHIFT_RIGHT_ASSIGN `>>=`;
# PUNCT_BIT_AND_ASSIGN `&=`; PUNCT_BIT_XOR_ASSIGN `^=`;
# PUNCT_BIT_OR_ASSIGN `|=`; PUNCT_AND_AND `&&`; PUNCT_OR_OR `||`;
# PUNCT_AT `@`; PUNCT_DOT `.`; PUNCT_COMMA `,`; PUNCT_COLON `:`;
# PUNCT_ASSIGN `=`; PUNCT_LPAREN `(`; PUNCT_RPAREN `)`;
# PUNCT_LBRACKET `[`; PUNCT_RBRACKET `]`; PUNCT_LBRACE `{`;
# PUNCT_RBRACE `}`; PUNCT_STAR `*`; PUNCT_SLASH `/`; PUNCT_PERCENT `%`;
# PUNCT_PLUS `+`; PUNCT_MINUS `-`; PUNCT_LT `<`; PUNCT_GT `>`;
# PUNCT_AMP `&`; PUNCT_CARET `^`; PUNCT_PIPE `|`; PUNCT_BANG `!`;
# PUNCT_TILDE `~`; PUNCT_UNDERSCORE `_`.
#
# The remaining public kinds are IDENTIFIER, INTEGER, FLOAT, STRING, RAW_STRING,
# CHARACTER, NEWLINE, EOF and INVALID. Exact `_` is PUNCT_UNDERSCORE rather than
# IDENTIFIER. Comments and spaces are trivia, never parser tokens.
#
# Public syntax-node kinds are exactly: File, UseDecl, Attribute, TypeDecl,
# ConstDecl, VarDecl, ErrorDecl, FnDecl, ExternDecl, ComptimeParam, Parameter,
# ReturnSpec, StructType, UnionType, EnumType, UnionEnumType, FieldDecl, EnumMember,
# UnionMember, PointerType, SliceType, ArrayType, FunctionType, NamedType, Block,
# Binding, BindingStmt, AssignmentStmt, CallStmt, TryStmt, ReturnStmt, DeferStmt,
# NocheckStmt, SharedVarStmt, BreakStmt, ContinueStmt, IfStmt, WhileStmt, ForStmt,
# WhenStmt, SwitchStmt, SwitchArm, UnaryExpr, BinaryExpr, FieldExpr,
# BracketPostfix, CallExpr, NameExpr, MemberExpr, GroupExpr, AggregateLiteral,
# LiteralItem, LiteralExpr and ErrorNode. Grammar helper productions are inlined;
# they do not create wrapper nodes. UnaryExpr and BinaryExpr carry their operator
# token as an ordered child. FieldExpr, BracketPostfix and CallExpr contain their
# receiver as their first child. Recovery creates ErrorNode and never a synthetic

```

```
# token. This closes both registries for tooling version 1 without making the public  
# tree mirror parser implementation detail.
```

pacman package manager

docs/pacman.md | 23421 bytes | sha256 11919ffd0144edcd

pacman — package manager design

Status: approved architecture for roadmap M6. This document fixes the package manager contract before implementation. It does not change neper v1 source semantics: `e.*`, `algo.*`, `text.*`, `crypto.*`, `fmt.*`, `gfx.*` and `ui.*` still ship with the toolchain, while pacman primarily acquires external `x.*` source.

1. Boundary and principles

Pacman is a dependency resolver, immutable source store, registry client, and publisher. It is not a second build system and never type-checks or compiles neper. The compiler remains deterministic and never accesses the network; pacman prepares a locked module map, then invokes the ordinary compiler command when asked.

The contract is:

1. The manifest records intent; the lockfile records the complete truth.
2. A locked build performs no dependency resolution and observes no mutable remote state.
3. Package contents are addressed and verified by cryptographic hash.
4. The global cache is immutable. A package cannot modify itself or another package after installation.
5. Module names remain path-derived inside a package. Pacman supplies an internal virtual source map; it does not copy a `node_modules` tree into the project.
6. The package graph is source-only and acyclic. Native libraries remain explicit compiler/linker inputs under the existing `extern` rules.
7. No dependency executes code merely because it was resolved, downloaded, or installed.

The package manager is called **pacman**, but its portable command spelling is `neper pacman`. A standalone `pacman` executable would collide with Arch Linux's system package manager. A distribution may provide a non-default `neper-pacman` launcher that forwards to the same command group.

2. Tier and namespace policy

- `e.*` is inseparable from a specific neper toolchain version. Pacman must reject a package that exports or replaces an `e.*` module.
- The toolchain's `algo.*`, `text.*`, `crypto.*`, `fmt.*`, `gfx.*` and `ui.*` modules follow the same rule. Their version is the language/toolchain version, not an independently resolved package version.
- Public third-party packages export modules beneath `x.<owner>.*`. The public registry verifies that the authenticated owner controls that prefix. Ownerless short names are invalid; curated packages use an explicit owner such as `x.microsoft.win32` or `x.khronos.vulkan`.
- A private registry may allocate its own owners, but two locked packages may not export the same fully qualified module name. Import aliases solve local qualifier collisions; they do not make duplicate module definitions legal.
- Package identity and module identity are separate. A package coordinate is `<registry>/<package-name>`, where a public package name begins `x.<owner>.`; one package may export several modules under its declared prefix.

This keeps the core tiers available offline with the compiler and focuses package resolution on `x.*`, where external versions and native dependencies actually exist.

3. Project manifest: `project.yaml`

`project.yaml` is human-authored intent. Pacman accepts a deliberately small YAML 1.2 subset: UTF-8 mappings, sequences, quoted or plain strings, booleans, and decimal integers. Anchors, aliases, tags, merge keys, duplicate keys, implicit timestamps, non-string mapping keys, and multiple documents are errors. Pacman rewrites the file in a canonical two-space layout without reordering comments across keys.

```
schema: neper-project-1
name: telemetry_agent
version: 0.4.0
```

```

language: "0.1"

targets:
  agent:
    root: src/main.e
    before: [generate_grpc]

registries:
  default: https://packages.neper.dev/v1
  internal: https://packages.example.test/neper/v1

dependencies:
  skia:
    package: x.graphics.skia
    version: ^1.2.0
  internal_tool:
    package: x.acme.custom_tool
    git: https://gitlab.example.test/acme/custom_tool.git
    ref: v1.0.4

assets:
  images/logo@1x:
    path: assets/logo.png
    media_type: image/png
    attributes: {base: images/logo, scale: "1", theme: any, locale: ""}
  images/logo@2x-dark:
    path: assets/logo@2x-dark.png
    media_type: image/png
    attributes: {base: images/logo, scale: "2", theme: dark, locale: ""}
  fonts/inter-regular:
    path: assets/Inter-Regular.ttf
    media_type: font/ttf
    attributes: {base: fonts/inter, scale: "1", theme: any, locale: ""}

tasks:
  generate_grpc:
    command: [tools/protoc, --neper_out, .neper/generated, api.proto]
    inputs: [api.proto]
    outputs: [.neper/generated/rpc/client.e]
    capabilities: [process]

```

Required keys are `schema`, `name`, `version`, and `language`. Optional keys are `targets`, `registries`, `dependencies`, `dev_dependencies`, `tools`, `assets`, and `tasks`. Unknown keys are errors, so a typo cannot silently change resolution. Pacman locates the project at the nearest ancestor containing `project.yaml`; that directory must also be the compiler project root containing `src/` or `lib/`.

Dependency keys such as `skia` are project-local labels used by CLI commands and lock records. A dependency selects exactly one source:

- Registry: package, version, and optional registry (default: default).
- Git: package, git, and exactly one of `ref`, `tag`, `branch`, or `rev`.
- Path: package and path, relative to the manifest. Path dependencies are for local development and make public publishing invalid.

`tools` is a mapping from a local label to `command`, exact `version`, and a `sha256` mapping keyed by host triple. Pacman resolves the executable without a shell, checks its reported version and bytes, and records the selected identity in the task result. A task using an external tool is portable only across the host triples for which the manifest supplies a digest; otherwise it is deliberately host-local and its outputs cannot satisfy a cross-host reproducibility claim.

Versions use SemVer 2.0.0. The supported constraints are an exact version, comparison sets, `^`, `~`, and comma-separated intersections. There are no optional dependencies, feature unification, target-conditional dependencies, or arbitrary version scripts in version 1. Target-specific `.e` files already express platform selection, and one platform-independent graph keeps a lockfile identical across hosts.

Every target has `root` and an optional ordered `before` task list. This is pacman metadata, not a new entry-point rule. `neper pacman build agent` runs only those root-declared tasks, resolves the root to `src/main.e`, and invokes `neper build src/main.e` with the locked module map. Direct `neper build <file.e>` remains valid and runs no task.

`assets` maps a unique normalized logical name to `path`, an ASCII-lowercase `media_type`, and an optional string-to-string `attributes` mapping. Paths are project-relative, may not escape the root after symlink resolution and must identify regular files. Logical names use `/` separators, contain no empty, `.` or `..` segment, and are sorted by UTF-8 bytes before linking. The raw file bytes are embedded unchanged; executable-section compression is deliberately outside version 1 so `e.asset.Asset.bytes` is always a zero-copy process-lifetime slice.

The conventional UI attributes are `base`, positive finite decimal `scale`, `theme` (`any`, `light` or `dark`) and a canonical BCP 47 `locale` or the empty fallback. `ui.asset` ignores unknown attributes but rejects malformed conventional ones. Ordinary `e.asset` callers may define other attributes. Asset names, metadata, sizes and SHA-256 values enter incremental and final link identity. Direct `neper build` outside a project embeds no assets. Dependency-package assets and external runtime assets are deferred; version 1 embeds only declarations in the root project.

Package manifests

Every dependency root contains `neper-package.json`, the canonical JSON package manifest already named by `docs/modules.md`. At M6 its version-1 fields are `schema:"neper-package"`, `version:1`, package name, `package_version`, language constraint, SPDX license, `exports`, and `dependencies`; source URL and digests are registry/lock metadata, not author-asserted identity. `exports` is the sorted complete list of modules in the archive. Dependency entries contain package coordinate, SemVer constraint, and registry name; public releases may not contain path or Git dependencies. This M6 constraint form supersedes the pre-pacman exact-dependency placeholder in the module plan—the lockfile, not a published package manifest, pins the selected transitive versions.

4. Lockfile: `project.lock`

`project.lock` is generated canonical JSON despite its extension: UTF-8, LF, sorted object keys where order is not prescribed below, two-space indentation, one final newline. JSON makes the truth format unambiguous and cheap for harnesses to validate; YAML's human conveniences are useful only in the intent file.

The root object contains:

- `schema:"neper-lock"` and `version: 1`;
- the canonical SHA-256 of `project.yaml` after parsing and schema normalization;
- the required language version and resolver version;
- every registry URL used, with its signed metadata snapshot/version;
- `packages`, sorted by package coordinate and version;
- the resolved direct dependency labels and the exact package record they select.

Every package record contains its coordinate, exact SemVer, source kind, immutable source identity, archive or tree SHA-256, normalized manifest SHA-256, licence identifiers, exported modules, and exact dependency edges. Registry records carry the registry URL and release digest. Git records carry URL, requested ref for audit only, resolved full commit ID, and canonical tree SHA-256. Path records carry the normalized project-relative path and tree SHA-256.

A branch or tag may move remotely; `sync` never follows it. The lockfile's commit and tree hash are authoritative. Git submodules are rejected in version 1 rather than introducing an unrecorded dependency graph. A yanked registry version remains installable from an existing lock but is not selected by new resolution.

The lockfile contains no machine-local cache path, credential, timestamp, host triple, or absolute project path. Resolving the same manifest against the same signed registry snapshot produces byte-identical lockfiles on Windows, Linux, macOS, and inside a container.

5. Resolution

Pacman uses PubGrub: it is fast for ordinary graphs and, unlike a bare SAT failure, produces a deterministic incompatibility explanation. Candidate ordering is highest non-yanked stable version first, then prereleases only when the manifest explicitly admits one. Package coordinate breaks all remaining ties by UTF-8 bytes.

Resolution reads immutable registry metadata snapshots into a local metadata cache. The chosen snapshot IDs enter the lockfile. Registry mirrors may serve identical content by digest, but fallback never changes package identity or the chosen version. Conflicting full module exports, dependency cycles, an incompatible language range, or two sources for one package coordinate are resolution errors.

Commands have deliberately different mutation rules:

- `add` edits the manifest and resolves a new lock.
- `remove` edits the manifest and resolves a new lock.
- `resolve` regenerates the lock from current manifest constraints.
- `upgrade` changes locked versions only within existing constraints.
- `upgrade --major <label>` edits that dependency's manifest constraint and resolves.
- `sync` never resolves or edits either file.

This makes review straightforward: constraint changes appear in `project.yaml`; newly selected transitive code appears in `project.lock`.

6. Sources: registry, Git, and path

Registries

The default public registry is configured by the toolchain, and a project may add named private registries. The lockfile always records the fully qualified registry URL; user-level aliases are never enough to reproduce a graph. Credentials come from the OS credential store or environment and never appear in project files, logs, or lockfiles.

Registry release records are immutable. An upload contains a normalized source archive, package manifest, SHA-256 digest, licence data, and publisher signature. Registry index metadata is signed and versioned; clients detect rollback and expired metadata. Publication is two-phase—upload and verify, then atomically expose the version—and an exposed coordinate/version can only be yanked, never replaced.

The protocol is static-HTTP/CDN friendly. Search is a separate convenience endpoint; resolution reads owner/package version metadata and content-addressed archives.

The release archive is deterministic `tar.zst`: path order by UTF-8 bytes, zero timestamps, uid and gid, regular files and directories only, modes `0644` or `0755`, and one `zstd` frame with a content checksum. Absolute paths, `./`, `..`, symlinks, hard links, device entries, case-folding collisions, and Windows reserved names are rejected. The release digest hashes the archive bytes; the tree digest hashes, in path order, each entry's path length and bytes, kind, normalized mode, content length, and content bytes. Git and path sources are converted to this same logical tree before hashing.

Git

Git is a first-class manifest source, but `pacman` does not embed a second Git implementation. It invokes a discovered `git` executable for Git dependencies, with arguments passed directly rather than through a shell. `neper pacman info` reports that capability. Registry and path dependencies require no Git installation.

`Pacman` maintains bare mirrors by normalized remote URL, fetches only the requested ref/commit, resolves it to a full commit, rejects submodules, exports a canonical tree, and stores that tree by SHA-256. SSH agent and credential behavior belongs to Git; `pacman` never copies credentials into its cache.

Paths

Path dependencies are read directly during local development. `sync --frozen` checks their locked tree hash and fails when contents differ. `publish` rejects a graph containing a path dependency. `neper pacman vendor` can replace a Git or registry dependency with an explicit project-relative path for audited or air-gapped use.

7. Global immutable cache

The default package store is:

- Windows: `%LOCALAPPDATA%\neper\pacman\v1`
- Linux and macOS: `$XDG_CACHE_HOME/neper/pacman/v1`, or the platform cache directory when `XDG_CACHE_HOME` is absent

`NEPER_PACMAN_CACHE` overrides it for CI and containers. A Distrobox or Docker setup mounts this directory as a cache volume; no project path is encoded within it.

Package trees live at `store/sha256/<digest>`, metadata at `metadata/<registry-id>`, and Git mirrors at `git/<remote-hash>`. Store entries are read-only after an atomic temporary-directory rename. Concurrent commands coordinate per digest,

verify after a contended install, and never expose partial content. A corrupt entry is quarantined, not repaired in place.

The compiler receives a read-only virtual module map from package module names to store files. It does not follow project-created symlinks and does not add arbitrary source roots. Diagnostics and build manifests identify dependency files by package coordinate plus root-relative path, never by a user-specific cache path.

cache verify, cache list, and cache gc are explicit. GC retains digests reached from supplied lockfiles or younger than a chosen age; builds never trigger it.

8. CLI

```
neper pacman init [--name NAME]
neper pacman add <package>[@constraint] [--registry NAME]
neper pacman add <package> --git URL (--ref REF|--tag TAG|--branch BRANCH|--rev COMMIT)
neper pacman add <package> --path PATH
neper pacman remove <label>
neper pacman resolve [--offline]
neper pacman sync [--frozen] [--offline]
neper pacman upgrade [label] [--offline]
neper pacman upgrade --major <label>
neper pacman build <target> [-- ARGS...]
neper pacman run <target> [-- ARGS...]
neper pacman task <name> [--allow CAP,...]
neper pacman vendor [label] [--into DIR]
neper pacman publish [--registry NAME]
neper pacman cache <verify|list|gc>
neper pacman info
```

init creates project.yaml, src/, and .gitignore entries for .neper/; it does not create a lock until dependencies exist. add, remove, resolve, and upgrade write both files through temporary files and atomically replace them only after the whole graph validates.

sync installs exactly the lock. It fails if manifest and lock disagree; --frozen also forbids path-tree changes and any metadata refresh. CI uses neper pacman sync --frozen, followed by ordinary build/test commands. --offline forbids all network access and reports every missing digest in one deterministic list.

The compiler never performs an implicit sync. If a locked digest is absent, build fails with the exact pacman command needed to obtain it. This keeps builds hermetic while making recovery obvious.

Every command supports the versioned --json stream conventions in docs/tooling.md. Progress is human stderr only; JSON mode emits typed progress and result records and leaves stderr empty under the same rules as other neper commands.

9. Tasks, not lifecycle hooks

Automatic dependency hooks are rejected. A transitive package must not run a shell, Node.js, or neper program during sync, install, or an ordinary build: that would make downloading code equivalent to executing it, mutate the global store, hide build control flow, and make host tools part of an unrecorded dependency graph.

Pacman instead provides **root-project tasks**:

- Only the root project.yaml may declare a task. Dependency manifests cannot inject one into a consumer.
- A task runs only through neper pacman task <name> or when a root target explicitly lists it in before. sync, install, and direct compiler commands never run tasks.
- command is an argument array, never a shell string. Shell interpretation requires the explicit shell capability. A command executable must be project-relative and included in inputs, name a neper target built from the locked graph, or reference a tools entry with exact version and per-host SHA-256. Node and similar host tools therefore work, but never through an unrecorded PATH lookup.
- Inputs, outputs, executable identity, arguments, allowed environment variables, and granted capabilities enter the task hash. Outputs live under .neper/generated/<task-hash>/ and are immutable inputs to compilation.
- An .e output's module name is its path below that generated directory. Pacman adds only the outputs declared by the selected target to the same read-only virtual module map as packages; duplicate project, package, or generated modules are an error before compilation.

- The default environment is empty except deterministic pacman variables; the working directory is a fresh scratch directory. Network, writable project files, shell, inherited environment, and processes beyond the named executable are denied unless individually declared and approved.
- The first interactive execution records capability approval in a user-local policy, never the lockfile. CI supplies explicit `--allow`; missing approval is an error, not a prompt.
- A task may use a neper executable that calls `e.proc`, but that grants no extra authority: the pacman sandbox still controls the child process.

There is no `post-install` hook. Generated code is a task output, and a package that needs generated release sources publishes those sources in its immutable archive.

10. Publishing and trust

`publish` performs format, test, licence, provenance, and dependency checks before any upload. It rejects dirty generated outputs, path dependencies, unpinned Git dependencies in the publish graph, files outside the package root, secrets matched by the registry's baseline scanner, duplicate module exports, and an archive whose normalized contents do not reproduce its declared digest.

The publisher signs the release digest with an account-bound key. Registry metadata uses separately rotated signing keys and a transparency log. The lockfile verifies content independently of transport security. `neper pacman audit` is reserved for the registry advisory protocol; M6 does not make vulnerability scanning part of compiler correctness.

11. Failure, offline, and reproducibility rules

- Network failure cannot alter an existing lockfile or store entry.
- Resolution and installation diagnostics are deterministic and redact credentials.
- Interrupted writes leave only removable temporary data.
- `sync --frozen --offline` succeeds from a populated cache without DNS, Git, registry, home-directory configuration, or wall-clock access.
- A clean cache populated from the lock produces the same virtual module map and package hashes on every supported host.
- Package source hashes, manifests, task outputs, and the exact module map enter the compiler build manifest and incremental cache identity.
- Registry timestamps and signature expiry affect acquisition, not a fully offline build from already verified content. Explicit `cache verify` can enforce current trust policy before an audited release.

12. Implementation sequence and acceptance

Roadmap M6 is delivered in four internal stages. P0 and P1 may start after M2 while GPU work proceeds; P2/P3 and the M6 completion gate require M4's mature platform and dynamic-linking surface, but do not wait for M5's Metal backend:

1. **P0 — formats and resolver:** strict manifest parser, canonical lock writer, SemVer, PubGrub, conflict explanations, module-export validation, golden fixtures.
2. **P1 — local store:** path dependencies, immutable content-addressed cache, atomic concurrency, offline/frozen sync, virtual module map, build-manifest integration.
3. **P2 — remote acquisition:** signed registry metadata and archives, Git adapter, credentials, mirrors, publish/yank, cache verification and garbage collection.
4. **P3 — controlled generation:** task hashing, scratch execution, capability policy, generated-source maps, JSON records, and cross-platform sandbox tests.

M6 is complete when:

- a clean machine can populate its cache from `project.lock`, disconnect, and produce byte-identical package maps and compiler outputs under `--frozen --offline`;
- Windows, Linux, macOS, and a container produce a byte-identical lock for the same manifest and signed registry snapshot;
- mutable Git refs cannot change a locked sync;

- concurrent syncs neither duplicate nor expose partial store entries;
- a malicious transitive package cannot execute during resolution or installation;
- denied task capabilities fail before the task starts and declared outputs are byte-identical on all hosts on which that task claims portability;
- the multi-package workload GP-12 in `docs/general-purpose-verification.md` passes.

13. Explicit deferrals

Version 1 deliberately excludes optional feature unification, platform-specific lock graphs, dependency plugins, dependency-provided hooks, Git submodules, binary package installation, automatic native-library discovery, workspace/monorepo manifests, federated identity, and a background daemon. Each adds hidden state or another graph; none is required to make external neper source reproducible.

Tooling protocol

docs/tooling.md | 19935 bytes | sha256 d00e697dc284e6d9

neper tooling protocol — version 1

Status: normative companion to `docs/spec.md` §13–§14. Nothing here requires an implementation before the compiler work begins; it fixes the interface that every implementation and harness must share.

The mandatory post-M2 M2.5 work in `post-m2-llm-hardening.md` specifies planned semantic context queries, causal diagnostics, snapshot-bound edits and analysis-completeness reporting. These are future requirements, not additions to the closed v1 protocol or currently advertised commands. Their implementation must version the affected schemas, capability records and compatibility identities before M3 begins.

The planned device-selection surface in [spec §10](#) also requires a versioned runtime inventory/report contract during M2.5 H18/H21, implemented with the backend in M3 (CUDA in M4). It is **not** an extra field or command in the closed v1 stream. The revised contract must expose:

- Requested backend/index or exact selector separately from the actual opened `DeviceInfo`; failed requests have no fabricated selected-device record.
- Canonical backend-prefixed UUID text, explicit key validity, display name, kind, capacity/known flag, capabilities and floor support. Encode `u64` capacities without losing JSON precision. CPU and Vulkan software devices are not labeled hardware GPU executions. Driver names are inert data, not harness instructions.
- Inventory observation identity and completeness, limits/errors, and explicit selection/fallback decisions made by the application. An inventory snapshot is not a reservation or the same thing as a compiler workspace snapshot.
- Distinct missing, ambiguous, unsupported, resource-exhausted and lost outcomes; indices never masquerade as persistent keys. Revalidate exact keys at open.

Hardware inventory is a runtime observation, not part of deterministic source compilation or a reason for ordinary `check` to probe drivers. Device-specific pipeline caches use H22's device/driver/options identity, not the selection UUID alone. Freeze exact record schemas and capability advertisement during M2.5; only advertise discovery when implemented. No compiler-wide automatic GPU-selection environment variable or unversioned `neper info` extension is introduced here.

1. Version and stream envelope

H11/H18 also require the installed-library capability inventory in `stdlib-hardening.md`. The module-plan `surface`, tier and milestone describe design/commitment, not current availability. The new versioned inventory binds compiler/language/target, qualified symbol/signature hash, canonical import/alias, implementation status, stability, ownership/allocation/failure contracts and test provenance. Planned-only records must never be returned as callable APIs. Freeze and validate its schema in M2.5 rather than adding undocumented fields to closed v1 `info` records. Static docs checks cannot manufacture implementation or execution evidence.

On every finite command, `--json` means UTF-8 JSON Lines on stdout and no human text on stdout. `neper dap` is the sole exception: it uses DAP's own framed JSON transport and rejects `--json` and `--absolute-paths`. The first line is always this header, with keys in the shown order:

```
{ "schema": "neper-stream", "version": 1, "record": "header", "command": "index", "tool_version": "0.1.0", "language_version": "0.1", "grammar_revision": 1 }
```

Every later object has a `record` discriminator. Diagnostics are records in the same stream; `stderr` is empty unless the process cannot initialize JSON output, in which case it contains one ASCII line and the process exits 2. A command emits its final `result` record last, including on source or option failure. Unknown record-level keys are forbidden at version 1. `result.data` is the sole extension map: its command-specific scalar keys are described with that command and unknown keys there must be ignored. Integers outside JSON's exactly representable range are decimal strings.

`neper info --json` emits the header followed by one `info` record and one successful `result`. `info` contains `tool_version`, `language_profiles`, `commands`, `host_target`, `build_targets`, `cpu_levels`, and `features`, with every collection sorted by UTF-8 bytes. Each `language_profiles` entry contains `language_version`, `grammar_revision`, `stream_version`, and `experimental`; versions are related by that entry rather than by parallel-array position. This is the capability query; harnesses never scrape `--help`.

--language-version MAJOR.MINOR selects one advertised version and defaults to the newest non-experimental version. An unsupported value is E-CLI-9999 before source is read. The selected version fixes its grammar revision, stream versions and conformance corpus; a build manifest records all three.

2. Sources and spans

A source identifier is:

```
{"root":"project-src","path":"parse/expr.e"}
```

root is one of project, project-src, project-lib, toolchain-lib, operand, generated, or external. project addresses non-neper inputs relative to the project root; external uses a generator-supplied stable URI as path. Every other path uses /, is relative to that root, contains no . or .. segment, and is NFC-normalized UTF-8. For a file outside all roots, operand uses the file's basename; a stdin source uses the normalized --path spelling. Only one such operand exists in a command, so that identity is unambiguous. Machine output contains no absolute path unless --absolute-paths is explicitly passed; then a separate absolute_path field is added and never replaces the source identifier.

A span is half-open and has this exact shape:

```
{
  "source":{"root":"project-src","path":"parse/expr.e"},
  "byte_start":128,
  "byte_end":133,
  "line":9,
  "column":5,
  "end_line":9,
  "end_column":10,
  "column_utf16":5,
  "end_column_utf16":10
}
```

Byte offsets address the original input bytes, including a BOM and physical CRLF; line and column values address the normalized source of spec §3. Lines and both column forms are one-based. column counts Unicode scalar values and column_utf16 counts UTF-16 code units. A zero-width location has equal byte offsets. Generated locations use the same representation. For recovery after invalid UTF-8, each Unicode maximal subpart occupies one logical scalar and one UTF-16 unit. A maximal subpart is the longest prefix, from one through three bytes, that could begin a valid sequence before the first missing, disallowed or non-continuation byte; an illegal lead or continuation byte therefore occupies one unit. This convention affects locations only, never acceptance or decoded source.

Captured bytes use one representation everywhere. Valid UTF-8 is a JSON string; otherwise the value is { "encoding":"base64", "data":"..." } using canonical padded RFC 4648 base64 with no whitespace. Empty bytes use the empty JSON string.

3. Diagnostics and fixes

A diagnostic record is:

```
{
  "record":"diagnostic",
  "severity":"error",
  "code":"E-TYPE-0003",
  "message":"cannot pass `i64` where `i32` is expected",
  "span":{"..."},
  "related":[{"message":"parameter declared here","span":{"...}}],
  "fixes":[{"
    "message":"convert explicitly",
    "applicability":"maybe",
    "edits":[{"span":{"..."},"replacement":"i32(value)"}]
  }]
}
```

severity is error or note. A note has no independent exit effect and carries parent, the zero-based diagnostic-record index of its error. applicability is machine only when applying all edits cannot change a valid program's behavior; otherwise it is maybe. Edits within one fix are non-overlapping and sorted by source then descending byte_start, so they can be applied without offset repair. Replacement text is normalized UTF-8 with LF endings.

Every diagnostic field is present: an error has `parent:null`, a location-free command diagnostic has `span:null`, and empty `related` or `fixes` arrays are `[]`.

Codes are allocated from the checked registry `docs/diagnostics.md` and have the form `E-<CATEGORY>-<NNNN>`. Categories are stable semantic names, not document section numbers. A code is never reused, even after its diagnostic is retired. Diagnostics sort by source identifier, `byte_start`, severity (error before its notes), code, then message. Parser recovery and the required primary diagnostics are part of the conformance corpus.

4. Token and syntax commands

```
neper tokens [--json] [--path VIRTUAL.e] <FILE.e|->
neper parse  [--json] [--path VIRTUAL.e] <FILE.e|->
```

- reads UTF-8 source from stdin. `--path` supplies its source identity and module name and is required with -; it has no filesystem effect. In text mode `tokens` prints one token per line and `parse` prints an indented tree. In JSON mode:

- `tokens` emits one token record per token, including `NEWLINE` and `EOF`. Fields are `record`, `index`, `kind`, `lexeme`, `span`, `leading_trivia`. Trivia items have `kind` (`space`, `comment`, or `bom`), exact text, and `span`. `bom` may occur only once, as the first token's first trivia item (the `EOF` token when the file contains only a BOM). Invalid bytes produce diagnostics and an `INVALID` token whose `lexeme` uses the base64 captured- bytes object from §2; other lexemes are strings. Token output therefore remains total.
- `parse` emits token records followed by one `syntax` record. A syntax node has `kind`, `span`, `token_start`, `token_end`, and `ordered_children`; a child is either `{"node":...}` or `{"token":N}`. The tree is lossless because tokens retain lexemes and trivia. Error recovery inserts an `ErrorNode` but never invented source text.

A token span covers its lexeme and excludes leading trivia. `EOF` has an empty lexeme and a zero-width span at the original byte length. String lexemes retain the original decoded source spelling: a `NEWLINE` lexeme is `"\\n"`, `"\\r\\n"`, or `"\\r"` according to the input, even though the parser sees normalized LF; a leading BOM is retained as `bom` trivia even though the parser ignores it. A malformed string, character or raw-string token that reaches its matching closing delimiter is one `INVALID` token through that delimiter. Without a closing delimiter, an ordinary string or character stops before the next physical line ending or at `EOF`, while a raw string extends through `EOF`. Leading trivia is ordered by byte offset: the BOM is one item, every maximal non-empty run of ASCII spaces is one `space` item, and each `//` comment through but excluding its physical line ending is one `comment` item. Because every physical line ending is a `NEWLINE` token, no trivia item crosses a normalized line boundary. If an `INVALID` token interrupts a comment, its valid prefix and suffix become separate `comment` items on the adjacent tokens; a suffix item therefore need not begin with `//`. Concatenating each trivia text and token lexeme in index order reconstructs every original byte (using decoded base64 for an object lexeme). A syntax-node span runs from its first token's `byte_start` through its last token's `byte_end`, excluding that first token's trivia; a recovery node with no token is zero-width at the recovery point.

Token kinds and syntax-node kinds are closed registries in `docs/grammar.ebnf`; a new kind requires a grammar-revision increase. Parsing is syntactic: the common `BracketPostfix` node is classified as indexing, slicing, or comptime arguments only during name resolution, so a parser never needs a symbol table.

5. Symbol and reference index

`neper index --json` emits a `symbol` record for every module, function, kernel, test, type, field, enum/union member, constant, module variable, parameter, local, error, extern and intrinsic. Its closed `kind` values are `module`, `fn`, `kernel`, `test`, `type`, `field`, `member`, `const`, `module_var`, `parameter`, `local`, `error`, `extern`, and `intrinsic`.

Each symbol contains `id`, `kind`, `name`, `qualified_name`, `module`, `signature`, `span`, `selection_span`, `container_id`, `attributes`, and nullable `documentation` from spec §3's attached `///` lines. `id` is the zero-based record number among symbol records and is stable for identical source. Nullable fields are present as JSON `null`; no field is omitted.

A reference record contains `source_span`, `role`, `spelling`, `target_id`, `target_qualified_name`, and `origin`. `role` is `import`, `type`, `call`, `read`, `write`, `address`, `protocol`, or `instantiate`; `origin` is `source` or `compiler`. Compiler-origin protocol, iterator and formatting calls use the smallest source span that caused the call and name the concrete target. This makes call graphs and protocol resolution inspectable without duplicating compiler logic. If source errors prevent resolution, both target fields are JSON `null`; the reference is still emitted so an editor can retain the occurrence.

Records sort by source identifier, span start, record kind, then qualified name. Null-span intrinsics sort first by qualified name. The stream ends with `{"record":"result","ok":true,"exit_code":0,"data":{"symbols":N,"references":N}}`.

6. Formatting contract

`neper fmt` is a canonical **layout** formatter, not a semantic normalizer. It does not remove explicit parentheses, change a numeric base, rename identifiers, reorder declarations, or rewrite ordinary strings as raw strings. Its complete v1 contract:

- UTF-8 without BOM, LF endings, one final LF, no trailing whitespace or trailing blank lines; indentation is four ASCII spaces and tabs are never emitted.
- One ASCII space surrounds binary and assignment operators and follows commas and colons. No space appears just inside delimiters, before comma, before a call or comptime-argument list, around `.`, or around a range's `...`
- Opening block braces remain on the declaration/control line; closing braces occupy their own indentation level. Empty blocks are `{}`. `else` follows `}` on the same line. A non-empty statement occupies its own line.
- A bracketed list that exceeds 100 Unicode-scalar columns breaks after its opening delimiter, contains one element per line with a trailing comma, and closes on its owning indentation level. Otherwise it is one line and has no trailing comma. The formatter never splits a token, comment, or expression outside a legal continuation delimiter; such an indivisible line may exceed 100 columns, making the width a deterministic wrapping threshold rather than a source-validity limit.
- Exactly one blank line separates top-level declarations. Attribute lines remain adjacent to their declaration, are sorted by attribute name, and duplicates are compile errors. Inside a block, runs of blank lines collapse to one.
- Comment text is preserved byte-for-byte after newline normalization. A trailing comment is preceded by one space. Comments are never column-aligned or moved across a declaration/statement. A comment between attributes and a declaration is illegal because attributes must be immediately adjacent.
- Contiguous comment-free `use` declarations at the start of a file sort by module path then alias. Other declarations retain source order. Raw strings use the smallest safe delimiter count; all other literal spellings are preserved.

`neper fmt --check` writes nothing and exits 1 if output would differ. `neper fmt -` reads stdin and writes only formatted source to stdout; `--json` instead emits one formatted record whose `text` contains that source. On syntax failure no formatted output is produced. Idempotence and every rule above are fixed by golden fixtures in the conformance corpus.

7. Test, build and command results

Test records are buffered and emitted in deterministic module/source order, never completion order. They contain `record:"test"`; the last test record is followed by one `record:"test_summary"`, then the command `result`. The summary contains explicit `passed`, `failed`, `crashed`, `timeout`, `total`, and `duration_ms` fields.

`run --json` captures the program's complete stdout and stderr and emits one `run` record with `process_exit_code`, `stdout`, `stderr`, and nullable `trap`; captured bytes use §2's representation. It does not stream raw child bytes into the JSONL channel. `dis --json` emits one disassembly record per function with `symbol`, `target`, and `text`. These records precede the final command `result`.

Every build writes `.neper/<mode>/build-manifest.json`. It is one canonical JSON object with `schema:"neper-build-manifest"`, `version:1`, `tool_version`, `language_version`, `grammar_revision`, `target`, `mode`, `root_module`, `inputs`, `dependencies`, `libraries`, `assets`, `artifacts`, and `options`. Inputs and dependencies carry source identifiers and SHA-256 hashes; libraries carry the requested name, ordered search roots, resolved source identifier or absolute external path, and SHA-256; assets carry logical name, source identifier, media type, sorted attributes, byte size and SHA-256; artifacts carry project-relative paths, kind, target and SHA-256. Arrays use the deterministic order in which their corresponding compiler operation is specified, and object keys use the order listed here.

Every JSON command ends with a `result` record containing `ok`, `exit_code`, and a `data` object holding command-specific counts or artifact identifiers. Exit statuses remain those in `spec.md §13`; the record and process status must agree.

8. Generated source maps

A generator may place `<file>.e.map.json` beside `<file>.e`. It contains `schema:"neper-source-map"`, `version:1`, the generated source identifier and hash, and `mappings`. Each mapping has `generated_span`, `original_span`, and nullable `name`; both spans use §2's exact shape and the generated span must be non-empty. Mappings are sorted, non-overlapping, and may leave generated regions unmapped. Paths obey §2 above.

When the generated hash matches, diagnostics use the original span as primary and include the generated span as a related location. A stale or malformed map produces `E-TOOL-0001`; the compiler still analyzes the generated file to report

independent source errors, but the command fails and writes no final artifact until the map is updated or removed. Source maps never affect parsing, type checking, code generation or cache identity.

9. Conformance and compatibility

`tests/conformance/` is a normative corpus shipped with the specification. It has `accept/`, `reject/`, `format/`, `tokens/`, `parse/`, and `tools/`. Every fixture has expected versioned output; rejected-source fixtures fix primary diagnostic codes and recovery points. Implementations must pass the corpus at each claimed language/grammar/stream version.

Source-derived output is compared byte-for-byte. Runtime `duration_ms` values are schema-checked as nonnegative and replaced with `0` before golden comparison; no other field is volatile.

The corpus also contains a generated-code benchmark reported, not gated by the language-conformance suite, for each release: tokens per syntax node under each named tokenizer, parse/type-check success rate, mean repair attempts, identifier-collision rate, and formatted diff size. The separate general-purpose claim is gated by `general-purpose-verification.md`. No claim of universal LLM-token optimality is made: tokenizers differ. Syntax changes are accepted only when this report shows the intended trade rather than inferring it from character count.

The machine-checkable JSON Schema for every record and document in this file is `schemas/neper-v1.schema.json`.

Sequence, key-order, sorting and cross-record constraints that JSON Schema cannot express remain normative in this prose.

UI framework

docs/ui-framework.md | 12939 bytes | sha256 a2e408a4dab4c22e

Neper declarative GPU UI framework

Status: experimental architecture proposal. The exact public declarations live in `module-apis.md`; module identities, tiers, dependency edges and blockers live in `modules.json`.

1. Goal and boundary

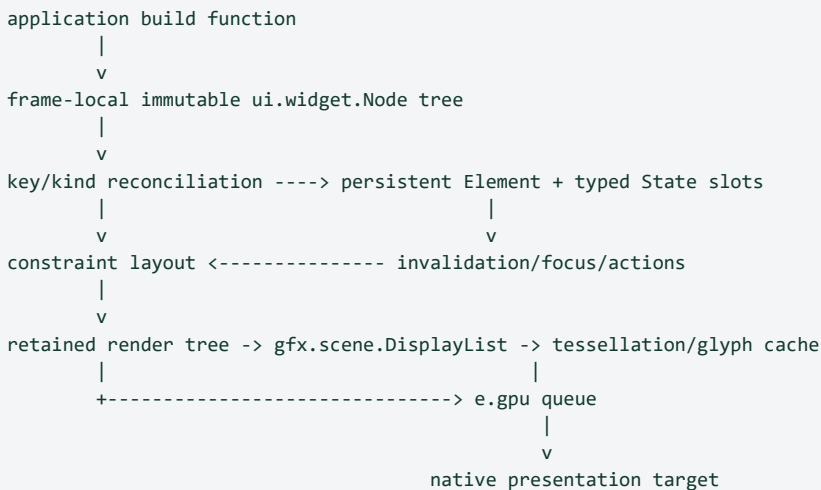
The framework builds portable desktop applications from declarative Neper values and renders them through `e.gpu`. It borrows Flutter's useful separation between immutable widget descriptions, persistent elements and render state without copying its managed-object runtime.

Version 1 targets native desktop windows on Windows, Linux and macOS. It provides:

- immutable widget trees written as ordinary Neper expressions;
- keyed reconciliation and explicit persistent state;
- constraint-based flex, grid, stack and scrolling layout;
- shaped international text, vector drawing, images and GPU composition;
- ordered pointer, keyboard, focus, text-input and IME events;
- an accessibility semantics tree;
- manifest-declared embedded assets with deterministic scale, theme and locale variants;
- deterministic animation and headless widget/snapshot testing.

It does not add language syntax, inheritance, runtime reflection, CSS, JavaScript, a browser DOM, web-page rendering, a tracing garbage collector or hidden unbounded allocation. Mobile, browser and embedded targets require separate proposals because their lifecycle and presentation contracts differ materially from desktop.

2. Architecture



Three lifetimes are deliberately distinct:

1. **Frame:** widget nodes, child slices and build-time strings occupy a bounded frame arena. The arena resets only after reconciliation and scene compilation.
2. **Runtime:** elements, state, focus, scroll offsets and retained render data occupy bounded generation-checked slots and size/alignment state cells in the application's arena. Retired cells return to bounded per-layout free lists.
3. **GPU:** textures, glyph atlases, meshes and compiled scenes are resources owned by `gfx.scene.Renderer` and explicitly released or closed.

No persistent layer stores a pointer into the frame arena. Debug builds poison the reset frame range, making violations visible under the existing arena rules.

Embedded assets

`project.yaml1` declares immutable assets that the linker places in the executable. `e.asset` exposes allocation-free lookup by logical name; its strings, metadata and bytes remain valid for the process lifetime and require no filesystem at runtime. Every input path, byte size and SHA-256 is recorded in the canonical build manifest.

`ui.asset` groups physical entries by their base attribute and deterministically selects locale, theme and display-scale variants. Fonts become zero-copy `text.shape.Font` values. Images use a caller-supplied decoder; standard adapters use `fmt.png`, `fmt.jpeg` or `fmt.webp`, while project-specific formats remain separate. Decoded pixels occupy a caller scratch arena and are uploaded before it resets. A caller-sized cache owns the GPU textures and explicitly evicts or releases them. There is no global asset manager.

```
use ui.asset as ui_asset

let request = ui_asset.Request{
  scale: window_metrics.scale,
  locale: "es-ES",
  theme: .Dark,
}
let logo = try ui_asset.texture(&cache, scratch, "images/logo", request, png_decoder)
let font = try ui_asset.font("fonts/inter", request, 0)
```

3. Declarative code

Declarative UI is a library convention expressed with literals and functions, not a new grammar form. A helper can return a complete value tree:

```
use e.mem
use gfx.paint
use text.layout as text_layout
use ui.input
use ui.layout as ui_layout
use ui.style
use ui.widget

type Counter = struct { value: i64 }

fn counter_action(ctx: *void, event: input.Event) -> err {
  let _ = event
  let counter = mem.cast[*Counter](ctx)
  counter.value = counter.value + 1
  ret ok
}

fn counter_view(counter: *Counter, label_style: text_layout.Style) -> widget.Node {
  let title = widget.Node{
    key: 1,
    kind: widget.Kind{ Text: widget.Text{
      value: "Count",
      style: label_style,
      color: paint.srgb8(20, 20, 20, 255),
    } },
    style: style.defaults(),
    children: nil,
  }
  let button = widget.Node{
    key: 2,
    kind: widget.Kind{ Button: widget.Button{
      action: widget.Action{ ctx: counter, invoke: counter_action },
      enabled: true,
    } },
    style: style.defaults(),
    children: nil,
  }
  ret widget.Node{
    key: 10,
    kind: widget.Kind{ Flex: ui_layout.Flex{ axis: .Vertical, main: .Start, cross: .Center, gap: 8.0 } },
    style: style.defaults(),
    children: [_]widget.Node{ title, button },
  }
}
```

```
}

```

The example uses explicit callback context because Neper has no closures. Higher-level helpers may reduce literal noise, but their result remains the same `Node` value and must not create hidden state.

4. Reconciliation and state

Within one parent, a nonzero `widget.Key` must be unique. Reconciliation matches a new child to an old element by (`key`, `kind`) when the key is nonzero and by (`position`, `kind`) otherwise. A duplicate key is `DuplicateKey`. A kind change replaces the element and retires its state generation.

`widget.state[T]` associates (`element`, `local key`) with one concrete type. Reusing the pair with another type returns `StateType`. The returned pointer is stable until that state is removed or the runtime closes, but application code must not retain it after either boundary. Removing an element recursively invalidates its state, captured pointers, focus and pending actions before its slot can be reused. State cells are recycled only for the same size/alignment class; `state_bytes` and `state_classes` bound both storage and fragmentation. Values that own external resources require an explicit application cleanup action because Neper has no destructors.

An `Action.ctx` retained by an element must remain valid until that element is reconciled away. It may point to application state or `widget.state` storage, never to the frame arena. Strings and slices needed after reconciliation are copied into bounded runtime storage or compiled into the retained render/semantics data; no persistent structure borrows a widget node.

Rebuild is explicit invalidation. Dispatching an action may mutate application or widget state and call `widget.invalidate`; `ui.animation.request` schedules another frame. There is no observation graph, implicit setter interception or global event bus.

5. Layout, paint and text

Layout uses Flutter-like downward constraints and upward sizes. Parents then assign child positions. Every constraint and result is finite and non-negative; invalid or contradictory limits return `Invalid`. Flex and grid distribute only bounded remaining space, with deterministic sibling-order rounding.

Painting emits a backend-neutral display list. Save/restore scopes transforms and clips; malformed nesting is `Invalid`. `gfx.scene` validates the whole list before submitting GPU work, tessellates paths, maintains bounded generation-checked texture and glyph caches, and rebuilds resources after a recoverable presentation change. Device loss remains `Lost` and is never silently hidden.

Text follows this pipeline:

```
UTF-8 + style
-> Unicode bidi/script segmentation and font fallback
-> OpenType glyph shaping
-> line breaking, alignment and caret geometry
-> glyph atlas entries and display-list commands

```

Fonts are caller-provided byte slices. Platform font discovery is intentionally not part of the pure shaping modules; a later host service may enumerate fonts through a reviewed `e.os` primitive. This keeps tests reproducible and prevents hidden file I/O.

6. Input, frames and accessibility

`ui.input.Queue` preserves native event order. Pointer coordinates are logical pixels. Pointer capture, keyboard focus and IME composition are explicit. Dispatch performs hit testing from the last committed render tree, then runs capture and bubble phases internally; version 1 exposes one action at the target rather than a mutable event-object hierarchy. A close request is data—the application decides when to close the window.

`ui.app.step` is the scheduling primitive. It waits up to the supplied duration, drains input, invokes actions, samples time once, rebuilds invalidated subtrees, reconciles, lays out and presents at most one frame per window. `run` merely repeats `step`, so games, editors and external event loops keep control.

Accessibility is derived from semantic widget properties and published as a separate stable-ID tree. Paint-only decorations do not appear. Platform requests such as focus, press or setting a value route back through the same action

system. The framework does not claim accessibility until the native bridge blocker is delivered and tested with each platform's assistive APIs.

7. Platform and GPU prerequisites

The proposal records four blockers instead of weakening the platform-boundary rule:

- `native-window-api`: reviewed `e.os` primitives for windows, monitors, event delivery, clipboard, cursor, pointer capture and IME;
- `gpu-presentation-api`: `e.gpu` support for presentation targets, textures, render passes, blending, synchronization, resize and device-loss recovery;
- `native-accessibility-api`: reviewed publication and action bridges for Windows, Linux and macOS accessibility systems.
- `embedded-asset-linking`: canonical manifest ingestion, incremental hashing, read-only executable sections and linker-generated `e.asset` registry symbols.

These primitives must be specified in `spec.md` and added to the exact `e.os` and `e.gpu` catalogues before the blocked modules can move from proposal to implementation. UI modules never declare platform externs directly.

8. Resource limits and failure

Every runtime is constructed with limits for elements, state entries, depth and display-list commands. Event queues, texture tables, scene tables, glyph atlases and frame arenas are also bounded at construction. Exceeding a bound returns `TooLarge` or `OutOfMemory`; the last presented frame remains valid. Reconciliation either commits a complete new tree or leaves the previous tree active.

Callbacks return `err`. The framework stops dispatch along the current event, preserves runtime invariants and returns the error from `step`; it does not throw, log globally or continue with a partially committed tree. Resource closure is explicit and logically linear under the language's existing handle rules.

9. Testing and delivery

The first useful vertical slice is deliberately narrower than the full catalogue:

1. CPU reference renderer, geometry, solid paint and deterministic snapshots;
2. synthetic window/input, box/flex layout, text, button and keyed state;
3. one native window plus one GPU presentation backend;
4. shaping/fallback, clipping, images, scrolling and animation;
5. accessibility bridge and keyboard/IME conformance;
6. remaining desktop backends and promotion review.

Required tests include keyed reorder/state preservation, state retirement, nested constraint cases, bidi and fallback goldens, clipping/compositing pixels, input ordering, focus and pointer capture, resize/DPI changes, device loss, every resource limit, accessibility tree/action fixtures, and leak-free repeated open/close. GPU backends are compared against the CPU reference with declared per-pixel tolerances.

The remaining `asset/text/UI/scene` modules remain experimental until a general-purpose application workload builds a multi-window, keyboard-accessible application on Windows, Linux and macOS and records stable memory, frame-time and compatibility results.

The pure `gfx.geometry`, `gfx.paint` and `gfx.image` dependency closure is instead extended and stabilizes together with `fmt.png`, `fmt.jpeg` and `fmt.webp` under `image-codec-conformance` (D84, `stdlib-hardening.md` SL10). It needs no native window, GPU initialization or GP-15 promotion. The UI consumes that stable value layer; its own rendering/presentation and lifecycle contracts remain experimental.

Source manifest

Every file rendered above, with its size and SHA-256. Re-running scripts/build-docs-pdf.py over the same inputs reproduces the same content.

Path	Bytes	SHA-256
docs/spec.md	284280	ceb18f69bd8454687544f379bbe6495a24c38448619f7f8f1ff17d48be3d412f
docs/modules.md	17305	4aa0a467c332183440554135224e4b60c19bfd78d894356fbfd91ac97dc41119
docs/module-apis.md	164529	c583bb4fdc9839550ee8875a3a6a32bc6a096b65e022404b78ad524316feda6f
docs/roadmap.md	65289	1c4ac20fcc8216e8aad311854a727682ed0a5d378dde8fec6d4709eb5544a7a2
docs/decisions.md	102251	8a97c866b02d739ef792840fd2a4d6187e3118362d5a019b18b5fcaa54f519b8
tests/selfhost/fixtures/diagnostics/src/broken.e	41	3a67b023b3c2691a65644f931b7878270b2849505df32386618f99ffb91db5a3
tests/selfhost/fixtures/diagnostics/src/main.e	80	537b5ec38009fdc2f4ceac8ede0928e36f403c17f58a51fc680e490a5c563121
docs/general-purpose-verification.md	20066	bf0a3306f4d0f68d089c5a9da61214d755ea1a6ae7da108e2d4032b963281d8e
docs/grammar.ebnf	14393	aca153df102d4e29faae6938d81d3e1e024f18d9a64da924174d0933bb0b5e4
docs/pacman.md	23421	11919ffd0144edcd4f48629fdcf7b4197747a97564f23e28663a94fc4735e276
docs/tooling.md	19935	d00e697dc284e6d976ba7cf90eff9c21d23587360ce844fd89560b87c12fdc2e
docs/ui-framework.md	12939	a2e408a4dab4c22ecc0bbbf8fcc679c80a53dd1573e86d24777f0c350cadb49